AVR Assembly Programming

Gary

Contents

Chapter 1

Why Assembly? Toolchain Setup

1.1 What Is Assembly Language?

Every program your computer runs is ultimately a sequence of binary numbers: machine code that the CPU decodes and executes. Assembly language is the human-readable form of that machine code. You are no longer asking a compiler to choose the instructions for you; you are choosing them yourself.

Higher-level languages (C, Python, Rust) are *compiled* or *interpreted* — they add layers of abstraction that trade control for convenience. Assembly gives you almost no abstraction: you decide every instruction, every register, and often every cycle count.

1.1.1 When do you actually need assembly?

Situation	Why assembly wins
Tight timing constraints	You control the exact cycle count
Hardware bring-up / no OS	No runtime support — you write everything
Interrupt service routines	Worst-case latency is deterministic
Extremely small flash budget	No compiler overhead or padding
Learning how CPUs work	Forces deep understanding of architecture
Hand-optimising hot paths	Compiler output is a starting point, not the ceiling

In the embedded world, assembly is not just historical trivia. It still shows up in bootloaders, startup code, interrupt paths, cycle-counted loops, and the smallest pieces of firmware where you cannot afford a vague understanding of what the CPU will do next.

1.2 Target: ATtiny3217

This book uses the **ATtiny3217** — an 8-bit AVR from Microchip's tinyAVR 1-series. It is a modern, capable microcontroller with:

Feature	Detail
Architecture	AVRxt (enhanced AVR core)

Feature	Detail
Flash	32 KB
SRAM	2 KB (at 0x3800–0x3FFF)
Clock	Up to 20 MHz internal oscillator
Programming	UPDI (1-wire, replaces ISP)
I/O	New-style PORT peripheral (not classic DDRx/PORTx)
Packages	24-pin VQFN, SOIC, SSOP

The **ATtiny3217 Curiosity Nano** development board is a USB-powered board with an on-board debugger/programmer. It has one yellow LED on **PA3** and exposes all device pins. This book uses this board for all hardware examples.

That choice is deliberate. The chip is modern enough to show the newer AVR peripheral model, but small enough that the architecture is still easy to hold in your head.

> **Note:** All concepts carry over to other tinyAVR 1-series devices (ATtiny3216, ATtiny1617, etc.) and with small adjustments to any AVR.

1.3 The AVR-GAS Toolchain

We use the **GNU Assembler** (`avr-as`), part of the `avr-gcc` toolchain. This is the same assembler `avr-gcc` uses under the hood when it compiles C for AVR. Using it directly keeps the toolchain familiar while removing the compiler as a middleman.

The toolchain has four main tools:

```
avr-as        Assembler: .S source → .o object file
avr-ld        Linker:    .o object(s) → .elf executable
avr-objcopy   Converter: .elf → .hex (Intel HEX for flashing)
avr-objdump   Inspector: disassemble, inspect sections, count cycles
```

You do not need to memorise all four on day one. The practical workflow is: assemble source into an object file, link it into an executable image, convert that image into something the programmer can flash, then inspect the result when you want to verify what really happened.

1.3.1 Installing the Toolchain

Debian / Ubuntu / Kali:

```
sudo apt install gcc-avr binutils-avr avr-libc avrdude
```

`avr-libc` is needed for the device header files (`<avr/io.h>`) that define register names and addresses.

Arch Linux:

```
sudo pacman -S avr-gcc avr-binutils avr-libc avrdude
```

macOS (Homebrew):

```
brew tap osx-cross/avr
brew install avr-gcc avrdude
```

Verify installation:

```
avr-as --version
avr-ld --version
avr-objcopy --version
```

Expected output (version numbers may differ):

```
GNU assembler (GNU Binutils) 2.42
GNU ld (GNU Binutils) 2.42
GNU objcopy (GNU Binutils) 2.42
```

1.4 The Build Pipeline

Source goes through four stages before reaching the MCU:

```
blink.S
  │
  │  avr-as -mmcu=attiny3217 -o blink.o blink.S
  ▼
blink.o         ← relocatable object (ELF, machine code + symbol table)
  │
  │  avr-gcc -mmcu=attiny3217 -nostartfiles -o blink.elf blink.o
  ▼
blink.elf       ← linked executable (ELF, absolute addresses assigned)
  │
  │  avr-objcopy -O ihex blink.elf blink.hex
  ▼
blink.hex       ← Intel HEX — what avrdude writes to flash
  │
  │  pymcuprog write -d attiny3217 -t uart -f blink.hex
  ▼
ATtiny3217 flash ← running program
```

If you are new to embedded builds, this pipeline is worth pausing over. The source file is not flashed directly. It is first turned into machine code, then assigned final addresses, then repackaged into a format the programmer can write into the device's flash memory.

1.4.1 Why two link steps?

We use `avr-gcc` as the linker driver rather than calling `avr-ld` directly because it already knows the right device-specific details: the memory layout, the default linker script, and the expected section placement. We could drive the linker manually, but that adds complexity before it teaches us anything useful.

The `-nostartfiles` flag matters because we are not building a normal C program. We do not want the C runtime to insert its own startup sequence; we want to write the reset path ourselves so we can see exactly how execution begins.

1.5 First Program: Blink

The classic "hello world" of embedded is not printing text. It is making a pin change state in a visible, repeatable way. Here, that means toggling the board LED forever.

Hardware: ATtiny3217 Curiosity Nano, LED0 on PA3.

File: `ch01_intro/src/blink.S`

1.5.1 Understanding the ATtiny3217 PORT Peripheral

The ATtiny3217 uses a **new-style** PORT peripheral, unlike older AVR devices (ATmega328P etc.) which used `DDRx`, `PORTx`, `PINx` registers in low I/O space.

On ATtiny3217, each port has a block of memory-mapped registers. PORTA lives at base address **0x0400**:

```
Offset  Register       Function
0x00    PORTA_DIR      Direction: 1 = output, 0 = input
0x01    PORTA_DIRSET   Write 1 to set pin as output (non-destructive)
0x02    PORTA_DIRCLR   Write 1 to set pin as input  (non-destructive)
0x03    PORTA_DIRTGL   Write 1 to toggle direction
0x04    PORTA_OUT      Output value
0x05    PORTA_OUTSET   Write 1 to drive pin HIGH (non-destructive)
0x06    PORTA_OUTCLR   Write 1 to drive pin LOW  (non-destructive)
0x07    PORTA_OUTTGL   Write 1 to toggle pin     (atomic!)
0x08    PORTA_IN       Read current pin state
```

The `SET`/`CLR`/`TGL` variants are key: they let you change one pin without reading the register first, eliminating the classic read-modify-write race condition.

That is one of the nicest features of the newer AVR GPIO design. Instead of reading the whole port, modifying one bit in software, and writing the whole byte back, the peripheral can perform the bit change for you.

Because these registers are above address 0x3F, we cannot use `IN`/`OUT` instructions (which only reach 0x00–0x3F). We use `STS` (store to SRAM address) instead. The header `<avr/io.h>` defines all register names for us.

So even in this tiny program, the hardware layout is already shaping the instruction choices. We are not using `STS` by style preference; we are using it because that is the instruction family that can reach these addresses.

1.5.2 The Code

```
#include <avr/io.h>

/* ── Vector table ──────────────────────────────────────────── */
.section .vectors, "ax", @progbits
.global __vectors

__vectors:
    jmp     main            ; reset vector: first instruction after power-on
```

When the ATtiny3217 resets, execution begins at **byte address 0x0000**. The CPU does not “look for `main`” by name. It simply fetches whatever instruction is stored at the reset vector and executes it. We place a `JMP main` there so the first thing the chip does is branch into our real program.

That is an important mental shift if you are coming from C: function names are for the assembler and linker, not for the CPU. The CPU only sees addresses and instructions.

```
.section .text

main:
    /* ── Stack pointer init ─────────────────────────────────────────── */
    ldi     r16, lo8(RAMEND)     ; RAMEND = 0x3FFF → lo8 = 0xFF
    out     SPL, r16
    ldi     r16, hi8(RAMEND)     ; hi8 = 0x3F
    out     SPH, r16
```

SP (the stack pointer) must be valid before any PUSH, CALL, or RCALL. If it is wrong, the CPU will store return addresses and saved registers in the wrong place, and the program will collapse quickly.

On the ATtiny3217, reset hardware already loads SP = RAMEND, so these writes are technically redundant in this exact program. We still do them because they make the startup state explicit, and because this is the pattern you need on many older AVR parts where the hardware does not fully set the stack up for you.

RAMEND expands to 0x3FFF (last byte of ATtiny3217 SRAM). lo8() and hi8() are assembler expressions that extract the low and high bytes of a 16-bit value.

SPL and SPH are in low I/O space (addresses 0x3D and 0x3E), so we reach them with OUT.

```
    ldi     r16, (1 << 3)        ; bitmask for PA3 = 0x08
    sts     PORTA_DIRSET, r16    ; set PA3 as output
```

LDI loads a constant into a register. STS stores that register value to a memory-mapped peripheral address. PORTA_DIRSET expands to 0x0401, and writing 0x08 there means "set bit 3 of the direction register."

In plain language: we are telling the chip that PA3 is now an output pin. We are not yet turning the LED on or off; we are only telling the hardware that this pin should be driven by the CPU rather than treated as an input.

```
loop:
    ldi     r16, (1 << 3)
    sts     PORTA_OUTTGL, r16    ; atomically toggle PA3
```

PORTA_OUTTGL flips PA3's output state. If it was high, it becomes low; if it was low, it becomes high. The useful part is that the flip happens inside the hardware peripheral. We do not have to read the old state first, and there is no risk of losing a concurrent change while doing a software read-modify-write.

```
    ldi     r19, 25
delay_outer:
    ldi     r25, 0
    ldi     r24, 0
delay_inner:
    sbiw    r24, 1               ; subtract 1 from r25:r24 (2 cycles)
    brne    delay_inner          ; branch if not zero     (2 cycles)
    dec     r19
    brne    delay_outer

    rjmp    loop
```

SBIW subtracts an immediate from a 16-bit register pair. Here, r25:r24 acts as the inner delay counter. It counts down from 65535 to 0. Each time it reaches zero, BRNE stops looping, DEC `r19` reduces the outer counter by one, and the whole process repeats until the outer loop also finishes.

This is not a precise timer. It is a simple busy-wait delay that burns CPU cycles. That makes it ideal for a first example because you can understand it with nothing more than arithmetic and branches.

After reset, the ATtiny3217 does not normally run at the full oscillator rate. It starts from `OSC20M` with a **divide-by-6 prescaler**. That gives **3.3 MHz** if the `FUSE.OSCCFG` frequency select is 20 MHz, or **2.66 MHz** if it is 16 MHz. Assuming the 20 MHz setting, the loop timing is roughly: 25 × 65536 × 4 cycles ÷ 3,330,000 ≈ **2 seconds** per toggle (about 1 Hz blink). If you later change the clock prescaler, oscillator source, or fuse setting, the blink rate changes proportionally.

That last point is worth remembering early: software delay loops are only as accurate as your clock configuration. Change the clock, and the visible timing changes too.

> **Instruction introduced: LDI** `LDI Rd, K` — Load immediate value K into register Rd. Rd must be r16–r31. 1 cycle. Affects no flags.

> **Instruction introduced: OUT** `OUT A, Rr` — Write register Rr to I/O address A (0x00–0x3F). 1 cycle. Affects no flags.

> **Instruction introduced: STS** `STS k, Rr` — Store register Rr to SRAM address k (0x0000–0xFFFF). 2 cycles. Affects no flags. 32-bit instruction (two 16-bit words).

> **Instruction introduced: SBIW** `SBIW Rd, K` — Subtract immediate K (0–63) from register pair Rd+1:Rd. Valid pairs: r25:r24, r27:r26 (X), r29:r28 (Y), r31:r30 (Z). 2 cycles. Affects C, Z, N, V, S flags.

> **Instruction introduced: BRNE** `BRNE k` — Branch if Z flag is clear (last result was not zero). 2 cycles if taken, 1 cycle if not. Range: ±63 words from current PC.

> **Instruction introduced: DEC** `DEC Rd` — Decrement register Rd by 1. 1 cycle. Affects Z, N, V, S flags. Does NOT affect C flag — careful when using in multi-byte arithmetic.

> **Instruction introduced: RJMP** `RJMP k` — Relative jump. PC = PC + k + 1. Range: ±2047 words. 2 cycles. Affects no flags.

> **Instruction introduced: JMP** `JMP k` — Absolute jump to any address in 64K word program space. 3 cycles. 32-bit instruction. Affects no flags.

1.6 Building and Flashing

1.6.1 Step 1: Assemble

```
cd book/ch01_intro/src
avr-as -mmcu=attiny3217 -o blink.o blink.S
```

`-mmcu=attiny3217` tells the tools exactly which device we are targeting. That choice affects instruction availability, predefined symbols, and which register definitions appear through `#include <avr/io.h>`.

No output on success. On error, `avr-as` prints the filename, line number, and problem.

1.6.2 Step 2: Link

```
avr-gcc -mmcu=attiny3217 -nostartfiles -o blink.elf blink.o
```

The linker is the stage where your program stops being a pile of pieces and becomes a real image with fixed addresses. It decides where `.vectors` and `.text` live in flash and resolves any symbol references between sections.

`-nostartfiles` suppresses the C runtime startup (`crt0.o`). Our `.vectors` section and `main` label take its place.

1.6.3 Step 3: Convert to HEX

```
avr-objcopy -O ihex blink.elf blink.hex
```

The `.elf` is the rich build artifact for tools and humans. The `.hex` is the lean transport format for programming the chip. Same program, different packaging.

1.6.4 Step 4: Inspect (optional but recommended)

```
avr-objdump -d blink.elf
```

Sample output:

```
blink.elf:     file format elf32-avr

Disassembly of section .vectors:

00000000 <__vectors>:
   0:   0c 94 02 00     jmp     0x4 <__ctors_end>

Disassembly of section .text:

00000004 <__ctors_end>:
   4:   0f ef           ldi     r16, 0xFF       ; RAMEND low byte
   6:   0d bf           out     0x3d, r16       ; SPL
   8:   0f e3           ldi     r16, 0x3F       ; RAMEND high byte
   a:   0e bf           out     0x3e, r16       ; SPH
   c:   08 e0           ldi     r16, 0x08       ; PIN3 mask
   e:   00 93 01 04     sts     0x0401, r16     ; PORTA_DIRSET
  ...
```

This is one of the healthiest habits you can build early: do not blindly trust source code. Inspect what the toolchain actually produced. `avr-objdump` shows the addresses, the machine-code bytes, and the final instruction sequence that will land in flash.

On this device/linker setup, `avr-objdump` may label the first `.text` address as `__ctors_end`; that is still the same address as our `main` label in this minimal program.

1.6.5 Step 5: Flash

ATtiny3217 Curiosity Nano (onboard debugger via USB):

```
pymcuprog write -d attiny3217 -t uart -f blink.hex
```

UPDI adapter (e.g. FT232R or CH340 with SerialUPDI):

```
avrdude -c serialupdi -p attiny3217 -P /dev/ttyUSB0 -b 115200 \
        -U flash:w:blink.hex:i
```

Successful flash output:

```
avrdude: AVR device initialized and ready to accept instructions
avrdude: writing flash (40 bytes):
avrdude: 40 bytes of flash written
avrdude: verifying flash memory against blink.hex:
avrdude: 40 bytes of flash verified
avrdude done.  Thank you.
```

The LED on PA3 should now blink at approximately 1 Hz.

1.7 A Minimal Makefile

Typing the build steps by hand is useful once because it teaches the pipeline. Typing them forever is just friction. Save this as `ch01_intro/src/Makefile`:

```
MCU     = attiny3217
TARGET  = blink
SRCS    = blink.S

AS      = avr-as
CC      = avr-gcc
OBJCOPY = avr-objcopy
OBJDUMP = avr-objdump
PROG    = serialupdi

ASFLAGS = -mmcu=$(MCU)
LDFLAGS = -mmcu=$(MCU) -nostartfiles

.PHONY: all flash disasm clean

all: $(TARGET).hex

$(TARGET).o: $(TARGET).S
    $(AS) $(ASFLAGS) -o $@ $<

$(TARGET).elf: $(TARGET).o
    $(CC) $(LDFLAGS) -o $@ $<

$(TARGET).hex: $(TARGET).elf
    $(OBJCOPY) -O ihex $< $@

flash: $(TARGET).hex
    pymcuprog write -d $(MCU) -t uart -f $<

disasm: $(TARGET).elf
    $(OBJDUMP) -d $<

clean:
    rm -f $(TARGET).o $(TARGET).elf $(TARGET).hex
```

Now build with `make`, flash with `make flash`, inspect with `make disasm`.

1.8 What Just Happened — Mental Model

Before moving on, fix this picture in your mind. This is the first real mental model of the whole book:

```
      Flash (32 KB)                    SRAM (2 KB)
  0x0000 ┌────────────────┐          0x3800 ┌────────────────┐
         │ JMP main       │ ← reset         │                │
  0x0004 │ main:          │                 │   stack grows  │
         │   ldi, out...  │                 │                │
         │   loop:        │          0x3FFF │ ← SP (RAMEND)  │
         │   sts, sbiw... │                 └────────────────┘
         │   rjmp loop    │
  0x7FFF └────────────────┘

CPU reads instructions from Flash.
CPU reads/writes variables in SRAM.
Registers (r0–r31) are inside the CPU — fastest possible storage.
```

The program counter starts at 0x0000 after reset, fetches the `JMP` at the reset vector, lands in `main`, runs straight through the setup code, and then spins forever around `loop`. Nothing mystical is happening. The CPU is just fetching, executing, and updating state one instruction at a time.

1.9 Summary

Concept	Key point
Assembly	One instruction = one CPU operation. No abstraction.
AVR-GAS	GNU Assembler, invoked as `avr-as`. GAS syntax.
Build pipeline	`.S` → assemble → `.o` → link → `.elf` → convert → `.hex` → flash
ATtiny3217 PORT	New-style: memory-mapped registers, use `STS`/`LDS`. `OUTTGL` is atomic.
Stack pointer	Must be valid before any subroutine call; on ATtiny3217 reset already loads `SP = RAMEND`.
Default clock	OSC20M ÷ 6 after reset: 3.3 MHz or 2.66 MHz, depending on `FUSE.OSCCFG`.

1.10 Exercises

1. Change the blink rate to 2 Hz (halve the delay loop count). Recalculate the required `r19` value.

2. Use `PORTA_OUTSET` and `PORTA_OUTCLR` instead of `PORTA_OUTTGL`. Make the LED on for a longer period than it is off (asymmetric blink).
3. Run `avr-objdump -d blink.elf` and find:
   - The byte address of the `rjmp loop` instruction.
   - The encoding of `sts 0x0407, r16` — how many bytes does it take?
   - The total size of the `.vectors` section.
4. Add a second LED on PA5. Both LEDs should blink in opposite phases (one on while the other is off).

Next: Chapter 2 — AVR Architecture Overview

Chapter 2

AVR Architecture Overview

Before writing a single instruction, you need a mental model of what the AVR CPU is actually doing: where code lives, where data lives, what the registers are for, and what hidden state changes after each instruction. Without that model, assembly feels like memorising magic spells. With it, most instructions become straightforward and predictable.

The target device in this book, the ATtiny3217, is a useful concrete example: it is an 8-bit AVR running at up to 20 MHz, with 32 KB of flash, 2 KB of SRAM, 256 bytes of EEPROM, a hardware multiplier, and a single-pin UPDI programming and debugging interface. Those numbers matter because assembly programming is always resource-aware. A “small” stack frame or lookup table is only small relative to the actual memory on the chip.

2.1 Harvard Architecture

The AVR is a **Harvard** architecture. This means it has **two separate memory spaces** — one for instructions (flash) and one for data (SRAM) — connected to the CPU by separate buses.

```
                    ┌─────────────────────┐
                    │       CPU           │
                    │  ┌──────────────┐   │
  Program bus ──────┼──┤ Instruction  │   │
  (read-only)       │  │  Register    │   │
                    │  └──────┬───────┘   │
                    │         │ decode    │
                    │  ┌──────▼───────┐   │
                    │  │ ALU / regs   │   │──── Data bus (R/W)
                    │  └──────────────┘   │
                    └─────────────────────┘
          │                                         │
    ┌─────▼─────┐                          ┌────────▼───────┐
    │   FLASH   │                          │      SRAM      │
    │  32 KB    │                          │      2 KB      │
    │ read-only │                          │   read/write   │
    │ at runtime│                          │                │
    └───────────┘                          └────────────────┘
```

Why does this matter to the programmer?

Because on AVR, "memory" is not one big uniform place. Where a value lives determines how you access it.

When your program is running, the CPU fetches instructions from **flash** automatically. You do not manually load the next instruction yourself; the program counter and instruction fetch hardware handle that in the background. That is why your code can live safely in non-volatile flash while your working data changes constantly in SRAM.

Your **variables**, your **stack**, and most of the peripheral register space you work with as data live in **SRAM/data space**. That is the memory area you read and write with ordinary load/store instructions such as `LD`, `LDS`, `ST`, and `STS`. So if you push a register, call a subroutine, store a loop counter, or write to a peripheral register, you are operating in the data space, not in program flash.

Flash is different. It holds your program, and on many AVR parts it also holds constant tables or strings that you do not want to waste SRAM on. But flash is not ordinary read/write RAM. You cannot treat it like a normal destination for `ST` and expect the program image to change. On the ATtiny3217, writes to flash and EEPROM go through the Nonvolatile Memory Controller (`NVMCTRL`), using protected command sequences covered later in the book. Older classic AVR devices expose flash self-programming through `SPM`; do not assume that model applies unchanged to tinyAVR 1-series devices.

This also affects **read-only data** such as lookup tables. On classic AVR, reading data back from flash requires `LPM`, because flash is on the program side of the machine. On AVRxt parts such as the ATtiny3217, flash is also mapped into the data address space, so some reads can be done through that mapping with ordinary data-memory instructions. Chapter 5 shows both styles and when each applies.

The practical rule is simple:

- If the value is temporary, changing, or part of the stack, think **SRAM/data space**.
- If the value is your code or a fixed table that should survive power loss, think **flash/program space**.

This separation is why you cannot simply think "an address is an address." On AVR, you must use the instruction that matches the address space you are trying to access.

2.2 ATtiny3217 Memory Map

The ATtiny3217 uses the **AVRxt** core (the GNU toolchain names this architecture `avrxmega3`). It has an important difference from classic AVR (ATmega328P): **flash is also mapped into the data address space** at offset 0x8000. This means you can read flash with ordinary `LD` instructions when you use the mapped address — no need for `LPM` for that access path.

That feature is convenient, but do not let it blur the architectural picture. Flash is still program memory. The chip is simply giving you a second way to read it. Thinking in terms of "program space" versus "data space" is still the right model.

```
Data Address Space (what LD/ST/LDS/STS see):

0x0000 ┌──────────────────────────────────┐
       │  I/O registers (IN/OUT range)    │  0x0000–0x003F  (64 bytes)
0x003F ├──────────────────────────────────┤
0x0040 │  Extended I/O / Peripherals      │  0x0040–0x0FFF
       │  (LDS/STS only, not IN/OUT)      │
0x0FFF ├──────────────────────────────────┤
```

```
0x1000 │  NVM I/O registers and data      │  0x1000–0x13FF
0x13FF ├──────────────────────────────────┤
0x1400 │  EEPROM — 256 bytes              │  0x1400–0x14FF
0x14FF ├──────────────────────────────────┤
0x1500 │  (reserved)                      │
0x37FF ├──────────────────────────────────┤
0x3800 │  SRAM — 2 KB                     │  0x3800–0x3FFF
0x3FFF ├──────────────────────────────────┤
0x4000 │  (reserved)                      │
0x7FFF ├──────────────────────────────────┤
0x8000 │  Flash — 32 KB (read-only)       │  0x8000–0xFFFF
       │  mapped into data space          │
0xFFFF └──────────────────────────────────┘

Program Address Space (what the PC addresses, separate bus):

0x0000 ┌──────────────────────────────────┐
       │  Flash — 16K words (32 KB)       │
       │  Byte addr = word addr × 2       │
0x3FFF └──────────────────────────────────┘
  (word addresses; byte addresses go to 0x7FFF)
```

Key numbers to memorise:

Symbol	Value	Meaning
`E2START` / `EEPROM_START`	`0x1400`	First EEPROM byte in data space
`E2END` / `EEPROM_END`	`0x14FF`	Last EEPROM byte
`FLASHEND`	`0x7FFF`	Last byte address in flash
`RAMSTART`	`0x3800`	First SRAM byte
`RAMEND`	`0x3FFF`	Last SRAM byte (init SP here)
`MAPPED_PROGMEM_START`	`0x8000`	Flash base in data address space

> **Key point:** When you see a linker script or header file refer to `MAPPED_PROGMEM_START`, it means 0x8000 — where flash appears in the data bus. On classic AVR, flash is *not* in the data bus at all.

If you are new to memory maps, read the diagram like this:

- The **low addresses** are the control area: CPU and peripheral registers.
- The **NVM/EEPROM region** is non-volatile data and device metadata, not general scratch RAM.
- The **middle SRAM block** is your working memory.
- The **high mapped-flash block** is read-only program memory made visible from the data side on AVRxt devices.

That one picture explains why some operations use `IN`/`OUT`, some use `LDS`/`STS`, and some use flash-specific mechanisms.

There are three common ways an address can appear in real code:

What you mean	Address base	Typical instruction
Program flash as code	`0x0000` word address	PC fetch, `RJMP`, `CALL`

What you mean	Address base	Typical instruction
Program flash as bytes via program bus	`0x0000` byte address	`LPM` with Z
Program flash mapped as data	`0x8000` byte address	`LD`, `LDS`

This is the source of many off-by-two and off-by-0x8000 mistakes. If a label is used as a branch target, the CPU treats it as a program address. If the same label is used as table data, you must be clear about whether the code is using `LPM` addressing or mapped-data addressing.

2.3 The Register File

The AVR CPU has **32 general-purpose 8-bit registers**, named `r0` through `r31`. They are directly connected to the ALU. In one CPU clock, the core can fetch register operands, execute an ALU operation, and write the result back to the register file. Compare that to SRAM: a load (`LDS`) takes 3 cycles; a store (`STS`) takes 2.

This is one of the main reasons assembly on AVR can be so efficient. If you can keep a value in a register, you avoid repeated trips to SRAM. A large part of "good AVR assembly" is really just good register management.

The ATtiny3217 instruction set has 135 instructions and includes a hardware multiplier. Multiplication is still slower than ordinary register ALU work, but it is a fixed two-cycle hardware operation rather than a long software shift-and-add routine. Chapter 6 uses that distinction when comparing arithmetic strategies.

```
Register file layout:

r0    ┌────────┐   General purpose. Some instructions can only use r0
r1    │        │   (e.g. MUL stores result in r1:r0).
r2    │        │
r3    │        │   r0 is often used as a temporary by the compiler.
...   │        │   By convention, keep r1 = 0x00 at all times when
r15   │        │   calling C code (GCC ABI requirement).
      ├────────┤
r16   │        │   ← LDI can only load immediate into r16–r31.
r17   │        │     Most two-operand arithmetic works on any register,
...   │        │     but immediate operands restrict you to r16–r31.
r23   │        │
      ├────────┤
r24   │        │   SBIW/ADIW work on r25:r24, r27:r26, r29:r28, r31:r30.
r25   │        │
      ├────────┤
r26   │   XL   │ ┐
r27   │   XH   │ ┘ X pointer register (r27:r26) — indirect addressing
      ├────────┤
r28   │   YL   │ ┐
r29   │   YH   │ ┘ Y pointer register (r29:r28) — indirect + stack frame
      ├────────┤
r30   │   ZL   │ ┐
r31   │   ZH   │ ┘ Z pointer register (r31:r30) — indirect + flash read
      └────────┘
```

2.3.1 Register Pairs (16-bit Pointers)

The top six registers form three **pointer pairs** used for indirect memory access:

Pair	Registers	Name	Special use
X	r27:r26	X-pointer	Indirect load/store
Y	r29:r28	Y-pointer	Indirect load/store, stack frame pointer
Z	r31:r30	Z-pointer	Indirect load/store, `LPM`, indirect jumps/calls

When you see `LD r0, X` in assembly, it means "load the byte at the address stored in the X register pair into r0." We cover this in depth in Chapter 5.

In practice, think of X, Y, and Z as your built-in address registers. The ordinary registers hold values; the pointer registers hold locations.

2.3.2 LDI Restriction

`LDI` — load immediate — **only works on r16–r31**. This is one of the most common beginner mistakes.

```
ldi r5, 42     ; ERROR: illegal operand — r5 not allowed with LDI
ldi r16, 42   ; OK
```

If you need a constant in r0–r15, load it into r16–r31 first, then `MOV` it:

```
ldi r16, 42
mov r5, r16    ; now r5 = 42
```

That may feel arbitrary at first, but it is simply part of the AVR instruction encoding. Some instructions have fewer bits available to describe operands, so they only work on part of the register file. You will see this kind of limit often in small instruction-set architectures.

2.4 The Status Register (SREG)

`SREG` is an 8-bit register at I/O address **0x3F** (accessible with `IN`/`OUT`). Every arithmetic and logic instruction updates some of its bits. Branch instructions read these bits to decide whether to jump.

You can think of `SREG` as the CPU's short-term memory of "what just happened." Did the last result become zero? Did it overflow? Was there a carry? Branches do not inspect your registers directly; they inspect these flag bits.

```
SREG bit layout:

Bit:  7    6    5    4    3    2    1    0
    ┌────┬────┬────┬────┬────┬────┬────┬────┐
    │ I  │ T  │ H  │ S  │ V  │ N  │ Z  │ C  │
    └────┴────┴────┴────┴────┴────┴────┴────┘
      │    │    │    │    │    │    │    │
      │    │    │    │    │    │    │    └── Carry
      │    │    │    │    │    │    └─────── Zero
      │    │    │    │    │    │
```

```
│     │     │     │     │     └───────────── Negative
│     │     │     │     └─────────────────── Overflow (two's complement)
│     │     │     └───────────────────────── Sign (N XOR V)
│     │     └─────────────────────────────── Half-carry (BCD arithmetic)
│     └───────────────────────────────────── Transfer (bit copy)
└─────────────────────────────────────────── Global Interrupt Enable
```

2.4.1 Flag by Flag

C — Carry Set when an addition produces a result > 255 (unsigned overflow) or a subtraction borrows. Also used by shift instructions (the bit shifted out). Used for multi-byte arithmetic (`ADC`, `SBC`).

```
ldi r16, 0xFF
ldi r17, 0x01
add r16, r17    ; result = 0x00, C=1 (carry out), Z=1
```

Z — Zero Set when the result of an operation is exactly zero. The most-used flag. `BRNE` (branch if not equal) branches when Z=0; `BREQ` branches when Z=1.

```
ldi r16, 1
dec r16         ; r16 = 0, Z=1
brne somewhere  ; NOT taken — Z is set
breq somewhere  ; taken   — Z is set
```

N — Negative Set when bit 7 of the result is 1 (result is negative in two's complement).

V — Overflow Set when two's-complement arithmetic overflows. For example, adding two positive numbers and getting a negative result.

```
ldi r16, 0x7F   ; +127 (largest positive signed 8-bit)
ldi r17, 0x01
add r16, r17    ; result = 0x80 = -128, V=1 (signed overflow)
```

S — Sign `S = N XOR V`. This is the "true" sign of the result after accounting for overflow. Use `BRGE`/`BRLT` (which test S) for signed comparisons, not N alone.

H — Half-carry Carry out of bit 3 into bit 4. It matters for nibble-oriented arithmetic and some compare/add/subtract cases. AVR does **not** have a `DAA` instruction, so you usually inspect H only in specialised code.

T — Transfer A single-bit scratch pad. `BST` copies a bit from a register into T; `BLD` copies T into a register bit. Useful for bit manipulation without clobbering other flags.

I — Global Interrupt Enable When I=1, the CPU responds to interrupts. `SEI` sets it; `CLI` clears it. Entering an ISR automatically clears I; `RETI` restores it. Chapter 10 covers this in detail.

2.4.2 Reading and Saving SREG

`SREG` is a normal I/O register. You can save and restore it:

```
in   r0, SREG      ; save SREG
cli                ; disable interrupts (clear I bit)
; ... critical section ...
out  SREG, r0      ; restore SREG (re-enables interrupts if I was set)
```

This pattern appears constantly in ISR prologues and critical sections.

If you remember only one thing about `SREG`, remember this: flags are part of program state. If your code depends on them, preserve them deliberately. The CPU does not automatically save or restore `SREG` when entering or leaving an ISR; that is your software's responsibility.

> **Instructions introduced: IN / OUT (revisited)** `IN Rd, A` — Load I/O register at address A into Rd. 1 cycle. `OUT A, Rr` — Write Rr to I/O register at address A. 1 cycle. Range: A = 0x00–0x3F only. For higher addresses, use LDS/STS.

> **Instruction introduced: SEI** `SEI` — Set Global Interrupt Enable (I bit in SREG). 1 cycle.

> **Instruction introduced: CLI** `CLI` — Clear Global Interrupt Enable. 1 cycle. Next instruction always executes before any interrupt is served (pipeline guarantee).

2.5 The Stack and Stack Pointer

The stack is a **last-in, first-out** (LIFO) region of SRAM. The AVR stack **grows downward** — pushing data decrements the stack pointer first, then writes.

That "grows downward" detail matters. On AVR, the top of SRAM is the natural starting point for the stack, and each push moves toward lower addresses. Beginners often imagine the stack growing upward because that feels more intuitive. AVR does the opposite.

```
SRAM at 0x3800–0x3FFF:

        0x3800  ┌──────────────┐
                │              │
                │    heap /    │
                │   globals    │
                │              │
                │              │
        SP ───► ├──────────────┤  ← stack pointer (grows ↑ as you pop)
                │   top of     │
                │   stack      │
                │              │  ← stack grows downward ↓ on PUSH
        0x3FFF  └──────────────┘  ← SP initialised here at startup
```

`SP` is a 16-bit register implemented as two I/O registers: - `SPL` at I/O address 0x3D (low byte of SP) - `SPH` at I/O address 0x3E (high byte of SP)

On the ATtiny3217, reset hardware loads `SP` with `RAMEND` automatically. You will still see startup code write it explicitly, because that is harmless and keeps examples portable to older AVR parts where software initialisation is required.

So there are really two questions:

- What value should `SP` start with? `RAMEND`.
- Who sets it? On ATtiny3217, the hardware does; on some older AVR parts, startup code must do it.

Both bytes are in the IN/OUT address range, so explicit initialisation uses `OUT`:

```
ldi r16, lo8(RAMEND)   ; 0xFF
out SPL, r16
ldi r16, hi8(RAMEND)   ; 0x3F
out SPH, r16
```

There is one subtle hardware assist here: when software writes `SPL`, the CPU automatically disables interrupts for up to four instructions, or until the next I/O-memory write, whichever comes first. That prevents an interrupt from observing a half-updated 16-bit stack pointer between the low-byte and high-byte writes. It is still good practice to initialise the stack before enabling interrupts or making subroutine calls.

2.5.1 PUSH and POP

```
push r16    ; SP -= 1, then write r16 to SRAM[SP]  (2 cycles)
pop  r16    ; read r16 from SRAM[SP], then SP += 1  (2 cycles)
```

Push decrements first, then writes. Pop reads first, then increments.

2.5.2 How CALL Uses the Stack

`CALL` / `RCALL` automatically pushes the **return address** onto the stack so `RET` can find it:

```
rcall my_function    ; push PC+1, jump to my_function
; RET in my_function pops the return address and jumps back here
```

On ATtiny3217 (16K word address space), return addresses are 2 bytes (16-bit). The return address stored on the stack is a **word pointer**, not a byte pointer. The least-significant byte is pushed first, at the higher stack address, then the most-significant byte is pushed at the next lower address. On devices with >128KB flash, return addresses are 3 bytes. Always account for this when calculating stack depth.

This is why even a program with "no local variables" still uses stack space. Every nested call consumes stack bytes for the return address, and any saved registers add to that cost.

2.6 The Program Counter (PC)

The Program Counter is a register inside the CPU that holds the **word address** of the next instruction to execute. It is not directly accessible via `IN`/`OUT` — you control it only through jump, branch, and call instructions.

In other words, the PC is where control flow lives. Arithmetic changes data; branches and calls change the PC.

```
After reset:       PC = 0x0000   (first word in flash)
After JMP main:    PC = address of first instruction in main
After RCALL foo:   PC = address of foo; old PC+1 on stack
After RET:         PC = popped from stack
```

The PC is 14 bits wide on ATtiny3217 (addressing 0x0000–0x3FFF in word units, covering the full 32KB flash).

Most AVR instructions are one 16-bit word. Some instructions, such as `JMP`, `CALL`, `LDS`, and `STS`, are two-word instructions. This matters for three reasons:

- The PC counts instruction words, not bytes.
- Skip instructions must skip either one word or two words depending on the next instruction.
- Disassembly addresses may be shown as byte addresses by tools, while the CPU return address is stored as a word pointer.

2.6.1 Instruction Fetch Pipeline

AVR uses a **single-stage pipeline**: while the CPU executes instruction N, it fetches instruction N+1 from flash. This is why most instructions execute in 1 cycle — the fetch is free. Instructions that change the PC (branches, jumps, calls, returns) take 2–4 cycles because the pipeline must be flushed and a new fetch begun.

At the device level, this is the basis for the usual AVR rule of thumb: straight-line register code can approach 1 MIPS per MHz of `CLK_CPU`. That is not a promise that every instruction is single-cycle; it is a reminder that branches, memory accesses, calls, returns, and peripheral waits are where the extra cycles usually appear.

You do not need to become a pipeline expert to use AVR well. The practical lesson is simpler: straight-line code is cheap, but changing direction costs extra cycles.

```
Cycle:   1       2       3       4
Fetch:   N       N+1     N+2     N+3
Execute:         N       N+1     N+2   ← 1 cycle/instruction (normal)

With a taken branch at instruction N:
Fetch:   N       N+1     target  target+1
Execute:         N       ---     target  ← N+1 fetch wasted; branch costs 2 cycles
```

This is why `BRNE` costs 1 cycle when not taken (no pipeline flush) and 2 cycles when taken (flush + re-fetch from target).

2.7 I/O Register Space

The ATtiny3217 peripherals are controlled through memory-mapped I/O registers. They fall into two zones:

```
Address range    Access method       Examples
0x0000–0x003F   IN / OUT (fast)     VPORTB_OUT, SREG, SPL, SPH, CCP
0x0040–0x0FFF   LDS / STS (2 words) PORTB_DIR, TCA0_CNT, USART0_RXDATAL
0x1000–0x13FF   LDS / STS           NVMCTRL, SIGROW, FUSES, USERROW
```

The low 64 addresses (0x00–0x3F) are the "classic" I/O space reachable with 1-word `IN`/`OUT` instructions in 1 cycle. The low half of that space (0x00–0x1F) is also reachable by `SBI`, `CBI`, `SBIC`, and `SBIS`. Everything above 0x3F needs the 2-word `LDS`/`STS` instructions, which take 3 and 2 cycles respectively.

This is one of the most important performance distinctions on AVR. Two registers may both control hardware, but if one is in low I/O space and the other is in extended space, the instruction choices and timing differ.

The ATtiny3217's important low I/O registers:

```
I/O Addr  Name    Description
0x05      VPORTB_OUT  Virtual PORTB output register (fast GPIO access)
0x04      VPORTB_DIR  Virtual PORTB direction register
0x3F      SREG    Status Register
0x3E      SPH     Stack Pointer High
0x3D      SPL     Stack Pointer Low
0x34      CCP     Configuration Change Protection (unlock protected registers)
0x1F      GPIOR3  General Purpose I/O Register 3 (scratch)
0x1E      GPIOR2  General Purpose I/O Register 2 (scratch)
```

```
0x1D      GPIOR1  General Purpose I/O Register 1 (scratch)
0x1C      GPIOR0  General Purpose I/O Register 0 (scratch)
```

`VPORTx_*` mirrors let you do single-cycle GPIO with `IN`/`OUT` and bit instructions, while the full `PORTx_*` peripheral block provides the richer `DIRSET`/`OUTCLR`/`OUTTGL` registers in extended I/O space.

`GPIOR0–3` are four registers with no peripheral function — pure scratch RAM accessible via `IN`/`OUT` in a single cycle. Useful for storing flags or counters that need fast ISR access.

Think of them as "fast little mailboxes" between parts of your program. They are not magic, but they can be handy when one-byte state needs to be very cheap to access.

The extended I/O region is where most modern tinyAVR peripherals live. A few important base addresses are:

Base	Peripheral	Why you care
`0x0400`	`PORTA`	Full GPIO configuration and atomic set/clear/toggle
`0x0600`	`ADC0`	ADC and Peripheral Touch Controller path
`0x0800`	`USART0`	Serial port used in later examples
`0x0810`	`TWI0`	I2C-compatible bus controller
`0x0820`	`SPI0`	SPI controller
`0x0A00`	`TCA0`	16-bit timer/counter
`0x1000`	`NVMCTRL`	Flash/EEPROM controller
`0x1100`	`SIGROW`	Factory calibration and signature data
`0x1280`	`FUSES`	Device configuration fuses

This is why examples often prefer header symbols such as `USART0_STATUS` or `PORTA_DIRSET` instead of raw numbers. The symbol tells you which peripheral block you are touching and lets the device header carry the exact address.

2.8 Putting It Together — A Register-State Walk-Through

Let's trace the CPU state through the blink program from Chapter 1, cycle by cycle through the setup section:

The goal here is not to memorise every state transition. It is to get used to thinking in terms of registers, memory side effects, and whether flags changed.

```
; State at reset:
; PC = 0x0000
; SP = 0x3FFF on ATtiny3217 reset hardware
; SREG = 0x00
; Treat general registers as undefined until your code sets them

ldi r16, lo8(RAMEND)   ; r16 = 0xFF. PC → next instr. 1 cycle.
out SPL, r16           ; explicit re-init of SP low byte. 1 cycle.
ldi r16, hi8(RAMEND)   ; r16 = 0x3F. PC → next. 1 cycle.
out SPH, r16           ; explicit re-init of SP high byte. 1 cycle.

ldi r16, (1 << 3)      ; r16 = 0x08. 1 cycle.
```

```
sts PORTA_DIRSET, r16  ; Write 0x08 to 0x0401.
                       ;   PA3 now output. 2 cycles.

; State now:
; r16 = 0x08, SP = 0x3FFF, PORTA_DIR bit 3 = 1, all flags unchanged
```

Flags: `LDI`, `OUT`, `STS` do **not** affect SREG at all. Only arithmetic, logic, and bit operations change flags.

2.9 Key Architectural Constraints Summary

These are the constraints you will bump into constantly. They look dry, but they explain a large share of beginner errors.

Constraint	Rule
LDI range	r16–r31 only
IN/OUT range	I/O addresses 0x00–0x3F only
SBIW/ADIW pairs	r25:r24, r27:r26, r29:r28, r31:r30 only
Pointer pairs	X = r27:r26, Y = r29:r28, Z = r31:r30
MUL result	Always in r1:r0
Stack direction	Grows downward; PUSH decrements first
Return address	2 bytes on stack (ATtiny3217, ≤128KB flash)
Interrupt response	6 cycles minimum on ATtiny3217 flash sizes > 8 KB
Flash read via LD	Use address + 0x8000 (mapped flash, AVRxt)
Flash read via LPM	Use Z = byte address, reads via program bus
EEPROM address	0x1400–0x14FF in data space, controlled by NVMCTRL
Flash/EEPROM writes	Use NVMCTRL command sequences, not ordinary SRAM stores

---

2.10 Summary

```
Harvard = two buses. Flash for code. SRAM for data.

ATtiny3217 memory map:
  0x0000–0x003F   I/O (IN/OUT)
  0x0040–0x0FFF   Peripherals (LDS/STS)
  0x1000–0x13FF   NVMCTRL / SIGROW / FUSES / USERROW region
  0x1400–0x14FF   EEPROM (256 bytes)
```

```
  0x3800–0x3FFF   SRAM (2 KB)
  0x8000–0xFFFF   Flash mapped in data space (AVRxt feature)

32 registers (r0–r31):
  r0–r15: general, no LDI
  r16–r31: general + LDI + most immediates
  r26:r27 = X, r28:r29 = Y, r30:r31 = Z (pointer pairs)

SREG (0x3F): C Z N V S H T I — set by ALU, read by branches.

SP = SPH:SPL, initialise to RAMEND. Stack grows down.
On ATtiny3217, reset hardware already loads SP = RAMEND.
PC = word address, 14 bits. Pipeline prefetch = 1 cycle/instr normal.
```

2.11 Exercises

1. What is the data-bus address of the first byte of flash on ATtiny3217? What is the data-bus address of the last byte of flash?

2. After `ldi r16, 0x80` and `ldi r17, 0x80` and `add r16, r17`, which SREG flags are set? Work it out by hand, then verify with:

   ```
   avr-as -mmcu=attiny3217 -o /tmp/t.o - <<'EOF'
   .text
   ldi r16, 0x80
   ldi r17, 0x80
   add r16, r17
   EOF
   avr-objdump -d /tmp/t.o
   ```

   (The disassembly won't show flag values — you must reason through them.)

3. Write a short assembly snippet that:

   - Saves SREG to r0
   - Disables interrupts
   - Does some work (e.g. `ldi r16, 42`)
   - Restores SREG (and thus the interrupt state)

4. Calculate the maximum stack depth for a program that calls a function, which calls another function. How many bytes does the stack use just for return addresses?

5. Why can't you write `ldi r15, 0xFF`? What two-instruction sequence achieves the same result?

Next: Chapter 3 — AVR-AS Syntax and Directives

Chapter 3

AVR-AS Syntax and Directives

Assembly source is more than a list of instructions. Before the CPU can run anything, the assembler has to answer basic questions: where does the code go, what data should be emitted, which names are constants, and which labels refer to real addresses? Those decisions are controlled by **directives** — assembler commands that begin with a dot.

This chapter is about that "language around the instructions." If Chapter 1 showed how to make the chip do something, Chapter 3 explains how to write assembly source in a way the toolchain can understand and organise correctly.

3.1 File Extensions: `.S` vs `.s`

GAS accepts two extensions:

Extension	Preprocessor	Use
`.S`	Yes — C preprocessor runs first	Always use this
`.s`	No — raw GAS input	Rarely; only for generated files

The uppercase `.S` lets you use `#include`, `#define`, `#ifdef`, and all other C preprocessor directives. Since we always want `#include <avr/io.h>` for register names, **always use `.S`**.

For a beginner, the practical rule is simple: if you are hand-writing AVR assembly for this book, name the file with an uppercase `.S` and do not think about it further.

The build flow preprocesses `.S` files before assembly. If you want to inspect that preprocessed output yourself, use the compiler driver:

```
# Inspect the preprocessed source that avr-as will assemble:
avr-gcc -x assembler-with-cpp -mmcu=attiny3217 -E blink.S
```

You normally do not need to run this manually if your build uses `avr-gcc` as the driver, because it sees `.S` and runs the preprocessor before invoking the assembler. `avr-as` itself does **not** run `cpp`; if you call `avr-as` directly, feed it preprocessed assembly or avoid `#include`/`#define`.

For hand-written examples in this book, prefer this pattern:

```
avr-gcc -x assembler-with-cpp -mmcu=attiny3217 -c blink.S -o blink.o
```

That keeps device-header symbols such as `PORTA_DIRSET`, `RAMEND`, `SREG`, `SPL`, `SPH`, and `MAPPED_PROGMEM_START` available in the source.

3.2 Comments

GAS accepts three comment styles. In `.S` files all three work:

```
; This is a comment — semicolon style, common in AVR assembly

// This is a comment — C++ line style

/* This is a
   block comment — C style */
```

Semicolon is traditional in AVR assembly and what you'll see most. Use block comments for file headers and section separators.

Style matters more in assembly than in many other languages because the code is so compact. A short comment can carry a lot of explanatory weight.

3.3 Instruction Syntax

AVR GAS uses a **destination-first, source-second** convention:

```
mnemonic  destination, source
```

Examples:

```
mov  r16, r17      ; r16 ← r17
add  r16, r17      ; r16 ← r16 + r17
ldi  r16, 42       ; r16 ← 42
sts  0x0421, r16   ; mem[0x0421] ← r16
ld   r16, X        ; r16 ← mem[X]
```

This matches GCC output and the Atmel/Microchip instruction set manual.

Instruction syntax is not just cosmetic; it reflects fields in the opcode. On AVR, many instructions cannot encode every register or address. The assembler will reject operands that do not fit the instruction format:

```
ldi r15, 0x55       ; ERROR: LDI can only encode r16-r31
ldi r16, 0x55       ; OK

out 0x3F, r16       ; OK: OUT reaches I/O address 0x00-0x3F
out 0x40, r16       ; ERROR: use STS for extended I/O
sts 0x0040, r16     ; OK: direct store to data space

sbi 0x1C, 0         ; OK: SBI/CBI/SBIC/SBIS reach 0x00-0x1F
sbi 0x20, 0         ; ERROR: above bit-addressable low I/O range
```

Microchip's ATtiny3217 peripheral map puts the four general-purpose I/O registers at `0x1C-0x1F`, which makes them directly usable with `SBI`, `CBI`, `SBIS`, and `SBIC`. Most larger peripherals, such as `PORTA` at `0x0400` and `USART0` at `0x0800`, are outside the `IN`/`OUT` range and require `LDS`/`STS` or pointer-based `LD`/`ST`.

3.4 Number Literals

GAS accepts numbers in four bases:

```
ldi r16, 42         ; decimal
ldi r16, 0x2A       ; hexadecimal
ldi r16, 0b00101010 ; binary
ldi r16, 052        ; octal (leading zero — easy to mistake for decimal!)
```

All four produce the same value (42). Avoid octal — the leading-zero trap causes real bugs. Hex and binary are clearest for hardware work.

Character literals also work:

```
ldi r16, 'A'        ; r16 = 65
ldi r16, '\n'       ; r16 = 10
```

3.5 Expressions

GAS evaluates constant expressions at assembly time. You can use:

```
ldi r16, (1 << 4)            ; bit shift  — evaluates to 16
ldi r16, (1 << 4) | (1 << 2) ; bitwise OR — evaluates to 20
ldi r16, RAMEND & 0xFF       ; mask       — evaluates to 0xFF
ldi r16, RAMEND / 256        ; division   — evaluates to 0x3F
ldi r16, 10 * 5 - 8          ; arithmetic — evaluates to 42
```

These are evaluated at **assembly time** — no runtime cost. They exist only to make source code readable.

That is an important distinction. These expressions do not generate arithmetic instructions. The assembler does the math once while building the file, and the CPU never sees the expression itself.

3.5.1 Built-in Expression Functions

GAS provides helper functions for splitting 16-bit values into bytes:

```
ldi r16, lo8(RAMEND)         ; low byte  of 0x3FFF = 0xFF
ldi r16, hi8(RAMEND)         ; high byte of 0x3FFF = 0x3F
ldi r16, lo8(0x1234)         ; = 0x34
ldi r16, hi8(0x1234)         ; = 0x12
```

Two more functions matter for AVRxt flash access (see Section 3.11):

```
; pm(label) — convert byte address to word address (for IJMP/ICALL targets)
ldi ZL, lo8(pm(my_func))     ; ZL = word address low byte of my_func
ldi ZH, hi8(pm(my_func))     ; ZH = word address high byte of my_func
```

For table addresses, be explicit about which address space you mean:

```
ldi ZL, lo8(table)                    ; flash byte address for LPM
ldi ZH, hi8(table)

ldi ZL, lo8(table + MAPPED_PROGMEM_START) ; data-space address for LD
ldi ZH, hi8(table + MAPPED_PROGMEM_START)
```

The ATtiny3217 datasheet describes both views: `LPM` sees flash starting at `0x0000`, while ordinary data-space loads see mapped flash starting at `0x8000`. Those two forms are not interchangeable.

3.6 Labels

A label marks a position in the output. It ends with a colon:

```
main:
    ldi r16, 0xFF

loop:
    dec r16
    brne loop       ; reference label by name — no colon when referencing
```

Labels have two uses: 1. **Jump/branch targets** — as above 2. **Data addresses** — when placed before `.byte`, `.word`, etc.

You can think of a label as a bookmark the assembler turns into an address. Sometimes that address is where execution should jump. Sometimes it is where data begins.

```
table:
    .byte 1, 2, 3, 4    ; "table" now equals the address of byte 1
```

3.6.1 Global vs Local Labels

By default, labels are **local to the object file** — the linker cannot see them unless you explicitly export them:

```
.global main       ; export "main" so the linker can use it as entry point
.global __vectors  ; export vector table symbol

main:              ; now visible to linker
    ...

internal_helper:   ; NOT global — invisible outside this file
    ...
```

For functions, you can also tell the linker/debugger what kind of symbol you are defining:

```
.global main
.type   main, @function
main:
    ; ...
    ret
.size   main, .-main
```

`.type` and `.size` do not change the emitted instructions. They improve symbol metadata, which makes `avr-objdump`, `avr-nm`, and debuggers report function boundaries more cleanly.

3.6.2 GAS Local Labels (Numeric)

Naming every loop, branch target, and temporary label creates a pollution of unique names. GAS solves this with **numeric local labels**:

```
1:                  ; define label "1"
    dec r16
    brne 1b         ; "1b" = most recent backward occurrence of label "1"
```

```
    tst r16
    breq 2f         ; "2f" = next forward occurrence of label "2"
    ldi r16, 0xFF
2:                  ; define label "2"
    ...
```

Rules: - Any number 0–9 (or longer: `10:`, `255:`) can be a local label - `Nb` = nearest backward occurrence of `N:` - `Nf` = nearest forward occurrence of `N:` - The same number can be reused anywhere in the file — each definition is distinct. GAS resolves `1b` to the *nearest* preceding `1:`.

This is invaluable in loops:

```
ldi r18, 10         ; outer loop count
1:                  ; outer loop top
    ldi r17, 0
    ldi r16, 0
2:                  ; inner loop top
    sbiw r16, 1
    brne 2b         ; back to inner "2:"
    dec r18
    brne 1b         ; back to outer "1:"
```

No naming conflict — `1b` and `2b` refer to the right labels unambiguously.

This style looks strange at first, but it becomes natural quickly. Named labels are best for important program locations such as `main` or `uart_tx`. Numeric labels are best for short internal loops where a descriptive name would add more clutter than clarity.

3.6.3 File-Scoped Local Labels (`.L` prefix)

Labels starting with `.L` are stripped from the symbol table by the linker and do not appear in debug info. Use them for internal subroutine labels you don't want to export:

```
my_subroutine:
.Lloop:
    dec r16
    brne .Lloop
    ret
```

`.Lloop` will not appear in `avr-nm` output or debugger symbol lists.

3.7 Sections

A **section** is a named region of output. The linker arranges sections into the final memory map. Every assembly statement belongs to the *current* section.

If labels are bookmarks, sections are containers. They tell the linker what kind of thing you are defining: executable code, read-only flash data, SRAM data, zero-initialised storage, and so on.

3.7.1 Core Sections

```
.section .text          ; code — goes to flash
.section .data          ; initialised SRAM data — startup code must copy it in
.section .bss           ; zero-initialised SRAM data — startup code must clear it
.section .rodata        ; read-only constants — stay in flash
```

For the ATtiny3217, the linker maps those abstract sections onto a concrete device memory map:

Section	Runtime location	ATtiny3217 address range
`.text`, `.rodata`, `.vectors`	Flash/program memory	Code space `0x0000-0x7FFF` bytes
`.data`	SRAM at runtime, initial image in flash	SRAM `0x3800-0x3FFF`
`.bss`	SRAM, no file contents	SRAM `0x3800-0x3FFF`
EEPROM data	EEPROM, NVMCTRL-controlled	Data space `0x1400-0x14FF`
Mapped flash reads	Flash through data bus	Data space `0x8000-0xFFFF`

The assembler does not know your intent beyond the section and bytes you emit. The linker script is what makes `.text` land in flash and `.bss` reserve SRAM. If you bypass the normal linker script or use custom sections, inspect the linker map and disassembly.

Shorthand forms (equivalent):

```
.text       ; same as .section .text
.data       ; same as .section .data
.bss        ; same as .section .bss
```

You can switch between sections multiple times in one file:

```
.text
foo:
    ldi r16, 5
    ret

.data
my_var: .byte 0

.text
bar:
```

```
    ldi r16, 10
    ret
```

The assembler concatenates all `.text` chunks together, all `.data` chunks together, etc. The order of sections in the output is determined by the linker script, not the source order.

That is why switching sections in the middle of a file is normal. You are not describing the final physical layout directly; you are describing which bytes belong to which category.

3.7.2 Section Flags and Types (Full Form)

The full `.section` directive takes optional flags:

`.section name, "flags", @type`

Flag	Meaning
`a`	Allocatable (takes space in memory)
`x`	Executable
`w`	Writable

Type	Meaning
`@progbits`	Contains data/code
`@nobits`	Takes space but has no data in file (BSS)

Most of the time you use the shorthand and GAS fills in reasonable defaults. You need the full form for special sections like `.vectors`:

```
.section .vectors, "ax", @progbits
```

This tells the linker: place this section at the start of flash (the linker script maps `.vectors` to address 0x0000), it contains code (`x`), and it is allocatable (`a`).

The vector table is executable code on this AVR target, not a table of raw ISR addresses. Each vector slot contains an instruction, commonly `jmp handler` on ATtiny3217-sized flash parts. That is why `.vectors` uses `"ax"` rather than plain read-only data flags.

You do not need to memorise all the flags immediately. What matters is knowing that the full form exists when you need something more specific than the common defaults.

3.8 Data Directives

These directives emit bytes into the current section.

That phrase "emit bytes" is worth taking literally. Directives such as `.byte` and `.word` do not create variables in the high-level-language sense. They cause exact byte patterns to be written into the object file.

3.8.1 `.byte` — 8-bit values

```
.byte 0x42              ; one byte: 0x42
.byte 1, 2, 3, 4        ; four bytes
.byte 'H', 'i', '!'     ; character literals
.byte (1 << 3)          ; expression — evaluates to 8
```

3.8.2 `.word` — 16-bit values (little-endian)

```
.word 0x1234            ; emits 0x34, 0x12 (little-endian)
.word 0, 1, 2           ; six bytes: three 16-bit words
```

> **Warning:** On AVR, `.word` is 16 bits. In non-AVR GAS targets `.word` can be 32 bits. Never assume word size when reading unfamiliar assembly.

In flash tables used by `LPM`, remember that bytes still come out in little endian order. If you write `.word 0x1234`, the first byte at the lower address is `0x34` and the second is `0x12`. If `Z` points at the label and you execute `lpm r16, Z+`, `r16` receives `0x34` first.

3.8.3 `.long` — 32-bit values

```
.long 0x12345678        ; emits 78 56 34 12 (little-endian)
```

3.8.4 `.ascii` and `.asciz` — Strings

```
.ascii "Hello"          ; 5 bytes: H e l l o — NO null terminator
.asciz "Hello"          ; 6 bytes: H e l l o 0x00 — WITH null terminator
.string "Hello"         ; alias for .asciz
```

Almost always use `.asciz` or `.string` so C string functions work correctly.

If you forget the terminating zero, the bytes are still valid assembly data, but any code that expects a C-style string will keep reading past the intended end.

3.8.5 `.skip` and `.space` — Reserve Bytes

```
buf:    .skip 16        ; reserve 16 bytes (content undefined in .text/.data)
buf:    .skip 16, 0     ; reserve 16 bytes, filled with 0x00
```

In `.bss`, content is always zeroed by startup code regardless of fill value.

Do not use `.skip` in `.text` as a cheap delay or timing pad. It emits bytes into flash; those bytes will be interpreted as instructions if execution falls through them. Use real instructions such as `nop`, or branch around data.

3.8.6 `.balign` — Alignment

```
.balign 2               ; align to 2-byte boundary (pad with 0x00 if needed)
.balign 4               ; align to 4-byte boundary
```

Required before `.word` or `.long` data in some contexts. Also used to align code for performance (though on AVR this matters less than on ARM).

AVR instructions are 16-bit words, and some are 32-bit two-word instructions. Keeping code and 16-bit flash data on even byte boundaries makes disassembly and label arithmetic easier to reason about. It also avoids accidental odd-byte entry points when mixing `.byte` tables and executable code in the same section.

3.9 Constant Directives

3.9.1 `.equ` — Define a Constant

```
.equ LED_PIN,  4
.equ LED_MASK, (1 << LED_PIN)   ; expressions work
.equ DELAY,    5000
```

`.equ` defines a **symbol** with a value. It cannot be redefined later. Use it for things that truly never change: pin numbers, masks, magic values.

In practice, `.equ` is how you turn anonymous numeric constants into named ideas. `0x08` is just a number; `LED_MASK` tells the reader what the number means.

Using the constant:

```
ldi r16, LED_MASK       ; assembler substitutes the value at assembly time
```

Prefer device-header names over hand-written `.equ` addresses whenever the name exists:

```
#include <avr/io.h>

ldi r16, (1 << 3)
sts PORTA_DIRSET, r16   ; better than .equ PORTA_DIRSET, 0x0401
```

The Microchip device header follows the datasheet address map. Using the header keeps source code tied to the selected `-mmcu=attiny3217` target instead of a copied constant that may be wrong on a different AVR.

3.9.2 `.set` — Reassignable Constant

```
.set count, 0
.set count, count + 1   ; OK — .set can be redefined
.set count, count + 1   ; count = 2 now
```

`.set` is useful for building values incrementally or for conditional definitions. Equivalent to `#define` + `#undef` patterns but within GAS.

3.9.3 `#define` — Preprocessor Macro

Because `.S` files go through `cpp`, you can also use:

```
#define LED_PIN   4
#define LED_MASK  (1 << LED_PIN)
#define SET_BIT(reg, bit) ((reg) |= (1 << (bit)))   ; functional macro
```

`#define` is processed *before* GAS sees the file; `.equ` is processed *by* GAS. For simple constants, `.equ` is idiomatic in assembly. For complex macros or conditional compilation, `#define`/`#ifdef` are necessary.

If you are unsure which to choose, prefer `.equ` for plain assembly constants. Reach for `#define` only when you really need preprocessor behavior.

3.10 The `.org` Directive

`.org` sets the **location counter** — the current output offset within the section. It can move the location counter **forward**, creating padding, but it cannot move it backward.

This is a more advanced tool than it first appears. It is useful when a layout must match hardware-defined slots, such as interrupt vectors, but it is also an easy way to create confusing gaps if used casually.

```
.org 0x0000
    jmp main            ; lands exactly at byte address 0

.org 0x0004
    jmp isr_int0        ; next 4-byte JMP slot

.org 0x0008
    jmp isr_pcint0      ; next slot
```

If you use `JMP`, each vector entry consumes 4 bytes. If you use `RJMP`, each entry consumes 2 bytes, so the `.org` spacing changes accordingly. The exact vector table is in the datasheet and in `<avr/iotn3217.h>` as `_VECTOR(n)` macros. Chapter 10 builds the full vector table.

The ATtiny3217 has 32 KB of flash, so the normal interrupt vector entries use `JMP`-sized slots in the examples. Microchip also documents a Compact Vector Table mode through `CPUINT.CTRLA.CVT`; when enabled, the interrupt table groups normal interrupts into three entries: NMI at vector address 1, priority level 1 at vector address 2, and all priority level 0 interrupts at vector address 3. This book uses the ordinary vector table unless a later chapter explicitly says otherwise.

So the real lesson is not "always use `.org`." The lesson is: use it when the hardware requires exact placement, and understand the instruction size when you do.

> **Warning:** `.org` without context is relative to the *start of the current section*. In `.vectors`, `.org 0` means "start of the vector section", which the linker places at flash address 0. In `.text`, `.org 0x100` would create a 0x100-byte gap from the section's start — usually not what you want.

3.11 Flash Data on AVRxt (ATtiny3217)

This is where AVRxt differs from classic AVR in an important way.

This section is easy to skim past, but it matters because it changes how you think about constants in flash. On older AVR parts, flash reads feel like a special case. On AVRxt, they can look much more like ordinary data reads.

Classic AVR (ATmega328P): Flash is NOT in the data address space. To read a byte from flash at runtime, you must use the `LPM` instruction with the Z register pointing to the flash byte address.

AVRxt (ATtiny3217): Flash IS mapped in the data address space at offset `MAPPED_PROGMEM_START` (= 0x8000). You can read flash with ordinary `LD` instructions by adding this offset to the flash address.

The concrete ATtiny3217 memory map behind that statement is:

```
0x0000-0x003F   I/O registers
0x0040-0x0FFF   Extended I/O / peripheral registers
0x1000-0x13FF   NVM I/O registers and data
0x1400-0x14FF   EEPROM, 256 bytes
0x3800-0x3FFF   SRAM, 2 KB
0x8000-0xFFFF   32 KB flash mapped into data space
```

Only the last range is the mapped view of program flash. EEPROM is a separate non-volatile memory region controlled by `NVMCTRL`; do not put ordinary `.rodata` there by accident.

```
.section .rodata

table:
    .byte 10, 20, 30, 40, 50

.section .text

read_table_entry:                  ; r24 = index, returns value in r25
    ldi ZL, lo8(table + MAPPED_PROGMEM_START)
    ldi ZH, hi8(table + MAPPED_PROGMEM_START)
    add ZL, r24                    ; Z now points to table[r24] in data space
    adc ZH, r1                     ; propagate carry (r1 = 0 by convention)
    ld  r25, Z                     ; read from flash via data bus
    ret
```

The assembler expression `table + MAPPED_PROGMEM_START` adds 0x8000 to the flash byte address of `table`. The result fits in 16 bits because the table starts in the first 32KB of flash.

Data-space `LD` access to mapped flash is convenient, but it is not SRAM. The AVR instruction set manual notes that data-memory cycle counts for `LD` assume internal RAM and that NVM accesses can add implementation-dependent time. For constant tables this usually does not change the code structure, but it matters when doing tight cycle accounting.

Conceptually, you are taking a flash address and translating it into the corresponding data-space view of the same bytes.

Old LPM method still works on AVRxt and is sometimes needed for compatibility:

```
    ldi ZL, lo8(table)             ; Z = flash byte address (no +0x8000)
    ldi ZH, hi8(table)
    lpm r25, Z                     ; read from flash via program bus
```

Both methods read the same data. On AVRxt, `LD` with mapped address is simpler and more natural. `LPM` is covered in Chapter 5.

That means you should understand both styles, even if the mapped-flash method is the one you use most often on the ATtiny3217.

3.12 The .include Directive

```
.include "macros.inc"       ; include a GAS file
```

Inserts the contents of another file at the point of the `.include`. The included file is processed by GAS (not `cpp`), so it cannot use `#define`.

For files that need preprocessor features, use `#include` instead:

```
#include <avr/io.h>         ; cpp include — register definitions
#include "my_macros.h"      ; cpp include — your own header
.include "data_tables.S"    ; GAS include — another assembly file
```

Use `#include <avr/io.h>` exactly once near the top of a `.S` file. That header selects the device-specific header from `-mmcu=attiny3217`; including `<avr/iotn3217.h>` directly works, but it makes the source less portable across build-system target changes.

3.13 Macro Directives

GAS has its own macro system separate from the C preprocessor.

```
.macro delay_ms ms_count
    ldi r19, \ms_count
.Lmacro_outer_\@:
    ldi r18, 0
    ldi r17, 0
.Lmacro_inner_\@:
    sbiw r17, 1
    brne .Lmacro_inner_\@
    dec r19
    brne .Lmacro_outer_\@
.endm
```

Using the macro:

```
delay_ms 100            ; expands inline — \ms_count → 100
delay_ms 500            ; second expansion — \@ ensures unique label names
```

Key syntax: - `\param` — substitute parameter value - `\@` — unique invocation counter (prevents label collisions across expansions) - `.endm` — end macro definition

GAS macros are powerful but can obscure what code is actually emitted. Prefer subroutines (`rcall`/`ret`) over macros for anything that runs more than once, to save flash space.

This is a good place to be disciplined. Macros are tempting because they feel convenient, but they duplicate code at every use site. In assembly, that cost is often visible.

3.14 Full Section Layout Example

This is the structure of a well-organised AVR assembly file:

```
/*
 * myprogram.S — description
```

```
 * Build: avr-gcc -x assembler-with-cpp -mmcu=attiny3217 -c myprogram.S
 */

#include <avr/io.h>

/* ── Constants ─────────────────────────────────────────────────────── */
.equ    MY_PIN,   5
.equ    MY_MASK,  (1 << MY_PIN)

/* ── Read-only data (flash) ────────────────────────────────────────── */
.section .rodata

my_table:
    .byte 1, 2, 4, 8, 16, 32, 64, 128

my_string:
    .asciz "ATtiny3217"

/* ── Uninitialised SRAM ─────────────────────────────────────────────── */
.section .bss

rx_buffer:  .skip 64        ; 64-byte ring buffer
rx_head:    .skip 1
rx_tail:    .skip 1

/* ── Initialised SRAM (requires startup copy code or CRT startup) ───── */
.section .data

state:      .byte 0x01      ; starts as 1 in SRAM

/* ── Interrupt vector table ────────────────────────────────────────── */
.section .vectors, "ax", @progbits
.global __vectors

__vectors:
    jmp     main            ; 0x0000 reset

/* ── Code ──────────────────────────────────────────────────────────── */
.section .text
.global main

main:
    ldi r16, lo8(RAMEND)
    out SPL, r16
    ldi r16, hi8(RAMEND)
    out SPH, r16
    ; ... rest of program
```

That example deliberately uses `.data` and `.bss` to show the sections, but a bare `-nostartfiles` assembly program must provide the startup behavior itself. If you define `.data`, somebody must copy its initial bytes from flash to SRAM. If you define `.bss`, somebody must clear it. The avr-libc C runtime normally does that before `main`; a minimal hand-written reset handler does not unless you write the code.

3.15 Examining the Output with `avr-objdump`

After assembling, inspect what the directives produced:

```
avr-gcc -x assembler-with-cpp -mmcu=attiny3217 -c syntax_demo.S -o syntax_demo.o
avr-gcc -mmcu=attiny3217 -nostartfiles -o syntax_demo.elf syntax_demo.o
avr-objdump -s -d syntax_demo.elf
```

The `-s` flag dumps the **raw contents** of every section as hex. Example output for `.rodata`:

```
Contents of section .rodata:
 0074 01000101 01000101 01010100 48656c6c  ........Hell
 0084 6f204156 5200                        o AVR.
```

Cross-reference that output with the source: `morse_dit = [01, 00]`, `morse_dah = [01, 01, 01, 00]`, `greeting = "Hello AVR\0"`. Each byte appears in order. This is how you confirm that your directives produced exactly the layout you intended.

The `-d` flag shows the **disassembly** of `.text`:

```
00000008 <main>:
   8:   0f ef           ldi     r16, 0xFF
   a:   0d bf           out     0x3d, r16
  ...
  1a:   e0 e7           ldi     r30, 0x70   ; ZL = lo8(morse_dit + 0x8000)
  1c:   f0 e8           ldi     r31, 0x80   ; ZH = hi8(...)
```

Notice ZH = 0x80 — the 0x8000 mapped flash offset is baked into the immediate at assembly time.

That is a perfect example of why inspecting output is useful: you can see a source-level idea become a real machine-level value.

Two other inspection commands are useful when debugging directives:

```
avr-objdump -h syntax_demo.elf   # section sizes, VMAs, file offsets
avr-nm -n syntax_demo.elf        # symbols sorted by address
```

Use `-h` to confirm that `.bss` takes SRAM space without occupying file bytes. Use `avr-nm` to confirm which labels survived linking; `.L` local labels should not clutter the public symbol list.

3.16 Quick Reference

Directive	Purpose	Example
`.equ`	Define constant	`.equ PIN, 4`
`.set`	Reassignable constant	`.set i, 0`
`.byte`	Emit bytes	`.byte 1, 2, 3`
`.word`	Emit 16-bit words	`.word 0x1234`
`.long`	Emit 32-bit words	`.long 0xDEADBEEF`
`.ascii`	String, no null	`.ascii "hi"`
`.asciz`	String with null	`.asciz "hi"`
`.skip`	Reserve bytes	`.skip 16`

Directive	Purpose	Example
`.balign`	Align location counter	`.balign 2`
`.org`	Set location counter	`.org 0x0000`
`.global`	Export symbol	`.global main`
`.section`	Switch/create section	`.section .text`
`.include`	Include GAS file	`.include "f.S"`
`.macro/.endm`	Define GAS macro	`.macro name p1`
`.type`	Mark symbol kind	`.type main, @function`
`.size`	Mark symbol size	`.size main, .-main`

Expression	Meaning	Result for RAMEND=0x3FFF
`lo8(x)`	Low byte	`0xFF`
`hi8(x)`	High byte	`0x3F`
`pm(label)`	Word address	`addr / 2`
`(1 << n)`	Bit mask	`(1 << 4)` = 0x10

3.17 Summary

```
.S extension → cpp runs first → #include works.

Sections:
  .text    = flash code
  .rodata  = flash constants (read via mapped addr + 0x8000 on AVRxt)
  .data    = SRAM, initialised if your startup code/CRT copies it
  .bss     = SRAM, zero-init if your startup code/CRT clears it
  .vectors = must be first in flash, holds the interrupt vector table

Labels:
  name:    = define label
  .Lname:  = local, stripped from symbol table
  1:       = numeric — reusable, resolved by 1f / 1b

Constants: .equ (fixed), .set (reassignable), #define (preprocessor).

AVRxt flash read: address + MAPPED_PROGMEM_START (0x8000), then LD.
ATtiny3217 data map: EEPROM is 0x1400-0x14FF; SRAM is 0x3800-0x3FFF.
Low I/O: IN/OUT reach 0x00-0x3F; SBI/CBI/SBIS/SBIC reach 0x00-0x1F.
```

3.18 Exercises

1. Write a `.rodata` section containing a table of the first 8 powers of 2 (`1, 2, 4, 8, ...`). Write code that reads `table[r24]` using the mapped flash address and returns the value in r25.
2. Add a `.bss` variable `byte_count` and a `.data` variable `max_count` initialised to 10. Write code that increments `byte_count` using `LDS`/ `STS` until it equals `max_count`, then halts.

3. Rewrite the Chapter 1 blink delay using a GAS `.macro` named `delay` that takes one parameter (outer loop count). Show that two calls `delay 25` and `delay 50` both assemble without label conflicts.

4. Assemble `syntax_demo.S` and run:

   ```
   avr-objdump -s syntax_demo.elf
   ```

   Verify the bytes in `.rodata` match the `morse_dit`, `morse_dah`, and `greeting` definitions in the source.

5. What happens if you put `.word 0x1234` inside a `.bss` section? Try it and inspect the linker output. Why does the linker warn or error?

Next: Chapter 4 — Instruction Set Basics

Chapter 3A: Debugging AVR Assembly with GDB

Assembly debugging is different from source-level debugging in a high-level language. There is no hidden runtime to explain what happened. The evidence is the program counter, the registers, the status flags, SRAM, I/O registers, the stack pointer, and the instruction stream in flash.

This chapter shows how to debug the assembly style used in this book: GNU AVR assembly syntax in `.S` files, preprocessed through `avr-gcc` and assembled by `avr-as`. The target board is the ATtiny3217 Curiosity Nano, using its on-board debugger over UPDI. The core habit is simple: build an ELF with symbols, keep labels at meaningful locations, stop at instruction boundaries, inspect the CPU state, and prove what changed after each instruction.

3.19 Why This Chapter Belongs Here

By this point you have seen the source format, sections, labels, symbols, and the build flow that produces an ELF file. That is the right moment to learn debugging. If you wait until after timers, USART, SPI, interrupts, and bootloaders, you will have too many moving parts. If you learn GDB now, every later chapter becomes easier to verify.

The goal is not to memorize every GDB command. The goal is to build a repeatable inspection loop:

1. Build an ELF with useful debug information.
2. Load the ELF into GDB.
3. Stop at a named label.
4. Inspect registers, flags, memory, and disassembly.
5. Step one instruction.
6. Inspect again.
7. Explain exactly what changed.

That loop catches most assembly mistakes faster than guessing.

3.20 The Files GDB Needs

GDB works from an ELF file, not from the final Intel HEX file alone.

The HEX file contains bytes to program into flash. It is good for programming the device, but it does not carry the full symbolic view you want while debugging. The ELF file can contain:

- section addresses
- symbol names such as `main`, `sum_loop`, and `break_here`
- line information connecting source lines to machine instructions
- relocation and address information useful to `objdump`, `avr-nm`, and GDB

For this book's assembly examples, keep the ELF file and debug that:

```
avr-gcc -mmcu=attiny3217 -x assembler-with-cpp -g3 -Wa,--gdwarf-2 \
    -c -o gdb_demo.o src/gdb_demo.S

avr-gcc -mmcu=attiny3217 -nostartfiles -g3 \
    -o gdb_demo.elf gdb_demo.o
```

The important pieces are:

Option	Purpose
`-mmcu=attiny3217`	Selects the device memory layout and instruction variant.
`-x assembler-with-cpp`	Treats the file as preprocessed assembly.
`-g3`	Keeps rich debug information in the object and ELF files.
`-Wa,--gdwarf-2`	Asks the assembler to emit DWARF line information.
`-nostartfiles`	Keeps the example's reset/vector code under your control.

Do not strip the ELF you plan to debug. Stripping removes the symbolic information that makes GDB useful.

Use `avr-objdump` beside GDB. It gives a static view of exactly what was linked:

```
avr-objdump -h -t -d -S gdb_demo.elf
```

That command shows section placement, symbols, raw instructions, and source interleaving when line information is present.

3.21 Keep Symbols That Help Debugging

GDB is much easier to use when the assembly file exposes useful names.

Prefer this:

```
.global main
.type   main, @function

main:
    ; setup

sum_loop:
    ; loop body

break_here:
    nop
```

Avoid debugging a file where every important place is an anonymous local label:

```
1:
    ; body
    brne 1b
```

Numeric local labels are fine for tight, obvious control flow, but they are poor debugger landmarks. A professional assembly file has labels at state changes, loop entries, peripheral writes, interrupt entries, error traps, and deliberate debug stops.

For functions and major routines, add `.type name, @function`. For data you intend to inspect, give it a symbol and place it in a named section:

```
.section .bss
.global sum_result
sum_result:
    .zero 1
```

Then GDB can inspect it by name:

```
(gdb) x/1xb &sum_result
```

3.22 Starting GDB

Use the AVR-targeted GDB when your toolchain provides it:

```
avr-gdb gdb_demo.elf
```

This chapter assumes `avr-gdb`. It matches the rest of the AVR GNU toolchain and loads the AVR register set and disassembler directly.

Inside GDB, confirm the file:

```
(gdb) info files
(gdb) info functions
(gdb) info variables
```

GDB startup scripts can change behavior. The local GDB tutorial included with this book notes the normal startup-script order: system init file, user init file, then the current directory's `.gdbinit`. If a debug session behaves strangely, start once with startup scripts disabled:

```
avr-gdb --nx gdb_demo.elf
```

Useful startup settings for assembly work:

```
set pagination off
set confirm off
set radix 16
set disassemble-next-line on
display/i $pc
```

The most important line is `display/i $pc`. GDB's own manual recommends it when stepping by machine instruction because it keeps the next instruction visible after every stop.

3.23 Debugging Targets Used Here

This book assumes one assembly syntax and one hardware debug board:

- source files use GNU AVR assembly accepted by `avr-as`
- the build driver is `avr-gcc`
- the debug image is the linked ELF file
- the hardware target is the ATtiny3217 Curiosity Nano and its on-board debugger

3.23.1 1. Static ELF Inspection

This does not run the program. It only inspects the linked ELF:

```
avr-gdb gdb_demo.elf
```

Useful commands:

```
(gdb) disassemble /r main
(gdb) info address sum_loop
(gdb) info files
(gdb) x/12i main
```

This mode is still valuable. Many assembly bugs are visible before execution: wrong section, unexpected instruction encoding, missing symbol, incorrect vector placement, a branch target in the wrong place, or data linked into flash when it was meant for SRAM.

3.23.2 2. ATtiny3217 Curiosity Nano Hardware

The ATtiny3217 uses UPDI, the Unified Program and Debug Interface. Microchip describes UPDI as the single-pin programming and debugging interface for this device family. The ATtiny3217 product documentation lists the device as an 8-bit AVR running up to 20 MHz, with up to 32 KB flash, 2 KB SRAM, 256 bytes of EEPROM, and a single-pin UPDI programming/debug interface.

The ATtiny3217 Curiosity Nano hardware guide documents the board's on-board debugger as the programming and debugging path for the ATtiny3217 using UPDI. That same on-board debugger also provides a virtual serial port over UART and debug GPIO. Those are the observation paths assumed in the rest of this book.

Debugger-to-target connections on the ATtiny3217 Curiosity Nano:

ATtiny3217 pin	Debugger function	Book use
`PA0`	`DBG0`, UPDI	program/debug lifeline
`PB2`	CDC RX, ATtiny3217 USART0 TX	serial output to host
`PB3`	CDC TX, ATtiny3217 USART0 RX	serial input from host
`PA3`	debug GPIO1	LED0 and timing/debug marks
`PB7`	debug GPIO0	optional timing/debug marks

Microchip documents that these debugger connections are tri-stated when the debugger is not actively using the interface. The board also exposes the signals on the edge connector, but the default assumption here is the unmodified Curiosity Nano board with the on-board debugger still connected.

Do not cut the debugger straps for the examples in this book. Microchip notes that disconnecting those straps disables programming, debugging, virtual serial port, and data streaming for the ATtiny3217 mounted on the board.

For this book, the practical rule is simple: treat PA0/UPDI as a debug lifeline. Do not use it as an ordinary GPIO pin in examples unless you also explain how the board can be recovered.

3.24 Core GDB Commands for Assembly

This table is the daily working set.

Command	Use
`help command`	Show built-in help for one command.
`apropos word`	Search command help by topic.
`file firmware.elf`	Load symbol and section information.
`info files`	Show loaded sections and entry information.
`info functions`	List known function symbols.
`info variables`	List known data symbols.
`break label`	Stop when execution reaches a label.
`hbreak label`	Request a hardware breakpoint when supported.
`delete`	Delete breakpoints.
`continue`	Resume execution until the next stop.
`stepi` or `si`	Execute one machine instruction, entering calls.
`nexti` or `ni`	Execute one machine instruction, stepping over calls when possible.
`finish`	Run until the current routine returns.
`disassemble /r label`	Show instructions and raw opcode bytes.
`x/8i $pc`	Examine eight instructions at the program counter.
`info registers`	Show register state.
`p/x $pc`	Print the program counter in hexadecimal.
`x/16xb address`	Examine 16 bytes of memory in hexadecimal.
`set $reg = value`	Change a register, using the target's register names.
`set {unsigned char}addr = value`	Patch one byte in writable memory.

Register names are target-specific. The GDB manual explicitly points out that `info registers` shows the canonical names for the selected machine. On AVR, check `info registers` before assuming exactly how your GDB build names `r0`, `r1`, `SREG`, `SP`, or `PC`.

3.25 Instruction-Level Stepping

Use instruction stepping when debugging assembly:

```
(gdb) break main
(gdb) continue
(gdb) display/i $pc
(gdb) info registers
(gdb) stepi
(gdb) info registers
```

The difference between `stepi` and `nexti` matters:

Command	Behavior
`stepi` / `si`	Execute one instruction. If the instruction calls a subroutine, stop inside it.
`nexti` / `ni`	Execute one instruction, but try to step over subroutine calls.

For branch debugging, use `stepi` until you trust the flags. A conditional branch is only as correct as the flags set by the previous instruction.

Example:

```
    dec     r18
    brne    sum_loop
```

After `dec r18`, inspect `SREG` or the named flags your GDB exposes. The branch does not test `r18` directly; it tests the Z flag produced by `DEC`.

For call/return debugging, inspect the stack pointer before and after `CALL`, `RCALL`, `RET`, and `RETI`. On AVR, the return address is stored on the stack. The return-address size depends on the device's program counter width, so avoid hard-coding a stack-byte assumption across all AVR families. On the ATtiny3217, the flash size is small enough that ordinary examples use a two-byte return address.

3.26 Reading Disassembly

GDB has two common disassembly views:

```
(gdb) disassemble main
(gdb) disassemble /r main
```

The `/r` form shows raw instruction bytes as well as mnemonics. That is useful when you are checking instruction encoding, word order, or whether the linker relaxed a branch or call.

To look around the current program counter:

```
(gdb) x/10i $pc
```

To look at a label:

```
(gdb) x/12i sum_loop
```

Remember that AVR flash is word-oriented at the instruction-set level but ELF tools and GDB often present addresses in bytes. The instruction set manual gives instruction sizes in words; the debugger may display byte addresses. When an address appears to move by 2 for a single instruction, that usually means one 16-bit instruction word.

3.27 Inspecting Registers and Flags

The fastest way to debug AVR assembly is to predict register changes before stepping:

```
(gdb) info registers
(gdb) stepi
(gdb) info registers
```

For a small sequence:

```
    ldi     r19, 0x11
    ldi     r20, 0x22
    add     r19, r20
```

You should predict:

- `r19` becomes `0x33`
- `r20` stays `0x22`
- `SREG` changes according to the `ADD` result
- `PC` advances by one instruction after each single-word instruction

For flag-heavy code, write down which instruction last set the flags. Common mistakes include:

- expecting `LDI` to set flags
- forgetting that `INC`, `DEC`, `ADD`, `SUB`, `CP`, `CPI`, `LSL`, and shifts affect flags
- placing a harmless-looking instruction between a compare and branch, then accidentally overwriting the flags
- treating carry as a generic error bit instead of the exact carry/borrow state produced by the last arithmetic instruction

When in doubt, single-step and read `SREG`.

3.28 Inspecting SRAM

Use named symbols for SRAM locations:

```
.section .bss
.global sum_result
sum_result:
    .zero 1
```

Then inspect the byte:

```
(gdb) x/1xb &sum_result
```

Other useful memory formats:

```
(gdb) x/16xb &sum_result     # 16 bytes, hex, byte width
(gdb) x/8xh  &sum_result     # 8 halfwords
(gdb) x/4xw  &sum_result     # 4 words in GDB's display format
```

For I/O registers, use the named address from the device header or a symbol you define yourself. Be aware that reading some peripheral registers has side effects. Status registers often clear flags by writing a one, and some data register reads advance FIFOs or clear interrupt states. A debugger memory window is not always passive.

This is especially important when debugging peripherals in later chapters:

- Read the datasheet before repeatedly inspecting a status or data register.
- Know whether a flag is cleared by writing one.
- Know whether a register is synchronized to a peripheral clock domain.
- Know whether a register is protected by a configuration-change mechanism.

3.29 Inspecting Flash Data

AVR has separate program and data address spaces. Instructions live in flash. Ordinary SRAM variables live in data memory. Constants may live in flash and be read by `LPM`.

GDB can disassemble flash reliably because instructions are the normal execution stream:

```
(gdb) x/8i main
```

Raw flash data can be more target-dependent because different tools expose AVR program memory through different address mappings. If GDB's memory view is not obvious, confirm flash contents with `avr-objdump`:

```
avr-objdump -s -j .text gdb_demo.elf
```

For assembly examples, make flash data easy to find:

```
.section .progmem.data, "a", @progbits
.global table_bytes
table_bytes:
    .byte 0x11, 0x22, 0x33, 0x44
```

Then use both tools:

```
avr-nm -n gdb_demo.elf
avr-objdump -s -j .text gdb_demo.elf
```

The default AVR linker script places program-memory input sections into the final flash image, commonly the `.text` output section. That is why the linked ELF is inspected with `-j .text` even though the source used `.progmem.data`. GDB shows what the current target exposes. `objdump` shows what the ELF contains.

3.30 Breakpoints on Microcontrollers

Breakpoints are easy on a desktop process and less abstract on a microcontroller.

A software breakpoint usually means patching code memory with a trap instruction. On a flash-based MCU, that may require special handling because flash is not ordinary writable SRAM. A hardware breakpoint uses on-chip debug resources. Those resources are limited.

Practical rules:

- Prefer breakpoints at labels, not raw numeric addresses.
- Use `hbreak` when the target requires hardware breakpoints.
- Do not assume unlimited breakpoints on a small MCU.
- Keep a deliberate `break_here:` label near important end states.
- For unexpected interrupts, route unused vectors to a trap label such as `unexpected_interrupt: rjmp unexpected_interrupt`.
- Remove stale breakpoints when changing code layout.

On the Curiosity Nano, treat breakpoints as a scarce debug resource and keep your assembly easy to inspect statically with `avr-objdump` even when the hardware session is not attached.

3.31 Watchpoints and Data Breaks

Watchpoints stop when memory changes:

```
(gdb) watch sum_result
```

On embedded targets, watchpoint support depends on the CPU and debug hardware. If a watchpoint is implemented in hardware, there may only be a small number available. If it is implemented in software, it may slow execution heavily.

When watchpoints are limited, instrument the assembly instead:

```
store_result:
    sts     sum_result, r19
break_after_store:
    nop
```

Then break at `break_after_store`.

The label costs nothing in the final instruction stream unless you add an instruction for it. Labels are free debugging handles.

3.32 Debugging the Stack

Stack bugs are common in assembly because the assembler will not protect you from an unbalanced `PUSH`, `POP`, `CALL`, `RET`, interrupt entry, or manual stack write.

Before stepping through subroutines, initialize the stack pointer:

```
    ldi     r16, lo8(RAMEND)
    out     SPL, r16
    ldi     r16, hi8(RAMEND)
    out     SPH, r16
```

Then in GDB:

```
(gdb) info registers
(gdb) p/x $sp
(gdb) x/16xb $sp
```

Register naming can vary, so use `info registers` to confirm whether your GDB build calls the stack pointer `$sp`, shows `SPL`/`SPH`, or exposes both forms.

Checklist for stack debugging:

- Is `SP` initialized before the first `CALL`, `PUSH`, or interrupt?
- Does every `PUSH` have a matching `POP` on every exit path?
- Is `SREG` saved before enabling nested behavior or modifying flags in an ISR?
- Does the routine return with `RET` while an ISR returns with `RETI`?
- Did the code accidentally store ordinary data into the current stack area?

If a return jumps to nonsense, inspect the bytes at the stack pointer before the return instruction. The return address is the evidence.

3.33 Debugging Interrupts

Interrupt debugging requires more discipline than straight-line code.

Give every vector a visible target:

```
.section .vectors, "ax", @progbits
__vectors:
    jmp     main
    jmp     unexpected_interrupt
    jmp     tca0_ovf_isr
```

Give every ISR a visible entry and exit:

```
.global tca0_ovf_isr
.type   tca0_ovf_isr, @function

tca0_ovf_isr:
    push    r16
    in      r16, SREG
    push    r16

    ; ISR body

tca0_ovf_isr_exit:
    pop     r16
    out     SREG, r16
    pop     r16
    reti
```

Then you can use:

```
(gdb) break tca0_ovf_isr
(gdb) break tca0_ovf_isr_exit
(gdb) continue
```

At ISR entry, inspect:

- `PC`
- `SP`
- saved return address bytes
- `SREG`
- peripheral interrupt flag register
- interrupt enable register

At ISR exit, inspect:

- whether the peripheral flag was cleared correctly
- whether saved registers were restored
- whether `SREG` was restored
- whether the stack pointer returned to its entry value
- whether the final instruction is `RETI`, not `RET`

When an interrupt never fires, debug in this order:

1. Is the peripheral clocked?
2. Is the peripheral configured?
3. Is the peripheral interrupt flag set?
4. Is the peripheral interrupt enable bit set?
5. Is the global interrupt enable bit set in `SREG`?
6. Is the vector table at the address the CPU is using?
7. Does the vector point to the intended ISR label?

3.34 Debugging Peripheral Code

For GPIO, timers, USART, SPI, TWI, and NVM code, the debugger can mislead you if you inspect only the instruction stream. Peripheral state often changes outside the CPU pipeline.

Use a three-part method:

1. Step the instruction that writes the control register.
2. Inspect the written register if it is safe to read.
3. Confirm the external effect when possible.

For example, after writing `PORTA_DIRSET`, read back the relevant direction state. After writing `PORTA_OUTTGL`, inspect the output latch and check the pin or LED. After enabling a timer, inspect the timer's control register, then let time pass and inspect the count or interrupt flag.

For communication peripherals, combine GDB with a second observation path:

- virtual COM port for USART
- logic analyzer for SPI/TWI
- oscilloscope for timing-sensitive pins
- Microchip Data Visualizer when using supported board debug features

GDB tells you what the CPU did. External tools tell you what the circuit saw.

3.35 A Minimal Assembly Debug Example

The companion file `src/gdb_demo.S` is intentionally small. It:

- installs a reset vector

- initializes the stack pointer
- loads four flash bytes with `LPM`
- accumulates them into an 8-bit sum
- stores the result in SRAM
- stops at a named debug label

Build it from this chapter directory:

```
avr-gcc -mmcu=attiny3217 -x assembler-with-cpp -g3 -Wa,--gdwarf-2 \
    -c -o gdb_demo.o src/gdb_demo.S

avr-gcc -mmcu=attiny3217 -nostartfiles -g3 \
    -o gdb_demo.elf gdb_demo.o
```

Inspect it statically:

```
avr-nm -n gdb_demo.elf
avr-objdump -d -S gdb_demo.elf
```

Start GDB:

```
avr-gdb --nx gdb_demo.elf
```

Then run these commands:

```
set pagination off
set radix 16
set disassemble-next-line on
display/i $pc

break main
break sum_loop
break break_here

info files
disassemble /r main
```

For hardware confirmation, program the same ELF-derived HEX image to the ATtiny3217 Curiosity Nano through its on-board debugger, then use the board's debugger-backed observation paths: breakpoints, register/memory views, the CDC serial port on USART0, and debug GPIO pins.

At `sum_loop`, step one instruction at a time:

```
info registers
si
info registers
x/8i $pc
```

At `break_here`, inspect the result:

```
x/1xb &sum_result
```

The expected result is `0xaa` because:

```
0x11 + 0x22 + 0x33 + 0x44 = 0xaa
```

If the result differs, debug from evidence:

- Was `Z` initialized to the flash table address?
- Did `LPM r20, Z+` load the expected byte?

- Did `ADD r19, r20` update the accumulator?
- Did `DEC r18` set Z only on the final iteration?
- Did `BRNE sum_loop` branch exactly three times?
- Did `STS sum_result, r19` write SRAM?

Do not guess. Step and inspect.

3.36 A Useful `.gdbinit` for AVR Assembly

For a project directory, this is a good starting point:

```
set pagination off
set confirm off
set radix 16
set disassemble-next-line on
display/i $pc

define regs
    info registers
end

define around
    x/8i $pc
end

define hook-stop
    x/i $pc
end
```

The `hook-stop` command runs every time execution stops. Keep it short. If it prints too much, stepping becomes noisy.

Start with `--nx` when you want to rule out startup-script effects:

```
avr-gdb --nx gdb_demo.elf
```

Start normally when you want the project helpers:

```
avr-gdb gdb_demo.elf
```

3.37 Failure Patterns and What to Inspect

Symptom	Likely area	First inspection
Program starts at the wrong code	vector table or entry address	`info files`, `x/4i 0`
Breakpoint never hits	wrong symbol, optimized/changed layout, wrong target image	`info break`, `info address label`, `compare-sections` if supported
Branch goes the wrong way	flags	`info registers` before and after the flag-setting instruction

Symptom	Likely area	First inspection
Return jumps to nonsense	stack corruption	`p/x $sp`, `x/16xb $sp`, inspect push/pop balance
SRAM variable unchanged	store not reached or wrong address	break before `STS`, inspect pointer/address
`LPM` reads unexpected value	flash table address or address-space mapping	`avr-nm`, `avr-objdump -s`, inspect `Z`
ISR never runs	interrupt enable chain	peripheral flag, peripheral enable bit, global I bit, vector
Curiosity Nano debugger cannot connect	UPDI wiring/fuse/power	target VCC, GND, PA0/UPDI path, fuse state
Debug works once then stops	UPDI pin reused or fuse changed	check PA0 configuration and fuse writes

The point of the table is not to replace thinking. It gives you the first piece of evidence to collect.

3.38 Professional Debug Habits

Write assembly so it can be debugged:

- Use stable labels at important states.
- Keep one instruction per source line.
- Use named SRAM symbols for values you will inspect.
- Keep reset and interrupt vectors readable.
- Save and restore registers in a visible order.
- Keep deliberate trap labels for impossible paths.
- Build with debug information during development.
- Keep the ELF, map file, and objdump listing for every important test image.
- Record the debugger commands that prove a bug is fixed.

When a bug is hard, make the code more observable. Add a label. Store an intermediate value. Toggle a debug pin. Split a dense block into smaller named blocks. Assembly is not self-explaining unless you make the important states visible.

3.39 Source Notes

Facts in this chapter are based on:

- Microchip ATtiny3217 product documentation for device memory sizes and UPDI.
- Microchip ATtiny3217 Curiosity Nano Hardware User Guide sections describing the on-board debugger, UPDI programming/debugging, virtual serial port, and debug GPIO, including PA0/UPDI, PB2/PB3 CDC UART, PA3 debug GPIO1, and PB7 debug GPIO0 connections.
- Microchip ATtiny3217 Curiosity Nano Hardware User Guide sections describing debugger strap disconnection and the warning that cutting the straps disables programming, debugging, virtual serial port, and data streaming for the on-board ATtiny3217.

- The local GDB 17.2 manual for `stepi`, `nexti`, `x/i`, `display/i $pc`, `info registers`, and `hook-stop`.
- The local Red Hat GDB tutorial PDF for practical startup-script behavior, `--nx`, `help`, `apropos`, and basic debug-session setup.

Chapter 4

Instruction Set Basics

The AVR instruction set has roughly 135 instructions, but a much smaller core set does most of the real work in ordinary firmware. This chapter focuses on that core: how instructions are encoded, which register restrictions matter, how arithmetic and logic actually affect the flags, and how to read the CPU's behaviour from the instruction sequence.

The goal is not to memorise a catalog. The goal is to make the instruction set feel predictable.

This chapter uses the **AVRxt** timing column from Microchip's AVR Instruction Set Manual, because the ATtiny3217 is an AVRxt-family device. Older AVR, AVRe, and AVRxm parts are machine-code compatible for most instructions, but some cycle counts differ. When cycle timing matters, check the column for the actual CPU family instead of copying a number from a different AVR generation.

4.1 Instruction Encoding

Every AVR instruction is encoded as one or two 16-bit words (2 or 4 bytes) stored in flash. The CPU fetches one word per clock cycle. Most instructions are a single word and execute in 1–2 cycles.

That encoding detail matters because it affects both code size and speed. On AVR, the shape of an instruction is often the reason for its operand limits. If an instruction only has a few bits available to describe a register or an immediate, the hardware simply cannot support every possible combination.

The instruction set manual reports instruction sizes in **words**, not bytes. One word is 16 bits. So "Words: 1" means 2 bytes in flash, and "Words: 2" means 4 bytes in flash. That same word-oriented view shows up in branch offsets and return addresses.

4.1.1 Single-Word Instructions (16-bit)

The vast majority. The 16 bits encode the opcode, destination register, and source register or small immediate.

```
LDI Rd, K  — Load Immediate
Encoding:  1110 KKKK dddd KKKK
           ─┬──      ─┬──
            └ opcode   └ d = Rd - 16  (only r16–r31 → d = 0–15)
```

```
                         K = 8-bit immediate split across bits

ldi r20, 0xAB:
  d = 20 - 16 = 4 = 0b0100
  K = 0xAB = 0b10101011
  → 1110 1010 0100 1011 = 0xEA4B
```

```
ADD Rd, Rr  — Add without Carry
Encoding:  0000 11rd dddd rrrr
           d = Rd (0–31), r = Rr (0–31)

add r16, r17:
  d = 16 = 0b10000, r = 17 = 0b10001
  → 0000 1101 0000 0001 = 0x0D01
```

4.1.2 Two-Word Instructions (32-bit)

These need a full 16-bit address or program address that doesn't fit in the first word:

Instruction	Why 32-bit
`LDS Rd, k`	Data address k is 16 bits — needs second word
`STS k, Rr`	Same
`JMP k`	Program address k is 22 bits — needs second word
`CALL k`	Same

```
STS k, Rr — Store to SRAM
Word 1:  1001 001r rrrr 0000   (r = Rr, 5 bits)
Word 2:  kkkk kkkk kkkk kkkk  (16-bit address k)

sts PORTB_DIRSET, r16:
  PORTB_DIRSET = 0x0421
  Word 1: 0x9200 | (16 << 4) = 0x9200
  Word 2: 0x0421
  → 4 bytes total in flash
```

This is why `STS` costs 2 cycles and takes twice the flash of `OUT`. Always use `OUT` when the I/O address is ≤ 0x3F.

This is a recurring AVR theme: the shorter instruction is usually faster, but only if the target address fits the shorter encoding.

The same size difference appears in disassembly. `OUT 0x3d, r16` is a single 16-bit word. `STS 0x0401, r16` is two words: one word for the opcode/register field and one word for the 16-bit data-space address. The programmer sees one source line either way, but flash sees different byte counts.

4.1.3 Cycle Counts at a Glance

Category	Typical cycles	Notes
Register ops (ADD, AND, MOV...)	1	Pipeline: fetch overlaps
Loads from SRAM (LD, LDS)	2–3	Memory access latency
Stores to SRAM (ST, STS)	2	
IN / OUT	1	Low I/O space only

Category	Typical cycles	Notes
Taken branch	2	Pipeline flush
Not-taken branch	1	No flush
RCALL / RET	4	Stack push/pop + flush
CALL / JMP (absolute)	4–5	32-bit instr + flush
MUL	2	Result in r1:r0
NOP	1	Explicit no-op

The table assumes ordinary internal SRAM/data accesses. Microchip notes that data-memory cycle counts for `LD`, `LDS`, `ST`, and `STS` assume internal RAM; accesses to non-volatile memory through the data space can take extra time depending on the `NVMCTRL` implementation. That matters later when reading mapped flash on AVRxt with ordinary `LD`.

4.2 Instruction Categories

```
┌─────────────────────────────────────────────────────────────┐
│                    AVR Instruction Categories                │
├─────────────────┬───────────────────────────────────────────┤
│ Data Transfer   │ MOV MOVW LDI LD ST LDS STS LPM PUSH POP  │
│                 │ IN OUT                                    │
├─────────────────┼───────────────────────────────────────────┤
│ Arithmetic      │ ADD ADC SUB SBC SUBI SBCI ADIW SBIW       │
│                 │ INC DEC NEG MUL MULS MULSU                │
├─────────────────┼───────────────────────────────────────────┤
│ Logic           │ AND ANDI OR ORI EOR COM TST CLR SER       │
├─────────────────┼───────────────────────────────────────────┤
│ Shift / Rotate  │ LSL LSR ROL ROR ASR SWAP                  │
├─────────────────┼───────────────────────────────────────────┤
│ Bit Operations  │ SBI CBI SBIC SBIS SBRC SBRS BST BLD       │
│                 │ SEC CLC SEI CLI BSET BCLR                 │
├─────────────────┼───────────────────────────────────────────┤
│ Branch / Jump   │ RJMP JMP IJMP RCALL CALL ICALL RET RETI   │
│                 │ BREQ BRNE BRCS BRCC BRSH BRLO BRMI BRPL   │
│                 │ BRGE BRLT BRVS BRVC BRIE BRID CPSE        │
├─────────────────┼───────────────────────────────────────────┤
│ MCU Control     │ NOP SLEEP WDR BREAK                       │
└─────────────────┴───────────────────────────────────────────┘
```

Chapters 5–8 cover Load/Store, Arithmetic, Branches, and Subroutines in depth. This chapter covers the core register-to-register operations that form the building blocks of everything else.

So think of this chapter as the grammar of AVR instructions. Later chapters combine these pieces into bigger programming patterns.

4.3 Data Transfer — Moving Values Around

4.3.1 MOV — Copy Register

```
MOV Rd, Rr          ; Rd ← Rr
```

Copies the 8-bit value from Rr to Rd. Source unchanged. No flags affected. Works on any register r0–r31.

`MOV` is boring in the best possible way. It does exactly what it says, and because it does not touch flags, it is safe to use in the middle of code that depends on the current condition state.

```
ldi r16, 42
mov r0, r16         ; r0 = 42, r16 still = 42
mov r5, r0          ; r5 = 42 (MOV works on r0–r15 unlike LDI)
```

1 cycle. Does not affect SREG.

4.3.2 MOVW — Copy Register Pair

```
MOVW Rd, Rr         ; Rd+1:Rd ← Rr+1:Rr
```

Copies two registers in one instruction. Rd and Rr must be even-numbered: r0, r2, r4, ... r30.

```
; Copy X pointer (r27:r26) into r1:r0
movw r0, r26        ; r1:r0 = r27:r26

; Copy Z into Y for a secondary pointer
movw r28, r30       ; r29:r28 = r31:r30  (Y = Z)
```

1 cycle. Does not affect SREG. Saves one instruction vs two `MOV`s.

Whenever you are copying a pointer pair or a 16-bit value, `MOVW` is worth remembering because it is both shorter and clearer than two separate moves.

Because `MOVW` uses even register numbers, `movw r24, r22` copies `r23:r22` into `r25:r24`. Think “low register named, high register implied.” This is the same low-byte-first convention used by pointer pairs X (`r27:r26`), Y (`r29:r28`), and Z (`r31:r30`).

4.3.3 LDI — Load Immediate

```
LDI Rd, K           ; Rd ← K   (K = 0..255)
```

Load a constant into a register. **r16–r31 only.**

```
ldi r16, 0xFF       ; OK
ldi r20, (1 << 4)   ; OK — expression evaluated at assembly time
ldi r0,  42         ; ERROR: r0 not allowed
```

1 cycle. Does not affect SREG.

`LDI` has an 8-bit immediate (`K = 0..255`) and a 4-bit encoded destination field. That 4-bit field is why only the upper half of the register file is legal: the encoded value is `Rd - 16`.

4.3.4 CLR and SER — Clear and Set All Bits

These are **pseudo-instructions** (GAS aliases, not separate opcodes):

```
clr r16     ; r16 = 0x00 — assembled as EOR r16, r16
ser r16     ; r16 = 0xFF — assembled as LDI r16, 0xFF (r16–r31 only)
```

CLR works on any register (r0–r31) because it uses EOR. SER only works on r16–r31 because it uses LDI.

This is a good example of how AVR pseudo-instructions work. They look like real instructions in source, but the assembler is translating them into more basic operations for you.

CLR sets Z=1, clears N, V, S. SER affects no flags.

Pseudo-instructions are useful, but do not forget their real flag behavior. CLR is not flag-neutral because it is really EOR Rd,Rd. SER is flag-neutral because it is really LDI Rd,0xFF.

4.4 Arithmetic Instructions

4.4.1 ADD — Add Without Carry

```
ADD Rd, Rr          ; Rd ← Rd + Rr
```

Adds two registers, stores result in Rd. Affects: H, S, V, N, Z, C.

If you are coming from a higher-level language, read that carefully: the result replaces the original destination register. AVR arithmetic is almost always destructive to the destination operand.

```
ldi r16, 100
ldi r17, 50
add r16, r17        ; r16 = 150, C=0, Z=0, N=0
```

Unsigned overflow (result > 255): C=1, result wraps.

```
ldi r16, 200
ldi r17, 100
add r16, r17        ; 200+100=300 → r16=44, C=1
```

For signed arithmetic, the carry flag is not the overflow test. Signed overflow is reported in V, and signed branches use S (N XOR V). For example, 0x7F + 0x01 = 0x80 sets V because +127 became -128 in 8-bit two's complement, even though that operation is also just an ordinary 8-bit wraparound.

4.4.2 ADC — Add with Carry

```
ADC Rd, Rr          ; Rd ← Rd + Rr + C
```

Identical to ADD but adds the carry flag too. Essential for **multi-byte arithmetic** — carry from the low byte automatically propagates into the high byte:

```
; 16-bit add: r17:r16 += r19:r18
add r16, r18        ; low bytes first — may set C
adc r17, r19        ; high bytes — C from previous ADD is included
```

```
; 24-bit add: r2:r1:r0 += r5:r4:r3
add r0, r3
adc r1, r4
adc r2, r5          ; each ADC picks up carry from the byte below
```

This low-byte-first pattern is one of the most important habits in AVR assembly. Once it becomes automatic, larger integer arithmetic stops feeling special.

Microchip's manual defines `ADC` as `Rd <- Rd + Rr + C`; the carry input is the current C bit before the instruction executes, and the output C bit is then ready for the next byte. That is why no separate "carry variable" exists in multi-byte arithmetic.

4.4.3 SUB — Subtract Without Carry

```
SUB Rd, Rr          ; Rd ← Rd - Rr
```

Subtracts Rr from Rd. C=1 means borrow (unsigned underflow).

That "C means borrow" convention is easy to trip over because it feels backwards if you learned carry only from addition. On AVR subtraction, carry set means the subtraction had to borrow.

```
ldi r16, 10
ldi r17, 20
sub r16, r17        ; 10-20 = -10 → r16=246 (0xF6), C=1 (borrow)
```

The same instruction can be read three ways:

Interpretation	Example result for `10 - 20`
Raw 8-bit bits	`0xF6`
Unsigned value	246, with C=1 showing borrow
Signed value	`-10`, with N=1 and S depending on V

The CPU only produces bits and flags. Your later branch instruction decides which interpretation matters.

4.4.4 SBC — Subtract with Carry (Borrow)

```
SBC Rd, Rr          ; Rd ← Rd - Rr - C
```

Like ADC for addition, SBC propagates borrow in multi-byte subtraction:

```
; 16-bit subtract: r17:r16 -= r19:r18
sub r16, r18        ; low bytes
sbc r17, r19        ; high bytes — includes borrow from low byte
```

Again, the pattern is the same as multi-byte addition: low byte first, then work upward using the carry/borrow chain.

4.4.5 SUBI — Subtract Immediate

```
SUBI Rd, K          ; Rd ← Rd - K   (r16–r31 only)
```

Subtract a constant without needing a second register. **No ADDI exists** — use `SUBI` with the negated value, or `ADIW`/`SBIW` for 16-bit register pairs.

That missing `ADDI` surprises many beginners. It is not an omission in the book; it is a real feature of the AVR ISA.

```
ldi r16, 50
subi r16, 7         ; r16 = 43
subi r16, -3        ; r16 = 46  (subtract -3 = add 3)
```

`SUBI r16, -3` works because the assembler encodes `-3` as the 8-bit constant `0xFD`. Subtracting `0xFD` from an 8-bit register is the same low-byte result as adding 3. This idiom is common, but use it deliberately; it can be less readable than `ADIW` when you really mean "add to a 16-bit pointer."

4.4.6 SBCI — Subtract Immediate with Carry

```
SBCI Rd, K          ; Rd ← Rd - K - C   (r16–r31 only)
```

Pair with `SUBI` for 16-bit immediate subtraction:

```
; r17:r16 -= 300 (0x012C)
subi r16, lo8(300)  ; r16 -= 0x2C
sbci r17, hi8(300)  ; r17 -= 0x01 - C
```

4.4.7 ADIW — Add Immediate to Word

```
ADIW Rd, K          ; Rd+1:Rd ← Rd+1:Rd + K   (K = 0..63)
```

Adds a small constant to a 16-bit register pair in one instruction. Only works on r25:r24, r27:r26 (X), r29:r28 (Y), r31:r30 (Z).

```
adiw r24, 1         ; r25:r24 += 1
adiw XL, 4         ; X pointer += 4 (advance 4 bytes)
adiw ZL, 16        ; Z pointer += 16
```

2 cycles. Affects C, Z, N, V, S.

`ADIW` is especially nice for pointer arithmetic, because X, Y, and Z all live in the supported register-pair set.

The immediate is only 6 bits, so `K` is limited to `0..63`. Larger pointer steps need repeated `ADIW`, register arithmetic, or explicit low/high-byte addition.

4.4.8 SBIW — Subtract Immediate from Word

```
SBIW Rd, K          ; Rd+1:Rd ← Rd+1:Rd - K   (K = 0..63)
```

Same register restrictions as ADIW. Used in delay loops (Chapter 1) and pointer adjustment.

```
sbiw r24, 1         ; r25:r24 -= 1 — sets Z when result = 0
sbiw ZL, 8         ; Z -= 8
```

4.4.9 INC and DEC — Increment / Decrement

```
INC Rd              ; Rd ← Rd + 1
DEC Rd              ; Rd ← Rd - 1
```

Works on r0–r31. **Critical gotcha: INC and DEC do not affect the carry flag (C).** This catches people using DEC in multi-byte loops.

```
; This loop is WRONG for multi-byte work:
ldi r16, 0
ldi r17, 0
dec r16             ; r16 = 255, C unchanged — cannot chain with SBC!

; Correct 16-bit decrement: use SBIW
sbiw r16, 1         ; r17:r16 -= 1, sets Z correctly
```

Flags affected by INC/DEC: S, V, N, Z — but NOT C.

That is why `INC` and `DEC` are great for single-byte counters and a bad fit for arithmetic that needs carry propagation.

Microchip calls out another subtlety: for unsigned values after `INC` or `DEC`, only `BREQ` and `BRNE` are generally reliable tests. Use them for "did the counter wrap/reach zero?" tests. Do not use `BRLO`/`BRSH` after `INC` or `DEC` and expect a meaningful unsigned less-than/greater-than result, because C was not updated.

4.4.10 NEG — Two's Complement Negate

```
NEG Rd              ; Rd ← 0x00 - Rd
```

Negates the register (flip all bits, then add 1). NEG 0 → 0 with C=0. All other values: C=1.

```
ldi r16, 10
neg r16             ; r16 = 246 (0xF6), which is -10 in two's complement
neg r16             ; r16 = 10 again
```

Useful for: converting subtraction direction, computing absolute value.

Whenever you see `NEG`, think "arithmetic sign change." Whenever you see `COM`, think "bitwise inversion." They are related, but not interchangeable.

There is one unavoidable two's-complement corner case: `NEG 0x80` leaves the bit pattern `0x80`. In signed 8-bit arithmetic, -128 has no positive counterpart representable in 8 bits. The V flag reports that overflow case.

4.5 Logic Instructions

4.5.1 AND / ANDI — Bitwise AND

```
AND  Rd, Rr         ; Rd ← Rd AND Rr          (r0–r31)
ANDI Rd, K          ; Rd ← Rd AND K           (r16–r31 only)
```

Use AND to **clear** (mask off) specific bits:

```
ldi r16, 0xAB       ; 0b10101011
andi r16, 0x0F      ; r16 = 0x0B (0b00001011) — upper nibble cleared
andi r16, 0b11111110 ; clear bit 0
```

Affects: S, V(=0), N, Z. Clears V.

Bit masking is one of the most common operations in embedded work. In practice: AND clears bits, OR sets bits, and EOR toggles bits.

`ANDI` is restricted to `r16-r31`, but `AND` works on all 32 registers. If your value is in `r0-r15`, either use a mask register with `AND`, or move the value to an upper register before using `ANDI`.

4.5.2 OR / ORI — Bitwise OR

```
OR   Rd, Rr         ; Rd ← Rd OR Rr
ORI  Rd, K          ; Rd ← Rd OR K            (r16–r31 only)
```

Use OR to **set** specific bits:

```
ldi r16, 0x2A
ori r16, 0x80       ; set bit 7 → r16 = 0xAA
ori r16, (1 << 3)   ; set bit 3
```

`ORI` has the same upper-register restriction as `LDI` and `ANDI`. The opcode has room for an 8-bit immediate and only the reduced destination encoding.

4.5.3 EOR — Bitwise XOR (Exclusive OR)

```
EOR Rd, Rr          ; Rd ← Rd XOR Rr
```

Use EOR to **toggle** bits or **invert** a value:

```
ldi r16, 0xFF
ldi r17, 0x0F
eor r16, r17        ; r16 = 0xF0 — lower nibble toggled
eor r16, r16        ; r16 = 0x00 — self-XOR always gives 0 (this is CLR)
```

4.5.4 COM — One's Complement (Bitwise NOT)

```
COM Rd              ; Rd ← 0xFF - Rd  (flip every bit)
```

```
ldi r16, 0b10101010
com r16             ; r16 = 0b01010101
```

Always sets C=1. Note: `COM` ≠ `NEG`. COM flips bits; NEG negates (COM + 1).

Because `COM` always sets C, avoid placing it between the low and high bytes of an `ADD`/`ADC` or `SUB`/`SBC` carry chain unless changing C is exactly what you intend.

4.5.5 TST — Test Register (Non-Destructive)

```
TST Rd              ; Rd AND Rd → set N, Z flags; Rd unchanged
```

Tests whether a register is zero or negative without modifying it:

```
lds r16, some_var
tst r16
breq var_is_zero    ; Z=1 if r16 was 0
brmi var_is_neg     ; N=1 if bit 7 was set (negative signed)
```

Assembled as `AND Rd, Rd`.

`TST` is one of those instructions that mostly exists to make your intent more obvious to a human reader.

Since `TST` is really `AND Rd,Rd`, it clears V and sets S from `N XOR V`. After `tst r16`, `BRMI` tests bit 7 directly because V is zero.

4.6 Comparison Instructions

These set flags **without storing any result**. Always followed by a conditional branch.

This is the AVR version of "ask a question, then branch on the answer." The question is encoded in the compare; the answer appears in the flags.

4.6.1 CP — Compare

```
CP Rd, Rr           ; Rd - Rr → flags only, result discarded
```

Exactly like SUB but throws away the result. Use before `BREQ`, `BRNE`, `BRLO`, `BRGE`, etc.

That makes `CP` conceptually simple: it performs a subtraction only for the purpose of updating flags.

```
ldi r16, 42
ldi r17, 100
cp  r16, r17        ; flags set as if 42 - 100
brlo r16_smaller    ; BRLO branches if C=1 (unsigned: Rd < Rr)
```

4.6.2 CPC — Compare with Carry

```
CPC Rd, Rr          ; Rd - Rr - C → flags only
```

For multi-byte comparison (after comparing low bytes with CP):

```
; Compare r17:r16 with r19:r18 (16-bit unsigned)
cp  r16, r18        ; compare low bytes
cpc r17, r19        ; compare high bytes including borrow
brlo less_than      ; taken if r17:r16 < r19:r18 unsigned
```

4.6.3 CPI — Compare with Immediate

```
CPI Rd, K           ; Rd - K → flags only    (r16–r31 only)
```

```
ldi r16, 10
cpi r16, 10
breq exactly_ten    ; taken
```

4.6.4 CPSE — Compare, Skip if Equal

```
CPSE Rd, Rr         ; if Rd == Rr: skip next instruction
```

Skips exactly one instruction when equal. Used for compact conditional code:

```
cpse r16, r17
rjmp not_equal      ; skipped if r16 == r17
; falls through here if equal
```

`CPSE` does **not** update SREG. It simply compares and adjusts the program counter if the registers are equal. It also has variable timing: 1 cycle if no skip happens, 2 cycles if it skips a one-word instruction, and 3 cycles if it skips a two-word instruction.

That makes `CPSE` compact, but not always ideal for cycle-stable code:

```
cpse r16, r17
sts  PORTA_OUTTGL, r18    ; two-word instruction: skip path is longer
```

If the next instruction might be two words (`LDS`, `STS`, `JMP`, `CALL`), count the skip path carefully.

4.7 Conditional Branches — Full Table

Every branch instruction tests one or more SREG flags. The mnemonic tells you what condition causes the branch.

Once you stop reading branch mnemonics as magic words and start reading them as “test this flag pattern,” control flow becomes much easier to reason about.

Mnemonic	Full name	Condition	Flag(s)	
BREQ	Branch if Equal	Z = 1	Z	
BRNE	Branch if Not Equal	Z = 0	Z	
BRCS	Branch if Carry Set	C = 1	C	
BRCC	Branch if Carry Cleared	C = 0	C	
BRSH	Branch if Same or Higher	C = 0	C	← unsigned ≥
BRLO	Branch if Lower	C = 1	C	← unsigned <
BRMI	Branch if Minus	N = 1	N	
BRPL	Branch if Plus	N = 0	N	
BRGE	Branch if ≥ (signed)	S = 0	N⊕V	← signed ≥
BRLT	Branch if < (signed)	S = 1	N⊕V	← signed <
BRVS	Branch if Overflow Set	V = 1	V	
BRVC	Branch if Overflow Clear	V = 0	V	
BRIE	Branch if Interrupts En.	I = 1	I	
BRID	Branch if Interrupts Dis.	I = 0	I	
BRHS	Branch if Half-carry Set	H = 1	H	
BRHC	Branch if Half-carry Clr	H = 0	H	
BRTS	Branch if T Set	T = 1	T	
BRTC	Branch if T Clear	T = 0	T	

All branch instructions: **-64 to +63 words** offset relative to the next instruction. In address terms, the destination must be within `PC - 63` through `PC + 64` after the branch instruction’s normal `PC + 1` step. If the target is farther, use `JMP` or `RJMP` with an inverted condition:

```
; Branch to far_label if r16 == r17 — but far_label > 63 words away:
cp   r16, r17
brne skip           ; if NOT equal, skip the jump
jmp  far_label
skip:
```

This branch-range limit is another place where instruction encoding leaks into everyday programming. The branch is short because the displacement field is short.

Conditional branches do not modify SREG. They only read the flags left by an earlier instruction. Any instruction inserted between the compare and the branch must be checked for flag effects:

```
cp   r16, r17
mov  r18, r16       ; OK: MOV does not affect flags
breq equal

cp   r16, r17
```

```
andi r18, 0x0F      ; BAD if you meant to branch on CP: ANDI changes Z/N/V/S
breq equal
```

4.8 Unsigned vs Signed Comparisons

This trips up almost every beginner. Unsigned and signed comparisons use different branch instructions after the same `CP`.

```
ldi r16, 0xF0   ; = 240 unsigned,  or -16 signed
ldi r17, 0x10   ; = 16  unsigned,  or  16 signed

cp  r16, r17    ; 0xF0 - 0x10 = 0xE0.  C=0 (no borrow), N=1, V=0, S=1

brlo below_u    ; C=1? NO → not taken.  Correct: 240 > 16 unsigned.
brlt below_s    ; S=1? YES → taken.    Correct: -16 < 16 signed.
```

Rule: **BRLO/BRSH for unsigned. BRLT/BRGE for signed.**

If you take only one rule from this section, take that one. The CPU does not know whether you intended a byte to mean 240 or -16. Your branch choice tells it how to interpret the flags.

Microchip's branch summary says the common tests (`Rd < Rr`, `Rd >= Rr`, and so on) are meaningful immediately after compare/subtract-style instructions such as `CP`, `CPI`, `SUB`, and `SUBI`. If the preceding instruction was a logical operation, `BRLO` still means "C=1", but it no longer means "less than" unless that flag pattern was intentionally prepared.

4.9 Flag Effects Summary

The table below covers every instruction in this chapter. "—" means the flag is not affected.

Instruction	C	Z	N	V	S	H	
ADD	●	●	●	●	●	●	
ADC	●	●	●	●	●	●	
SUB	●	●	●	●	●	●	
SBC	●	●	●	●	●	●	
SUBI	●	●	●	●	●	●	
SBCI	●	●	●	●	●	●	
ADIW	●	●	●	●	●	–	
SBIW	●	●	●	●	●	–	
INC	–	●	●	●	●	–	← C NOT affected
DEC	–	●	●	●	●	–	← C NOT affected
NEG	●	●	●	●	●	●	
AND / ANDI	0	●	●	0	●	–	← V cleared
OR / ORI	0	●	●	0	●	–	← V cleared
EOR	0	●	●	0	●	–	← V cleared
COM	1	●	●	0	●	–	← C always 1, V cleared
TST	0	●	●	0	●	–	
CP / CPI	●	●	●	●	●	●	
CPC	●	●	●	●	●	●	

```
MOV/MOVW/LDI  –    –    –    –    –    –
CLR           0    1    0    0    0    –
```

The full SREG also contains I, T, H, S, V, N, Z, and C. This table focuses on the arithmetic/logic flags used in this chapter. `SEI`, `CLI`, `BSET`, `BCLR`, `BST`, and `BLD` manipulate the other status bits and are handled more deeply in the interrupt and bit-manipulation chapters.

4.10 Worked Example: Compare, Clamp, Negate

Goal: read an 8-bit value, clamp it to 0–100 range, then negate it if a flag register bit is set.

This example matters because it combines several ideas you will use constantly: compare, branch, clamp by assignment, test a control bit, and optionally apply an arithmetic transform.

```
/*
 * clamp_and_negate:
 *   Input:  r24 = raw value (unsigned), r25 = flags (bit 0 = negate)
 *   Output: r24 = clamped value, optionally negated
 *   Clobbers: r16
 */
clamp_and_negate:
    /* Step 1: clamp to 100 max */
    cpi     r24, 100            ; compare r24 with 100
    brlo    clamped             ; if r24 < 100 (unsigned), skip clamp
    ldi     r24, 100            ; else clamp to 100
clamped:

    /* Step 2: clamp to 0 min – already unsigned so r24 >= 0 always.
     * If the raw value were signed we'd add BRGE check here.          */

    /* Step 3: negate if bit 0 of r25 is set */
    sbrs    r25, 0              ; Skip if Bit in Register Set (bit 0 of r25)
    ret                         ; bit 0 clear – return as-is
    neg     r24                 ; negate: r24 = 0 - r24
    ret
```

New instructions introduced:

> **SBRS Rr, b** Skip next instruction if bit b of Rr is set. 1 cycle (not skipped), 2 cycles (skipped over 1-word instr), 3 cycles (skipped over 2-word instr). Does not affect SREG.

The skip family all follows the same timing idea on AVRxt:

Instruction	Test location	Skips when
`SBRC Rr,b`	Register bit	bit is clear
`SBRS Rr,b`	Register bit	bit is set
`SBIC A,b`	Low I/O bit `0x00-0x1F`	bit is clear
`SBIS A,b`	Low I/O bit `0x00-0x1F`	bit is set
`CPSE Rd,Rr`	Register compare	registers equal

All skip exactly one following instruction, not a block. If you need to skip multiple operations, skip over a branch:

```
sbrs r25, 0
rjmp no_negate
neg  r24
no_negate:
```

Trace through the example:

```
Call: r24=150, r25=0b00000001 (negate flag set)

cpi r24, 100     → 150-100=50, C=0, Z=0
brlo clamped     → C=0: NOT taken (150 >= 100, no branch)
ldi r24, 100     → r24 = 100
clamped:
sbrs r25, 0      → bit 0 of r25 is 1: SKIP next instr
                   (ret is skipped)
neg r24          → r24 = 0 - 100 = 156 (0x9C = -100 signed)
ret
```

Walking through examples like this is how you build intuition. Do not just read the final code; read the flags and the decision points as they evolve.

4.11 Operand Restrictions Cheat-Sheet

This is worth turning into instinct:

Instruction	Rd range	Rr range	Immediate range
MOV	r0–r31	r0–r31	–
MOVW	r0,r2..r30	r0,r2..r30	– (even only)
LDI	r16–r31	–	0–255
CLR	r0–r31	–	–
SER	r16–r31	–	–
ADD/ADC	r0–r31	r0–r31	–
SUB/SBC	r0–r31	r0–r31	–
SUBI/SBCI	r16–r31	–	0–255
ADIW/SBIW	r24,X,Y,Z	–	0–63
INC/DEC	r0–r31	–	–
NEG/COM	r0–r31	–	–
AND/OR/EOR	r0–r31	r0–r31	–
ANDI/ORI	r16–r31	–	0–255
CP	r0–r31	r0–r31	–
CPI	r16–r31	–	0–255
CPC	r0–r31	r0–r31	–
CPSE	r0–r31	r0–r31	–
SBRS/SBRC	r0–r31	–	bit 0–7

4.12 Summary

```
Instructions: 1 or 2 words (16 or 32 bits).
Two-word: LDS/STS carry a 16-bit data address; JMP/CALL carry a larger
```

```
program address field.
AVRxt timing in this chapter: LDS=3 cycles, STS=2 cycles, LD/ST=2 cycles
for internal SRAM; mapped NVM can add cycles.

Register rules:
  LDI, SUBI, SBCI, ANDI, ORI, SER, CPI → r16–r31 only.
  ADIW, SBIW → r24, X(r26), Y(r28), Z(r30) only.
  Everything else → r0–r31.

Carry chains:  ADD+ADC, SUB+SBC, SUBI+SBCI — always low byte first.
INC/DEC do NOT touch C — never use for multi-byte counters.
NEG ≠ COM: NEG = two's complement, COM = bitwise invert.

Comparisons:
  Unsigned: BRLO (< ), BRSH (≥ ), BRCS/BRCC
  Signed:   BRLT (< ), BRGE (≥ ), BRMI/BRPL

Branch range: -64..+63 word offset. Use JMP/RJMP patterns for farther targets.
Skip instructions skip one instruction only and cost 1/2/3 cycles depending
on whether no skip, one-word skip, or two-word skip occurs.
```

4.13 Exercises

1. Add two 24-bit numbers stored in `r2:r1:r0` and `r5:r4:r3`. Store the result in `r2:r1:r0`. What happens to the carry if the result exceeds 24 bits?
2. Write a sequence to compute `r16 = r16 * 4` without using `MUL`. (Hint: shifting left by 1 is the same as multiplying by 2.) Use only `ADD` instructions.
3. Write a loop that counts from 0 to 255 in `r16`, then wraps and repeats forever. Use `INC` and `BREQ`. Why is `BREQ` the right branch here?
4. Given `r16 = 0xAB` and `r17 = 0x3C`, predict the result and all six flag values (C, Z, N, V, S, H) after each operation — before assembling:
   - `ADD r16, r17`
   - `SUB r16, r17` (fresh r16=0xAB each time)
   - `AND r16, r17`
   - `EOR r16, r17`
5. Write `clamp_and_negate` to also handle the case where `r24` starts as 0 and the negate flag is set. What does `NEG 0` produce and is that correct for your use case?
6. Why does `SUBI r16, -1` add 1 to r16? What is the bit pattern of -1 as an 8-bit two's complement number, and what does subtracting it do?

Next: Chapter 5 — Load and Store

Chapter 5

Load and Store

Every useful program moves data. In AVR assembly, **every data movement is explicit**: you choose the exact instruction that moves each byte, the exact addressing mode, and often the exact pointer update behaviour. This chapter covers the main ways to get data into registers, get it back out to memory, and read constants from flash on the ATtiny3217.

5.1 The Memory-Register Model

The AVR CPU can only operate on data that is in **registers** (r0–r31). To work on a byte in SRAM, you must load it into a register first, operate, then store it back. There is no "operate directly on memory" instruction.

That rule is so central that it is worth repeating in plain language: if the value is not in a register yet, the ALU cannot touch it.

```
SRAM / I/O regs / Flash
         │
         │  Load (LD, LDS, LPM, IN)
         ▼
   Registers r0–r31   ──── ALU (ADD, AND, EOR...) ────► result in Rd
         │
         │  Store (ST, STS, OUT)
         ▼
SRAM / I/O regs
```

Every instruction in this chapter is either a load (memory → register) or a store (register → memory).

Once you see that pattern, a lot of AVR code becomes easier to read. A routine is often just alternating phases: bring bytes into registers, do work there, write results back out.

5.1.1 ATtiny3217 Data Space Is Not Just SRAM

The load/store instructions use the AVR **data address space**, not just the 2 KB internal SRAM block. On ATtiny3217, Microchip's memory map puts several different resources into that 16-bit address space:

Data address range	Resource
`0x0000-0x003F`	64 I/O registers
`0x0040-0x0FFF`	960 extended I/O/peripheral registers
`0x1000-0x13FF`	NVM I/O registers and data
`0x1400-0x14FF`	EEPROM, 256 bytes
`0x3800-0x3FFF`	Internal SRAM, 2 KB
`0x8000-0xFFFF`	Flash mapped into data space, 32 KB

That is why the same `LD` instruction can read SRAM, an I/O register, EEPROM space, or mapped flash. The address decides which hardware block answers.

For normal variables, the assembler and linker place `.bss` and `.data` in SRAM at `0x3800-0x3FFF`. Do not treat EEPROM or flash as scratch SRAM: they are nonvolatile memory regions controlled by NVMCTRL and have different write rules and timing.

5.2 Addressing Modes

The AVR supports five ways to specify a memory address:

Mode	Syntax	Address source	Instruction
Direct	`LDS Rd, k`	16-bit constant `k` in instruction	LDS / STS
Indirect	`LD Rd, X`	Value in X, Y, or Z pointer pair	LD / ST
Post-increment	`LD Rd, X+`	X (used), then X += 1	LD / ST
Pre-decrement	`LD Rd, -X`	X -= 1, then X (used)	LD / ST
Indirect + displacement	`LDD Rd, Y+q`	Y + constant q (0–63)	LDD / STD

These are not five completely different ideas. They are variations on one theme: where does the address come from, and should the pointer change as a side effect?

5.3 Direct Addressing: LDS and STS

```
LDS Rd, k           ; Rd ← SRAM[k]     (k = 16-bit address)
STS k, Rr           ; SRAM[k] ← Rr
```

The address is encoded directly in the instruction as a second 16-bit word. Both are **32-bit (two-word) instructions**.

That makes `LDS`/`STS` very convenient to write, but relatively expensive in both flash space and cycles.

```
.section .bss
my_var: .skip 1         ; reserve one byte in SRAM
```

```
.section .text
    ldi  r16, 0xAB
    sts  my_var, r16    ; write 0xAB to my_var's SRAM address
    lds  r17, my_var    ; r17 = 0xAB
```

Cycle costs:

Instruction	Cycles
`LDS Rd, k`	3
`STS k, Rr`	2

These AVRxt cycle counts assume internal SRAM or ordinary data-memory access. Microchip's instruction manual notes that data-memory accesses to NVM need at least one extra cycle, with the exact timing depending on the device's NVMCTRL implementation. Treat EEPROM and mapped flash as slower than SRAM unless the device data sheet gives a tighter bound for the specific access.

When to use LDS/STS: accessing a single named variable. For loops over arrays, indirect addressing (Section 5.4) is faster and smaller.

So `LDS`/`STS` are great for "go touch this exact place once." They are a poor fit for repeated traversal through memory.

5.3.1 I/O Registers Above 0x3F

`STS`/`LDS` are the only way to reach peripheral registers with addresses above 0x3F. This includes all the PORT, timer, and USART registers on ATtiny3217:

```
ldi  r16, (1 << 4)
sts  PORTB_DIRSET, r16      ; PORTB_DIRSET = 0x0421, above 0x3F → STS
```

For addresses 0x00–0x3F, always prefer `IN`/`OUT` (1 cycle, 1 word) over `LDS`/`STS` (3/2 cycles, 2 words).

That is one of the easiest AVR optimisations to make because it is mechanical: if the register is in low I/O space, use the I/O instructions.

There is one more I/O-address boundary to remember:

- `IN` and `OUT` can address I/O locations `0x00-0x3F`.
- `SBI`, `CBI`, `SBIC`, and `SBIS` can address only `0x00-0x1F`.

ATtiny3217 exposes virtual port registers (`VPORTx`) in the low I/O range so simple GPIO operations can use `IN`/`OUT`. The full `PORTx` peripheral registers live in extended I/O space, so examples such as `PORTB_DIRSET` use `STS`.

5.4 Pointer Registers: X, Y, Z

The three pointer register pairs are 16-bit registers built from adjacent general-purpose registers:

```
X = r27 : r26       XH : XL
Y = r29 : r28       YH : YL
Z = r31 : r30       ZH : ZL
```

To load an address into a pointer pair, use two `LDI` instructions:

```
; Point X at buf (a label in .bss)
ldi  XL, lo8(buf)       ; XL = r26 = low byte of buf's address
ldi  XH, hi8(buf)       ; XH = r27 = high byte
```

Now X holds the 16-bit address of `buf` and can be used with `LD`/`ST`.

This is the beginning of pointer-based programming in AVR. Instead of naming an address directly in every instruction, you put the address in X, Y, or Z once and then operate through that pointer.

Practical convention:

- Use X for simple source or destination pointers.
- Use Y when you want a stable base pointer for `LDD`/`STD`, especially for structs or stack-frame-style code.
- Use Z for flash table reads, because `LPM` requires Z and mapped-flash examples commonly use Z too.

The ATtiny3217 data space fits in 64 KB, so X, Y, and Z are enough. Larger AVR devices can add `RAMPX`, `RAMPY`, `RAMPZ`, or `RAMPD` to extend addressing beyond 64 KB, but that extra machinery is not part of normal ATtiny3217 code.

5.5 Indirect Addressing: LD and ST

5.5.1 Basic Indirect — No Change to Pointer

```
LD  Rd, X           ; Rd ← SRAM[X]      (X unchanged)
LD  Rd, Y           ; Rd ← SRAM[Y]
LD  Rd, Z           ; Rd ← SRAM[Z]
ST  X, Rr           ; SRAM[X] ← Rr      (X unchanged)
ST  Y, Rr
ST  Z, Rr
```

2 cycles each. Use when you want to read/write the same address multiple times without moving the pointer.

This is the pointer equivalent of “stay where you are and keep working here.”

```
; Read, modify, write a single SRAM byte
ldi  XL, lo8(my_var)
ldi  XH, hi8(my_var)
ld   r16, X             ; r16 = my_var
ori  r16, 0x80          ; set bit 7
st   X, r16             ; write back — X still points to my_var
```

5.5.2 Post-Increment — Walk Forward Through Memory

```
LD  Rd, X+          ; Rd ← SRAM[X], then X += 1
ST  X+, Rr          ; SRAM[X] ← Rr, then X += 1
```

The increment happens **after** the access. The full 16-bit pointer is incremented, so `0x38FF` correctly becomes `0x3900`. X is ready to point to the next byte. Perfect for forward array traversal:

This is one of the most natural addressing modes on AVR. It lets the memory access and the pointer update happen in the same instruction.

```
; Zero-fill buf (8 bytes)
ldi  XL, lo8(buf)
ldi  XH, hi8(buf)
ldi  r16, 0
ldi  r18, 8
1:
    st   X+, r16        ; write 0, advance X
    dec  r18
    brne 1b
```

5.5.3 Pre-Decrement — Walk Backward Through Memory

```
LD  Rd, -X          ; X -= 1, then Rd ← SRAM[X]
ST  -X, Rr          ; X -= 1, then SRAM[X] ← Rr
```

The decrement happens **before** the access. The full 16-bit pointer is decremented before the memory cycle. To walk an array backwards, point X one past the last element:

Pre-decrement often feels odd the first time you see it, but it matches how stacks and reverse traversals naturally work.

```
; Sum array backwards into r17
ldi  XL, lo8(buf + 8)   ; one past end
ldi  XH, hi8(buf + 8)
clr  r17
ldi  r18, 8
1:
    ld   r16, -X        ; X--, then load
    add  r17, r16
    dec  r18
    brne 1b
```

5.5.4 All Three Pointers, All Three Modes

X, Y, and Z all support `LD`/`ST`, `LD+`/`ST+`, and `LD-`/`ST-`. The only difference is which register pair holds the address:

So the real question is rarely "which instruction family do I need?" More often it is "which pointer register should own this job?"

```
ld  r16, X+     ; load via X, post-increment
ld  r16, Y+     ; load via Y, post-increment
ld  r16, Z+     ; load via Z, post-increment — also used for LPM
```

Avoid using a pointer register as both the pointer and the source or destination register in auto-increment/decrement forms. For example, the AVR instruction manual marks combinations such as `LD r30, Z+` and `ST Z+, r30` as undefined because r30 is part of Z itself.

5.6 Displacement Addressing: LDD and STD

Y and Z support a **constant displacement** added to the pointer without modifying it. X does **not** support displacement.

```
LDD Rd, Y+q         ; Rd ← SRAM[Y + q]   (Y unchanged, q = 0–63)
LDD Rd, Z+q         ; Rd ← SRAM[Z + q]
STD Y+q, Rr         ; SRAM[Y + q] ← Rr
STD Z+q, Rr
```

2 cycles. Displacement q must be a constant 0–63 — it cannot be a register.

Displacement mode is useful because it lets one pointer stay parked at a base address while you reach nearby fields by constant offset.

The 0-63 displacement limit is encoded directly in the 16-bit instruction. If a field is at offset 64 or later, adjust the base pointer first with `ADIW` or use normal pointer arithmetic, then access through `Y` or `Z`.

Primary use: struct member access. Point Y at the base of a struct, then access each member by offset:

```
; Struct layout in SRAM:
;   offset 0: status (1 byte)
;   offset 1: count  (1 byte)
;   offset 2: value  (2 bytes, little-endian)
;   offset 4: flags  (1 byte)

.equ STATUS_OFF, 0
.equ COUNT_OFF,  1
.equ VALUE_OFF,  2
.equ FLAGS_OFF,  4

; Point Y at the struct
ldi  YL, lo8(my_struct)
ldi  YH, hi8(my_struct)

; Read all fields without moving Y
ldd  r16, Y+STATUS_OFF     ; r16 = status
ldd  r17, Y+COUNT_OFF      ; r17 = count
ldd  r18, Y+VALUE_OFF      ; r18 = value low byte
ldd  r19, Y+VALUE_OFF+1    ; r19 = value high byte
ldd  r20, Y+FLAGS_OFF      ; r20 = flags
```

Secondary use: stack frame pointer. The compiler uses Y as the frame pointer, accessing local variables via `LDD r, Y+offset`. Chapter 8 covers this in the context of subroutines.

In other words, `LDD`/`STD` are the AVR equivalent of "base pointer + fixed offset" addressing.

5.7 Cycle and Size Comparison

Mode	Instruction	Cycles	Words	Pointer moves?
Direct	LDS / STS	3 / 2	2	No
Indirect	LD / ST	2	1	No

```
Post-increment      LD+ / ST+       2       1       +1 after
Pre-decrement       LD- / ST-       2       1       -1 before
Displacement        LDD / STD       2       1       No (Y/Z only)
I/O register        IN / OUT        1       1       N/A (addr 0–3F)
```

Cycle counts in this table are the AVRxt base costs for internal SRAM and low I/O style accesses. Accesses that hit NVM regions, including EEPROM and mapped flash, can take additional cycles.

Rule of thumb: - **Named variable, one access:** `LDS`/`STS` - **Named variable, repeated access:** load address into pointer, then `LD`/`ST` - **Array traversal:** `LD+`/`ST+` or `LD-`/`ST-` - **Struct fields:** `LDD`/`STD` with Y as base - **I/O registers ≤ 0x3F:** `IN`/`OUT`

Those rules are worth internalising because they cover most real choices in ordinary firmware.

5.8 Reading Flash: LPM

On all AVR devices, `LPM` reads one byte from the **program bus** (flash) into a register. Z holds the byte address of the flash location to read.

This is the classic AVR way to read constants stored in flash. If you learned AVR on older parts, `LPM` is probably the method you saw first.

```
LPM Rd, Z          ; Rd ← Flash[Z]      (Z unchanged)
LPM Rd, Z+         ; Rd ← Flash[Z], Z++ (post-increment)
LPM                ; r0  ← Flash[Z]     (deprecated form)
```

3 cycles. Z must be the byte address (not word address).

That byte-address detail matters because some other AVR concepts, like jump targets, use word-oriented program addressing instead.

On ATtiny3217, flash is 32 KB, so ordinary `LPM` is enough for the whole program memory. `ELPM` and `RAMPZ` matter on larger AVR devices with more than 64 KB of program memory, but they do not buy anything on this MCU.

```
.section .rodata
lookup:
    .byte 0, 1, 1, 2, 3, 5, 8, 13   ; Fibonacci

.section .text

    ; Load lookup[3] into r16
    ldi  ZL, lo8(lookup + 3)
    ldi  ZH, hi8(lookup + 3)
    lpm  r16, Z                     ; r16 = 2 (lookup[3])
```

Copying a flash table to SRAM with LPM:

```
    ldi  ZL, lo8(lookup)
    ldi  ZH, hi8(lookup)
    ldi  XL, lo8(dest_buf)
    ldi  XH, hi8(dest_buf)
    ldi  r18, 8

copy:
    lpm  r16, Z+            ; load from flash, Z++
```

```
    st   X+,  r16           ; store to SRAM, X++
    dec  r18
    brne copy
```

5.9 Reading Flash: Mapped Access (AVRxt Only)

On AVRxt (ATtiny3217), flash is mapped into the **data address space** at offset `MAPPED_PROGMEM_START` = **0x8000**. You can read it with ordinary `LD` instructions — no LPM needed.

This is one of the nicest quality-of-life improvements on the ATtiny3217. It lets flash reads look much more like normal memory reads.

`LD Rd, Z            ; Rd ← DataMem[Z]`

If Z ≥ 0x8000, the data bus reads from flash. If Z < 0x8000, it reads SRAM or I/O registers. The CPU doesn't care which — the address selects the bus.

That means one instruction family, `LD`, can read from very different physical resources depending on the address you place in the pointer.

The ATtiny3217 data sheet states that the whole 32 KB flash is mapped from `0x8000` and is accessible with normal `LD`/`ST` instructions as well as `LPM`. In ordinary application code, use this mapped view mainly for reads. Flash writes are NVMCTRL-controlled operations, not arbitrary byte stores like SRAM writes.

```
.section .rodata
lookup:
    .byte 0, 1, 1, 2, 3, 5, 8, 13

.section .text

    ; Load lookup[3] into r16 via mapped flash
    ldi  ZL, lo8(lookup + MAPPED_PROGMEM_START + 3)
    ldi  ZH, hi8(lookup + MAPPED_PROGMEM_START + 3)
    ld   r16, Z             ; r16 = 2
```

Copying with mapped LD — same loop as SRAM copy:

```
    ldi  ZL, lo8(lookup + MAPPED_PROGMEM_START)
    ldi  ZH, hi8(lookup + MAPPED_PROGMEM_START)
    ldi  XL, lo8(dest_buf)
    ldi  XH, hi8(dest_buf)
    ldi  r18, 8

copy_mapped:
    ld   r16, Z+            ; load from mapped flash, Z++
    st   X+,  r16           ; store to SRAM, X++
    dec  r18
    brne copy_mapped
```

5.9.1 LPM vs Mapped LD — Which to Use?

	LPM	Mapped LD
Available on	All AVR	AVRxt only (tinyAVR 1-series, megaAVR 0-series)
Cycles	3	2 base, plus NVM wait time if any
Words	1	1
Z value	Flash byte address	Flash byte address + 0x8000
Pointer modes	Z, Z+ only	X, Y, Z — all modes including displacement
ATtiny3217 coverage	Whole 32 KB flash	Whole 32 KB flash at `0x8000-0xFFFF`

On ATtiny3217 with 32 KB flash: **prefer mapped LD** when AVRxt-specific code is acceptable. It uses all pointer modes and reads flash and SRAM with the same instruction family, which is usually the simpler mental model. It is also usually faster than `LPM`, but remember the NVM wait-state caveat above.

`LPM` is still worth knowing because it is portable across AVR families, but for this chip, mapped `LD` is usually the cleaner choice.

5.10 Pointer Arithmetic

5.10.1 Advancing a Pointer by N Bytes

Post-increment advances by 1 per cycle. To skip N bytes, use `ADIW`:

```
adiw ZL, 4          ; Z += 4 (skip 4 bytes)
adiw XL, 16         ; X += 16
```

`ADIW` only handles values 0–63. For larger steps, use `ADD`/`ADC` on the pair directly:

So pointer arithmetic on AVR comes in two flavors: compact/specialized (`ADIW`/`SBIW`) and general/manual (`ADD`/`ADC`).

```
; Z += 200
ldi  r16, 200
add  ZL, r16
ldi  r16, 0
adc  ZH, r16        ; propagate carry into high byte
```

Or use r1 (which the ABI keeps at 0) for the carry:

```
ldi  r16, 200
add  ZL, r16
adc  ZH, r1         ; r1 = 0 by convention — just propagates carry
```

5.10.2 Computing an Array Element Address

```
; Address of array[i]: base + i * element_size (element_size = 1 here)
; r24 = index, Z = array base
add  ZL, r24        ; Z += index (1-byte elements)
adc  ZH, r1         ; carry
ld   r16, Z         ; load array[i]
```

For 2-byte elements:

```
; i * 2 = i << 1
lsl  r24            ; r24 *= 2 (left shift)
rol  r25            ; shift carry into r25 (if index > 127)
add  ZL, r24
adc  ZH, r25
```

This is the same reasoning as in multi-byte arithmetic: build the low byte result first, then propagate carry into the high byte.

5.11 Atomicity on ATtiny3217

Some AVR families implement `XCH`, `LAC`, `LAS`, and `LAT`, but the ATtiny3217's AVRxt core does **not**. If you need interrupt-safe bit updates on GPIO, use the dedicated peripheral registers such as `PORTx_OUTSET`, `PORTx_OUTCLR`, and `PORTx_OUTTGL` instead of a read-modify-write sequence.

This matters because "load-modify-store" is not automatically safe when an ISR or another hardware path can touch the same state concurrently.

```
ldi  r16, (1 << 3)
sts  PORTA_OUTSET, r16      ; set PA3 without reading PORTA_OUT first
sts  PORTA_OUTCLR, r16      ; clear PA3 atomically
sts  PORTA_OUTTGL, r16      ; toggle PA3 atomically
```

For ordinary SRAM bytes shared with an ISR, there is no single instruction on this MCU that performs an atomic arbitrary read-modify-write. Use a critical section (`IN/CLI/OUT SREG`) when the update must not be interrupted.

So the practical split is:

- for GPIO, prefer the hardware set/clear/toggle registers;
- for shared SRAM state, protect the update with interrupt control when needed.

5.12 Full Addressing Mode Reference

Instruction	Addr calc	Pointer after	Cycles	Words
`LDS Rd, k`	k	–	3	2
`STS k, Rr`	k	–	2	2
`LD Rd, X`	X	unchanged	2	1
`LD Rd, X+`	X	X + 1	2	1
`LD Rd, -X`	X - 1	X - 1	2	1
`ST X, Rr`	X	unchanged	2	1
`ST X+, Rr`	X	X + 1	2	1
`ST -X, Rr`	X - 1	X - 1	2	1
`LD Rd, Y`	Y	unchanged	2	1
`LD Rd, Y+`	Y	Y + 1	2	1
`LD Rd, -Y`	Y - 1	Y - 1	2	1

```
LDD  Rd, Y+q        Y + q          unchanged      2       1
ST   Y,  Rr         Y              unchanged      2       1
ST   Y+, Rr         Y              Y + 1          2       1
ST   -Y, Rr         Y - 1          Y - 1          2       1
STD  Y+q, Rr        Y + q          unchanged      2       1

LD   Rd, Z          Z              unchanged      2       1
LD   Rd, Z+         Z              Z + 1          2       1
LD   Rd, -Z         Z - 1          Z - 1          2       1
LDD  Rd, Z+q        Z + q          unchanged      2       1
ST   Z,  Rr         Z              unchanged      2       1
ST   Z+, Rr         Z              Z + 1          2       1
ST   -Z, Rr         Z - 1          Z - 1          2       1
STD  Z+q, Rr        Z + q          unchanged      2       1

LPM  Rd, Z          Flash[Z]       unchanged      3       1
LPM  Rd, Z+         Flash[Z]       Z + 1          3       1

IN   Rd, A          I/O[A] (0–3F)  –              1       1
OUT  A,  Rr         I/O[A] (0–3F)  –              1       1

PUSH Rr             SRAM[SP-1]     SP - 1         2       1
POP  Rd             SRAM[SP]       SP + 1         2       1
```

5.13 Worked Example: Reverse a String In-Place

Reverse an ASCII string stored in SRAM. Classic two-pointer approach.

This example is useful because it combines several addressing modes in one small routine: forward scan with Z+, fixed access through X and Z, then converging pointers for the swap loop.

```
/*
 * str_reverse — reverse a null-terminated string in SRAM in-place.
 * Input:  X = pointer to first byte of string
 * Clobbers: X, Z, r16, r17, r18
 */
str_reverse:
    ; Find the end: walk Z to the null terminator
    movw ZL, XL             ; Z = X (copy pointer)
1:
    ld   r16, Z+            ; load byte, Z++
    tst  r16                ; null?
    brne 1b                 ; no — keep walking

    ; Z is now one past the null. Step back twice: past null, to last char.
    sbiw ZL, 2              ; Z now points to last character

    ; Now swap X[front] and Z[back], moving inward
2:
    cp   XL, ZL             ; has front crossed back? (low bytes)
    cpc  XH, ZH             ; (high bytes)
```

```
    brsh done               ; BRSH: unsigned >=, front >= back → done

    ld   r16, X             ; r16 = front char
    ld   r17, Z             ; r17 = back char
    st   X+, r17            ; store back char at front, advance front
    st   Z,  r16            ; store front char at back
    sbiw ZL, 1              ; retreat back pointer
    rjmp 2b

done:
    ret
```

Trace with "AVR\0" at address 0x3810:

```
Initial: X = 0x3810 ('A'), Z = 0x3812 ('R')

Iter 1: swap 'A' and 'R' → X=0x3811, Z=0x3811
Iter 2: XL=0x11 >= ZL=0x11 → BRSH taken → done

Result in SRAM: "RVA\0"  OK
```

Notice how little arithmetic the routine needs. Most of the real work is being done by the addressing modes themselves.

5.14 Summary

```
LDS/STS    – direct, 16-bit address in instruction, 3/2 cycles, 2 words.
             Use for named variables, single accesses.

LD/ST      – indirect via X, Y, or Z. 2 cycles, 1 word.
  no mode – pointer unchanged
  X+/Y+/Z+ – post-increment (forward traversal)
  -X/-Y/-Z  – pre-decrement  (backward traversal)

LDD/STD    – Y or Z plus constant offset 0–63. Struct member access.
             X does NOT support displacement.

LPM        – flash read via program bus. Z = byte address. 3 cycles.
Mapped LD – AVRxt: LD with Z = flash addr + 0x8000. 2 cycles. Preferred.

ADIW/SBIW – adjust pointer by constant 0–63 in 2 cycles.
ADD+ADC    – adjust pointer by any 16-bit value.

ATtiny3217 data space: I/O 0x0000-0x003F, extended I/O 0x0040-0x0FFF,
EEPROM 0x1400-0x14FF, SRAM 0x3800-0x3FFF, mapped flash 0x8000-0xFFFF.

ATtiny3217 has no XCH/LAC/LAS/LAT. Use PORTx set/clear/toggle registers for
GPIO, and critical sections for shared SRAM updates.
```

5.15 Exercises

1. Write a subroutine `memset(X, val, n)` that fills n bytes starting at address X with value `val` (in r16). Use `ST X+` in the loop.
2. Write `memcpy(X, Z, n)` that copies n bytes from Z to X. Both source and destination are SRAM. Then modify it so the source is flash (use mapped LD for ATtiny3217).
3. Given the struct:

   ```
   offset 0: type   (1 byte)
   offset 1: length (1 byte)
   offset 2: data   (4 bytes)
   ```

   Write code using `LDD`/`STD` with Y to read `type`, increment `length`, and zero out the `data` field — all without moving Y.
4. What is the maximum array size (in elements, 1 byte each) you can access with `LDD Y+q`? What happens if you need element 64? Write code that handles this using `ADIW` and basic `LD`.
5. Assemble `loadstore.S` and run `avr-objdump -s loadstore.elf`. Find the `.rodata` section. Verify the bytes match `1, 2, 4, 8, 16, 32, 64, 128`. Then find the address of `buf_lpm` in the `.bss` section — what address does it get?
6. Why does the `str_reverse` subroutine use `CP`+`CPC` instead of just `CP` to compare X and Z? What would go wrong with only `CP XL, ZL`?

Next: Chapter 6 — Arithmetic and Logic (16-bit and beyond)

Chapter 6

Arithmetic and Logic: 16-bit and Beyond

Chapter 4 introduced 8-bit arithmetic. Real programs quickly outgrow that. Addresses are 16-bit, timers are often 16-bit, counters spill past 255, and scaled sensor values rarely fit in a single byte for long. This chapter shows how AVR handles arithmetic once one byte is no longer enough.

The key idea is simple: AVR does not magically become a 16-bit machine when you want a 16-bit result. You build larger arithmetic explicitly out of 8-bit pieces, and the carry flag is what ties those pieces together.

6.1 Multi-Byte Addition and Subtraction

The AVR has no 16-bit ADD instruction. You build it from two 8-bit instructions using the carry flag as a bridge between bytes.

That sounds more awkward than it is. Once you trust the carry chain, multi-byte arithmetic becomes a repeatable pattern rather than a special trick.

Microchip's instruction summary is useful here because it separates the two classes of arithmetic flags:

Flag	Meaning in this chapter
C	Carry for unsigned addition; borrow for unsigned subtraction
Z	Result is zero
N	Result bit 7 is set for 8-bit ops, or bit 15 for `ADIW`/`SBIW`
V	Two's-complement signed overflow
S	`N xor V`, used by signed branches such as `BRLT`/`BRGE`
H	Half-carry/half-borrow between bits 3 and 4, mainly for BCD-style code

For multi-byte arithmetic, the important flags are C for unsigned carry/borrow and V/S/N for signed interpretation of the final high byte.

6.1.1 The Carry Chain Pattern

Always start with the least-significant byte. Never with the most-significant.

```
; 16-bit add: r25:r24 += r23:r22
add  r24, r22       ; low bytes — may produce carry
adc  r25, r23       ; high bytes — ADC includes carry from ADD
```

```
; 16-bit subtract: r25:r24 -= r23:r22
sub  r24, r22       ; low bytes — may produce borrow (C=1)
sbc  r25, r23       ; high bytes — SBC subtracts borrow
```

The carry flag is the pipeline between bytes. Break the chain and the high byte has the wrong value.

That is why instruction order matters here. Arithmetic is not just about the operands; it is also about preserving the flag state from one byte to the next.

6.1.2 24-bit and 32-bit

The pattern extends to any width:

```
; 32-bit add: r3:r2:r1:r0 += r7:r6:r5:r4
add  r0, r4
adc  r1, r5
adc  r2, r6
adc  r3, r7
```

```
; 32-bit subtract: r3:r2:r1:r0 -= r7:r6:r5:r4
sub  r0, r4
sbc  r1, r5
sbc  r2, r6
sbc  r3, r7
```

After the chain: if C=1, unsigned overflow occurred (result too large for N bytes). If V=1, signed overflow occurred.

So after a multi-byte operation, the flags still matter. The CPU is telling you whether the final N-byte result wrapped or overflowed in the signed sense.

One subtle detail from the AVR instruction manual: `SBC` preserves the previous Z value when its own byte result is zero. That is deliberate. It means a multi-byte subtract or compare leaves Z=1 only if **all** bytes in the chain were zero/equal.

6.1.3 Adding a Small Constant to a 16-bit Register Pair

`ADIW` handles constants 0–63 in 2 cycles, but only on the upper even register pairs:

Low register operand	Full pair
`r24`	`r25:r24`
`r26` / `XL`	`XH:XL`
`r28` / `YL`	`YH:YL`
`r30` / `ZL`	`ZH:ZL`

```
adiw r24, 1         ; r25:r24 += 1  — 2 cycles, 1 word
adiw ZL,  16        ; Z += 16
```

`SBIW` has the same register-pair and immediate limits, but subtracts:

```
sbiw r24, 1         ; r25:r24 -= 1
sbiw ZL,  16        ; Z -= 16
```

`ADIW`/`SBIW` update Z, C, N, V, and S. They do not update H. That differs from byte `ADD`/`ADC`/`SUB`/`SBC`, which do update H.

For constants 64–255, use `SUBI`+`SBCI` with negated values — because there is no `ADDI`:

```
; r25:r24 += 100
subi r24, lo8(-100)     ; subtract -100 = add 100
sbci r25, hi8(-100)     ; propagate carry
```

`SUBI` and `SBCI` only work on registers r16-r31. If your value is in a lower register pair, move it or use a register-register add/subtract sequence.

For constants > 255, load into a register pair and use `ADD`+`ADC`:

```
; r25:r24 += 1000
ldi  r22, lo8(1000)
ldi  r23, hi8(1000)
add  r24, r22
adc  r25, r23
```

This gives you three practical tools for "add a constant":

- `ADIW` when the constant is small and the register pair is supported
- `SUBI`/`SBCI` when the constant fits in one byte but `ADIW` is not enough
- `ADD`/`ADC` when you want the general case

6.1.4 Negating a 16-bit Value

Two's complement negate: invert all bits, add 1.

```
; negate r25:r24
com  r24            ; ~low
com  r25            ; ~high
adiw r24, 1         ; + 1  (or: ldi r16,1 / add r24,r16 / adc r25,r1)
```

Or equivalently using `NEG` + `SBC`:

```
neg  r25            ; high = 0 - high
neg  r24            ; low  = 0 - low, C=1 if low was non-zero
sbc  r25, r1        ; subtract the borrow from high (r1 must be zero)
```

That order looks backwards, but it is intentional: `NEG r24` must run before the final `SBC` so the carry flag tells whether the low-byte negate borrowed from the high byte.

The reason is that it fits naturally into the carry/borrow model the CPU already uses for subtraction.

A correct register-register form that avoids `ADIW` is:

```
com  r24
com  r25
ldi  r16, 1
add  r24, r16
adc  r25, r1        ; r1 must be zero
```

For signed 8-bit values, `NEG` has one important edge case: `0x80` (-128) cannot be represented as +128 in signed 8-bit two's complement, so the result stays `0x80` and V is set.

6.2 Shift and Rotate Instructions

Shifts move bits left or right. The bit shifted out goes into the carry flag. Rotates put the old carry flag back in on the vacated side.

If addition and subtraction are about numeric value, shifts and rotates are about bit position. They are the bridge between arithmetic and bit-level work.

```
LSL Rd — Logical Shift Left
     C ← [7][6][5][4][3][2][1][0] ← 0
     Rd *= 2 (unsigned). Old bit 7 → C.

LSR Rd — Logical Shift Right
     0 → [7][6][5][4][3][2][1][0] → C
     Rd /= 2 (unsigned). Old bit 0 → C. Bit 7 forced to 0.

ASR Rd — Arithmetic Shift Right
    b7 → [7][6][5][4][3][2][1][0] → C
     Rd /= 2 (signed). Bit 7 preserved (sign extension).

ROL Rd — Rotate Left through Carry
     C ← [7][6][5][4][3][2][1][0] ← C(old)
     Like LSL but old C enters at bit 0.

ROR Rd — Rotate Right through Carry
     C(old) → [7][6][5][4][3][2][1][0] → C
     Like LSR but old C enters at bit 7.
```

All shift/rotate instructions: **1 cycle**. The instruction summary lists C, Z, N, and V as directly affected; S follows from `N xor V`. H, T, and I are not affected. `SWAP` is separate: it affects no flags.

That makes them cheap enough to use constantly, especially for scaling by powers of two or for moving carries across byte boundaries.

6.2.1 16-bit Shift Left (Multiply by 2)

```
lsl  r24        ; low byte: bit 7 → C
rol  r25        ; high byte: C → bit 0 (carry from low byte propagates)
```

6.2.2 16-bit Shift Right, Logical (Unsigned Divide by 2)

```
lsr  r25        ; high byte: bit 0 → C (MSB filled with 0)
ror  r24        ; low byte: C → bit 7
```

6.2.3 16-bit Shift Right, Arithmetic (Signed Divide by 2)

```
asr  r25        ; high byte: sign bit preserved, bit 0 → C
ror  r24        ; low byte: C → bit 7
```

6.2.4 Shifting by N Bits

GAS does not have a multi-bit shift instruction (unlike x86's `SHL r, cl`). Repeat for each bit:

```
; r16 <<= 3  (multiply by 8)
lsl r16
lsl r16
lsl r16
```

For large shifts on 8-bit values, `SWAP` is faster when shifting by 4:

```
; r16 <<= 4
swap  r16       ; swap nibbles: bits 7:4 ↔ bits 3:0
andi  r16, 0xF0 ; zero lower nibble (was upper before swap)
```

```
; r16 >>= 4 (logical)
swap  r16
andi  r16, 0x0F ; zero upper nibble
```

This is a good reminder that on AVR, repeated simple instructions often replace one "fancier" instruction from a larger ISA.

6.2.5 Extracting a Bit Field

Example: extract bits 5:3 of r16 into bits 2:0 of r17:

```
mov   r17, r16
andi  r17, 0b00111000   ; mask bits 5:3
lsr   r17               ; shift right 3 times
lsr   r17
lsr   r17               ; r17 = bits 5:3 in positions 2:0
```

6.3 SWAP — Swap Nibbles

```
SWAP Rd         ; bits 7:4 ↔ bits 3:0
```

1 cycle. Does not affect any flags. Pure bit rearrangement.

```
ldi  r16, 0xAB
swap r16        ; r16 = 0xBA
```

Primary uses: - Fast ×16 or ÷16 on values that fit in a nibble - BCD digit manipulation - Building two-nibble values from separate sources:

```
; Pack two 4-bit values (r16 low nibble, r17 low nibble) into one byte
andi r16, 0x0F      ; keep only low nibble of r16
swap r17            ; move r17's low nibble to high position
andi r17, 0xF0      ; keep only high nibble of r17
or   r16, r17       ; r16 = r17_nibble : r16_nibble
```

`SWAP` is a good example of an instruction that looks niche until you start doing display code, BCD work, or byte packing. Then it becomes surprisingly useful.

6.4 MUL — Unsigned 8×8 Multiply

```
MUL Rd, Rr      ; r1:r0 ← Rd × Rr  (unsigned)
```

2 cycles. Result always in **r1:r0** regardless of operands.

ATtiny3217 has the hardware multiplier block, so this is a real two-cycle instruction, not a library loop. The multiplier takes two 8-bit inputs and produces a 16-bit result; the integer multiply instructions differ only in how they interpret the inputs.

The fixed result location is the most important thing to remember about `MUL`. The product does not stay near the source operands. It always lands in `r1:r0`.

```
ldi  r16, 12
ldi  r17, 25
mul  r16, r17       ; r1:r0 = 300 = 0x012C
movw r24, r0        ; copy result to a usable register pair
clr  r1             ; RESTORE r1 = 0 — this is mandatory (see below)
```

Why must you clear r1?

By the GCC AVR ABI, r1 must always be 0 when calling C functions or when C code calls assembly. `MUL` sets r1 to the high byte of the product. If you forget `CLR r1`, the next multiply or any code that assumes r1=0 (like `ADC Rr, r1` for carry propagation) will produce wrong results.

Develop the habit: every `MUL`/`MULS`/`MULSU` is immediately followed by `CLR r1` unless you are certain no C code is involved and you consume r1 before any carry-chain arithmetic.

Even if you are not mixing with C today, it is still a good discipline. The "r1 should be zero" convention is useful across the whole codebase.

6.4.1 Self-Multiply (Square)

```
; r25:r24 = r16 * r16  (r16 squared)
mul  r16, r16
movw r24, r0
clr  r1
```

`MUL` allows both operands to be the same register.

6.4.2 16-bit Result Scaling

A common embedded pattern: scale a 16-bit value by an 8-bit fraction using the high byte of the product as a "multiply then shift right 8" operation:

This is a classic fixed-point trick: keep the multiply, throw away the low byte, and treat the high byte as the scaled result.

```
; Approximate: result = (val * fraction) >> 8
; val in r16, fraction in r17 (0x00=0%, 0xFF≈100%, 0x80=50%)
mul  r16, r17
```

```
mov  r16, r1        ; take only the high byte → effectively >> 8
clr  r1
```

6.5 MULS — Signed 8×8 Multiply

```
MULS Rd, Rr     ; r1:r0 ← (signed)Rd × (signed)Rr
```

Both operands treated as signed (-128..127). Result is signed 16-bit. **r16–r31 only** for both operands.

```
ldi  r16, 0xF6      ; -10 in two's complement
ldi  r17, 5
muls r16, r17       ; r1:r0 = -50 = 0xFFCE
movw r24, r0
clr  r1
```

```
ldi  r16, 0xF6      ; -10
ldi  r17, 0xFB      ; -5
muls r16, r17       ; r1:r0 = 50 = 0x0032
movw r24, r0
clr  r1
```

6.6 MULSU — Signed × Unsigned Multiply

```
MULSU Rd, Rr    ; r1:r0 ← (signed)Rd × (unsigned)Rr
```

Rd (r16–r23) is signed; Rr (r16–r23) is unsigned. Result is signed 16-bit.

Used in fixed-point arithmetic where one operand is a coefficient (signed scaling factor) and the other is a measurement (unsigned raw value):

That signed/unsigned mix shows up often in calibration and correction code.

```
; Scale a raw ADC reading (0–255, unsigned) by a signed calibration
; coefficient (-128..127), result in r25:r24.
; Example: reading=200 (0xC8), coeff=-3 (0xFD)
ldi  r16, 0xFD      ; signed coefficient -3
ldi  r17, 200       ; unsigned reading
mulsu r16, r17      ; r1:r0 = -600 = 0xFDA8
movw  r24, r0
clr   r1
```

6.7 Fractional Multiply: FMUL, FMULS, FMULSU

The ATtiny3217 multiplier also supports fractional multiply instructions:

```
FMUL   Rd, Rr      ; unsigned fractional
FMULS  Rd, Rr      ; signed fractional
FMULSU Rd, Rr      ; signed × unsigned fractional
```

These still compute an 8×8 product into `r1:r0` in 2 cycles, but they shift the 16-bit product left by one bit before storing it. Microchip describes the typical input format as 1.7 fixed point, where the binary point is between bits 6 and 7. The left shift puts the high byte back into the same fractional scale that DSP-style code usually wants.

Operand limits:

Instruction	Rd range	Rr range
`FMUL`	r16-r23	r16-r23
`FMULS`	r16-r23	r16-r23
`FMULSU`	r16-r23	r16-r23

Like the integer multiply family, fractional multiply writes `r1:r0` and affects only Z and C. Clear `r1` after consuming the product if the surrounding code follows the GCC AVR ABI.

Example: signed Q1.7-style scale. `0x40` represents +0.5, so multiplying 0.5 by 0.5 gives 0.25, represented by `0x20` in the high byte:

```
ldi   r16, 0x40     ; +0.5 in signed 1.7 fixed point
ldi   r17, 0x40     ; +0.5
fmuls r16, r17      ; r1:r0 = (0x40 * 0x40) << 1 = 0x2000
mov   r18, r1       ; high byte = 0x20, which represents +0.25
clr  r1
```

Fractional multiply is powerful, but be precise about your fixed-point format. For signed fractional edge cases such as `0x80 * 0x80`, Microchip notes that the shifted result can overflow two's-complement interpretation and software must handle that if it matters.

6.8 Multiply Summary

```
Instruction    Rd range     Rr range     Signed?         Result
─────────────────────────────────────────────────────────────────────
MUL   Rd, Rr   r0–r31       r0–r31       both unsigned   r1:r0 unsigned
MULS  Rd, Rr   r16–r31      r16–r31      both signed     r1:r0 signed
MULSU Rd, Rr   r16–r23      r16–r23      Rd signed,      r1:r0 signed
                                          Rr unsigned
FMUL*          r16–r23      r16–r23      fractional      (product << 1) in r1:r0

All: 2 cycles. Result always in r1:r0. Always CLR r1 after.
All multiply instructions affect only C and Z.
```

6.9 Division — Software Algorithm

AVR has no divide instruction. Division is implemented with a shift-subtract loop: the same long division algorithm you learned in school, but in binary.

So division is not something the hardware does for you in one step. You are trading simplicity of concept for cost in cycles.

6.9.1 Unsigned 8-bit Division

`dividend / divisor → quotient, remainder`

The algorithm processes one bit at a time from MSB to LSB:

```
For each bit (MSB to LSB):
  1. Shift MSB of dividend into remainder LSB  (remainder <<= 1, r[0] = d_msb)
  2. If remainder >= divisor:
       remainder -= divisor
       set current quotient bit = 1
     Else:
       quotient bit = 0
```

If that seems abstract, think of it this way: each iteration asks, “Can the current remainder afford one more divisor?” If yes, subtract and record a 1 in the quotient. If not, record a 0 and continue.

Assembly implementation (clean restoring version):

```
/*
 * div8u — unsigned 8-bit division
 *   Input:  r16 = dividend, r17 = divisor
 *   Output: r18 = quotient, r19 = remainder
 *   Clobbers: r20
 *   Cycles: roughly 9-10 cycles per bit, plus call/return if used as a subroutine
 */
div8u_clean:
    clr  r18                 ; quotient = 0
    clr  r19                 ; remainder = 0
    ldi  r20, 8              ; 8 bits
1:
    lsl  r16                 ; MSB of dividend → C
    rol  r19                 ; remainder = (r19 << 1) | C
    lsl  r18                 ; quotient <<= 1, make room for next bit
    cp   r19, r17
    brlo 2f                  ; remainder < divisor: quotient bit stays 0
    sub  r19, r17            ; remainder -= divisor
    ori  r18, 1              ; quotient bit = 1
2:
    dec  r20
    brne 1b
    ret
```

Trace: 7 / 3:

```
bit 7: remainder=0→0, 0<3 → quotient bit 0 → r18=0b00000000
bit 6: remainder=0→0, 0<3 → quotient bit 0 → r18=0b00000000
bit 5: remainder=0→0, 0<3 → quotient bit 0 → r18=0b00000000
bit 4: remainder=0→0, 0<3 → quotient bit 0 → r18=0b00000000
bit 3: remainder=0→0, 0<3 → quotient bit 0 → r18=0b00000000
bit 2: remainder=0→1, 1<3 → quotient bit 0 → r18=0b00000000
bit 1: remainder=1→3, 3>=3 → sub→0, quotient bit 1 → r18=0b00000001
bit 0: remainder=0→1, 1<3 → quotient bit 0 → r18=0b00000010

quotient = 0b00000010 = 2 OK
remainder = 1 OK  (7 = 2×3 + 1)
```

Tracing a few divisions by hand is worth the time. After that, the loop stops looking magical and starts looking mechanical.

6.9.2 Unsigned 16-bit Division

Extend the pattern to 16-bit dividend, 8-bit divisor (common for scaled sensor values):

This is the same algorithm again, just stretched over more bits.

```
/*
 * div16u8 — divide 16-bit by 8-bit
 *   Input:  r25:r24 = dividend, r16 = divisor
 *   Output: r25:r24 = quotient, r18 = remainder
 *   Clobbers: r19, r20, r21
 */
div16u8:
    clr  r18                ; remainder = 0
    ldi  r19, 16            ; 16 bits
    clr  r20                ; quotient low
    clr  r21                ; quotient high
1:
    lsl  r24                ; shift MSB of dividend → C
    rol  r25
    rol  r18                ; remainder <<= 1, | C
    lsl  r20                ; quotient <<= 1
    rol  r21
    cp   r18, r16
    brlo 2f
    sub  r18, r16
    ori  r20, 1             ; quotient bit = 1
2:
    dec  r19
    brne 1b
    movw r24, r20           ; quotient → r25:r24
    ret
```

Both routines assume the divisor is nonzero. AVR has no hardware divide-by-zero trap; if zero is possible, test it before entering the loop and choose an explicit policy such as saturating the quotient, returning zero, or branching to an error handler.

6.9.3 Power-of-2 Division — Just Shift

If the divisor is a compile-time power of 2, skip the loop entirely:

```
; Unsigned: r16 /= 8  (= 2^3)
lsr r16
lsr r16
lsr r16

; Signed: r16 /= 4  (rounds toward -∞, which is floor division)
asr r16
asr r16

; 16-bit unsigned: r25:r24 /= 4
lsr r25
```

```
ror r24
lsr r25
ror r24
```

Always use shifts for power-of-2 division. The loop costs ~72 cycles; three LSRs cost 3.

That is not a small optimization. It is the difference between "expensive software division" and "almost free."

6.10 Worked Example: 8×8 Unsigned Multiply to 16-bit Result

Full annotated example with verification by objdump.

```
/*
 * Input:  r24 = a (0–255), r22 = b (0–255)
 * Output: r25:r24 = a * b  (0–65025)
 * ABI-compliant: returns 16-bit result in r25:r24.
 */
mul8_to_16:
    mul   r24, r22      ; r1:r0 = a * b
    movw  r24, r0       ; r25:r24 = result
    clr   r1            ; restore ABI r1=0
    ret
```

Disassembly of this function (`avr-objdump -d`):

```
<mul8_to_16>:
   0:   86 9f           mul     r24, r22
   2:   c0 01           movw    r24, r0
   4:   11 24           eor     r1, r1      ; CLR alias
   6:   08 95           ret
```

`MUL` is a single 16-bit word (2 bytes). The two-cycle cost comes from the hardware multiplier computing the 16-bit product in parallel with the next fetch. `MOVW` is another single word. The entire routine is 5 instructions, 8 bytes, 6 cycles.

This is a good example of why knowing the ISA matters. A tiny hand-written routine can be both obvious and efficient once you know the dedicated instruction exists.

6.11 Signed Arithmetic Pitfalls

6.11.1 Two Mistakes with Signed Division

Mistake 1: Using `LSR` (logical) instead of `ASR` (arithmetic) to divide a signed value.

```
ldi  r16, 0xFE      ; = -2 signed
lsr  r16            ; r16 = 0x7F = 127  ← WRONG (unsigned right shift)
asr  r16            ; r16 = 0xFF = -1   ← CORRECT (-2 / 2 = -1)
```

Mistake 2: Using `BRLO`/`BRSH` (unsigned) instead of `BRLT`/`BRGE` (signed) for signed comparisons. Covered in Chapter 4 but worth repeating whenever you work with signed arithmetic.

The CPU does not know your intent. A byte is just a bit pattern until your instruction choice tells the machine whether to treat it as signed or unsigned.

6.11.2 Overflow Detection

After a signed operation, V=1 means overflow. The result is wrong as a signed value:

```
ldi  r16, 100       ; +100
ldi  r17, 100       ; +100
add  r16, r17       ; result = 200, but 200 > 127 → V=1 (signed overflow)
brvs overflow_handler
```

After unsigned operations, C=1 means overflow (result > 255 / 65535).

That signed/unsigned split is one of the most common sources of subtle bugs in embedded arithmetic.

6.12 Useful Idioms

6.12.1 Test if a value is zero without clobbering flags from the previous op

```
tst  r16            ; Z=1 if r16=0, N=1 if negative — no flags from ADD etc.
```

Strictly speaking, TST does set flags. The real point is that it lets you ask "is this register zero or negative right now?" without changing the register value itself.

6.12.2 Conditional absolute value

```
; |r16|
sbrc r16, 7         ; skip if bit 7 clear (positive)
neg  r16            ; negate if negative
```

6.12.3 Saturating add (clamp at 255 instead of wrapping)

```
add  r16, r17       ; add
brcc 1f             ; no carry: result fits in 8 bits
ser  r16            ; carry: clamp to 0xFF
1:
```

This is a very common embedded pattern because sensor and UI code often prefers clamping over wraparound.

6.12.4 Saturating subtract (clamp at 0 instead of wrapping)

```
sub  r16, r17
brcc 1f             ; no borrow (C=0): result >= 0
clr  r16            ; borrow: clamp to 0
1:
```

6.12.5 Sign extension: 8-bit signed → 16-bit signed

```
; r16 (signed) → r17:r16 (signed 16-bit)
clr   r17
sbrc  r16, 7        ; if bit 7 clear: r17 = 0 (positive, done)
ser   r17           ; bit 7 set: r17 = 0xFF (negative, sign extend)
; r17:r16 is now the correct signed 16-bit representation
```

6.13 Summary

```
Multi-byte arithmetic:
  Always low byte first. Use ADC/SBC to chain carry between bytes.
  ADIW/SBIW: ±0-63 to 16-bit pair, 2 cycles.
  SUBI+SBCI: add any constant (negate the immediate).

Shifts:
  LSL/LSR — logical (unsigned), 0 inserted.
  ASR     — arithmetic (signed), sign bit preserved.
  ROL/ROR  — rotate through carry — chains shifts across bytes.
  SWAP    — nibble swap, 1 cycle, use for ×16 / ÷16.

Multiply:
  MUL   — unsigned × unsigned → r1:r0. Always CLR r1 after.
  MULS  — signed × signed → r1:r0. r16–r31 only.
  MULSU — signed × unsigned → r1:r0. r16–r23 only.
  FMUL* — fractional multiply, r16–r23 only, product shifted left once.
  All: 2 cycles. Result in r1:r0.

Division: no hardware DIV. Use shift-subtract loop (~72 cycles for 8-bit).
Power-of-2 divisors: use LSR/ASR instead — O(log₂ n) cycles.

Signed pitfalls:
  Use ASR not LSR for signed right shift.
  Use BRLT/BRGE not BRLO/BRSH for signed branch.
  V flag = signed overflow, C flag = unsigned overflow.
```

6.14 Exercises

1. Write a 16-bit multiply: `r25:r24 = r25:r24 × r23:r22` (both inputs unsigned). You cannot use `MUL` for the full 16×16 product directly (it only handles 8×8). Decompose into four 8×8 multiplies and accumulate. Hint: `(a_hi×256 + a_lo) × (b_hi×256 + b_lo)`.

2. Extend `div16u8` to handle divide-by-zero explicitly. Choose a policy (for example: quotient = 0xFFFF, remainder = dividend low byte, or branch to an error handler) and document it.

3. Write a function that counts the number of 1-bits in r16 (popcount). Use `LSL` + `ADC r17, r1` pattern: after each LSL, C holds the shifted-out bit; `ADC r17, r1` adds C to the accumulator (r1=0).

4. Implement saturating 16-bit addition: `r25:r24 + r23:r22`, clamped to 0xFFFF if overflow. The 16-bit carry flag is in C after the second `ADC`.
5. Verify the `div8u_clean` routine:
   - Trace 100 / 7 by hand (expected: quotient=14, remainder=2).
   - Assemble and simulate: use `avr-objdump -d` to confirm the instruction sequence, then trace the bit loop manually.
6. Without `MUL`, compute `r16 × 10` using only shifts and adds. (10 = 8 + 2 = (1«3) + (1«1).)

Next: Chapter 7 — Branches and Control Flow

Chapter 7

Branches and Control Flow

Every non-trivial program has decisions and loops. In assembly, all of them reduce to one fundamental act: change the program counter, or deliberately let it continue to the next instruction. This chapter covers the main ways AVR does that — unconditional jumps, conditional branches, and single-instruction skips — and shows the assembly patterns behind decisions, loops, and multi-way dispatch.

The key mental shift is that high-level control flow is just structured PC manipulation.

7.1 Unconditional Jumps

7.1.1 RJMP — Relative Jump

```
RJMP k      ; PC ← PC + k + 1   (k = signed offset, −2048..+2047 words)
```

2 cycles. 1 word. Range: **±2047 words** (±4094 bytes) from the instruction.

```
rjmp loop       ; jump to label "loop" — GAS computes the offset
rjmp .          ; infinite loop (jump to current address, offset = -1)
```

`RJMP` covers most local control-flow transfers — nearly 4K instruction words (just under 8 KB of code bytes). Use it by default.

That “use it by default” rule is practical: `RJMP` is smaller and faster than `JMP`, and most control-flow targets are close enough to fit.

7.1.2 JMP — Absolute Jump

```
JMP k      ; PC ← k   (k = 22-bit program word address)
```

3 cycles. 2 words (32-bit instruction). Range: entire 64K-word (128KB) flash.

```
jmp main            ; jump anywhere — no range limit
jmp reset_handler
```

Use `JMP` only when the target is out of `RJMP` range, or in the vector table where you need a guaranteed 2-word slot. For everything else, `RJMP` is smaller and faster.

So the usual decision is simple:

- nearby target: `RJMP`

- far target or fixed-size vector entry: `JMP`

7.1.3 IJMP — Indirect Jump via Z

```
IJMP        ; PC ← Z   (Z holds word address of target)
```

2 cycles. 1 word. Z contains the **word address** (byte address ÷ 2) of the destination. Use `pm(label)` to convert a label to its word address:

```
ldi ZL, lo8(pm(target))
ldi ZH, hi8(pm(target))
ijmp                        ; jump to target
```

Primary use: **jump tables**. Covered in Section 7.7.

Indirect jumps are less common in beginner code, but they are the step that takes you from simple branch chains to table-driven dispatch.

7.2 Conditional Branches — Mechanics

All conditional branches work identically:

1. Test one or more bits in SREG.
2. If the condition is true: `PC += offset + 1` (branch taken, 2 cycles).
3. If the condition is false: `PC += 1` (not taken, 1 cycle).

Range: **±63 words** (±126 bytes). If the target is farther, invert the condition and use `JMP`/`RJMP`:

```
; Branch to far_target if r16 == 0 — but far_target > 63 words away:
tst   r16
brne  skip          ; if NOT zero, skip the jump (inverted condition)
jmp   far_target    ; now unconditionally jump to far target
skip:
```

This pattern — invert condition, skip over jump — is the standard way to handle far conditional transfers.

7.2.1 Exact Branch Range

The branch operand is a 7-bit signed word offset: `-64..+63`. Because the CPU adds the offset to `PC + 1`, the reachable destination range is:

```
PC - 63  through  PC + 64    ; destination instruction word addresses
```

That is why datasheets often describe conditional branches as reaching `PC - 63` to `PC + 64`, while programmers usually summarize the range as “about ±64 words.” Both descriptions refer to the same encoding.

All AVR branch and jump offsets are in **instruction words**, not bytes. This matters when you compare a disassembly address, a linker error, and the instruction manual:

- `RJMP` offset: signed word offset, `-2048..+2047`
- conditional branch offset: signed word offset, `-64..+63`
- flash byte addresses: twice the word address on classic AVR notation

GAS hides most of this with labels, but the distinction matters for jump tables, manual address arithmetic, and datasheet tables that list program addresses in words.

7.2.2 Signed vs Unsigned Conditions

The CPU does not know whether a byte is signed or unsigned. You choose the branch mnemonic that interprets the flags the way your data should be read.

After `CP Rd, Rr` or `CPI Rd, K`, the comparison is internally `Rd - Rr` or `Rd - K`.

Relation	Unsigned branch	Signed branch
<	BRLO / BRCS	BRLT
>=	BRSH / BRCC	BRGE
==	BREQ	BREQ
!=	BRNE	BRNE

Example: `0x80` is 128 unsigned but -128 signed.

```
ldi r16, 0x80
cpi r16, 1
brlo unsigned_less     ; false: 128 is not < 1
brlt signed_less       ; true: -128 is < 1
```

This is the most common comparison bug in hand-written assembly: using BRLO/BRSH for signed values or BRLT/BRGE for unsigned values.

7.3 Complete Branch Instruction Table

Mnemonic	Alias for	Condition	Flags tested	Typical use after
BREQ	BRBS 1,k	Z = 1	Z	CP, CPI, TST, SUB
BRNE	BRBC 1,k	Z = 0	Z	CP, CPI, DEC, INC
BRCS	BRBS 0,k	C = 1	C	ADD, SUB, LSL, LSR
BRCC	BRBC 0,k	C = 0	C	ADD, SUB
BRLO	BRCS	C = 1	C	CP, CPI (unsigned <)
BRSH	BRCC	C = 0	C	CP, CPI (unsigned ≥)
BRLT	BRBS 4,k	S = 1 (N⊕V)	N, V	CP, CPI (signed <)
BRGE	BRBC 4,k	S = 0 (N⊕V)	N, V	CP, CPI (signed ≥)
BRMI	BRBS 2,k	N = 1	N	ADD, SUB (signed neg)
BRPL	BRBC 2,k	N = 0	N	ADD, SUB (signed pos)
BRVS	BRBS 3,k	V = 1	V	ADD, SUB (ovf detect)
BRVC	BRBC 3,k	V = 0	V	ADD, SUB
BRIE	BRBS 7,k	I = 1	I	SEI/CLI context
BRID	BRBC 7,k	I = 0	I	SEI/CLI context
BRHS	BRBS 5,k	H = 1	H	BCD arithmetic

BRHC	BRBC 5,k	H = 0	H	BCD arithmetic
BRTS	BRBS 6,k	T = 1	T	BST/BLD sequences
BRTC	BRBC 6,k	T = 0	T	BST/BLD sequences

`BRBS b, k` (Branch if Bit in SREG Set) and `BRBC b, k` (Branch if Bit in SREG Clear) are the generic forms. Every named branch is an alias for one of these two — GAS assembles `BREQ` as `BRBS 1, k` (bit 1 = Z flag).

In practice you almost always write the named forms, because `BREQ` expresses intent better than "branch if SREG bit 1 is set."

7.4 Skip Instructions

Skips are single-instruction conditional skips — they bypass exactly the next instruction if a condition is met. No label needed. They are the most compact way to write small conditionals.

If branches are for "go somewhere else," skips are for "ignore the next thing if this condition holds."

7.4.1 CPSE — Compare, Skip if Equal

```
CPSE Rd, Rr     ; if Rd == Rr: skip next instruction
```

1 cycle (no skip), 2 cycles (skip 1-word), 3 cycles (skip 2-word).

That timing detail matters because the skipped instruction still occupies space in the stream even though it does not execute.

```
cpse r16, r17
rjmp not_equal  ; skipped if r16 == r17 — falls through to equal path
; equal path here
```

`CPSE` does **not** set SREG. It compares only for the purpose of deciding whether to skip the next instruction. If later code needs Z, C, N, V, or S from the comparison, use `CP`/`CPI` instead.

7.4.2 SBRC — Skip if Bit in Register is Clear

```
SBRC Rr, b      ; if bit b of Rr == 0: skip next instruction
```

Tests bit b (0–7) of register Rr (r0–r31). No flags affected.

```
; Process bit 0 of flags register r16
sbrc r16, 0         ; skip if bit 0 clear
rcall handle_flag0  ; only called if bit 0 was set
```

7.4.3 SBRS — Skip if Bit in Register is Set

```
SBRS Rr, b      ; if bit b of Rr == 1: skip next instruction
```

```
sbrs r16, 7         ; skip if bit 7 set (value is negative signed)
rjmp positive_path  ; only taken if bit 7 was clear
; negative path falls through here
```

7.4.4 SBIC — Skip if Bit in I/O Register is Clear

```
SBIC A, b       ; if bit b of I/O[A] == 0: skip   (A = 0x00–0x1F only)
```

Tests a bit in an I/O register **in a single cycle without loading into a register first**. The address range is limited to 0x00–0x1F (the lower half of the IN/OUT space).

```
; Wait until a flag bit in GPIOR0 is set by an ISR
wait_for_flag:
    sbis GPIOR0, 0      ; skip if bit 0 of GPIOR0 is set
    rjmp wait_for_flag  ; not set: loop
; bit 0 now set — proceed
```

7.4.5 SBIS — Skip if Bit in I/O Register is Set

```
SBIS A, b       ; if bit b of I/O[A] == 1: skip   (A = 0x00–0x1F only)
```

```
; Check if a GPIOR flag is clear
sbic GPIOR0, 2      ; skip if bit 2 clear
rcall clear_handler ; only if bit 2 was set
```

7.4.6 SBIC/SBIS Address Limit on ATtiny3217

On ATtiny3217, `SBIC/SBIS` reach I/O addresses **0x00–0x1F** only. The regular PORT peripheral registers (`PORTA_IN`, `PORTB_IN`, etc.) are at 0x0400+ — far beyond this limit. To test a regular PORT pin register, you must load it first:

```
; Read PA3 state into r16, test bit 3
lds  r16, PORTA_IN      ; load port input register
sbrc r16, 3             ; skip if PA3 is low
rcall pa3_high_handler
```

The registers that ARE reachable with `SBIC/SBIS` on ATtiny3217:

I/O Addr	Register	Useful bits
0x02	`VPORTA_IN`	Fast pin reads for Port A
0x06	`VPORTB_IN`	Fast pin reads for Port B
0x0A	`VPORTC_IN`	Fast pin reads for Port C
0x1C	`GPIOR0`	User-defined flags (bit 0–7)
0x1D	`GPIOR1`	User-defined flags
0x1E	`GPIOR2`	User-defined flags
0x1F	`GPIOR3`	User-defined flags

`GPIOR0–3` are key scratch flag registers in low I/O space: readable in 1 cycle with `SBIC/SBIS`, no load required. `VPORTx_IN` is the other major use: it gives you fast bit tests on real pins without touching the slower `PORTx_IN` registers in extended I/O space.

```
; Wait while PA3 is low using the virtual port input register
wait_pa3_high:
    sbis VPORTA_IN, 3       ; VPORTA_IN is I/O address 0x02
    rjmp wait_pa3_high
```

The regular `PORTx` registers are still the full peripheral interface. The virtual ports map only the common `DIR`, `OUT`, `IN`, and `INTFLAGS` registers into low I/O space so bit instructions can reach them.

7.4.7 Skip Over 2-Word Instructions Carefully

Skip instructions can skip either a 1-word instruction or a 2-word instruction. On AVRxt, the timing is:

Case	Cycles
Condition false, no skip	1
Skip a 1-word instruction	2
Skip a 2-word instruction	3

This is legal:

```
sbrc r16, 0
jmp  far_handler      ; 2-word instruction, skipped as a whole if bit 0 clear
```

But it is easy to misread during review because the skipped instruction takes two words in flash. For small conditionals, prefer skipping a 1-word `RJMP` or `RCALL` when the target range allows it.

7.4.8 Skip vs Branch — When to Use Which

Situation	Use
Skip 1 instruction on a bit condition	`SBRC/SBRS/SBIC/SBIS`
Skip 1 instruction on equality	`CPSE`
Branch to a label	`BREQ`, `BRNE`, etc.
More than 1 instruction in the conditional body	Branch

Skips are best for guard clauses and flag-polling loops. Branches are best for bodies with multiple instructions.

That is the simplest rule for choosing between them: one-instruction body, consider a skip; larger body, use a branch.

7.5 Structured Control Flow Patterns

7.5.1 Two-way decision

```
; r16 = a, r17 = b, result → r18
cp   r16, r17          ; a - b
brsh use_a             ; unsigned a >= b
mov  r18, r17
rjmp done
use_a:
mov  r18, r16
done:
```

Pattern: compare/test -> branch to one path -> execute the other path -> `rjmp` over the first path -> join at a common label.

That "invert the condition" step is one of the most common assembly habits. You often branch around the code you do not want rather than branch directly into every desired path.

7.5.2 Multi-byte comparisons

For 16-bit values, compare the low byte first with CP, then the high byte with CPC so the carry/borrow flows into the high-byte comparison.

```
; Compare unsigned a = r17:r16 with b = r19:r18
cp   r16, r18          ; low byte
cpc  r17, r19          ; high byte with carry from low-byte compare
brlo a_below_b         ; unsigned a < b
brsh a_same_or_above_b ; unsigned a >= b
```

For equality, the same CP/CPC sequence leaves Z set only if both bytes are equal:

```
cp   r16, r18
cpc  r17, r19
breq equal16
```

For signed 16-bit compares, use the same CP/CPC sequence but finish with BRLT or BRGE. The signed condition comes from the final high-byte comparison, which is the byte containing the sign bit.

7.5.3 Guard branch

```
tst  r16
brne continue           ; NOT zero: skip the return
ret                     ; zero: return early
continue:
```

For this pattern, a branch is clearer than forcing CPSE into it:

```
tst  r16
breq early_return
rjmp continue
early_return:
ret
continue:
```

This is a good example of a broader rule: the shortest control-flow sequence is not always the clearest one.

7.5.4 Top-tested loop

```
top_test:
    tst  r16
    breq top_done       ; condition false: exit
    dec  r16            ; body
    rjmp top_test
top_done:
```

Pre-test — loop may execute zero times. Costs one extra RJMP per iteration.

That is why a top-tested loop is often not the most efficient assembly shape.

7.5.5 Bottom-tested loop (preferred in assembly)

```
do_body:
    dec  r16            ; body
    brne do_body        ; condition: not zero → repeat
```

This is the most efficient loop form. No `RJMP` back — the branch IS the loop-back. One instruction of overhead per iteration instead of two.

This is one of the most useful structural transformations in assembly: if the loop body must run at least once, put the test at the bottom.

Use this shape when the body runs at least once:

```
; Count-down loop: r18 iterations, guaranteed >= 1
ldi  r18, N
loop:
    ; body
    dec  r18
    brne loop           ; one branch instruction, no rjmp
```

7.5.6 Counted loop

Count **down** rather than up — comparison against 0 is free (BRNE after DEC):

```
; Count-down (preferred):
ldi  r18, N
1:
    ; body
    dec  r18
    brne 1b

; Count-up (necessary when index used inside body):
clr  r18
1:
    ; body (r18 = current index)
    inc  r18
    cpi  r18, N         ; costs one extra CPI per iteration
    brne 1b
```

That is why countdown loops appear everywhere in AVR code. Zero is the cheapest comparison target on the machine.

7.5.7 Nested loops

```
ldi r19, 8              ; outer counter
outer:
    ldi r18, 64         ; inner counter — reload every outer iteration
    inner:
        ; body
        dec  r18
        brne inner
    dec  r19
    brne outer
```

Critical: reload the inner counter at the top of the outer loop body, **before** the inner loop starts. A common mistake is loading it once before the outer loop — it reaches zero after the first outer iteration and never loops again.

This bug is common because the loop still looks nested at a glance. The reload placement is what actually makes it nested.

7.6 Worked Example: Sum an Array

Sum 8 unsigned bytes from flash, store 16-bit result in SRAM.

```
/* sum_array — sums ARRAY_LEN bytes from flash
 * Uses LPM: Z is the flash byte address of test_array.
 * Result in r25:r24
 */

.section .progmem.data
test_array:
    .byte 10, 20, 30, 40, 50, 60, 70, 80    ; sum = 360 = 0x0168
.equ ARRAY_LEN, 8

.section .text

sum_array:
    ldi  ZL, lo8(test_array)
    ldi  ZH, hi8(test_array)
    clr  r24                ; sum_lo = 0
    clr  r25                ; sum_hi = 0
    ldi  r18, ARRAY_LEN     ; counter
loop:
    lpm  r16, Z+            ; byte = flash[Z], Z++
    add  r24, r16           ; sum_lo += byte
    adc  r25, r1            ; sum_hi += carry (r1 = 0)
    dec  r18
    brne loop               ; bottom-tested loop: no rjmp needed

    ; r25:r24 = 360 = 0x0168
    ret
```

`LPM` reads through the program-memory bus. On AVRxt devices such as the ATtiny3217, flash is also mapped into the data address space starting at `0x8000`, but `LPM` is the portable and explicit way to say "read from flash" in a small example like this.

Step-by-step trace:

Iter	r18	Z points to	r16	r25:r24
1	7	[20]	10	0x000A
2	6	[30]	20	0x001E
3	5	[40]	30	0x003C
4	4	[50]	40	0x0064
5	3	[60]	50	0x0096
6	2	[70]	60	0x00D2
7	1	[80]	70	0x0118

Iter	r18	Z points to	r16	r25:r24
8	0	—	80	0x0168 OK

Why `ADC r25, r1` and not `ADC r25, r17`? `ADD r24, r16` sets C if the low-byte sum overflows, and `ADC r25, r1` immediately consumes that carry. Using r1 (=0) adds only the carry bit, correctly propagating any overflow from the low byte without needing a dedicated zero register. The later `DEC r18` updates N, Z, and V for the loop test, but it does not affect C.

This is a good example of how flag discipline and dataflow interact. The right instruction is not just about values in registers; it is about which flag bit you are intentionally consuming next.

7.7 Jump Tables

For multi-way branches, a jump table is faster than a chain of comparisons. In the compact form below, each table entry is an executable `RJMP` instruction. `IJMP` jumps into the selected table entry, and that `RJMP` then jumps to the handler.

Conceptually, a jump table turns a small command number into an indexed branch.

```
; r16 = command index (0, 1, or 2)

; Bounds check first:
cpi  r16, 3             ; 3 = number of table entries
brsh jtable_default     ; >= 3: out of range

; Build Z = pm(jtable) + r16
; Each RJMP in the table is 1 word, so index directly.
ldi  ZL, lo8(pm(jtable))
ldi  ZH, hi8(pm(jtable))
add  ZL, r16            ; offset by selector index
adc  ZH, r1             ; carry
ijmp                    ; jump to selected RJMP entry in the table

jtable:
    rjmp cmd0           ; selector 0
    rjmp cmd1           ; selector 1
    rjmp cmd2           ; selector 2

jtable_default:
    rcall cmd_default
    rjmp  jtable_done

cmd0: rcall handler0  ; rjmp table_done implicit via fall-through pattern
      rjmp jtable_done
cmd1: rcall handler1
      rjmp jtable_done
cmd2: rcall handler2
      rjmp jtable_done
jtable_done:
```

How it works:

`pm(jtable)` gives the **word address** of `jtable`. Adding the selector index (in words) moves Z to the corresponding `RJMP` instruction. `IJMP` loads the program counter with that word address, so execution continues at the selected table entry. The table entry then executes and jumps to the actual handler. Each entry is 1 word (`RJMP` = 1 word), so the index scales correctly.

That two-step jump looks strange the first time you see it, but it buys you compact entries and simple indexing.

Cycle cost: bounds check (2) + LDI×2 (2) + ADD+ADC (2) + IJMP (2) + RJMP from table (2) = **10 cycles**. A chain of `CPI`/`BREQ` for 3 entries costs up to 3 × (1+2) = 9 cycles on the slowest path — comparable. Jump tables win decisively with more entries.

So jump tables are not automatically better for tiny dispatches. They become attractive once the table is large enough that linear comparison chains stop being cheap.

7.7.1 Address Tables with LPM

Another jump-table style stores raw handler addresses as data and loads the selected entry with `LPM`. That is useful when you want the table to be data rather than executable branch instructions, but it costs more instructions because each address is two bytes and must be loaded into Z before `IJMP`.

```
; r16 = selector index, handlers are within 64K words
lsl  r16                    ; byte offset = index * 2
ldi  ZL, lo8(addr_table)    ; LPM uses a flash byte address
ldi  ZH, hi8(addr_table)
add  ZL, r16
adc  ZH, r1

lpm  r30, Z+                ; low byte of handler word address
lpm  r31, Z                 ; high byte of handler word address
ijmp

addr_table:
    .word pm(cmd0)
    .word pm(cmd1)
    .word pm(cmd2)
```

Use the executable-`RJMP` table when handlers are within `RJMP` range and code size matters. Use an address table when you need data-like table handling or when the table is generated by other tools.

7.7.2 Large Jump Tables with JMP Entries

If handlers are far from the table (`RJMP` out of range), use `JMP` (2 words) instead. Then each table entry is 2 words, so multiply the index by 2:

```
; Index × 2 for 2-word entries
lsl  r16                ; r16 *= 2
ldi  ZL, lo8(pm(jtable))
ldi  ZH, hi8(pm(jtable))
add  ZL, r16
adc  ZH, r1
ijmp
```

```
jtable:
    jmp cmd0            ; 2 words each
    jmp cmd1
    jmp cmd2
```

On devices with more than 64K words of program memory, indirect jumps may also need `EIJMP` and the `EIND` register. ATtiny3217 has 32 KB of flash, so plain `IJMP` is enough for all program-memory destinations on this device.

7.8 Polling Loops

A polling loop spins until a condition becomes true — set by hardware or an ISR.

Polling is simple and sometimes appropriate, but it is also the most CPU-hungry waiting strategy because the core keeps executing the test loop continuously.

```
; Wait for USART0 receive complete (bit 7 of USART0_STATUS = RXCIF)
; Cannot use SBIS: address too high. Must use LDS + SBRS.
wait_rxc:
    lds  r16, USART0_STATUS     ; load USART status
    sbrs r16, 7                 ; skip if RXCIF bit set
    rjmp wait_rxc               ; not set: loop
    lds  r16, USART0_RXDATAL    ; read the received byte
```

For low I/O registers (GPIOR), `SBIS` eliminates the load:

```
; ISR sets bit 0 of GPIOR0 when data ready
wait_ready:
    sbis GPIOR0, 0      ; 1 cycle: skip if bit 0 set
    rjmp wait_ready     ; 2 cycles: loop back
; 3 cycles per iteration vs 5 cycles with LDS+SBRS
```

That difference is exactly why low-I/O flag registers are so useful on AVR.

For real pins on ATtiny3217, prefer `VPORTx_IN` when the loop only needs a single pin state:

```
; Wait for PB2 high through the virtual port
wait_pb2:
    sbis VPORTB_IN, 2       ; VPORTB_IN is I/O address 0x06
    rjmp wait_pb2
```

Use regular `PORTx.IN` with `LDS` when you need the full peripheral register view, or when the code is shared with a device that lacks the same virtual-port layout.

7.9 Branch Optimisation Notes

7.9.1 Bottom-tested saves one instruction per loop

```
; top-tested: 3 instructions per iteration (body + dec + brne + rjmp)
; bottom-tested: 2 instructions per iteration (body + dec + brne)
```

For a loop that runs 1000 times, that's 1000 saved cycles.

Over one or two iterations this does not matter. Over hot loops, it matters a lot.

7.9.2 Count down, not up

`DEC r18` + `BRNE` is **two instructions with no extra comparison**. Counting up needs `INC` + `CPI N` + `BRNE` — three instructions.

This is one of those patterns that looks like a small optimization until you write enough firmware to see it everywhere.

7.9.3 Use SBRC/SBRS for single-bit tests

Instead of:

```
andi r16, (1 << 3)
brne bit3_set
```

Write:

```
sbrs r16, 3
rjmp bit3_clear
; bit3_set path falls through
```

Saves one instruction and doesn't destroy r16.

Preserving the register value is often as important as saving the instruction.

7.9.4 Branch delay slots do not exist on AVR

Unlike ARM or MIPS, the AVR has no branch delay slot. The instruction after a branch is NOT executed if the branch is taken. No surprises.

That makes AVR control flow pleasantly literal: if you branch away, the next sequential instruction does not secretly run first.

7.10 Summary

```
Unconditional:
  RJMP  – ±2047 words, 2 cycles, 1 word. Default choice.
  JMP   – full range, 3 cycles, 2 words. For far targets or vector table.
  IJMP  – Z = word address, 2 cycles. For jump tables.

Conditional branches: ±63 words, 2 cycles taken / 1 not taken.
  Unsigned: BRLO (<), BRSH (≥), BREQ (=), BRNE (≠)
  Signed:   BRLT (<), BRGE (≥), BRMI (neg), BRPL (pos)
  Far target: invert condition, skip over JMP.

Skip instructions (1–3 cycles, no label needed):
  CPSE  Rd, Rr       – skip if Rd == Rr
  SBRC  Rr, b        – skip if bit b of Rr clear
  SBRS  Rr, b        – skip if bit b of Rr set
  SBIC  A, b         – skip if bit b of I/O[A] clear  (A ≤ 0x1F)
  SBIS  A, b         – skip if bit b of I/O[A] set    (A ≤ 0x1F)
```

```
Loop patterns:
  bottom-tested: most efficient — branch is the loop-back (no extra RJMP)
  counted: count down with DEC+BRNE — no CPI needed
  nested: reload inner counter at start of each outer iteration

Jump tables: IJMP with Z = pm(table) + index. O(1) dispatch for table entries.

Low-I/O targets such as `VPORTx_IN` and `GPIOR0–3` are reachable with
`SBIC`/`SBIS`. `GPIORx` make especially good fast flag bytes between ISR and
main loop.
```

7.11 Exercises

1. Rewrite the array-sum example to sum **signed** 8-bit values into a signed 16-bit result. What changes? (Hint: `ADC r25, r1` is still correct for unsigned carry, but how do you handle sign extension of each negative byte before adding?)
2. Write a function `find_min(X, n)` that finds the minimum byte value in an array of `n` bytes at SRAM address X. Return the minimum in r24. Use a bottom-tested loop.
3. Extend the jump table example to 8 entries. What is the cycle cost of the dispatch compared to a chain of 8 `CPI`/`BREQ` pairs (slowest path)?
4. Write a zero-terminated byte-string length routine: given X pointing to the first byte in SRAM, return the length (not counting the zero terminator) in r24. Use a bottom-tested loop with `LD X+` and `TST`.
5. Implement a byte-search routine: scan n bytes at X for value r22, return pointer to first match in X (or 0 if not found). Use `CPSE` to detect a match.
6. The polling loop `sbis GPIOR0, 0 / rjmp wait` burns CPU cycles continuously. Name one hardware peripheral on ATtiny3217 that would let you sleep the CPU instead of polling, waking only when the condition is true. (Covered in Chapter 10 — but think about it now.)

Next: Chapter 8 — Subroutines and the Stack

Chapter 8

Subroutines and the Stack

Subroutines let you package a sequence of instructions once and reuse it from many places. The stack is what makes that possible: it records the return address, holds temporary saved registers, and gives each active call its own scratch space when registers are not enough.

This chapter stays at the assembly level. The goal is to understand exactly what `RCALL`, `CALL`, `RET`, `PUSH`, `POP`, and stack frames do on ATtiny3217.

8.1 CALL and RCALL

```
RCALL k     ; push return address, PC <- PC + k + 1   (-2048..+2047 words)
CALL  k     ; push return address, PC <- k             (absolute target)
```

Both instructions push the return address, then branch to the target. The return address is the program-memory word address of the instruction after the call.

On ATtiny3217, the Program Counter is 14 bits wide for 32 KB of flash, so the stored return address is **2 bytes**. Larger AVR devices with wider program counters can use larger return addresses, but this chapter assumes the ATtiny3217.

Instruction	Words	Cycles	Range
`RCALL k`	1	3	relative, -2048..+2047 words
`CALL k`	2	4	absolute program address

Use `RCALL` for nearby subroutines. Use `CALL` only when the target is outside the relative range or when a fixed 2-word instruction is required.

```
rcall delay_tick       ; compact local call
call  reset_handler    ; absolute call
```

The return address is pushed as a **word pointer**, not as a flash byte address. For example, if the next instruction is at byte address `0x0006`, the stored return word address is `0x0003`.

8.1.1 Return Address Layout

The ATtiny3217 data sheet specifies that the least significant byte of the return address is pushed first and ends up at the higher SRAM address. The stack grows downward, so after a call the stack looks like this:

```
Before RCALL, SP = 0x3FFF

0x3FFF  [free]
0x3FFE  [free]

After RCALL target, return word address = 0x1234

0x3FFF  0x34       ; return low byte, higher address
0x3FFE  0x12       ; return high byte
0x3FFD  [free]     ; SP now points here
```

You normally let `RET` handle this, but the byte order matters when debugging a corrupted stack in memory.

8.1.2 ICALL

```
ICALL       ; push return address, PC <- Z
```

`ICALL` calls the word address held in Z (`r31:r30`). It is the subroutine version of `IJMP`.

```
ldi  ZL, lo8(pm(handler))
ldi  ZH, hi8(pm(handler))
icall
```

Use it for dispatch through a table or for a selected handler address. On ATtiny3217, plain `ICALL` is enough for all program-memory destinations. Devices with more than 64K words of program memory may need `EICALL` and the `EIND` register.

8.2 RET and RETI

```
RET         ; pop return address into PC
RETI        ; pop return address into PC, then set SREG.I
```

`RET` returns from a normal subroutine. `RETI` returns from an interrupt handler and re-enables global interrupts by setting the I bit in `SREG`.

Instruction	Cycles on AVRxt with 16-bit PC	Use
`RET`	4	normal subroutine return
`RETI`	4	interrupt return

Do not use `RETI` for ordinary subroutines. Do not use `RET` at the end of an interrupt handler unless you intentionally want interrupts to remain globally disabled.

Every call path must return with the stack pointer at the same position it had on subroutine entry. If a subroutine pushes three bytes and pops only two, `RET` will fetch one byte of ordinary data as part of the return address.

8.3 PUSH and POP

```
PUSH Rr     ; SRAM[SP] <- Rr, then SP decreases by 1
POP  Rd     ; SP increases by 1, then Rd <- SRAM[SP]
```

On AVRxt, `PUSH` and `POP` are each 2 cycles and 1 word. The stack grows from higher SRAM addresses toward lower SRAM addresses.

```
push r16
push r17
; use r16 and r17
pop  r17        ; last pushed, first popped
pop  r16
```

Example starting with `SP = 0x3FFF`:

```
push r16:
  SP = 0x3FFE
  SRAM[0x3FFF] = r16

push r17:
  SP = 0x3FFD
  SRAM[0x3FFE] = r17
  SRAM[0x3FFF] = r16

pop r17:
  r17 = SRAM[0x3FFE]
  SP = 0x3FFE
```

The practical rule is simple: pop in the exact reverse order of pushes.

8.4 Stack Pointer Facts on ATtiny3217

The stack pointer is the 16-bit `CPU.SP` register, exposed as `SPL` and `SPH` in I/O space. The ATtiny3217 SRAM range is:

```
SRAM start: 0x3800
SRAM end:   0x3FFF
Size:       2 KB
```

After reset, the stack pointer is automatically set to the highest internal SRAM address. In standalone examples this book still initializes it explicitly:

```
ldi r16, lo8(RAMEND)
ldi r17, hi8(RAMEND)
out SPH, r17
out SPL, r16
```

If you move the stack yourself, set it before any subroutine call executes and before interrupts are enabled. A valid custom stack pointer must point inside SRAM and leave enough room above the lowest possible stack address.

8.4.1 Updating SP Safely

When writing a 16-bit stack pointer value manually, write `SPH` before `SPL`:

```
out SPH, r29
out SPL, r28
```

Microchip documents a hardware protection detail: writing SPL automatically disables interrupts for up to four instructions, or until the next I/O-memory write, whichever comes first. Writing high first and low second keeps the observable update coherent during that protected window.

8.5 Register Ownership

A reusable subroutine needs a register contract. This book uses the following assembly convention unless a routine documents otherwise:

Register	Meaning in this book's examples
r0	Scratch, often used by MUL. Not preserved.
r1	Zero register. Keep as 0 except during MUL sequences.
r2-r17	Preserved by the subroutine if used.
r18-r27	Scratch for the subroutine.
r28:r29 (Y)	Preserved if used; preferred frame pointer.
r30:r31 (Z)	Scratch pointer; used by LPM, ICALL, IJMP, tables.
SREG	Not preserved by ordinary routines unless documented.
SP	Must be balanced on every return path.

That gives two useful habits:

- short-lived temporary values go in scratch registers
- values that must survive a nested call should be pushed or kept in preserved registers saved by the callee

Example: a subroutine that uses r14 and r15 must save them:

```
worker:
    push r14
    push r15

    ; body may use r14/r15

    pop  r15
    pop  r14
    ret
```

Example: a caller that needs r20 after a nested call must save it itself:

```
push r20
rcall helper          ; helper may use scratch registers
pop  r20
```

8.5.1 The r1 = 0 Rule

AVR multiply instructions store their result in r1:r0. That means r1 may become non-zero after MUL, MULS, or MULSU.

Many AVR idioms use r1 as a zero register:

```
add r24, r16
adc r25, r1           ; add carry only
```

If r1 is not zero, this sequence is wrong. Clear r1 after every multiply sequence unless the routine immediately consumes and overwrites it before any code can observe it.

```
mul  r16, r17         ; r1:r0 = product
movw r24, r0
clr  r1               ; restore zero register
```

8.6 Prologue and Epilogue Patterns

A prologue is the entry sequence that saves state and allocates stack space. An epilogue restores that state and returns.

8.6.1 Minimal Subroutine

```
small_worker:
    ; use only scratch registers
    ret
```

If a routine only uses scratch registers and has no nested state to preserve, it needs no pushes.

8.6.2 Saving Preserved Registers

```
accumulate:
    push r12
    push r13

    ; body uses r12:r13

    pop  r13
    pop  r12
    ret
```

The order is part of correctness. Push low/high or high/low as you prefer, but the pops must be the exact reverse.

8.6.3 Stack Frame with Y

Use a stack frame when a routine needs per-call storage that cannot live in registers. Y (r29:r28) is the natural frame pointer because LDD and STD support displacement addressing from Y.

This prologue allocates four local bytes:

```
with_frame:
    push r28
    push r29
    in   r28, SPL       ; Y = SP, currently below saved r29
    in   r29, SPH
```

```
    sbiw r28, 4         ; reserve 4 bytes below saved Y
    out  SPH, r29
    out  SPL, r28
```

After this prologue, Y equals the new SP. The local bytes are addressed as `Y+1` through `Y+4`:

```
Higher addresses

Y+8   return low byte
Y+7   return high byte
Y+6   saved r28
Y+5   saved r29
Y+4   local byte 4
Y+3   local byte 3
Y+2   local byte 2
Y+1   local byte 1
Y+0   current SP value, below the allocated local area

Lower addresses
```

The saved Y bytes are above the local area. After `sbiw`, the active local bytes are normally accessed through `Y+1..Y+N`. Keep the layout written down when using stack frames. Off-by-one errors here corrupt saved registers or return addresses.

Access example:

```
ldi  r16, 10
std  Y+1, r16
ldi  r16, 20
std  Y+2, r16
ldd  r17, Y+1
```

Epilogue:

```
    adiw r28, 4         ; Y points back at saved r29
    out  SPH, r29
    out  SPL, r28
    pop  r29
    pop  r28
    ret
```

8.6.4 When a Frame Is Worth It

Situation	Pattern
no saved state	no prologue
one or two preserved registers	`push`/`pop` only
several per-call temporary bytes	Y frame
recursion	stack storage for every value needed after the nested call
interrupt handler	save SREG and every register the handler modifies

8.7 Stack Depth Analysis

Stack overflow on a small AVR usually has no exception or fault signal. The stack simply overwrites SRAM data. You need to estimate the worst case.

For one active call:

```
stack bytes = return address bytes + pushed registers + local allocation
```

For ATtiny3217:

```
return address = 2 bytes
SRAM           = 0x3800..0x3FFF
```

Examples:

```
simple_add:
  2 return bytes + 0 pushes + 0 locals = 2 bytes

with_frame:
  2 return bytes + 2 saved Y bytes + 4 locals = 8 bytes

factorial recursive frame:
  2 return bytes + 1 saved n byte = 3 bytes
```

Peak stack use is the deepest active chain, not the total number of calls over time. If a main routine calls `alpha`, `alpha` calls `beta`, and `beta` calls `gamma`, add the frame sizes for all three at the deepest point.

Inspect SRAM allocation with:

```
avr-nm -n subroutines.elf
avr-size -A subroutines.elf
```

Then reserve a safety margin for interrupts. If an interrupt can occur at the deepest subroutine point, add the interrupt handler's pushes and return address to the worst-case stack budget.

8.8 Worked Example: Recursive Factorial

Factorial is a compact example because every recursive level needs to remember one byte: the current `n`.

```
0! = 1
1! = 1
n! = n * (n - 1)!
```

This routine uses:

```
Input:   r24 = n
Output:  r25:r24 = n!
Limit:   n <= 8 for a 16-bit result
```

`8! = 40320 = 0x9D80`, which fits in 16 bits. `9! = 362880`, which does not.

```
factorial:
    cpi   r24, 2
    brlo  .Lbase            ; 0! and 1! return 1
```

```
    push  r24               ; save n for after the recursive call
    dec   r24
    rcall factorial         ; r25:r24 = (n - 1)!

    pop   r16               ; r16 = original n
    rcall mul8x16           ; r25:r24 = r16 * r25:r24, low 16 bits
    ret

.Lbase:
    clr   r25
    ldi   r24, 1
    ret
```

8.8.1 Stack Trace for factorial(4)

This trace shows only the saved `n` bytes. Each `RCALL` also pushes a 2-byte return address.

```
factorial(4):
  push 4
  factorial(3):
    push 3
    factorial(2):
      push 2
      factorial(1):
        return 1
      pop 2, compute 2 * 1 = 2
    pop 3, compute 3 * 2 = 6
  pop 4, compute 4 * 6 = 24
```

Peak active calls for `factorial(4)`:

```
factorial(4), factorial(3), factorial(2), factorial(1)
4 return addresses = 4 * 2 = 8 bytes
saved n values     = 3 * 1 = 3 bytes
mul8x16 call       = 2 bytes while multiply helper is active
```

At the deepest recursive point before unwinding, the saved-`n` cost is 3 bytes. During each multiply helper call, one additional return address is active.

8.8.2 8 x 16 Multiply Helper

```
/*
 * mul8x16 - 8-bit x 16-bit -> low 16 bits
 * Input:   r16 = 8-bit factor
 *          r25:r24 = 16-bit factor
 * Output:  r25:r24 = low 16 bits of product
 * Clobbers r0, r1, r22, r23
 */
mul8x16:
    mul   r16, r24          ; low factor byte
    movw  r22, r0           ; r23:r22 = low partial
    mul   r16, r25          ; high factor byte
    add   r23, r0           ; add shifted contribution
    clr   r1
```

```
    movw  r24, r22
    ret
```

The helper intentionally truncates the product to 16 bits. That is fine for the documented factorial range, but it would be wrong for a routine promising a full 24-bit result.

8.9 Tail Jumps

A tail jump is a jump to another subroutine as the final action. It avoids creating another return address on the stack.

```
; extra return layer:
wrapper:
    rcall target
    ret

; tail jump:
wrapper:
    rjmp target          ; target's RET returns to wrapper's caller
```

Savings:

```
RCALL + RET avoided = 7 cycles
temporary return address avoided = 2 stack bytes on ATtiny3217
```

Use a tail jump only when the current routine has already restored any saved registers and returned SP to its entry value. If there is cleanup left to do, use a normal call and return.

8.10 Routine Template

Use this as a checklist, not as mandatory boilerplate:

```
.global routine_name
routine_name:
    ; Save preserved registers used by this routine.
    ; push r14
    ; push r15

    ; Optional Y frame.
    ; push r28
    ; push r29
    ; in   r28, SPL
    ; in   r29, SPH
    ; sbiw r28, N
    ; out  SPH, r29
    ; out  SPL, r28

    ; Body.
    ; Put result in the documented output registers.

    ; Optional Y frame cleanup.
```

```
    ; adiw r28, N
    ; out  SPH, r29
    ; out  SPL, r28
    ; pop  r29
    ; pop  r28

    ; Restore preserved registers in reverse order.
    ; pop  r15
    ; pop  r14
    ret
```

For every routine, document:

- input registers
- output registers
- clobbered registers
- preserved registers used internally
- maximum stack use
- numerical limits, if any

8.11 Summary

```
RCALL k  - call relative target, 3 cycles, 1 word, +/-2048-word offset.
CALL k   - call absolute target, 4 cycles, 2 words.
ICALL    - call target in Z, 3 cycles.
RET      - pop return address, 4 cycles.
RETI     - RET plus set SREG.I; interrupt exit only.

ATtiny3217 return address:
  2 bytes, stored as a program-memory word pointer.
  Low byte is pushed first and resides at the higher SRAM address.

Stack:
  SRAM range: 0x3800..0x3FFF.
  Grows downward.
  PUSH stores at SP, then decreases SP by 1.
  POP increases SP by 1, then loads from SP.
  CALL/RCALL/ICALL decrease SP by 2.
  RET/RETI increase SP by 2.

SP updates:
  CPU.SP is SPL/SPH.
  Write SPH before SPL for manual 16-bit updates.
  Writing SPL masks interrupts for up to four instructions or until the next
  I/O-memory write.

Register discipline:
  r1 is the zero register; clear it after multiply instructions.
  Push/pop preserved registers in strict reverse order.
  SP must be balanced on every return path.

Stack usage:
```

```
return address bytes + pushed registers + local allocation.
Add interrupt-handler stack use to the worst-case budget when interrupts can
fire at the deepest point.
```

8.12 Exercises

1. Write `sum_range`: input `r24 = start`, `r22 = end`; output `r25:r24` = sum of all integers from start through end inclusive. Verify 1 through 10 gives 55.
2. For `with_frame`, draw the stack after `push r28`, after `push r29`, after `sbiw r28, 4`, and after each `std Y+q` store. Assume SP starts at `0x3FFF`.
3. Calculate peak stack use for `factorial(8)`, including recursive return addresses, saved `n` bytes, and the active `mul8x16` call during unwinding.
4. Rewrite `factorial` as a counted loop. Compare stack use against the recursive version.
5. Write a byte-swap routine: input `X` points to byte A and `Z` points to byte B. Swap the two SRAM bytes using one scratch register pair and no stack local storage.
6. Add a stack guard byte at the low end of an allowed stack region. Write a small check routine that branches to `stack_error` if the guard byte has changed.

Next: Chapter 9 - GPIO

Chapter 9

GPIO

GPIO (General-Purpose Input/Output) is the mechanism that connects the CPU to the physical world: sensors, LEDs, switches, relays, and anything else wired to a pin. This chapter covers the four dedicated GPIO instructions — `IN`, `OUT`, `SBI`, `CBI` — together with `SBIC`/`SBIS` for pin-state testing, and builds up to a fully debounced button-controlled LED.

The ATtiny3217 uses a **dual-register model** (VPORT + PORT) that differs from the classic DDRx/PORTx/PINx model found on ATmega parts. Both models are covered here; the worked example targets ATtiny3217.

That difference matters because a lot of AVR examples online assume the older ATmega naming. If you only memorize the old names, modern tinyAVR code will look stranger than it really is.

9.1 GPIO Fundamentals

Every GPIO pin has three properties the firmware controls:

```
Direction  0 = input (pin floats / follows external signal)
           1 = output (pin is driven high or low by the CPU)

Output     0 = drive low  (when direction = output)
value      1 = drive high (when direction = output)

           When direction = input:
           0 = no pull-up (pin floats if nothing drives it)
           1 = weak pull-up to VCC (prevents floating)

Input      Read-only: reflects actual voltage on pin
```

On all AVR devices, GPIO is controlled through I/O registers. The exact register names and addresses differ between classic ATmega and the newer tinyAVR 1-series, but the concepts are identical.

So before worrying about the register map, keep the simple mental model in mind: configure direction, choose driven value or pull-up behavior, then read the pin state back when needed.

9.2 Classic DDRx / PORTx / PINx (ATmega-Style)

Most AVR examples online target **ATmega328P** (used on the Arduino Uno). That device uses this three-register model per port:

```
DDRx   — Data Direction Register.  Bit n: 0 = input, 1 = output.
PORTx  — Output Data / Pull-Up.    Bit n: driven value (output) or
                                    pull-up enable (input).
PINx   — Input Data (read-only).   Bit n: actual pin state.
```

On ATmega328P these are in the low I/O space (addresses 0x00–0x1F), so `IN`, `OUT`, `SBI`, `CBI`, `SBIC`, and `SBIS` all work directly:

```
/* ATmega328P example — NOT ATtiny3217 */

/* Configure PB5 (Arduino LED) as output */
sbi  DDRB,  5          ; DDRB bit 5 = 1 (output)

/* Drive LED on */
sbi  PORTB, 5          ; PORTB bit 5 = 1 (high)

/* Drive LED off */
cbi  PORTB, 5          ; PORTB bit 5 = 0 (low)

/* Read button on PD2 (input, pull-up enabled) */
cbi  DDRD,  2          ; DDRD bit 2 = 0 (input)
sbi  PORTD, 2          ; enable pull-up (input + PORTx bit = 1)
in   r16,   PIND       ; r16 = all 8 bits of PIND
sbrc r16,   2          ; skip if bit 2 = 0 (button pressed)
rjmp not_pressed
```

ATtiny3217 does not have DDRx/PORTx/PINx registers. It uses the PORT peripheral described in the next section. If you see online examples that reference `DDRB` or `PORTC`, they are targeting classic ATmega devices — not the ATtiny3217 used in this book.

That is one of the easiest ways to get misled by otherwise-correct AVR code: it may be valid for the wrong AVR family.

9.3 ATtiny3217 Port Architecture

The ATtiny3217 exposes each physical port through **two register sets**:

9.3.1 VPORT — Virtual Port (Fast Path)

VPORT registers are in the low I/O address space (0x00–0x0B). Because they are below address 0x20, all six GPIO instructions work on them: `IN`, `OUT`, `SBI`, `CBI`, `SBIC`, `SBIS`.

Register	I/O addr	Memory addr	Width	Access
VPORTA_DIR	0x00	0x0000	8-bit	R/W
VPORTA_OUT	0x01	0x0001	8-bit	R/W
VPORTA_IN	0x02	0x0002	8-bit	R
VPORTA_INTFLAGS	0x03	0x0003	8-bit	R/W(W1C)
VPORTB_DIR	0x04	0x0004	8-bit	R/W

VPORTB_OUT	0x05	0x0005	8-bit	R/W
VPORTB_IN	0x06	0x0006	8-bit	R
VPORTB_INTFLAGS	0x07	0x0007	8-bit	R/W(W1C)
VPORTC_DIR	0x08	0x0008	8-bit	R/W
VPORTC_OUT	0x09	0x0009	8-bit	R/W
VPORTC_IN	0x0A	0x000A	8-bit	R
VPORTC_INTFLAGS	0x0B	0x000B	8-bit	R/W(W1C)

W1C = Write 1 to Clear (write 1 to a flag bit to clear it)

> **Note on tinyAVR 1-series addressing:** On classic ATmega, there is a 0x20 offset between I/O addresses and memory addresses (I/O 0x00 = memory 0x0020). On ATtiny3217, this offset does not exist — I/O address N = memory address N. `_SFR_MEM8(0x0000)` and I/O address 0x00 are the same location.

VPORT registers are **aliases** of the full PORT registers. Writing to `VPORTA_DIR` is identical to writing to `PORTA_DIR`.

Microchip documents the VPORT mapping as four regular PORT registers:

Regular PORT register	Virtual PORT register
PORTx.DIR	VPORTx.DIR
PORTx.OUT	VPORTx.OUT
PORTx.IN	VPORTx.IN
PORTx.INTFLAGS	VPORTx.INTFLAGS

Only those high-traffic registers are mirrored in VPORT. Pin configuration still lives in the full `PORTx.PINnCTRL` registers, so pull-ups, input inversion, and interrupt-sense setup require `LDS`/`STS`.

The big idea is that VPORT gives you the fast instruction-set view of a port, while the full PORT block gives you the richer configuration interface.

9.3.2 PORT — Full Port Registers (Configuration and Atomic Operations)

The full PORT peripheral sits in the extended memory range (above 0x3F) and cannot be accessed with `IN`/`OUT`/`SBI`/`CBI`. Use `LDS`/`STS`.

```
PORTA base: 0x0400
PORTB base: 0x0420
PORTC base: 0x0440
```

Offset	Register	Description
+0x00	DIR	Direction (0=in, 1=out) — full byte read/write
+0x01	DIRSET	Atomic direction set (write 1 to set bits)
+0x02	DIRCLR	Atomic direction clear (write 1 to clear bits)
+0x03	DIRTGL	Atomic direction toggle
+0x04	OUT	Output value — full byte read/write
+0x05	OUTSET	Atomic output set (write 1 to set bits)
+0x06	OUTCLR	Atomic output clear (write 1 to clear bits)
+0x07	OUTTGL	Atomic output toggle (write 1 to toggle bits)
+0x08	IN	Input (read-only: actual pin voltage)
+0x09	INTFLAGS	Interrupt flags — write 1 to clear a flag
+0x0A	Reserved	Reserved on ATtiny3217

```
+0x0B   Reserved
+0x0C   Reserved
+0x0D   Reserved
+0x0E   Reserved
+0x0F   Reserved
+0x10   PIN0CTRL      Pin 0: ISC[2:0] + PULLUPEN + INVEN
+0x11   PIN1CTRL      Pin 1 configuration
...
+0x17   PIN7CTRL      Pin 7 configuration
```

The **SET/CLR/TGL** variants perform atomic read-modify-write in hardware: a single `STS PORTA_OUTSET, r16` can set individual bits without disabling interrupts. Use these for multi-bit changes where atomicity matters (e.g., if an ISR also touches the same port).

This is one of the nicest parts of the newer AVR GPIO design. Instead of forcing software to read a whole port byte, modify one bit, and write the whole byte back, the peripheral can do the bit update internally.

The **PINnCTRL** registers at offsets 0x10–0x17 hold: - `ISC[2:0]` (bits 2:0) — interrupt sense (covered in Chapter 10) - `PULLUPEN` (bit 3) — enables the internal pull-up resistor - `INVEN` (bit 7) — inverts the pin logic

After reset, Microchip specifies all PORT outputs as tri-stated and the digital input buffers enabled. The `PINnCTRL` reset value is `0x00`, which means no inversion, pull-up disabled, and `ISC=0x0` (`INTDISABLE`: interrupt disabled, input buffer enabled).

Writing 1 to `OUTTGL` toggles the corresponding `OUT` bit. Microchip also documents that writing 1 to bit `n` of `PORTx.IN` or `VPORTx.IN` toggles the matching `OUT` bit, but this book uses `OUTTGL` because it makes the intent obvious and keeps input reads separate from output writes.

9.4 IN — Read I/O Register

```
IN Rd, A
   Operands: Rd ∈ {R0..R31},  A ∈ {0..63}  (I/O address)
   Operation: Rd ← I/O(A)
   Flags:     None
   Cycles:    1
   Encoding:  1011 0AAd dddd AAAA
```

`IN` reads one byte from an I/O register into a general-purpose register. On ATtiny3217, VPORT registers sit at I/O addresses 0x00–0x0B and are within this range.

Think of `IN` as "take a snapshot of this register so the CPU can inspect it."

```
in   r16, VPORTA_IN        ; r16 = current state of PORTA pins (8 bits)
in   r17, VPORTB_OUT       ; r17 = current output values of PORTB
in   r18, SREG             ; r18 = Status Register (a critical use of IN)
```

One common idiom: read SREG before a critical section, write it back after.

This shows up here because GPIO work and interrupt control often meet in the same small critical sections.

```
in   r24, SREG             ; save interrupt enable state
cli                         ; disable interrupts (Chapter 10)
```

```
; ... critical section ...
out  SREG, r24            ; restore interrupt enable state exactly
```

9.5 OUT — Write I/O Register

```
OUT A, Rr
   Operands: A ∈ {0..63},  Rr ∈ {R0..R31}
   Operation: I/O(A) ← Rr
   Flags:     None
   Cycles:    1
   Encoding:  1011 1AAr rrrr AAAA
```

`OUT` writes one byte from a register to an I/O register.

Where `IN` captures a register value, `OUT` pushes a whole register value back into the I/O space in one step.

```
out  VPORTA_OUT, r16      ; drive all 8 PORTA output pins at once
out  VPORTB_DIR, r17      ; set direction for all 8 PORTB pins
out  SPL, r16             ; set Stack Pointer Low byte
```

Writing all 8 bits at once with `OUT` is often less convenient than the single-bit `SBI`/`CBI` instructions. Use `OUT` when you want to change multiple pins simultaneously (e.g., output a byte to an 8-bit bus).

So `OUT` is not the default for every GPIO task. It is the right tool when the whole byte matters.

9.6 SBI and CBI — Set / Clear Bit in I/O Register

```
SBI A, b
   Operands: A ∈ {0..31},  b ∈ {0..7}   ← I/O addresses 0x00–0x1F only
   Operation: I/O(A).b ← 1
   Flags:     None
   Cycles:    1 on AVRxt (ATtiny3217), 2 on older AVRe devices

CBI A, b
   Operands: A ∈ {0..31},  b ∈ {0..7}
   Operation: I/O(A).b ← 0
   Flags:     None
   Cycles:    1 on AVRxt (ATtiny3217), 2 on older AVRe devices
```

`SBI` and `CBI` are **atomic** single-bit operations. On ATtiny3217, the hardware internally does read-modify-write in one indivisible step: no interrupt can observe a partial state between the read and the write.

That atomicity is why these instructions are still so valuable even though the newer PORT peripheral has richer register options.

> **Range restriction:** `SBI`/`CBI` only reach I/O addresses 0x00–0x1F (the lower 32 I/O registers). The VPORT registers (0x00–0x0B) are all within this range. Full PORT registers (0x0400+) require `LDS`/`STS`.

```
sbi  VPORTA_DIR, 3         ; set PA3 as output   (VPORTA_DIR = I/O 0x00)
cbi  VPORTA_OUT, 3         ; drive PA3 low        (VPORTA_OUT = I/O 0x01)
sbi  VPORTA_OUT, 3         ; drive PA3 high
sbi  VPORTB_DIR, 3         ; set PB3 as output
cbi  VPORTB_DIR, 5         ; set PB5 as input
```

When to use SBI/CBI vs. full PORT OUTSET/OUTCLR:

Situation	Instruction	Why
Set or clear one pin in VPORT	`SBI/CBI`	1-word instruction; 1 cycle on ATtiny3217
Set/clear bits, ISR may also touch port	`STS PORTA_OUTSET`	Guaranteed atomic
Set multiple bits at once	`STS PORTA_OUTSET`	Single write, no loop

If you remember only one distinction here, remember this one:

- `SBI/CBI` are ideal for one bit in low I/O space
- `OUTSET/OUTCLR/OUTTGL` are ideal for atomic updates in the full port block

9.7 SBIC and SBIS — Skip if Bit Clear / Set

```
SBIC A, b
   Operands: A ∈ {0..31},  b ∈ {0..7}
   Operation: if I/O(A).b = 0, skip next instruction
   Flags:     None
   Cycles:    1 (no skip),  2 (skip 1-word instruction),  3 (skip 2-word)

SBIS A, b
   Operands: A ∈ {0..31},  b ∈ {0..7}
   Operation: if I/O(A).b = 1, skip next instruction
   Flags:     None
   Cycles:    1 (no skip),  2 (skip 1-word instruction),  3 (skip 2-word)
```

These are the fastest way to branch on a single I/O pin state. Instead of `IN` + `SBRC/SBRS`, a single instruction reads and conditionally skips in one operation.

That is why `SBIC` and `SBIS` feel so natural in polling loops: test plus control flow happens in one instruction.

```
Wait for a button press (PB2 active-low):

.Lwait:
    sbis VPORTB_IN, 2      ; skip if PB2 = 1 (not pressed)
    rjmp .Lpressed          ; PB2 = 0: button is pressed
    rjmp .Lwait

Equivalent without SBIS:
    in   r16, VPORTB_IN    ; 1 cy: read all PORTB pins
```

```
    sbrs r16, 2             ; 1 cy: skip if bit 2 = 1
    rjmp .Lpressed          ; 2 cy: branch if bit was 0 (pressed)
    rjmp .Lwait
```

The SBIC/SBIS version saves one register (r16 untouched) and avoids the IN instruction.

Small savings like that matter in tight loops and ISR-adjacent code.

Skip behavior for 2-word instructions: SBIC/SBIS skip exactly one instruction slot. If the instruction immediately after is a 2-word instruction (JMP, CALL, LDS, STS, LDD, STD), the skip costs 3 cycles (the full 2-word instruction is skipped). Skip over a 1-word instruction costs 2 cycles.

So the timing of a skip depends not just on the skip instruction itself, but also on what comes immediately after it.

```
; Good: skip over a 1-word RJMP (common pattern)
sbic VPORTB_IN, 2          ; skip if PB2 = 0 (pressed)
rjmp .Lnot_pressed          ; 1 word — skipped in 2 cycles

; Also fine: skip over a 2-word STS
sbis VPORTA_IN, 3          ; skip if PA3 = 1
sts  PORTA_OUTTGL, r16     ; 2 words — skipped in 3 cycles
```

9.8 Configuring a GPIO Output

Two equivalent approaches on ATtiny3217:

Method A — SBI on VPORT (single pin, 1 cycle on ATtiny3217):

```
sbi  VPORTA_DIR, 3         ; PA3 = output
sbi  VPORTA_OUT, 3         ; initial value: high (Curiosity Nano LED off)
```

Method B — STS with PORT DIRSET/OUTCLR (atomic, any number of pins):

```
ldi  r16, (1 << 3)
sts  PORTA_DIRSET, r16     ; PA3 = output, other bits unchanged
sts  PORTA_OUTSET, r16     ; PA3 driven high: Curiosity Nano LED off
```

Both methods produce identical hardware results for a single pin. Method B is preferred when configuring multiple pins in a single write:

That means you can choose the style based on clarity and scaling, not because one is inherently more “correct” for a single pin.

```
ldi  r16, (1 << 3) | (1 << 5)  ; configure PA3 and PA5
sts  PORTA_DIRSET, r16         ; both pins become outputs
sts  PORTA_OUTCLR, r16         ; both driven low
```

9.9 Configuring a GPIO Input with Pull-Up

On ATtiny3217, pin direction defaults to input (0) at reset, so only the pull-up needs configuration. Pull-up lives in the PINnCTRL register, which is above the I/O space — LDS/STS required.

```
PINnCTRL register layout:
Bit:  7      6:4       3         2:0
      INVEN  reserved  PULLUPEN  ISC[2:0]

PULLUPEN (bit 3) = 0: pull-up off (pin floats if nothing drives it)
                 = 1: pull-up on  (pin pulled toward VCC)
ISC[2:0]         = 0x00: interrupt disabled, input buffer enabled
                 = 0x04: interrupt and digital input buffer disabled
                 (other values in Chapter 10)
PORT_PULLUPEN_bm = 0x08
```

The floating input problem: If a digital input pin is not connected to anything and has no pull-up, its voltage floats between GND and VCC. The CPU reads random logic levels that change with temperature, noise, and nearby switching signals. Always enable the pull-up when using a button wired to GND, add an external pull resistor, or drive the pin from a source you control.

An unconnected digital input is not neutral; it is unreliable.

```
; PB2 as input with pull-up (button wired to GND)
; Direction is already 0 (input) — nothing to do for DIR.
ldi  r16, PORT_PULLUPEN_bm     ; = 0x08
sts  PORTB_PIN2CTRL, r16       ; PORTB_PIN2CTRL = memory 0x0432

; PB3 as input, NO pull-up (externally driven signal, e.g. UART RX)
ldi  r16, 0
sts  PORTB_PIN3CTRL, r16       ; ISC = disabled, PULLUPEN = 0
```

Reading a pin:

```
in   r16, VPORTB_IN            ; read all 8 PORTB pins
sbrc r16, 2                    ; skip if bit 2 = 0 (button pressed)
; ... bit 2 = 1: button not pressed ...

; Or more directly:
sbis VPORTB_IN, 2              ; skip if PB2 = 1 (not pressed)
rjmp .Lpressed
```

9.10 Debounce

Mechanical switch contacts **bounce**: when you press or release a button, the contacts make and break contact several times in rapid succession before settling. The CPU is fast enough to see each bounce as a separate button event.

So debouncing is not a cosmetic improvement. It is what turns a noisy physical event into a single logical event.

```
Ideal button press:      Actual button press:
PB2 voltage              PB2 voltage

1 ─────┐                1 ─────┐  ┌┐  ┌┐
       │                       │  ││  ││  ┌────
0      └─────             0    └──┘└──┘└──┘
```

```
        time                            time
                                     ←bounce→
```

The standard software fix: after detecting a press, wait a few milliseconds (bounce settles in < 10 ms for most switches), then re-check. Do the same on release.

Simple time-delay debounce:

```
; Wait for press
.Lpoll:
    sbis VPORTB_IN, BTN_BIT    ; skip if not pressed
    rjmp .Lpressed
    rjmp .Lpoll

.Lpressed:
    rcall delay_20ms            ; wait for bounce to settle

    ; Optional: confirm still pressed (guards against glitches)
    sbic VPORTB_IN, BTN_BIT    ; skip if still pressed (PB2 = 0)
    rjmp .Lpoll                 ; was a glitch — restart

    ; Confirmed: genuine press
    ; ... handle event ...

    ; Wait for release
.Lwait_release:
    sbis VPORTB_IN, BTN_BIT    ; skip if released (PB2 = 1)
    rjmp .Lwait_release
    rcall delay_20ms            ; debounce the release
    rjmp .Lpoll
```

For typical tactile buttons, 20 ms is a safe debounce window. Some mechanical switches need 50 ms.

That delay is a human-timescale compromise: long enough to outlast bounce, but short enough that the user does not notice it.

9.11 Worked Example: Button-Controlled LED

LED0 on PA3 (Curiosity Nano, active-low). External button on PB2 (active-low, internal pull-up). Each confirmed press toggles the LED state.

This example combines almost every GPIO concept in one small program: output setup, input pull-up, active-low logic, skip-based polling, debounce, and atomic output update.

Hardware:

```
ATtiny3217
                      VCC
                       |
PA3 ── Curiosity Nano LED0 (on board, active-low)

PB2 ──[internal pull-up]──────── VCC
PB2 ──[button]──────────────── GND
```

Source file: `src/gpio.S`

```
#include <avr/io.h>

.equ LED_BIT,         3          ; PA3, Curiosity Nano LED0
.equ BTN_BIT,         2          ; PB2
.equ DEBOUNCE_COUNT,  16666      ; ~20 ms at 3.3 MHz (sbiw+brne loop)

.section .vectors, "ax", @progbits
.global __vectors
__vectors:
    rjmp main

.section .text
.global main

main:
    ldi  r16, hi8(RAMEND)
    out  SPH, r16
    ldi  r16, lo8(RAMEND)
    out  SPL, r16

    sbi  VPORTA_DIR, LED_BIT   ; PA3 = output
    sbi  VPORTA_OUT, LED_BIT   ; LED off (active-low)

    ldi  r16, PORT_PULLUPEN_bm ; 0x08
    sts  PORTB_PIN2CTRL, r16   ; PB2: input + pull-up, no interrupt

.Lloop:
    sbis VPORTB_IN, BTN_BIT    ; skip if PB2 = 1 (not pressed)
    rjmp .Lpressed
    rjmp .Lloop

.Lpressed:
    rcall delay_20ms
    sbic VPORTB_IN, BTN_BIT    ; skip if PB2 is still low (still pressed)
    rjmp .Lloop                ; bounce/glitch: ignore

    ldi  r16, (1 << LED_BIT)
    sts  PORTA_OUTTGL, r16     ; atomic toggle PA3

.Lwait_release:
    sbis VPORTB_IN, BTN_BIT    ; skip if PB2 = 1 (released)
    rjmp .Lwait_release
    rcall delay_20ms
    rjmp .Lloop

delay_20ms:
    ldi  r26, lo8(DEBOUNCE_COUNT)
    ldi  r27, hi8(DEBOUNCE_COUNT)
.Ldly_loop:
    sbiw r26, 1                ; 2 cycles
    brne .Ldly_loop             ; 2 cycles taken
    ret
```

9.11.1 Why PORTA_OUTTGL instead of SBI/CBI?

`SBI VPORTA_OUT, 3` sets the bit; `CBI VPORTA_OUT, 3` clears it. Neither alone toggles — you'd need to read the current state first. The naive approach:

```
; Non-atomic toggle — dangerous if ISR touches PORTA_OUT
in   r16, VPORTA_OUT
ldi  r17, (1 << LED_BIT)
eor  r16, r17              ; flip LED bit
out  VPORTA_OUT, r16       ; write back
```

Three instructions. An interrupt that fires between `IN` and `OUT` and modifies another PA pin will have its change overwritten by the `OUT`. The `PORTA_OUTTGL` approach is atomic: the hardware performs the read, XOR, and write in one step.

That is the real lesson: the shorter code is not just shorter, it is safer when something else can touch the same port state.

9.11.2 Build and Inspect

```
avr-gcc -mmcu=attiny3217 -x assembler-with-cpp -c -o gpio.o src/gpio.S
avr-gcc -mmcu=attiny3217 -nostartfiles -o gpio.elf gpio.o
avr-objdump -d gpio.elf      # inspect disassembly
avr-objcopy -O ihex gpio.elf gpio.hex   # generate flashable HEX
```

Check the disassembly around `.Lpressed` to confirm that `sts PORTA_OUTTGL` encodes as a 2-word `sts` instruction (opcode `0x92..`) with the 16-bit address 0x0407.

9.12 Summary

```
IN  Rd, A      — read I/O register A (0–63) into Rd.    1 cycle.
OUT A, Rr      — write Rr to I/O register A (0–63).     1 cycle.
SBI A, b       — set bit b of I/O register A (0–31).    1 cycle on AVRxt. Atomic.
CBI A, b       — clear bit b of I/O register A (0–31).  1 cycle on AVRxt. Atomic.
SBIC A, b      — skip next if I/O(A).b = 0.             1/2/3 cycles.
SBIS A, b      — skip next if I/O(A).b = 1.             1/2/3 cycles.

SBI/CBI/SBIC/SBIS range: I/O addresses 0x00–0x1F only.
VPORT registers (0x00–0x0B) are within this range: all six instructions apply.
Full PORT registers (0x0400+) need LDS/STS.

ATtiny3217 GPIO quick-reference:
  Configure output:   sbi VPORTA_DIR, n   or  sts PORTA_DIRSET, r16
  Drive high:         sbi VPORTA_OUT, n   or  sts PORTA_OUTSET, r16
  Drive low:          cbi VPORTA_OUT, n   or  sts PORTA_OUTCLR, r16
  Toggle output:      sts PORTA_OUTTGL, r16  (atomic)
  Configure input:    (DIR default = 0 — no action required)
  Enable pull-up:     sts PORTB_PIN2CTRL, r16  (PORT_PULLUPEN_bm = 0x08)
  Read input:         in r16, VPORTB_IN   or  sbic/sbis VPORTB_IN, n

Debounce: after detecting edge, wait ~20 ms, re-check, wait for release,
          wait ~20 ms again before returning to poll loop.
```

9.13 Exercises

1. Reconfigure the example so the LED is on PC0 instead of PA3. Which registers change? Which instructions change? Is there any difference in the generated machine code size?
2. Write a routine `toggle_n_times` that accepts a count in r24 and toggles PA3 that many times with a 100 ms delay between each toggle. Use `PORTA_OUTTGL` and a subroutine-based delay.
3. The debounce loop calls `delay_20ms` twice: once after press, once after release. Under what button-press pattern could the LED be toggled without a real press if you removed the release debounce?
4. Rewrite the button poll using `SBIC` instead of `SBIS`. Draw the new control flow. Are the two approaches equivalent in terms of cycles per loop iteration?
5. How many cycles does the `delay_20ms` subroutine take from `RCALL` to the final `RET`? Show the sum: prologue, loop body × N, final loop exit, return.
6. Add a second button on PA5 (active-low, pull-up). When BTN1 (PB2) is pressed, turn the LED on; when BTN2 (PA5) is pressed, turn it off. Use only `SBIC`/`SBIS` — no `IN` instruction.

Next: Chapter 10 — Interrupts

Chapter 10

Interrupts

Polling burns CPU cycles checking whether something has happened. An **interrupt** lets hardware signal the CPU directly: the CPU pauses whatever it was doing, runs a short handler (ISR), and resumes exactly where it left off. This chapter covers the interrupt mechanism, the ATtiny3217 vector table, the `SEI`/`CLI`/`RETI` instructions, and the discipline required to write a correct ISR.

This is where the program stops being a single straight-line thread of control and starts reacting to the outside world asynchronously.

10.1 Why Interrupts?

The polling loop from Chapter 9:

```
.Lloop:
    sbis VPORTB_IN, BTN_BIT    ; check button every iteration
    rjmp .Lpressed
    rjmp .Lloop
```

This consumes 100% of the CPU checking one pin. In a real application, the CPU may need to: update a display, process serial data, manage a state machine — all while remaining responsive to button presses. Polling every possible event source and handling each one in priority order becomes unwieldy.

Interrupts invert the model: the CPU runs its main work and is **notified** when an event occurs. The ISR handles the event in a bounded time, then execution resumes with main code unmodified.

That last requirement is the hard part. An interrupt is only useful if the main code can continue as though nothing important was damaged by the pause.

10.2 The Interrupt Mechanism

10.2.1 What Happens When an Interrupt Fires

On ATtiny3217, think of interrupt entry as a short fixed hardware sequence: finish the current instruction, push the PC, then execute the vector entry. With the `RJMP`-based vector table used in this book, the first ISR instruction starts after **5 cycles minimum**: one cycle to finish the current instruction, two cycles to store the PC, and two cycles to execute the vector `RJMP`.

That means interrupt latency is not arbitrary. It is mostly predictable, which is one reason small microcontrollers work well for real-time event handling.

```
1. CPU finishes the current instruction.
2. PC is pushed onto the stack (2 bytes on ATtiny3217).
3. CPUINT records the active interrupt level.
4. The interrupt vector entry executes (`RJMP` in this chapter's layout).
5. CPU begins the first instruction of the ISR body.
```

When the ISR executes `RETI`:

```
1. PC is popped from the stack (SP += 2).
2. CPUINT returns to the previous interrupt state.
3. CPU resumes the instruction after where it was interrupted.
```

The interrupted code is **unaware** an interrupt occurred — as long as the ISR leaves all registers and SREG flags exactly as it found them.

That “as long as” is the whole discipline of ISR writing.

10.2.2 The I Flag (Global Interrupt Enable)

```
SREG bit layout:
Bit:  7   6   5   4   3   2   1   0
      I   T   H   S   V   N   Z   C
      ↑
      Global Interrupt Enable
      1 = interrupts enabled
      0 = interrupts disabled (masked)
```

The I flag is a master switch. Even if a peripheral has its interrupt configured and pending, the CPU will not respond until I = 1. On ATtiny3217, Microchip specifies that this bit is **not** cleared by hardware when entering an ISR and is **not** set by `RETI`; control it explicitly with `SEI`, `CLI`, or a direct write to `SREG`.

So enabling a peripheral interrupt is never the whole story. The peripheral must be configured, and the global interrupt gate must also be open.

Microchip gives the initialization order explicitly:

1. Configure CPUINT only if the defaults are not enough.
2. Configure the peripheral interrupt condition and enable that source.
3. Set `SREG.I` globally with `SEI`.

For this chapter, the CPUINT defaults are enough: normal vector table, fixed priority by vector address, no round-robin scheduling, and no level-1 vector. Advanced CPUINT bits such as `IVSEL`, `CVT`, `LVL0RR`, and `LVL1VEC` are left unchanged.

10.2.3 Interrupt Priority

On ATtiny3217, lower vector address = higher priority. RESET (0x0000) has the highest priority. If two interrupts become pending simultaneously, the one with the lower vector address fires first.

CPUINT tracks the current interrupt level. By default, all maskable interrupt vectors are level 0, so another level 0 interrupt is kept pending until the current handler returns. One vector can optionally be assigned to level 1 (high priority), and a level 1 interrupt can interrupt a level 0 handler.

For most firmware, leave the default level-0 behavior alone unless there is a clear timing reason to introduce a high-priority interrupt.

CPUINT also has a Compact Vector Table mode (`CPUINT.CTRLA.CVT`). When enabled, all level-0 interrupts share one vector. This saves flash but requires a shared dispatcher that checks interrupt flags. This book uses the normal table because one vector per source is easier to inspect in assembly.

10.3 SEI and CLI

```
SEI
   Operation: SREG.I ← 1
   Flags:     I = 1
   Cycles:    1
   Note:      The instruction immediately after SEI executes before any
              pending interrupt fires.

CLI
   Operation: SREG.I ← 0
   Flags:     I = 0
   Cycles:    1
   Note:      Guaranteed to prevent any interrupt from firing after CLI.
```

The one-instruction delay after `SEI` is important:

```
; Guaranteed: the RJMP executes before any pending interrupt fires.
sei
rjmp .Lidle
```

That behavior prevents awkward races where an interrupt could fire immediately between `SEI` and the next instruction.

10.3.1 Critical Sections

`CLI`/`SEI` create a **critical section** — a code region that cannot be interrupted:

```
cli                     ; disable interrupts — begin critical section
; ... read or write shared variables that the ISR also touches ...
sei                     ; re-enable interrupts — end critical section
```

A cleaner pattern that preserves the interrupt state rather than unconditionally re-enabling:

```
in   r24, SREG          ; save current SREG (including I flag)
cli                     ; disable interrupts
```

```
; ... critical section ...
out  SREG, r24          ; restore SREG exactly (may or may not re-enable)
```

This is safe if the caller had interrupts disabled — `SEI` would have incorrectly re-enabled them, but restoring SREG preserves their disabled state.

This is the pattern to prefer in reusable code, because it does not assume that interrupts were enabled before the critical section began.

10.4 RETI — Return from Interrupt

```
RETI
   Operation: PC ← stack (pop 2 bytes), SP ← SP + 2; CPUINT exits ISR state
   Flags:     SREG is not restored automatically
   Cycles:    4
```

On ATtiny3217, `RETI` is still required even though it does not set the I flag. It tells CPUINT that the interrupt handler has completed and returns the interrupt controller to the previous interrupt level. Never use `RET` to exit an ISR:

```
Using RET at end of ISR:          Using RETI at end of ISR:
  PC returns, but CPUINT still      PC returns and CPUINT exits
  thinks an ISR is active.          the interrupt state cleanly.
  Same/lower-priority requests      Pending interrupts can be
  may remain blocked.               serviced after one instruction.
```

The gap between executing the `OUT SREG, r24` (in the epilogue) and `RETI` is safe because CPUINT has not yet completed the current interrupt. Pending interrupts are not serviced until after `RETI`, and Microchip documents that pending interrupts are served only after one instruction of the resumed program has executed.

That is why restoring SREG before `RETI` is correct even when the saved I bit is 1.

10.5 The Interrupt Vector Table

10.5.1 Layout on ATtiny3217

The vector table occupies the first 62 bytes of flash (31 entries × 2 bytes each). Every entry is one RJMP instruction — `JMP` (4 bytes) would overflow the 2-byte slot.

Byte addr	Vector num	Peripheral	Trigger condition
0x0000	–	RESET	Power-on, external reset, WDT
0x0002	1	CRCSCAN_NMI	CRC scan failure (non-maskable)
0x0004	2	BOD_VLM	Voltage level monitor
0x0006	3	PORTA_PORT	Any PORTA pin with ISC configured
0x0008	4	PORTB_PORT	Any PORTB pin with ISC configured
0x000A	5	PORTC_PORT	Any PORTC pin with ISC configured
0x000C	6	RTC_CNT	RTC counter/compare
0x000E	7	RTC_PIT	RTC periodic interrupt
0x0010	8	TCA0_OVF	Timer/Counter A overflow
0x0012	9	TCA0_HUNF	TCA high-byte underflow (split)

0x0014	10	TCA0_CMP0	TCA compare channel 0
0x0016	11	TCA0_CMP1	TCA compare channel 1
0x0018	12	TCA0_CMP2	TCA compare channel 2
0x001A	13	TCB0_INT	Timer/Counter B0
0x001C	14	TCB1_INT	Timer/Counter B1
0x001E	15	TCD0_OVF	Timer/Counter D overflow
0x0020	16	TCD0_TRIG	TCD trigger
0x0022	17	AC0_AC	Analog Comparator 0
0x0024	18	AC1_AC	Analog Comparator 1
0x0026	19	AC2_AC	Analog Comparator 2
0x0028	20	ADC0_RESRDY	ADC result ready
0x002A	21	ADC0_WCOMP	ADC window compare
0x002C	22	ADC1_RESRDY	ADC1 result ready
0x002E	23	ADC1_WCOMP	ADC1 window compare
0x0030	24	TWI0_TWIS	TWI slave
0x0032	25	TWI0_TWIM	TWI master
0x0034	26	SPI0_INT	SPI
0x0036	27	USART0_RXC	USART RX complete
0x0038	28	USART0_DRE	USART data register empty
0x003A	29	USART0_TXC	USART TX complete
0x003C	30	NVMCTRL_EE	EEPROM ready

10.5.2 Defining the Vector Table in .S

```
.section .vectors, "ax", @progbits
.global __vectors
__vectors:
    rjmp main               /* 0x0000: RESET — must be first              */
    rjmp isr_default        /* 0x0002: CRCSCAN_NMI                         */
    rjmp isr_default        /* 0x0004: BOD_VLM                             */
    rjmp isr_default        /* 0x0006: PORTA_PORT                          */
    rjmp portb_isr          /* 0x0008: PORTB_PORT  ← our handler           */
    rjmp isr_default        /* 0x000A: PORTC_PORT                          */
    /* ... continue for all 30 remaining vectors ... */
```

Each consecutive `RJMP` is exactly 2 bytes, so the nth entry lands at byte address n×2 automatically — no `.org` directives needed.

Unhandled vectors should point to a catch-all handler that just `RETI`s, not to `main` (re-running `main` on a spurious interrupt would re-initialize your whole program). A simple trap for debugging: `rjmp .` (infinite loop).

Which default you choose depends on your goal:

- `reti` if you want unused interrupts to be harmless
- `rjmp .` if you want unexpected interrupts to halt the system for debugging

```
isr_default:
    reti                    /* ignore unexpected interrupts silently        */
```

10.6 ISR Prologue and Epilogue

An ISR may fire at any point during the main code: between any two instructions. The main code has no way to know whether an ISR ran during a particular period. Therefore:

Every register written by an ISR must be saved before use and restored before `RETI`.

SREG must always be saved. Any instruction that modifies flags (arithmetic, logic, shifts, compares, even `SBIW`) will corrupt the flags that the interrupted code was relying on.

These are the core ISR rules. Violating them may not fail immediately, which is why ISR bugs can be so frustrating to diagnose.

10.6.1 Minimal ISR (uses r16 only)

```
my_isr:
    push  r16              ; save r16 (2 cycles, 1 byte stack)
    in    r16, SREG        ; save SREG into r16
    push  r16              ; save SREG on stack (2 cycles, 1 byte)

    ; ... ISR body, may freely use r16 ...

    pop   r16              ; restore SREG value into r16
    out   SREG, r16        ; restore SREG
    pop   r16              ; restore r16
    reti
```

10.6.2 Why This Order?

```
Step                 Stack contents (grows down)       SP points to
--------------------------------------------------------------------
(on ISR entry)       [ ret_hi ][ ret_lo ]              top of ret addr
push r16             [ ret_hi ][ ret_lo ][ r16 ]       saved r16
in r16, SREG         r16 = SREG value
push r16             [ ret_hi ][ ret_lo ][ r16 ][ SREG_val ]  ← SP
... body ...
pop r16              r16 = SREG_val, stack shrinks
out SREG, r16        SREG restored
pop r16              r16 = original r16 value
reti                 pop return addr, exit CPUINT ISR state, jump back
```

On ATtiny3217, `OUT SREG, r16` restores the I bit to whatever value it had when the ISR saved SREG. `RETI` does not repair SREG; it returns the CPUINT state to the previous interrupt level. The two operations handle different state, so both are required.

That separation is one of the main reasons `RETI` exists as a distinct instruction instead of reusing ordinary `RET`.

10.6.3 Saving Multiple Registers

```
my_isr:
    push  r16
    push  r17
    push  r18
```

```
    in    r16, SREG
    push  r16              ; SREG last — now on top of stack

    ; ... ISR body using r16, r17, r18 ...

    pop   r16              ; SREG value
    out   SREG, r16
    pop   r18              ; restore in reverse order
    pop   r17
    pop   r16
    reti
```

10.6.4 ISR Stack Budget

Every PUSH costs 2 cycles and 1 byte of stack. The two-register minimal ISR (r16 + SREG) uses: - 2 bytes for the return address (pushed automatically by CPU) - 2 bytes for the two PUSH instructions

Total: 4 bytes of stack per invocation. If main code uses 200 bytes of stack and ISRs nest, add the ISR frame size to your stack budget.

So ISR design is not just about speed. It is also about bounded stack usage.

10.7 Configuring Pin Interrupts on ATtiny3217

Classic ATmega devices have dedicated `INT0`/`INT1` external interrupt pins. ATtiny3217 takes a different approach: **any pin** can generate an interrupt by configuring its `PINnCTRL` register.

This is more flexible than the old dedicated-pin model, but it also means you think in terms of per-pin configuration rather than special global interrupt pins.

10.7.1 PINnCTRL Layout

```
Bit:   7      6:4      3         2:0
       INVEN  reserved PULLUPEN  ISC[2:0]

PULLUPEN = 1: enable internal pull-up (bit 3 = 0x08)

ISC[2:0]:  Interrupt Sense Configuration
  0x00 = PORT_ISC_INTDISABLE_gc  — interrupt disabled, input buffer on
  0x01 = PORT_ISC_BOTHEDGES_gc   — interrupt on both edges (rise + fall)
  0x02 = PORT_ISC_RISING_gc      — interrupt on rising edge  (low → high)
  0x03 = PORT_ISC_FALLING_gc     — interrupt on falling edge (high → low)
  0x04 = PORT_ISC_INPUT_DISABLE_gc — input buffer off (ADC pins, saves power)
  0x05 = PORT_ISC_LEVEL_gc       — interrupt while pin is low (level triggered)
```

For a button wired to GND (active-low, fires on press = falling edge):

```
ldi  r16, (PORT_PULLUPEN_bm | PORT_ISC_FALLING_gc)  ; = 0x08 | 0x03 = 0x0B
sts  PORTB_PIN2CTRL, r16   ; PB2: pull-up + falling edge interrupt
```

Microchip describes the PORT interrupt source as `INTn` in `PORTx.INTFLAGS` being raised according to `ISC` in `PORTx.PINnCTRL`. One `PORTx_PORT` vector covers the whole port, so the ISR must inspect INTFLAGS if more than one pin on that port can interrupt.

When changing interrupt sense settings, avoid doing it while an interrupt is being synchronized. The datasheet notes that changing ISC during synchronization may lose a request, and re-enabling an input after INPUT_DISABLE may request an interrupt even with different settings.

10.7.2 PORT INTFLAGS — Clearing Interrupt Flags

When an interrupt fires, the corresponding bit in PORT_INTFLAGS is set. The flag is **sticky**: it stays set until explicitly cleared. If you return from the ISR without clearing the flag, the interrupt fires again immediately.

That sticky-flag behavior is easy to forget the first time you write an ISR. If the flag is not cleared, the hardware still believes the event is pending.

Clearing: write 1 to the flag bit (write-1-to-clear, or W1C semantics).

Method A — STS with full PORT address (any bit):

```
ldi  r16, (1 << BTN_BIT)       ; bit 2 = PB2
sts  PORTB_INTFLAGS, r16       ; write 1 to bit 2 → clears flag
```

Method B — SBI on VPORTB_INTFLAGS (single bit, faster):

```
sbi  VPORTB_INTFLAGS, BTN_BIT  ; VPORTB_INTFLAGS = I/O 0x07
                                ; SBI writes 1 to bit 2 → clears flag
```

SBI writing a 1 to a W1C register clears the flag. This is counterintuitive (SBI normally "sets" a bit) but correct: the hardware interprets a write-1 as "clear this flag".

This is one of those peripheral rules you simply have to memorize: W1C means "write one to clear," even if the instruction name sounds like the opposite.

The flag for PORTB_PORT covers all PORTB pins. If multiple pins are configured with ISC ≠ 0, read PORTB_INTFLAGS first to determine which pin(s) fired before clearing.

10.8 ISR Latency and Timing

With the RJMP vector style used here, the CPU reaches the first ISR instruction in **5 cycles minimum** from acknowledgement on ATtiny3217: one cycle to finish the ongoing instruction, two cycles to store the PC on the stack, and two cycles for the vector-table RJMP. If the interrupt arrives during a multi-cycle instruction, that instruction must finish first, so total observed latency is higher.

```
Finish current instruction
Push 2-byte PC to stack
Execute vector-table RJMP
Begin ISR body
```

At 3.3 MHz, one cycle is about 303 ns, so 5 cycles is about 1.5 us before the ISR body starts, not counting any extra wait for a multi-cycle interrupted instruction or synchronizer delay in the peripheral itself.

That is fast enough for many GPIO, timer, and serial-response tasks, but still slow enough that cycle counting matters in tight timing work.

The minimal ISR prologue shown here (`PUSH r16`, `IN r16,SREG`, `PUSH r16`) adds **5 cycles** before the first useful ISR work: - `PUSH` = 2 cycles - `IN` = 1 cycle - `PUSH` = 2 cycles

For time-critical ISRs, minimize the prologue: save only what you use.

In ISR code, every unnecessary instruction is paid on every event.

10.9 Worked Example: Pin Interrupt Toggles LED

Curiosity Nano LED0 on PA3, external button on PB2 (falling edge). The ISR toggles the LED; main loops forever doing nothing.

This is intentionally minimal so the interrupt path is easy to see in isolation.

Source file: `src/interrupts.S`

```
#include <avr/io.h>

.equ LED_BIT,   3
.equ BTN_BIT,   2
.equ BTN_CTRL,  0x0B       /* PORT_PULLUPEN_bm | PORT_ISC_FALLING_gc */

/* ── Vector table ─────────────────────────────────────────────────────── */
.section .vectors, "ax", @progbits
.global __vectors
__vectors:
    rjmp main               /* 0x0000: RESET                            */
    rjmp isr_default        /* 0x0002: CRCSCAN_NMI                      */
    rjmp isr_default        /* 0x0004: BOD_VLM                          */
    rjmp isr_default        /* 0x0006: PORTA_PORT                       */
    rjmp portb_isr          /* 0x0008: PORTB_PORT                       */
    rjmp isr_default        /* 0x000A: PORTC_PORT                       */
    /* ... full source lists all remaining vectors as isr_default ... */

/* ── Catch-all ISR ─────────────────────────────────────────────────────── */
.section .text

isr_default:
    reti

/* ── PORTB ISR ─────────────────────────────────────────────────────────── */
/*
 * Prologue:     PUSH r16, IN r16←SREG, PUSH r16        (5 cycles, 2 stack)
 * Body:         LDI r16, STS PORTA_OUTTGL, SBI VPORTB_INTFLAGS
 * Epilogue:     POP r16, OUT SREG←r16, POP r16, RETI   (9 cycles, 0 stack)
 */
portb_isr:
    push  r16
    in    r16, SREG
    push  r16

    ldi   r16, (1 << LED_BIT)
    sts   PORTA_OUTTGL, r16      /* toggle PA3 / LED0                    */
    sbi   VPORTB_INTFLAGS, BTN_BIT  /* clear flag (W1C via SBI)          */
```

```
    pop   r16
    out   SREG, r16
    pop   r16
    reti

/* ── main ────────────────────────────────────────────────────────────── */
.global main

main:
    ldi   r16, hi8(RAMEND)
    out   SPH, r16
    ldi   r16, lo8(RAMEND)
    out   SPL, r16

    sbi   VPORTA_DIR, LED_BIT   /* PA3 = output                          */
    sbi   VPORTA_OUT, LED_BIT   /* LED off (active-low)                  */

    ldi   r16, BTN_CTRL
    sts   PORTB_PIN2CTRL, r16   /* PB2: pull-up + falling edge interrupt   */

    sei                         /* enable global interrupts               */

.Lidle:
    rjmp  .Lidle                /* CPU idles, ISR handles everything      */
```

10.9.1 Execution Flow Walkthrough

```
Power-on:
  RESET vector (0x0000) → RJMP main → main executes
  Configures PA3 output, PB2 pull-up + ISC, SEI
  Enters .Lidle loop

User presses button:
  PB2 falls from 1 → 0 (falling edge)
  PORTB_PIN2CTRL ISC detects edge → sets PORTB_INTFLAGS bit 2
  CPU detects pending interrupt (I=1, so enabled)
  Finishes current RJMP .Lidle instruction
  Pushes PC onto stack and records the active interrupt level in CPUINT
  Fetches PORTB_PORT vector (0x0008) → RJMP portb_isr
  Executes portb_isr:
    push r16 / save SREG
    sts PORTA_OUTTGL → PA3 toggles (LED changes state)
    sbi VPORTB_INTFLAGS, 2 → clears flag
    restore SREG / pop r16
    RETI → pop PC, exit CPUINT ISR state, jump back to .Lidle

User presses button again:
  Same sequence, LED toggles back
```

10.9.2 Why Not Debounce in the ISR?

The Chapter 9 polling example used a 20 ms delay loop after each press. That loop busy-waits, which is unacceptable inside an ISR (other time- critical events would be missed, and ISR latency would explode to 20 ms).

Options for interrupt-based debouncing: 1. **Hardware debounce**: RC + Schmitt trigger on the button line. 2. **Timer debounce**: in the ISR, start a timer; the real handler fires when the timer expires (Chapter 11). Until the timer expires, disable the pin interrupt. 3. **Level-based re-arm**: change ISC to `ISC_LEVEL` or use a state machine in `main` that re-enables the interrupt after a delay.

For this example, the button is held long enough that multiple edges do not cause visible problems. Finished firmware should use method 2 or 3.

The example demonstrates the interrupt mechanism cleanly; a finished button interface should add a real debounce strategy.

10.9.3 Build and Inspect

```
avr-gcc -mmcu=attiny3217 -x assembler-with-cpp -c -o interrupts.o src/interrupts.S
avr-gcc -mmcu=attiny3217 -nostartfiles -o interrupts.elf interrupts.o
avr-objdump -d interrupts.elf
avr-objcopy -O ihex interrupts.elf interrupts.hex
```

Inspect the disassembly to confirm: - The vector table starts at address 0x0000 with a sequence of `rjmp`s. - `portb_isr` is at the address pointed to by the entry at 0x0008. - The `sts PORTA_OUTTGL` inside `portb_isr` is a 4-byte instruction. - `sbi VPORTB_INTFLAGS, 2` is a 2-byte instruction (opcode `0x9ABF` or similar — check with `avr-objdump -d`).

10.10 Summary

```
SEI      — set I flag (enable global interrupts). 1 cycle.
CLI      — clear I flag (disable global interrupts). 1 cycle.
RETI    — return from ISR: pop PC, exit CPUINT ISR state. 4 cycles. Never use RET here.

Interrupt response sequence (ATtiny3217, 5 cycles minimum with RJMP vectors):
  finish current instruction → push PC → execute vector RJMP → ISR

ISR discipline:
  1. PUSH all registers used.         (2 cy, 1 stack byte each)
  2. IN r16, SREG  →  PUSH r16.       (save SREG — always)
  3. ISR body.
  4. POP r16  →  OUT SREG, r16.       (restore SREG)
  5. POP all registers in reverse order.
  6. RETI.                            (not RET!)

Vector table: .section .vectors — sequential RJMP instructions.
  2 bytes each. Entry n at byte address n×2. RESET at 0x0000.
  Unhandled vectors → isr_default: reti.

ATtiny3217 pin interrupt config:
  sts PORTx_PINnCTRL, r16
```

```
    PORT_PULLUPEN_bm      = 0x08  (bit 3: pull-up)
    PORT_ISC_FALLING_gc   = 0x03  (bits 2:0: falling edge)
    PORT_ISC_RISING_gc    = 0x02
    PORT_ISC_BOTHEDGES_gc = 0x01

Clearing INTFLAGS (W1C):
  sts  PORTx_INTFLAGS, r16       (STS to full PORT — any bit)
  sbi  VPORTx_INTFLAGS, n        (faster — SBI writes 1 → clears flag)
  Must be done before RETI or interrupt fires again.
```

10.11 Exercises

1. The current ISR saves r16 and SREG. If the ISR body also needed to use r17 and r18, where exactly (before/after which existing instruction) should the PUSH r17 and PUSH r18 go? Write the complete modified prologue and epilogue.
2. What happens if you replace `reti` with `ret` at the end of `portb_isr`? Describe the symptom you would observe and explain why it occurs in terms of CPUINT interrupt state.
3. The idle loop is `rjmp .Lidle`. If you replaced it with the AVR `sleep` instruction (opcode 0x9588), the CPU would enter idle sleep mode and halt until the next interrupt. What changes would be required in `main` to enable sleep mode? (Hint: the SLPCTRL peripheral needs to be configured — look at Chapter 1's linker-script walkthrough or the ATtiny3217 datasheet section on SLPCTRL.)
4. Configure PB3 for a second button that also fires on falling edge. In `portb_isr`, read `PORTB_INTFLAGS` to determine which pin fired, clear only that flag, and call one of two handlers: `led_on` or `led_off`. Write the complete modified ISR.
5. Using `avr-objdump -d interrupts.elf`, count the exact number of bytes used by the vector table. Does it match 31 vectors × 2 bytes = 62 bytes? What byte address does the first non-vector instruction appear at?
6. The ISR uses `sts PORTA_OUTTGL, r16` (a 4-byte instruction, 2 cycles). Could you replace it with a 2-byte instruction using the VPORT registers? If so, write the alternative. If not, explain why.

Next: Chapter 11 — Timer/Counter

Chapter 11

Timer/Counter

Timers are the heartbeat of embedded firmware. They let you generate precise waveforms, schedule periodic work, measure elapsed time, and produce PWM signals — all without busy-waiting and without tying up the CPU. This chapter covers the TCA0 (Timer/Counter Type A) peripheral on the ATtiny3217, its main waveform-generation modes, and the interrupts each mode can generate. The worked example produces a 1 Hz timer tick that toggles LED0 from a CMP0 compare-match interrupt.

This is where you start handing time-sensitive repetition over to hardware instead of burning CPU cycles in delay loops.

11.1 Timer Fundamentals

11.1.1 What a Timer Actually Is

A timer is a hardware counter clocked by a divided version of the CPU clock. Each clock tick, the counter register (CNT) increments by 1. When CNT reaches a programmed limit (TOP), something happens — an interrupt fires, an output pin toggles, CNT resets — depending on the mode. The CPU is not involved; the peripheral manages everything in hardware.

That is the mental model to keep in view: a timer is just a counter plus rules about what to do when the counter reaches certain values.

The key equation:

```
f_interrupt = f_cpu / (prescaler × (TOP + 1))

Rearranged for TOP given a desired frequency:
  TOP = (f_cpu / (prescaler × f_desired)) − 1
```

Most timer configuration problems reduce to this equation plus careful attention to which register plays the role of TOP in the chosen mode.

11.1.2 Prescaler

The prescaler divides the CPU clock before it reaches the counter. Without it, a 16-bit counter at 20 MHz would overflow in 3.2 ms — useful for microsecond timing but not for a 1 Hz blink. Available prescaler values on TCA0:

Prescaler	Divides by	Max period @ 3.333 MHz (16-bit counter)
DIV1	1	19.7 ms (65535 ticks)
DIV2	2	39.3 ms
DIV4	4	78.6 ms
DIV8	8	157.3 ms
DIV16	16	314.6 ms
DIV64	64	1.26 s
DIV256	256	5.03 s
DIV1024	1024	20.1 s

For a 1 Hz timer tick, `DIV1024` gives 20 seconds of headroom with a 16-bit counter — plenty of room.

So the prescaler is what lets one timer peripheral cover both fast and slow timescales.

11.2 TCA0 on ATtiny3217

ATtiny3217 has one Timer/Counter Type A (TCA0) and two Timer/Counter Type B (TCB0, TCB1). TCA0 is the most versatile:

- 16-bit counter (CNT)
- Three compare channels (CMP0, CMP1, CMP2) each with an output pin option
- Three waveform generation modes
- Single mode (16-bit) or split mode (two independent 8-bit timers)

This chapter uses **single mode** (the default).

That is the easiest mode to reason about because the whole 16-bit counter stays together as one timer.

11.2.1 TCA0 Register Map

Register	Address	Width	Purpose
TCA0_SINGLE_CTRLA	0x0A00	8	Prescaler, enable
TCA0_SINGLE_CTRLB	0x0A01	8	Waveform generation mode, compare enable
TCA0_SINGLE_CTRLC	0x0A02	8	Compare output value override
TCA0_SINGLE_CTRLD	0x0A03	8	Split mode enable (leave 0 for single)
TCA0_SINGLE_CTRLECLR	0x0A04	8	Command/update bits clear side
TCA0_SINGLE_CTRLESET	0x0A05	8	Command/update bits set side
TCA0_SINGLE_CTRLFCLR	0x0A06	8	Buffer-valid flags clear side
TCA0_SINGLE_CTRLFSET	0x0A07	8	Buffer-valid flags set side
Reserved	0x0A08	8	Reserved
TCA0_SINGLE_EVCTRL	0x0A09	8	Event control
TCA0_SINGLE_INTCTRL	0x0A0A	8	Interrupt enable bits
TCA0_SINGLE_INTFLAGS	0x0A0B	8	Interrupt flags (W1C)
TCA0_SINGLE_DBGCTRL	0x0A0E	8	Debug run override
TCA0_SINGLE_TEMP	0x0A0F	8	Temporary bits for 16-bit access
TCA0_SINGLE_CNT	0x0A20	16	Current counter value (L at 0x20, H at 0x21)
TCA0_SINGLE_PER	0x0A26	16	Period (TOP in normal/PWM mode)
TCA0_SINGLE_CMP0	0x0A28	16	Compare 0 (also TOP in FRQ mode)
TCA0_SINGLE_CMP1	0x0A2A	16	Compare 1
TCA0_SINGLE_CMP2	0x0A2C	16	Compare 2

```
TCA0_SINGLE_PERBUF    0x0A36    16    Period buffer (double-buffered update)
TCA0_SINGLE_CMP0BUF   0x0A38    16    CMP0 buffer
TCA0_SINGLE_CMP1BUF   0x0A3A    16    CMP1 buffer
TCA0_SINGLE_CMP2BUF   0x0A3C    16    CMP2 buffer
```

All TCA0 registers are outside the I/O space (addresses ≥ 0x0060), so they require `LDS`/`STS` — not `IN`/`OUT`. The 16-bit registers use two consecutive bytes: the L suffix register at the lower address must be read or written first (low byte first for 16-bit accesses on AVR).

That means timer code is one of the places where the full memory-mapped peripheral model becomes very visible.

11.2.2 CTRLA — Control A

```
Bit:   7:4       3:1       0
       ─────     ──────    ──────
       reserved  CLKSEL    ENABLE

CLKSEL[3:1]:
  TCA_SINGLE_CLKSEL_DIV1_gc    = (0x00 << 1)  prescaler /1
  TCA_SINGLE_CLKSEL_DIV2_gc    = (0x01 << 1)  prescaler /2
  TCA_SINGLE_CLKSEL_DIV4_gc    = (0x02 << 1)  prescaler /4
  TCA_SINGLE_CLKSEL_DIV8_gc    = (0x03 << 1)  prescaler /8
  TCA_SINGLE_CLKSEL_DIV16_gc   = (0x04 << 1)  prescaler /16
  TCA_SINGLE_CLKSEL_DIV64_gc   = (0x05 << 1)  prescaler /64
  TCA_SINGLE_CLKSEL_DIV256_gc  = (0x06 << 1)  prescaler /256
  TCA_SINGLE_CLKSEL_DIV1024_gc = (0x07 << 1)  prescaler /1024

ENABLE (bit 0):
  1 = timer running
  0 = timer stopped (CNT holds its value)
```

Write `ENABLE` last after configuring the mode and TOP — avoids glitches where the timer runs briefly with incorrect settings.

This is a recurring embedded pattern: configure first, enable last.

11.2.3 CTRLB — Control B

```
Bit:   7      6      5      4      3      2:0
       —      CMP2EN CMP1EN CMP0EN ALUPD  WGMODE[2:0]

CMP2EN/CMP1EN/CMP0EN: 1 = compare output drives the associated pin
ALUPD: auto-lock update of double-buffered registers on OVF/TOP

WGMODE[2:0]:
  TCA_SINGLE_WGMODE_NORMAL_gc   = 0x00  Normal — count to PER, interrupt on OVF
  TCA_SINGLE_WGMODE_FRQ_gc      = 0x01  Frequency — count to CMP0, toggle pin,
                                         interrupt on CMP0
  0x02                                 Reserved
  TCA_SINGLE_WGMODE_SINGLESLOPE_gc = 0x03  PWM — count to PER, duty via CMPn
  TCA_SINGLE_WGMODE_DSTOP_gc    = 0x05  Dual-slope PWM (top)
  TCA_SINGLE_WGMODE_DSBOTH_gc   = 0x06  Dual-slope (top + bottom)
  TCA_SINGLE_WGMODE_DSBOTTOM_gc = 0x07  Dual-slope (bottom)
```

11.2.4 INTCTRL and INTFLAGS

```
Bit:   6      5      4      0
       CMP2   CMP1   CMP0   OVF

INTCTRL: writing 1 to a bit enables that interrupt.
INTFLAGS: bit set when interrupt fires. Clear by writing 1 (W1C).

TCA_SINGLE_OVF_bm  = 0x01   Overflow / Underflow
TCA_SINGLE_CMP0_bm = 0x10   Compare 0 match
TCA_SINGLE_CMP1_bm = 0x20   Compare 1 match
TCA_SINGLE_CMP2_bm = 0x40   Compare 2 match
```

The TCA interrupt flag bits can be set even before the matching interrupt is enabled. An interrupt request is generated only when the enable bit in `INTCTRL` is 1 and the corresponding flag in `INTFLAGS` is set. The request remains active until the flag is cleared by writing 1 to that flag bit.

11.3 Waveform Generation Modes

11.3.1 Normal Mode (WGMODE = 0x00)

```
CNT: 0──────────────────────────────────────────────PER──▶ 0
                                                       ↑
                                               OVF interrupt fires,
                                               CNT resets to 0
```

CNT counts from 0 to PER (the TOP value), then wraps to 0 and fires the **OVF** (overflow) interrupt. Use this mode when you need an interrupt at a fixed period and don't need compare-match outputs.

Normal mode is the simplest conceptual model: count up, hit the top, wrap, raise the event.

Interrupt frequency:

```
f_ovf = f_cpu / (prescaler × (PER + 1))
```

Hardware behavior: increment `CNT`, compare it with `PER`, raise the OVF condition at TOP, then restart from zero.

11.3.2 FRQ Mode — Frequency Generation (WGMODE = 0x01)

```
CNT: 0─────────────CMP0──▶ 0─────────────CMP0──▶ 0 ...
                     ↑                      ↑
               CMP0 interrupt         CMP0 interrupt
               (if WO0 pin set, it toggles here too)
```

CNT counts from 0 to **CMP0** (not PER), then resets. The **CMP0** interrupt fires on every reset. If the CMP0 output pin is enabled (CTRLB.CMP0EN = 1), WO0 (Wave Output 0) toggles on every compare match, producing a square wave. That square wave completes one full high-low cycle every two compare matches.

FRQ mode is the simplest way to get a periodic interrupt: - No separate PER register needed — CMP0 is both the match value and the TOP. - One register write sets both the period and the interrupt rate. - PER is unused (or can be set to 0xFFFF — it has no effect in FRQ mode).

- CMP1 and CMP2 may still compare, but because CNT resets at CMP0, they only make sense when their compare value is lower than CMP0.

Interrupt frequency:

```
f_cmp0 = f_cpu / (prescaler × (CMP0 + 1))
```

This is the mode used in the chapter example.

If your job is "interrupt me every fixed interval," FRQ mode is often the most direct fit.

11.3.3 Single-Slope PWM Mode (WGMODE = 0x03)

```
CNT: 0──────────────────────────────────────────PER──► 0
           ↑                           ↑         ↑
        CMP0 match                  CMP0 match  OVF
        (pin goes low)              (output     (pin goes high,
                                    low again)   OVF interrupt)

WO0 output: ████████████████░░░░░░░░░░░░░░░░████████...
                                  ↑              ↑
                                 CMP0          PER (OVF)
Duty cycle = CMP0 / PER
```

CNT counts to PER. WO0 is high from 0 to CMP0, then low from CMP0 to PER. Duty cycle is `CMP0 / PER`. OVF interrupt fires when CNT wraps. This is the standard PWM mode for motor control, LED dimming, etc.

PWM mode is where the timer stops being just a scheduler and starts being a waveform generator.

PWM frequency:

```
f_pwm = f_cpu / (prescaler × (PER + 1))
Duty  = CMP0 / PER  (0.0 = 0%, 1.0 = 100%)
```

11.4 Computing Timer Values

For a 1 Hz interrupt from a 3.333 MHz CPU using FRQ mode and prescaler 1024:

```
TOP = (f_cpu / (prescaler × f_desired)) − 1
    = (3333333 / (1024 × 1)) − 1
    = 3255.2 − 1
    = 3254.2  →  round to 3254

Actual frequency check:
  f = 3333333 / (1024 × (3254 + 1))
    = 3333333 / 3332096
    = 1.000371 Hz  (error < 0.04%)
```

The 16-bit value 3254 in hex is `0x0CB6`:

```
3254 = 0x0CB6
  lo8(3254) = 0xB6
  hi8(3254) = 0x0C
```

Low byte must be written first. The ATtiny3217 uses a 16-bit write latch: writing CMP0L latches the value, and writing CMP0H completes the transfer to the 16-bit register atomically. Reversing the order produces a glitch where the intermediate (wrong) value is briefly seen by the timer.

This is not stylistic ceremony. The write order is part of the hardware interface contract.

```
ldi  r16, lo8(3254)
sts  TCA0_SINGLE_CMP0L, r16    /* writes 0xB6, latches */
ldi  r16, hi8(3254)
sts  TCA0_SINGLE_CMP0H, r16    /* writes 0x0C, transfers 0x0CB6 atomically */
```

11.5 TCA0 Interrupt Vectors

From the ATtiny3217 vector table (Chapter 10):

Byte addr	Vector	Peripheral	Trigger
0x0010	8	TCA0_OVF	Overflow (normal/PWM modes)
0x0012	9	TCA0_HUNF	Split mode high-byte underflow
0x0014	10	TCA0_CMP0	Compare 0 match
0x0016	11	TCA0_CMP1	Compare 1 match
0x0018	12	TCA0_CMP2	Compare 2 match

In FRQ mode, the **TCA0_CMP0** interrupt (vector 10, address 0x0014) fires on every compare match. FRQ mode also reaches TOP at CMP0, so the OVF flag can be set at TOP, but this example does not enable the OVF interrupt. It enables only CMP0 and clears only CMP0.

So the interrupt source depends on the chosen mode, not just on the peripheral name.

To place `tca0_cmp0_isr` at 0x0014, the vector table needs `rjmp tca0_cmp0_isr` as vector entry 10 (counting RESET as entry 0):

```
__vectors:
    rjmp  main           /* 0x0000: RESET     */
    rjmp  isr_default    /* 0x0002: CRCSCAN   */
    rjmp  isr_default    /* 0x0004: BOD       */
    rjmp  isr_default    /* 0x0006: PORTA     */
    rjmp  isr_default    /* 0x0008: PORTB     */
    rjmp  isr_default    /* 0x000A: PORTC     */
    rjmp  isr_default    /* 0x000C: RTC_CNT   */
    rjmp  isr_default    /* 0x000E: RTC_PIT   */
    rjmp  isr_default    /* 0x0010: TCA0_OVF  */
    rjmp  isr_default    /* 0x0012: TCA0_HUNF */
    rjmp  tca0_cmp0_isr  /* 0x0014: TCA0_CMP0 ← */
```

11.6 The ISR

The TCA0 CMP0 ISR follows the same prologue/epilogue discipline from Chapter 10. It adds one step: **clear the INTFLAGS bit before returning**, or the interrupt fires again immediately.

```
tca0_cmp0_isr:
    push  r16
    in    r16, SREG
    push  r16

    ldi   r16, (1 << LED_BIT)
    sts   PORTA_OUTTGL, r16         /* toggle LED0 on PA3 (active low)      */

    ldi   r16, TCA_SINGLE_CMP0_bm   /* = 0x10                               */
    sts   TCA0_SINGLE_INTFLAGS, r16 /* clear CMP0 flag (W1C)                */

    pop   r16
    out   SREG, r16
    pop   r16
    reti
```

`PORTA_OUTTGL` is a toggle register — writing a 1 to bit n toggles PORTAn without affecting other bits. It is at address 0x0407 (outside I/O space, requires STS). Writing `(1 << LED_BIT)` = `0x08` toggles only PA3.

`TCA0_SINGLE_INTFLAGS` uses W1C semantics: writing `TCA_SINGLE_CMP0_bm` (bit 4 = 1) clears only the CMP0 flag. If you needed to clear all flags simultaneously: `ldi r16, 0xFF; sts TCA0_SINGLE_INTFLAGS, r16`.

As with every interrupt source on AVR, forgetting to clear the flag turns one interrupt into an endless interrupt loop.

11.7 Main: Timer Initialization Sequence

Order matters. The safe sequence is:

```
1. Set CTRLB (waveform mode) — timer is not running yet
2. Write CMP0L then CMP0H — latch and load TOP value
3. Set INTCTRL — enable the interrupt we want
4. Set CTRLA — prescaler + enable bit (starts the timer)
5. SEI — enable global interrupts
```

Starting the timer before configuring CMP0 would cause the timer to run briefly with CNT=0 and CMP0=0, firing immediately. Starting before INTCTRL is set means the first match is missed silently (no interrupt enabled yet).

Initialization order matters because hardware begins reacting as soon as the enable bit is set.

```
main:
    ldi   r16, hi8(RAMEND)
    out   SPH, r16
    ldi   r16, lo8(RAMEND)
    out   SPL, r16

    sbi   VPORTA_DIR, LED_BIT       /* PA3 = output (LED0 on Curiosity Nano) */
    sbi   VPORTA_OUT, LED_BIT       /* LED off initially (active low: HIGH = off) */

    /* 1. Waveform mode */
    ldi   r16, TCA_SINGLE_WGMODE_FRQ_gc
```

```
    sts   TCA0_SINGLE_CTRLB, r16

    /* 2. TOP value: CMP0 = 3254 */
    ldi   r16, lo8(3254)
    sts   TCA0_SINGLE_CMP0L, r16
    ldi   r16, hi8(3254)
    sts   TCA0_SINGLE_CMP0H, r16

    /* 3. Enable CMP0 interrupt */
    ldi   r16, TCA_SINGLE_CMP0_bm
    sts   TCA0_SINGLE_INTCTRL, r16

    /* 4. Start timer: prescaler 1024, enable */
    ldi   r16, (TCA_SINGLE_CLKSEL_DIV1024_gc | TCA_SINGLE_ENABLE_bm)
    sts   TCA0_SINGLE_CTRLA, r16

    /* 5. Global interrupt enable */
    sei

.Lidle:
    rjmp  .Lidle
```

11.8 Worked Example: 1 Hz Timer Tick

Full source: `src/timer_ctc.S`

The complete program combines the vector table, the CMP0 ISR, and main into one file. No shared state is needed between main and the ISR — main initializes hardware and parks in the idle loop; the ISR does all the work.

That makes this example a good demonstration of timer-driven design: main sets the policy, the timer peripheral generates the schedule, and the ISR performs the periodic action.

11.8.1 Register Usage

```
r16:  scratch — used in prologue/epilogue and ISR body
      (saved/restored each invocation)
```

No other registers needed. The simplest possible ISR.

LED0 on Curiosity Nano is active low (PA3 high = LED off, PA3 low = LED on). `PORTA_OUTTGL` toggles PA3 on every CMP0 match. With a 1 Hz interrupt, the LED changes state once per second, so a complete on-off cycle takes about two seconds.

11.8.2 Timing Analysis

```
Event                           Cycles
─────────────────────────────────────────────────────────
TCA0 CMP0 match (hardware)       0
Interrupt request to CPU         1–2 (sync)
CPU finishes current RJMP        2
Push PC, enter CPUINT ISR state, fetch vector  3–5
```

```
RJMP tca0_cmp0_isr              1
Total to ISR first instruction  ~7–9

ISR prologue (PUSH + IN + PUSH)  6 cycles (2+1+2+1)
Toggle (LDI + STS)              3 cycles (1+2)
Clear flag (LDI + STS)          3 cycles (1+2)
ISR epilogue (POP + OUT + POP + RETI)  9 cycles
Total ISR duration              ~21 cycles

At 3.333 MHz: 21 cycles × 300 ns ≈ 6.3 µs ISR runtime per tick
```

The timer period is 3333333 / (1024 × 3255) ≈ 1.0 s. The ISR runtime (~6.3 µs) is 0.00063% of the period — nearly zero CPU overhead for the tick.

This is exactly why timers are so valuable. The CPU only pays for the tiny ISR runtime, not for the whole one-second wait.

11.8.3 Build Commands

```
avr-gcc -mmcu=attiny3217 -x assembler-with-cpp -c -o timer_ctc.o src/timer_ctc.S
avr-gcc -mmcu=attiny3217 -nostartfiles -o timer_ctc.elf timer_ctc.o
avr-objcopy -O ihex timer_ctc.elf timer_ctc.hex

# Inspect disassembly
avr-objdump -d timer_ctc.elf
```

11.8.4 Verifying with avr-objdump

```
Expected output (excerpts):
  00000000 <__vectors>:
     0:  XX XX  rjmp .+N   ; → main
     4:  ...
    14:  XX XX  rjmp .+N   ; → tca0_cmp0_isr   (TCA0_CMP0 vector at 0x0014)

  <tca0_cmp0_isr>:
    push r16
    in   r16, 0x3f         ; SREG = 0x3f in I/O space
    push r16
    ldi  r16, 0x08         ; (1 << 3) = PA3
    sts  0x0407, r16       ; PORTA_OUTTGL
    ldi  r16, 0x10         ; TCA_SINGLE_CMP0_bm
    sts  0x0a0b, r16       ; TCA0_SINGLE_INTFLAGS
    pop  r16
    out  0x3f, r16         ; restore SREG
    pop  r16
    reti
```

Check that `tca0_cmp0_isr` is at the address that vector 0x0014 points to.

11.9 Extending: Normal Mode with OVF Interrupt

For cases where you want a fixed period and don't need compare-match output, use Normal mode with the OVF interrupt. Change two registers:

```
/* Normal mode — OVF fires when CNT reaches PER */
ldi   r16, TCA_SINGLE_WGMODE_NORMAL_gc   /* = 0x00 */
sts   TCA0_SINGLE_CTRLB, r16

ldi   r16, lo8(3254)
sts   TCA0_SINGLE_PERL, r16             /* PER (not CMP0) */
ldi   r16, hi8(3254)
sts   TCA0_SINGLE_PERH, r16

ldi   r16, TCA_SINGLE_OVF_bm            /* = 0x01: overflow interrupt */
sts   TCA0_SINGLE_INTCTRL, r16
```

The ISR changes slightly — clear OVF instead of CMP0:

```
tca0_ovf_isr:
    push  r16
    in    r16, SREG
    push  r16

    ldi   r16, (1 << LED_BIT)
    sts   PORTA_OUTTGL, r16

    ldi   r16, TCA_SINGLE_OVF_bm       /* = 0x01 */
    sts   TCA0_SINGLE_INTFLAGS, r16    /* clear OVF flag */

    pop   r16
    out   SREG, r16
    pop   r16
    reti
```

And the vector changes to entry 8 (0x0010 — TCA0_OVF):

```
    rjmp  tca0_ovf_isr   /* 0x0010: TCA0_OVF */
    rjmp  isr_default    /* 0x0012: TCA0_HUNF */
    rjmp  isr_default    /* 0x0014: TCA0_CMP0 */
```

Both modes produce identical 1 Hz interrupt behavior. FRQ mode is marginally simpler (no PER register to configure separately).

That kind of tradeoff is common in timer work: several modes can solve the same problem, but one usually fits the mental model more directly.

11.10 Extending: Single-Slope PWM

To dim an LED with 50% duty cycle at ~16.3 kHz using PA0 (WO0 pin):

```
/* PER = 204 → f_pwm = 3333333 / (1 × 205) ≈ 16.26 kHz */
/* CMP0 = 102 → duty = 102/204 ≈ 50% */
```

```
ldi   r16, TCA_SINGLE_WGMODE_SINGLESLOPE_gc   /* CTRLB: SS-PWM + CMP0EN */
ori   r16, TCA_SINGLE_CMP0EN_bm
sts   TCA0_SINGLE_CTRLB, r16

ldi   r16, lo8(204)
sts   TCA0_SINGLE_PERL, r16
ldi   r16, hi8(204)
sts   TCA0_SINGLE_PERH, r16

ldi   r16, lo8(102)
sts   TCA0_SINGLE_CMP0L, r16
ldi   r16, hi8(102)
sts   TCA0_SINGLE_CMP0H, r16

/* Start timer with prescaler 1 (no division) */
ldi   r16, (TCA_SINGLE_CLKSEL_DIV1_gc | TCA_SINGLE_ENABLE_bm)
sts   TCA0_SINGLE_CTRLA, r16
```

No interrupt needed for PWM — the hardware toggles WO0 automatically. The PA0 pin must be set as output (SBI VPORTA_DIR, 0) before enabling the timer output, or WO0 is disconnected from the pin.

This is an important milestone: once PWM is configured, the peripheral can keep producing the waveform even if the CPU goes off and does something unrelated.

To change duty cycle at runtime, write a new value to CMP0BUF (the buffer). The ALUPD bit in CTRLB causes the buffer to transfer to CMP0 on the next OVF, preventing glitches mid-cycle.

11.11 Summary

```
TCA0 modes:
  NORMAL  (0x00): CNT → PER → reset; OVF interrupt.
  FRQ     (0x01): CNT → CMP0 → reset; CMP0 interrupt; WO0 toggles (if enabled).
  SS-PWM  (0x03): CNT → PER → reset; WO0 high from 0→CMP0, low CMP0→PER; OVF int.

Frequency formula:
  f = f_cpu / (prescaler × (TOP + 1))
  TOP = (f_cpu / (prescaler × f)) − 1

16-bit register write order:  LOW byte first, then HIGH byte (latch).

Initialization order:
  1. CTRLB (mode)   2. CMP0/PER (TOP)   3. INTCTRL   4. CTRLA (start)   5. SEI

Interrupt flag clear: write TCA_SINGLE_CMP0_bm (or TCA_SINGLE_OVF_bm) to TCA0_SINGLE_INTFLAGS
  before RETI — or interrupt fires again immediately.

ISR discipline: identical to Chapter 10.
  PUSH all used regs → IN/PUSH SREG → body → POP/OUT SREG → POP regs → RETI.

ATtiny3217 TCA0 vectors:
```

```
0x0010  TCA0_OVF   — overflow
0x0014  TCA0_CMP0  — compare 0 match (FRQ mode trigger)
0x0016  TCA0_CMP1
0x0018  TCA0_CMP2
```

11.12 Exercises

1. The example uses prescaler 1024 and CMP0 = 3254. Calculate the CMP0 value needed for a 2 Hz blink with the same prescaler. What is the exact frequency error as a percentage?
2. Modify the example to blink the LED three times fast (150 ms on, 150 ms off) then pause for 700 ms. First reconfigure the timer for a 150 ms tick, then use a counter variable in SRAM to step through the pattern.
3. Calculate PER and CMP0 values for a 1 kHz PWM signal on WO0 with 25% duty cycle, using prescaler DIV1 at 3.333 MHz. Write the complete initialization sequence in assembly.
4. The ISR clears `TCA0_SINGLE_INTFLAGS` with an STS instruction. STS is a 4-byte, 2-cycle instruction. TCA0_SINGLE_INTFLAGS is at 0x0A0B — not in the VPORT I/O range (0x00–0x1F), so SBI cannot be used here. Confirm this by checking whether 0x0A0B mapped to any VPORT address. What is the smallest instruction count you can use to clear the flag?
5. Add a second compare channel (CMP1) to the FRQ-mode example. CMP1 should fire at 2 Hz (while CMP0 still fires at 1 Hz). What constraint does FRQ mode place on CMP1's relationship to CMP0? (Hint: re-read the FRQ mode description — CNT resets at CMP0, not PER.)
6. Using `avr-objdump -d timer_ctc.elf`, verify that the `sts TCA0_SINGLE_CMP0L` instruction is encoded as a 4-byte instruction. What are the two 16-bit words of its encoding? (Refer to the AVR instruction set manual for the STS 32-bit format.)

Next: Chapter 12 — USART (Serial Communication)

Chapter 12

USART (Serial Communication)

USART (Universal Synchronous/Asynchronous Receiver/Transmitter) is the simplest way to communicate between a microcontroller and a PC. One wire transmits (TX), one receives (RX), and data flows as a timed sequence of bits. No clock line is needed — both sides agree on a baud rate (bits per second) in advance. This chapter covers the ATtiny3217's USART0 peripheral: register setup, polling transmit, polling receive, ISR-driven receive, and a complete echo terminal example.

This is often the first peripheral that makes a microcontroller feel "interactive" because it gives you a direct text path to a terminal.

12.1 UART Framing

12.1.1 The Bit Stream

USART uses NRZ (Non-Return-to-Zero) encoding: the line is high (idle) when no data is being sent. Each byte is framed with:

```
Idle: ─────────────┐
Start:              │ ← start bit (line goes low, 1 bit period)
                    └─ D0 ─ D1 ─ D2 ─ D3 ─ D4 ─ D5 ─ D6 ─ D7 ─┐
Stop:                                                          └──── (line high)

Timeline at 9600 baud (1 bit = 104.2 µs):
  │  1 bit  │  1 bit  │ ... 8 bits ... │  1 stop  │ idle ... │
  │  start  │  D0 LSB │                │  D7 MSB  │          │
```

One character transmission = 1 start + 8 data + 1 stop = **10 bit periods**. At 9600 baud: 10 × 104.2 µs = 1.042 ms per byte → maximum ~960 bytes/second.

That simple 10-bit framing cost is worth remembering. Even before software overhead, UART throughput is lower than the raw baud number suggests.

12.1.2 8N1 Format

`8N1` means: - **8** data bits (LSB first) - **N** — no parity bit - **1** stop bit

This is the most common UART configuration and the ATtiny3217 USART0 default after reset.

So for ordinary 8-bit serial text, the default framing is already the framing you want.

12.2 USART0 Registers on ATtiny3217

Register	Address	Purpose
USART0_RXDATAL	0x0800	Receive data low byte (read = receive byte)
USART0_RXDATAH	0x0801	Receive data high byte (error flags + 9th bit)
USART0_TXDATAL	0x0802	Transmit data (write = queue byte to send)
USART0_TXDATAH	0x0803	Transmit data high byte (9th bit when used)
USART0_STATUS	0x0804	Status flags (RXCIF, TXCIF, DREIF, etc.)
USART0_CTRLA	0x0805	Interrupt enables, RS-485 mode
USART0_CTRLB	0x0806	TX/RX enable, start-frame detect, RX mode
USART0_CTRLC	0x0807	Frame format: baud mode, char size, parity, stop bits
USART0_BAUDL	0x0808	Baud rate register low byte
USART0_BAUDH	0x0809	Baud rate register high byte
USART0_CTRLD	0x080A	Auto-baud width field
USART0_DBGCTRL	0x080B	Debug run override
USART0_EVCTRL	0x080C	IrDA event input enable
USART0_TXPLCTRL	0x080D	IrDA transmitter pulse length
USART0_RXPLCTRL	0x080E	IrDA receiver pulse length

All outside I/O space: require `LDS`/`STS`.

That means USART code on ATtiny3217 naturally uses the "full peripheral register" access style rather than the short `IN`/`OUT` style.

12.2.1 STATUS — Status Register (0x0804)

```
Bit:   7      6      5      4      3      2      1      0
       RXCIF  TXCIF  DREIF  RXSIF  ISFIF  —      BDF    WFB

RXCIF  – RX Complete Interrupt Flag: 1 when unread byte in RXDATAL.
           Cleared automatically by reading RXDATAL.
TXCIF  – TX Complete Interrupt Flag: 1 when last byte shifted out fully.
           Cleared by writing 1 (W1C) or enabling TXCIE.
DREIF  – Data Register Empty Flag: 1 when TXDATAL is ready for new byte.
           Cleared automatically when TXDATAL is written.
           On reset, DREIF = 1 (TX is ready immediately).
RXSIF  – Receive Start Interrupt Flag, used with start-frame detection.
ISFIF  – Inconsistent Sync Field Interrupt Flag, used with auto-baud/LIN modes.
BDF    – Break Detected Flag, used with break/sync detection.
WFB    – Wait For Break control/status bit.
```

Received-byte error flags are not in `STATUS`. They are stored with the received byte in `RXDATAH`: `BUFOVF` at bit 6, `FERR` at bit 2, `PERR` at bit 1, and `DATA[8]` at bit 0 for 9-bit frames. Read `RXDATAH` before `RXDATAL` when you need those error bits.

For polling TX: wait until `DREIF = 1`, then write `TXDATAL`. For polling RX: wait until `RXCIF = 1`, then read `RXDATAL`. For ISR-driven RX: enable `RXCIE` in CTRLA; ISR fires when `RXCIF` sets.

Those three sentences are the whole USART mental model in miniature: one flag for "ready to transmit," one flag for "data received," and one interrupt-enable bit if you want hardware to notify you instead of polling.

12.2.2 CTRLB — Control B (0x0806)

```
Bit:   7        6        5         4      3       2:1     0
       RXEN     TXEN     SFDEN     ODME   —       RXMODE  MPCM

RXEN   — 1 = receive enabled (USART monitors RXD pin)
TXEN   — 1 = transmit enabled (USART drives TXD pin)
SFDEN  — 1 = start-frame detection enabled
ODME   — 1 = open-drain mode enabled
RXMODE — 0 = normal (16x oversampling), 1 = CLK2X (8x), 2 = LINAUTO, 3 = GENAUTO
MPCM   — 1 = multiprocessor communication mode

Bit constants (from avr/io.h):
  USART_TXEN_bm  = 0x40
  USART_RXEN_bm  = 0x80
  USART_RXCIE_bm is in CTRLA (see below)
```

12.2.3 CTRLA — Control A / Interrupt Enables (0x0805)

```
Bit:   7       6       5       4       3      2      1:0
       RXCIE   TXCIE   DREIE   RXSIE   LBME   ABEIE  RS485

RXCIE  — RX Complete Interrupt Enable: fires ISR when RXCIF = 1
TXCIE  — TX Complete Interrupt Enable
DREIE  — Data Register Empty Interrupt Enable
RXSIE  — Receive Start Interrupt Enable
LBME   — Loop-back Mode Enable
ABEIE  — Auto-baud Error Interrupt Enable

Bit constants:
  USART_RXCIE_bm = 0x80
  USART_TXCIE_bm = 0x40
  USART_DREIE_bm = 0x20
```

12.2.4 CTRLC — Frame Format (0x0807)

```
Bit:   7:6      5:4      3      2:1    0
       CMODE    —        SBMODE PMODE  CHSIZE

CMODE (bits 7:6):  communication mode
  0x00 = USART_CMODE_ASYNCHRONOUS_gc — async (normal UART)
  0x01 = USART_CMODE_SYNCHRONOUS_gc  — sync
  0x02 = USART_CMODE_IRCOM_gc        — IrDA
  0x03 = USART_CMODE_MSPI_gc         — SPI master

SBMODE (bit 3):  stop bits
  0 = 1 stop bit
  1 = 2 stop bits

PMODE (bits 2:1):  parity
  0x00 = none
  0x02 = even
  0x03 = odd
```

```
CHSIZE:
  0x03 = 8-bit (USART_CHSIZE_8BIT_gc) — most common

Default after reset = 0x03 = async, 8-bit, no parity, 1 stop = 8N1.
No write needed if 8N1 is desired.
```

12.3 Baud Rate Calculation

12.3.1 The Formula

USART0 on ATtiny3217 uses a 16-bit baud register (BAUDL + BAUDH) with a fractional format. For **asynchronous normal mode** (16× oversampling):

```
BAUD_REG = (f_cpu × 64) / (16 × baud_rate)
         = (f_cpu × 4) / baud_rate
```

The × 64 and ÷ 16 come from the internal fractional divider format — the top 12 bits of BAUD_REG are the integer part, the bottom 4 bits are the fractional part (1/16 increments).

You do not need to memorize the internal fractional details to use the peripheral well, but they explain why the formula looks stranger than a simple clock-divider equation.

For 9600 baud at 3.333 MHz:

```
BAUD_REG = (3333333 × 64) / (16 × 9600)
         = 213333312 / 153600
         = 1388.88875
         → round to 1389 = 0x056D

Actual baud rate = (3333333 × 64) / (16 × 1389)
                 = 213333312 / 22224
                 = 9599.23 bps  (-0.008% error — negligible)
```

Common baud rates at 3.333 MHz:

Baud	BAUD_REG (decimal)	BAUD_REG (hex)	Error
9600	1389	0x056D	-0.01%
19200	694	0x02B6	0.06%
38400	347	0x015B	0.06%
57600	231	0x00E7	0.21%
115200	116	0x0074	-0.22%

Writing BAUD (low byte first):

```
ldi   r16, lo8(1389)     /* = 0x6D */
sts   USART0_BAUDL, r16
ldi   r16, hi8(1389)     /* = 0x05 */
sts   USART0_BAUDH, r16
```

As with other 16-bit peripheral values on AVR, the write order is part of the hardware interface, not just style.

12.4 TX/RX Pins on ATtiny3217

USART0 uses **PORTB** by default:

Function	Pin	Direction	Notes
TXD	PB2	Output	Must set VPORTB_DIR bit 2 = 1
RXD	PB3	Input	Leave as input (default); no pull-up needed

The USART peripheral drives TXD when TXEN=1, but only if the pin is configured as output. If PB2 is left as input, TXD is disconnected from the pin driver and the line floats.

That is an easy detail to miss: enabling the USART transmitter is not the same thing as configuring the port pin direction.

```
sbi   VPORTB_DIR, 2      /* PB2 = output for TXD */
/* PB3 (RXD) = input by default — no configuration needed */
```

Alternative pin mapping (PORTMUX_USARTROUTEA register) allows moving USART0 to other pins, but this chapter uses the default mapping.

12.5 Polling Transmit

The simplest TX approach: spin until DREIF = 1, then write the byte.

Polling transmit is usually acceptable because the program already knows when it wants to send something.

```
AVR Instruction: LDS Rd, A   — load from SRAM/peripheral
                 SBRS Rd, b  — skip next if bit b of Rd is set
```

```
/*
 * usart_tx_byte — transmit r16 via polling
 * Clobbers: r17
 */
usart_tx_byte:
    push  r17
.Lwait_dreif:
    lds   r17, USART0_STATUS
    sbrs  r17, USART_DREIF_bp    /* bit 5 — skip if DREIF = 1              */
    rjmp  .Lwait_dreif           /* DREIF = 0: TX busy, loop               */
    sts   USART0_TXDATAL, r16    /* write byte: clears DREIF, starts TX    */
    pop   r17
    ret
```

`USART_DREIF_bp` is the bit position (5) of DREIF in the STATUS register. `SBRS r17, 5` skips the `RJMP` when bit 5 is 1 (DREIF set = TX ready).

So the loop is doing exactly one question over and over: “is the transmit data register empty yet?”

12.5.0.1 Why Not SBIS?

`SBIS` and `SBIC` only work with I/O space registers (addresses 0x00–0x1F in I/O space, or 0x20–0x3F in data space). USART0_STATUS is at 0x0804 — far outside I/O space. Use `LDS` + `SBRS` for peripheral registers.

12.5.0.2 Cycle Cost of Polling

Each loop iteration: LDS (2 cy) + SBRS (1 cy, not skipping) + RJMP (2 cy) = 5 cycles. At 3.333 MHz, 5 cycles = 1.5 µs per iteration. At 9600 baud, one bit period is 104 µs — the polling loop runs ~69 times per bit period. Acceptable for simple programs; for power-sensitive applications, use DREIE interrupt instead.

That is the tradeoff in one paragraph: polling is simple, but it spends CPU time checking a condition that usually is not ready yet.

12.5.1 Transmitting a String from Flash

```
/*
 * usart_tx_string — send bytes from flash
 * ZL:ZH = pointer to first byte, r18 = length
 * Clobbers: r16, r17, r18, Z
 */
usart_tx_string:
    tst   r18
    breq  .Ltx_done
.Ltx_loop:
    lpm   r16, Z+              /* load byte from flash, post-increment Z   */
    rcall usart_tx_byte
    dec   r18
    brne  .Ltx_loop
.Ltx_done:
    ret
```

String data in flash:

```
.Lready_str:
    .byte 'R', 'e', 'a', 'd', 'y', '\r', '\n'
.equ READY_LEN, 7
.balign 2
```

Calling it:

```
ldi   ZL, lo8(.Lready_str)
ldi   ZH, hi8(.Lready_str)
ldi   r18, READY_LEN
rcall usart_tx_string
```

The `.byte` directive places raw bytes in the `.text` section at the label address. `LPM Z+` loads each byte from flash and advances Z. `READY_LEN` is a constant counting the bytes (7 = 5 chars + CR + LF). The `.balign 2` keeps the next instruction word-aligned after the 7-byte string.

This is a nice place where Chapter 5's flash-reading model becomes immediately useful in application code.

12.6 Polling Receive

Wait until `RXCIF = 1`, then read `RXDATAL`. Reading clears the flag.

Polling receive is conceptually symmetric with polling transmit, but it is much more wasteful in practice because the CPU does not know when the next incoming byte will arrive.

```
/*
 * usart_rx_byte — receive one byte into r16 (polling)
 * Clobbers: r17
 */
usart_rx_byte:
    push  r17
.Lwait_rxcif:
    lds   r17, USART0_STATUS
    sbrs  r17, USART_RXCIF_bp    /* bit 7 — skip if RXCIF = 1 (byte ready) */
    rjmp  .Lwait_rxcif
    lds   r16, USART0_RXDATAL    /* read byte — clears RXCIF               */
    pop   r17
    ret
```

`USART_RXCIF_bp` = 7 (bit position). `SBRS r17, 7` skips the loop branch when the RX byte is ready.

12.6.1 Error Checking

Before reading `RXDATAL`, the error flags in `STATUS` should be checked:

```
.Lwait_rxcif:
    lds   r17, USART0_STATUS
    sbrs  r17, USART_RXCIF_bp
    rjmp  .Lwait_rxcif
    lds   r17, USART0_RXDATAH       /* read high byte first for error flags */
    andi  r17, (USART_FERR_bm | USART_BUFOVF_bm | USART_PERR_bm)
    brne  .Lrx_error                /* jump to error handler if any flag set */
    lds   r16, USART0_RXDATAL       /* no error — read data                  */
```

`RXDATAH` contains the error flags (FERR, BUFOVF, PERR) and optionally the 9th data bit. For 8N1, only the error bits matter. `RXDATAH` must be read before `RXDATAL` — reading RXDATAL consumes the byte and discards the error flags.

That read order is another hardware contract: get the status context first, then consume the byte.

For the simple echo example, error checking is omitted for clarity.

12.7 ISR-Driven Receive

Polling TX is fine for transmit — the CPU controls when it sends. But polling RX is wasteful: the CPU spins doing nothing until a byte arrives, unable to perform other work. The RXCIE interrupt solves this: the ISR fires only when a byte is ready, and main code runs uninterrupted between bytes.

This is one of the clearest examples of when interrupts are worth the added complexity.

12.7.1 USART0_RXC Vector

From the ATtiny3217 vector table (Chapter 10):

Byte addr	Vector	Peripheral	Trigger

```
0x0036     27        USART0_RXC         RXCIF = 1 (byte received)
0x0038     28        USART0_DRE         DREIF = 1 (TX data register empty)
0x003A     29        USART0_TXC         TXCIF = 1 (TX shift register empty)
```

The USART0_RXC vector is at byte address 0x0036. In the vector table, it is the 28th entry (counting RESET as entry 0):

```
__vectors:
    rjmp  main          /* 0x0000: entry 0  RESET    */
    ...                 /* entries 1–26              */
    rjmp  usart0_rxc_isr /* 0x0036: entry 27 USART0_RXC */
    rjmp  isr_default    /* 0x0038: entry 28 USART0_DRE */
    rjmp  isr_default    /* 0x003A: entry 29 USART0_TXC */
    rjmp  isr_default    /* 0x003C: entry 30 NVMCTRL_EE */
```

12.7.2 RXC ISR

Reading `RXDATAL` inside the ISR **automatically clears `RXCIF`**. No explicit flag clear is needed (unlike TCA0 INTFLAGS which requires W1C).

That makes USART receive interrupts a little cleaner than many timer or GPIO interrupt sources.

```
usart0_rxc_isr:
    push  r16
    push  r17
    in    r16, SREG
    push  r16

    lds   r16, USART0_RXDATAL    /* read byte – clears RXCIF automatically   */
    rcall usart_tx_byte          /* echo it back (polling TX)                */

    pop   r16
    out   SREG, r16
    pop   r17
    pop   r16
    reti
```

Note `r17` is pushed/popped because `usart_tx_byte` clobbers it. The ISR calls a subroutine — this is legal but requires that the callee's clobbers are also saved in the prologue.

The important rule is local and mechanical: if an ISR calls a helper, the ISR must also preserve any registers the helper uses.

12.7.2.1 Calling Subroutines from ISRs

Calling `RCALL` inside an ISR pushes another 2 bytes onto the stack. The total stack depth for this ISR:

```
On ISR entry (CPU pushes return addr):   2 bytes
PUSH r16:                                1 byte
PUSH r17:                                1 byte
PUSH r16 (SREG):                         1 byte
RCALL usart_tx_byte (CPU pushes addr):   2 bytes
PUSH r17 (in usart_tx_byte):             1 byte
------------------------------------------------
Total stack depth:                        8 bytes
```

On ATtiny3217, SRAM is 2 KB. Typical use of a few hundred bytes of stack is safe, but deep call trees inside ISRs require careful accounting.

So ISR design is not just about correctness and latency; it is also about bounded stack growth.

12.7.2.2 Blocking TX Inside an ISR

The ISR calls `usart_tx_byte`, which polls `DREIF`. While polling, the ISR is still active, so another USART0_RXC interrupt will not run until `RETI`. This is safe here because at 9600 baud the TX wait is at most one byte period (~1.042 ms) and no other time-critical ISR is competing. If more bytes arrive while the ISR is blocked, they wait in the receive path; if software does not drain them in time, `BUFOVF` is reported with the received frame. In higher-throughput designs, use a circular buffer: the RXC ISR puts the byte in SRAM, the DREIE interrupt sends from the buffer, and the RXC ISR returns immediately.

That distinction is important: a simple echo ISR is fine for a teaching example, but blocking inside an ISR does not scale well.

12.8 Initialization Sequence

```
main:
    /* Stack */
    ldi   r16, hi8(RAMEND)
    out   SPH, r16
    ldi   r16, lo8(RAMEND)
    out   SPL, r16

    /* TXD pin = output */
    sbi   VPORTB_DIR, TXD_BIT        /* PB2 */

    /* Baud rate: 9600 at 3.333 MHz → BAUD_REG = 1389 = 0x056D */
    ldi   r16, BAUD_LO               /* 0x6D */
    sts   USART0_BAUDL, r16
    ldi   r16, BAUD_HI               /* 0x05 */
    sts   USART0_BAUDH, r16

    /* CTRLC defaults = 8N1 async — no write needed */

    /* Enable TX + RX */
    ldi   r16, (USART_TXEN_bm | USART_RXEN_bm)
    sts   USART0_CTRLB, r16          /* TXEN + RXEN */
    /* RXCIE lives in CTRLA — separate write */
    ldi   r16, USART_RXCIE_bm
    sts   USART0_CTRLA, r16

    sei

    /* Send startup message */
    ldi   ZL, lo8(.Lready_str)
    ldi   ZH, hi8(.Lready_str)
    ldi   r18, READY_LEN
    rcall usart_tx_string
```

```
.Lidle:
    rjmp  .Lidle
```

Note on CTRLB vs CTRLA: `TXEN` and `RXEN` are in CTRLB (0x0806). `RXCIE` is in CTRLA (0x0805). They are separate registers — two STS writes.

This is exactly the sort of small register-detail bug that can waste a lot of debug time if you assume all enable bits live together.

12.9 Worked Example: Echo Terminal

Full source: `src/usart_echo.S`

The program transmits `Ready\r\n` on startup, then echoes every received byte back to the sender. No main-loop processing — all receive logic is in the ISR.

That makes the data path very easy to reason about: receive a byte, trigger the ISR, send the same byte back.

12.9.1 Data Flow

```
PC terminal → USB-UART adapter → RXD (PB3) → USART0_RXDATAL
                                 RXCIF = 1  → USART0_RXC ISR fires
                                              reads RXDATAL (clears RXCIF)
                                              calls usart_tx_byte(r16)
                                              polls DREIF → writes TXDATAL
TXD (PB2) → USB-UART adapter → PC terminal  (sees same char echoed back)
```

12.9.2 Build and Test

```
# Build
avr-gcc -mmcu=attiny3217 -x assembler-with-cpp -c -o usart_echo.o src/usart_echo.S
avr-gcc -mmcu=attiny3217 -nostartfiles -o usart_echo.elf usart_echo.o
avr-objcopy -O ihex usart_echo.elf usart_echo.hex

# Flash
avrdude -c serialupdi -p attiny3217 -P /dev/ttyUSB0 -U flash:w:usart_echo.hex

# Connect serial terminal at 9600 8N1
# Linux/macOS:
screen /dev/ttyUSB0 9600
# or:
minicom -b 9600 -D /dev/ttyUSB0
```

12.9.3 Disassembly Check

```
avr-objdump -d usart_echo.elf
```

Key things to verify: - Vector at 0x0036 (`rjmp usart0_rxc_isr`) — counts 27 entries from RESET. - `sts 0x0808, r16` for BAUDL, `sts 0x0809, r16` for BAUDH. - `sts 0x0806, r16` for

CTRLB, `sts 0x0805, r16` for CTRLA. - `lds r16, 0x0800` for RXDATAL inside the ISR. - `lds r17, 0x0804` for STATUS in the TX polling loop.

12.10 Circular Buffer for High-Throughput RX

The simple ISR copies one byte directly. At 115200 baud with rapid bursts, the CPU may not finish the TX echo before the next byte arrives — BUFOVF sets and bytes are lost.

A circular buffer decouples RX from TX:

```
SRAM layout:
  rx_buf:    .space 16       ; 16-byte circular buffer
  rx_head:   .space 1        ; write index (ISR writes here)
  rx_tail:   .space 1        ; read index (main reads here)
```

The ISR writes to `rx_buf[rx_head]` and increments `rx_head` (modulo 16). Main code checks `rx_head != rx_tail` to know a byte is ready, reads from `rx_buf[rx_tail]`, and increments `rx_tail`.

This is the standard way to decouple "data arrived now" from "main code is ready to process it now."

```
/* ISR stores byte in circular buffer */
usart0_rxc_isr:
    push  r16
    push  r17
    push  ZL
    push  ZH
    in    r16, SREG
    push  r16

    lds   r16, USART0_RXDATAL        /* read byte, clear RXCIF              */

    lds   r17, rx_head               /* load write index                    */
    ldi   ZL, lo8(rx_buf)
    ldi   ZH, hi8(rx_buf)
    add   ZL, r17                    /* Z = &rx_buf[rx_head]                */
    adc   ZH, __zero_reg__
    st    Z, r16                     /* store byte                          */

    inc   r17
    andi  r17, 0x0F                  /* modulo 16 (power-of-2 wrap)         */
    sts   rx_head, r17

    pop   r16
    out   SREG, r16
    pop   ZH
    pop   ZL
    pop   r17
    pop   r16
    reti
```

Full circular buffer implementation is left as Exercise 5.

12.11 Summary

```
USART0 pins (default):  TXD = PB2 (output), RXD = PB3 (input)

Baud rate register:
  BAUD_REG = (f_cpu × 64) / (16 × baud)  [async normal mode]
  At 3.333 MHz, 9600 bps → BAUD_REG = 1389 = 0x056D
  Write: BAUDL first (low byte), then BAUDH.

Registers to configure:
  USART0_BAUDL / BAUDH  — baud rate (write low byte first)
  USART0_CTRLC          — frame format (default = 8N1, no write needed)
  USART0_CTRLB          — TXEN_bm (0x40) | RXEN_bm (0x80)
  USART0_CTRLA          — RXCIE_bm (0x80) for ISR-driven RX

STATUS flags:
  DREIF (bit 5) — TX data register empty: 1 = ready for new byte
  RXCIF (bit 7) — RX complete: 1 = byte ready in RXDATAL
  Both checked with: LDS Rd, USART0_STATUS  then SBRS/SBRC

Polling TX:
  wait DREIF = 1 → STS USART0_TXDATAL, Rr → DREIF auto-clears

Polling RX:
  wait RXCIF = 1 → LDS Rd, USART0_RXDATAL → RXCIF auto-clears

ISR-driven RX (USART0_RXC, vector 27, address 0x0036):
  Enable: USART_RXCIE_bm in USART0_CTRLA
  ISR reads RXDATAL → clears RXCIF automatically
  No manual flag clear needed (unlike TCA0 INTFLAGS)

ISR rules: same as always.
  Save r16+SREG minimum. Save any register clobbered by callees.
  Stack depth = 2 (ret addr) + saved regs + callee frames.
```

12.12 Exercises

1. Calculate BAUD_REG for 115200 baud at 3.333 MHz. What is the percentage error? If the error exceeds 2%, the link is likely to be unreliable — is 115200 feasible at this clock speed?

2. The polling TX function checks DREIF each iteration. At 9600 baud and 3.333 MHz, how many poll iterations occur while waiting for a byte to finish transmitting? (Use the LDS+SBRS+RJMP cycle count.)

3. The ISR calls `usart_tx_byte`, which polls DREIF. During this polling, could a second USART0_RXC interrupt fire? Explain why or why not. What would happen to the second received byte?

4. Modify `usart_echo.S` to echo each byte with its hex representation. Receiving `A` (0x41) should transmit the string `41\r\n`. Write the `byte_to_hex` subroutine that converts r16 (0x00–0xFF) into two ASCII hex digits in r17 (high nibble) and r16 (low nibble).

5. Implement the 16-byte circular buffer described in Section 12.10. Write:
   - the SRAM variable declarations (`.bss` section)
   - the full ISR using the buffer
   - a `main_loop` that calls a `process_byte` subroutine for each byte in the buffer (use `rx_head != rx_tail` as the ready check)
6. Using `avr-objdump -d usart_echo.elf`, locate the `LDS r17, USART0_STATUS` instruction in `usart_tx_byte`. What is its 32-bit encoding? What address (0x0804) appears in which bytes of the instruction?

Next: Chapter 13 — SPI & TWI (I2C) Overview

Chapter 13

SPI & TWI (I2C) Overview

Serial communication protocols let microcontrollers talk to sensors, memory chips, displays, and other devices using only a handful of wires. This chapter covers the two most common synchronous serial buses used in embedded systems: SPI (Serial Peripheral Interface) and TWI (Two-Wire Interface, Microchip's name for I²C). Both are present on the ATtiny3217 as hardware peripherals; both are accessed through memory-mapped registers using `LDS`/`STS`, exactly as with USART0 in Chapter 12.

This chapter is less about talking to a terminal and more about talking to other chips.

13.1 SPI Fundamentals

SPI is a full-duplex, synchronous serial bus. Four wires connect one master to one or more slave devices:

```
Master (ATtiny3217)         Slave (25LC010A EEPROM)
───────────────────         ───────────────────────
MOSI (PA1) ──────────────→  SI   (Master Out Slave In)
MISO (PA2) ←──────────────  SO   (Master In Slave Out)
SCK  (PA3) ──────────────→  SCK  (clock, generated by master)
CS   (PA4) ──────────────→  CS̄   (chip select, active-low)
```

Full duplex: every clock pulse simultaneously shifts one bit out on MOSI and samples one bit in on MISO. An 8-bit transfer is 8 clock pulses. If the slave has nothing meaningful to return (a pure command), the byte received by the master is discarded (or a dummy byte 0xFF is sent to clock the slave's response out).

That full-duplex behavior is one of the first things that makes SPI feel different from UART. You are always sending and receiving at the same time, even if one direction is only carrying dummy data.

Chip select: SPI has no built-in addressing. Instead, the master drives CS low to select a particular slave before the transfer begins, then drives CS high after the transfer ends. Each slave needs its own CS line.

So SPI trades protocol simplicity for pin count: the bus itself is simple, but every slave needs a dedicated select signal.

13.1.1 SPI Clock Polarity and Phase (Mode)

The idle state of SCK and the edge on which data is sampled define four modes:

Mode	CPOL	CPHA	SCK idle	Data captured on
0	0	0	low	rising edge ← 25LC010A compatible
1	0	1	low	falling edge
2	1	0	high	falling edge
3	1	1	high	rising edge ← also 25LC010A compatible

The 25LC010A EEPROM used in this chapter works in Mode 0 or Mode 3. We use Mode 0 (CPOL=0, CPHA=0) — SCK idles low, data captured on the rising edge.

In practice, “which SPI mode?” is one of the first things you check in a datasheet when a device does not respond correctly.

On ATtiny3217, the default SPI0 pins are PA1=MOSI, PA2=MISO, PA3=SCK, and PA4=SS. The datasheet also lists alternate SPI0 positions on PORTC: PC2=MOSI, PC1=MISO, PC0=SCK, and PC3=SS, selected through PORTMUX. This chapter keeps the default PA pins and uses PA4 as a software-controlled chip select.

13.1.2 Timing Diagram (Mode 0)

```
CS̄   ─┐                                            ┌─
      │                                            │
      └────────────────────────────────────────────┘

SCK  ──┐ ┌─┐ ┌─┐ ┌─┐ ┌─┐ ┌─┐ ┌─┐ ┌─┐ ┌────────
       └─┘ └─┘ └─┘ └─┘ └─┘ └─┘ └─┘ └─┘
         7   6   5   4   3   2   1   0   ← bit number (MSB first)

MOSI ──X───────────────────────────────X──────────
         output bit 7 ... output bit 0

MISO ────X───────────────────────────────X────────
           input  bit 7 ... input  bit 0
```

Data is set up before the rising edge and sampled on the rising edge.

13.2 SPI0 Registers on the ATtiny3217

The ATtiny3217 SPI0 peripheral uses five memory-mapped registers:

Register	Address	Purpose
SPI0_CTRLA	0x0820	Master/slave, clock, data order, enable
SPI0_CTRLB	0x0821	Buffer mode, slave-select, SPI mode
SPI0_INTCTRL	0x0822	Interrupt enables
SPI0_INTFLAGS	0x0823	Transfer-complete flag (IF), write-collision (WRCOL)
SPI0_DATA	0x0824	Transmit / receive data register

13.2.1 SPI0_CTRLA — Control A

```
Bit:   7      6      5       4      3    2     1     0
```

```
      —     DORD   MASTER  CLK2X  —    PRESC[1:0]  ENABLE
```

Field	Bits	Description
DORD	6	Data order: 0 = MSB first (normal), 1 = LSB first
MASTER	5	1 = master mode, 0 = slave mode
CLK2X	4	Double the clock speed (halves effective prescaler)
PRESC	2:1	Clock prescaler: 00=/4, 01=/16, 10=/64, 11=/128
ENABLE	0	1 = enable SPI peripheral

At 3.333 MHz with PRESC=DIV4 and CLK2X=0: SCK = 3.333 MHz / 4 ≈ **833 kHz**. The 25LC010A maximum SCK is 10 MHz, so the default chapter setting is well inside the EEPROM limit. On this CPU clock, SPI0 can run as fast as f_CPU/2 ≈ 1.667 MHz by setting `CLK2X`.

13.2.2 SPI0_CTRLB — Control B

```
Bit:  7     6     5   4   3   2     1    0
      BUFEN BUFWR —   —   —   SSD   MODE[1:0]
```

Field	Bits	Description
BUFEN	7	Enable buffer mode (more complex; not used here)
BUFWR	6	Buffer write mode enable (buffer mode only)
SSD	2	Slave Select Disable — set to 1 when driving CS manually
MODE	1:0	SPI mode: 00=Mode0, 01=Mode1, 10=Mode2, 11=Mode3

SSD must be set when driving CS as a GPIO (as we do in this chapter). If SSD=0 and the hardware SS pin (PA4) is driven low by external logic, the ATtiny3217 will switch itself to slave mode — a common source of confusion.

This is one of those details that feels obscure until it breaks the whole bus.

13.2.3 SPI0_INTFLAGS — Interrupt / Status Flags (non-buffered mode)

```
Bit:  7     6    5   4   3   2   1    0
      IF    WRCOL —   —   —   —   —    —
```

Flag	Bit	Set when	Cleared by
IF	7	8-bit transfer complete	Reading SPI0_DATA
WRCOL	6	Write collision (SPI0_DATA written mid-transfer)	Reading SPI0_DATA

`IF` at bit position `SPI_IF_bp` (= 7). Poll with `SBRS`.

13.2.4 SPI0_DATA — Transmit / Receive

Reading SPI0_DATA after IF is set returns the received byte and clears IF. Writing SPI0_DATA (when IF is set or before the first transfer) queues the byte to be shifted out. Both transmit and receive share the same register address — write to transmit, read to receive.

13.3 SPI Initialisation (Master Mode)

Before any transfer, configure the three output pins, leave MISO as input, set SSD, set MODE, then enable as master.

That order is deliberate: get the pins and mode right first, then turn the peripheral on.

```
/* Pin directions: MOSI (PA1), SCK (PA3), CS (PA4) → output */
ldi   r16, (1 << 1) | (1 << 3) | (1 << 4)
out   VPORTA_DIR, r16

/* CS high (deselect) */
sbi   VPORTA_OUT, 4

/* SPI0_CTRLB: SSD=1 (manual CS), MODE=0 (SPI mode 0) */
ldi   r16, SPI_SSD_bm
sts   SPI0_CTRLB, r16

/* SPI0_CTRLA: master, DIV4 prescaler, MSB first, enable */
ldi   r16, (SPI_MASTER_bm | SPI_ENABLE_bm)
sts   SPI0_CTRLA, r16
```

`SPI_MASTER_bm = (1 << SPI_MASTER_bp) = 0x20`. `SPI_ENABLE_bm = (1 << SPI_ENABLE_bp) = 0x01`. Both are defined in <`avr/io.h`> for the ATtiny3217.

13.4 Transferring a Byte

The hardware handles the 8-bit shift; the CPU's job is to: 1. Write the byte to `SPI0_DATA` (starts the transfer). 2. Poll `SPI0_INTFLAGS` until `IF` = 1 (transfer done). 3. Read `SPI0_DATA` (clears `IF`; returns received byte).

Once you see SPI transfers this way, most device protocols reduce to "send a command byte, maybe send an address byte, maybe send or receive data bytes."

```
/*
 * spi_transfer — exchange one byte over SPI
 * In:  r16 = byte to send
 * Out: r16 = byte received
 * Clobbers: r17
 */
spi_transfer:
    sts   SPI0_DATA, r16        /* write tx byte → starts 8-bit transfer    */
.Lwait_if:
    lds   r17, SPI0_INTFLAGS
    sbrs  r17, SPI_IF_bp        /* spin until IF (bit 7) = 1                */
    rjmp  .Lwait_if
    lds   r16, SPI0_DATA        /* read received byte — clears IF            */
    ret
```

Cycle count at 3.333 MHz with SCK = f/4:

Phase	Duration
8 SPI bits	8 × 4 CPU cycles = 32 cycles (SCK)

Phase	Duration
Poll loop	typically 1–3 iterations = 6 cycles
`STS+LDS`	2 + 2 = 4 cycles
Total	≈ 42 CPU cycles (12.6 µs)

13.5 The 25LC010A SPI EEPROM

The 25LC010A is a 1-Kbit (128 × 8-bit) SPI EEPROM. It communicates using single-byte commands followed by optional address and data bytes.

13.5.1 Key Characteristics

Parameter	Value
Capacity	128 bytes (1 Kbit)
Address width	7 bits (0x00 – 0x7F)
Page size	16 bytes (for page write)
SPI modes	Mode 0 and Mode 3
Max SCK	10 MHz
Write cycle time	5 ms maximum
Supply voltage	1.8 V – 5.5 V

13.5.2 Command Set

```
Opcode   Hex    Bytes after opcode   Description
--------------------------------------------------------------------
READ     0x03   addr, [data...]      Read one or more bytes
WRITE    0x02   addr, data[...]      Write one byte (or page)
WREN     0x06   –                    Set Write Enable Latch (WEL)
WRDI     0x04   –                    Clear Write Enable Latch
RDSR     0x05   [dummy]              Read Status Register
WRSR     0x01   data                 Write Status Register
```

Write Enable Latch (WEL): the EEPROM resets WEL after every write. You must send WREN (its own CS-framed transaction) before every WRITE command.

That makes accidental writes less likely, but it also means your software has to treat “enable writing” as part of every write sequence, not as a one-time setup step.

13.5.3 Status Register (returned by RDSR)

```
Bit:  7    6    5    4    3    2    1      0
      –    –    –    –    BP1  BP0  WEL    WIP
```

Flag	Bit	Meaning
WIP	0	Write In Progress — 1 while write cycle is running
WEL	1	Write Enable Latch — 1 after WREN, 0 after write completes
BP	3:2	Block-protect bits (control write-protect regions)

Poll WIP = 0 before issuing another WRITE command.

In other words, the WRITE command only starts the EEPROM's internal work. The chip still needs time afterwards to commit the byte into nonvolatile storage.

13.5.4 CS Framing Rules

Each command must be bracketed by CS low/high transitions. Raising CS before a complete transfer aborts the current command. The typical patterns:

This is the other half of SPI correctness besides mode. Many slave devices care not just about the bytes, but about the exact CS framing around them.

```
Write Enable:            CS↓ → WREN(0x06) → CS↑
Write byte:              CS↓ → WRITE(0x02) → ADDR → DATA → CS↑
Read status (poll WIP):  CS↓ → RDSR(0x05) → DUMMY → CS↑
                         read status byte from DUMMY transfer
```

13.6 Example: Write Byte to SPI EEPROM

The full annotated source is at `ch13_spi_twi/src/spi_eeprom.S`. Here we walk through the key subroutines.

13.6.1 eeprom_wren — Send Write Enable

```
eeprom_wren:
    rcall cs_assert            /* CS low                              */
    ldi   r16, EEPROM_WREN    /* 0x06                                */
    rcall spi_transfer         /* shift out opcode (received byte discarded)*/
    rcall cs_deassert          /* CS high — WEL is now set            */
    ret
```

13.6.2 eeprom_write_byte — Write One Byte

```
/*
 * r24 = 8-bit address, r22 = data byte
 */
eeprom_write_byte:
    rcall eeprom_wren          /* WEL must be set before each write       */

    rcall cs_assert
    ldi   r16, EEPROM_WRITE   /* 0x02                                    */
    rcall spi_transfer         /* opcode                                  */
    mov   r16, r24
    rcall spi_transfer         /* address                                 */
    mov   r16, r22
    rcall spi_transfer         /* data                                    */
    rcall cs_deassert          /* CS↑ latches the write; 5 ms cycle starts */
    ret
```

13.6.3 eeprom_wait_wip — Poll WIP Until Write Done

```
eeprom_wait_wip:
.Lwip_poll:
    rcall cs_assert
    ldi   r16, EEPROM_RDSR     /* 0x05                                    */
    rcall spi_transfer          /* send RDSR opcode                        */
    ldi   r16, 0xFF             /* dummy: clocks out the status register   */
    rcall spi_transfer          /* r16 = status byte after return          */
    rcall cs_deassert
    sbrc  r16, WIP_BIT          /* skip rjmp if WIP = 0 (write done)       */
    rjmp  .Lwip_poll
    ret
```

After `spi_transfer` with the dummy byte, r16 holds the status register. `SBRC r16, 0` skips the `RJMP` when bit 0 (WIP) is clear — loop exits.

This is a common SPI pattern: send a command, then send a dummy byte only to clock the response back from the slave.

13.6.4 main — Putting It Together

```
main:
    /* SP, pins, SPI0 init (see full source) */

    ldi   r24, 0x00         /* EEPROM address                              */
    ldi   r22, 0xA5         /* data byte                                   */
    rcall eeprom_write_byte /* WREN + WRITE command sequence               */
    rcall eeprom_wait_wip   /* wait up to 5 ms for write cycle             */

.Lhalt:
    rjmp  .Lhalt
```

13.6.5 Build and Inspect

```
# Assemble and link with the GCC driver so <avr/io.h> is preprocessed
avr-gcc -mmcu=attiny3217 -x assembler-with-cpp -c -o spi_eeprom.o src/spi_eeprom.S
avr-gcc -mmcu=attiny3217 -nostartfiles -o spi_eeprom.elf spi_eeprom.o

# Disassemble — verify instruction encoding
avr-objdump -d spi_eeprom.elf

# Generate flashable HEX
avr-objcopy -O ihex spi_eeprom.elf spi_eeprom.hex
```

Expected objdump output for `spi_transfer`:

```
0000003e <spi_transfer>:
  3e:  00 93 24 08   sts  0x0824, r16   ; SPI0_DATA ← tx byte
  42:  10 91 23 08   lds  r17, 0x0823   ; read SPI0_INTFLAGS
  46:  17 ff         sbrs r17, 7        ; wait for IF (bit 7)
  48:  fc cf         rjmp 0x42          ; loop
  4a:  00 91 24 08   lds  r16, 0x0824   ; read received byte, clears IF
  4e:  08 95         ret
```

(Exact symbol labels and addresses vary; with `-nostartfiles`, `objdump` may also show `__ctors_end` at the same address as the first text symbol. The important part is the `LDS`/`STS` encoding: `24 08` encodes address 0x0824 in little-endian.)

That is another good reminder that on AVR, peripheral access cost depends on where the register lives.

13.7 TWI (I²C) Fundamentals

TWI uses two wires: SDA (data) and SCL (clock). Both lines are open-drain with pull-up resistors — any device can pull a line low, but no device drives it high (the pull-up does that). This allows multi-master arbitration and clock stretching by slaves.

On ATtiny3217, the default TWI0 pins are PB1=SDA and PB0=SCL. Alternate TWI0 signals are available on PA1=SDA and PA2=SCL through PORTMUX. The bus still needs pull-up resistors in either location.

This is a very different bus philosophy from SPI. TWI uses fewer wires and built-in addressing, but the protocol is more stateful and the electrical behavior is less direct.

13.7.1 Bus Topology

```
VCC
 │
 ├── 4.7 kΩ ──── SDA ────┬─────────────┬──── ...
 │                         │             │
 ├── 4.7 kΩ ──── SCL ────┼──┬──────────┼──┬─ ...
 │                         │  │          │  │
 │                   ATtiny3217      Slave 1   Slave 2
 │                   (master)       (0x27)    (0x4E)
```

Every device has a 7-bit address (0x00–0x7F). The master sends the address at the start of each transaction; only the matching slave responds.

So TWI solves the "which slave am I talking to?" problem in-band, with an address byte, instead of with separate CS wires.

13.7.2 Transaction Structure

A master write transaction (master sends data to slave) looks like:

```
S  ADDR  W  A  DATA  A  DATA  A  ...  P
│  ────  │  │  ────  │  ────  │       │
│        │  │        │        │       └─ STOP condition
│        │  │        │        └─ ACK from slave
│        │  │        └─ data byte
│        │  └─ ACK from slave (address acknowledged)
│        └─ 7-bit address + W bit (0 = write)
└─ START condition
```

- **START (S)**: SDA falls while SCL is high.
- **Address + R/W**: 7-bit address followed by a Read(1) or Write(0) bit.
- **ACK (A)**: slave pulls SDA low during the 9th SCK pulse to acknowledge.
- **STOP (P)**: SDA rises while SCL is high.

13.7.3 I²C Speeds

Mode	Max SCL	Common use
Standard	100 kHz	Most sensors, EEPROMs
Fast	400 kHz	High-speed peripherals
Fast-Plus	1 MHz	ATtiny3217 supports this

13.8 TWI0 Master Mode Registers on ATtiny3217

The ATtiny3217's TWI0 peripheral has separate register sets for master and slave operation. We use master-mode registers only.

```
Register         Address   Purpose
-----------------------------------------------------------------------
TWI0_CTRLA       0x0810    SDA hold time, fast-mode-plus enable
TWI0_DBGCTRL     0x0812    Debug run control
TWI0_MCTRLA      0x0813    Master enable, interrupts, smart mode
TWI0_MCTRLB      0x0814    Flush, ACK action, bus command (STOP etc.)
TWI0_MSTATUS     0x0815    WIF, RIF, CLKHOLD, RXACK, ARBLOST, BUSERR, BUSSTATE
TWI0_MBAUD       0x0816    SCL baud rate register
TWI0_MADDR       0x0817    Send START + address (write to initiate)
TWI0_MDATA       0x0818    Master data (read = receive, write = transmit)
```

13.8.1 TWI0_MCTRLA — Master Control A

```
Bit:   7     6     5    4      3      2          1      0
       RIEN  WIEN  —    QCEN   TIMEOUT[1:0]      SMEN   ENABLE
```

Field	Description
RIEN	Read Interrupt Enable (fires on RIF)
WIEN	Write Interrupt Enable (fires on WIF)
QCEN	Quick Command Enable (ACK sent automatically)
TIMEOUT	Bus timeout (00 = disabled)
SMEN	Smart Mode Enable (ACK sent automatically on read)
ENABLE	Enable TWI master

For polling mode (no interrupts): set only `TWI_ENABLE_bm`.

13.8.2 TWI0_MSTATUS — Master Status

```
Bit:   7    6    5         4        3          2         1      0
       RIF  WIF  CLKHOLD   RXACK    ARBLOST    BUSERR    BUSSTATE[1:0]
```

Flag	Description
RIF	Read Interrupt Flag — byte received and ready in MDATA
WIF	Write Interrupt Flag — address/data write accepted, ACK received
CLKHOLD	Clock hold — SCL is being stretched; write MDATA/MCTRLB to release

Flag	Description
RXACK	Received ACK: **0 = ACK** (slave acknowledged), 1 = NACK
ARBLOST	Arbitration lost (multi-master: another master won the bus)
BUSERR	Bus error (illegal START/STOP condition detected)
BUSSTATE	00=unknown, 01=idle, 10=owner, 11=busy

`WIF` is the flag you poll most: it sets after the master successfully sends an address byte or a data byte. Check `RXACK` immediately after — if `RXACK=1` the slave sent NACK (not present or busy).

That `RXACK` polarity is easy to misread because success is encoded as zero.

13.8.3 TWI0_MBAUD — Baud Rate Register

`f_SCL = f_CPU / (10 + 2 × BAUD + f_CPU × T_rise)`

Ignoring `T_rise` (assume short wires, small capacitance):

`BAUD = (f_CPU / f_SCL - 10) / 2`

At 3.333 MHz for 100 kHz:

`BAUD = (3333333/100000 - 10) / 2 = 11.67`

Use **12** if you want to stay just below 100 kHz when ignoring rise time: `3333333 / (10 + 2 × 12) ≈ 98.0 kHz`.

```
.equ TWI_BAUD_100K, 12      /* 3.333 MHz → ~98 kHz SCL, ignoring rise time */
```

13.8.4 TWI0_MCTRLB — Master Control B (Bus Commands)

Write to `TWI0_MCTRLB` to issue a bus command after each transaction phase. The `MCMD` field (bits 1:0) controls the state machine:

MCMD value	Symbolic name	Effect
0b00	TWI_MCMD_NOACT_gc	No action
0b01	TWI_MCMD_REPSTART_gc	Repeated START (re-address without STOP)
0b10	TWI_MCMD_RECVTRANS_gc	ACK received byte + clock next (read op)
0b11	TWI_MCMD_STOP_gc	Issue STOP condition, release bus

To send STOP after a write: `sts TWI0_MCTRLB, TWI_MCMD_STOP_gc`

13.9 TWI Initialisation

```
/* TWI0 pins (default mux): SDA = PB1, SCL = PB0
 * Both open-drain — no DDR configuration needed.
 * The TWI peripheral takes control of the pins when ENABLE = 1.
 * External 4.7 kΩ pull-ups to VCC required. */

/* Force bus state to IDLE before we start */
ldi   r16, TWI_BUSSTATE_IDLE_gc     /* 0x01                              */
sts   TWI0_MSTATUS, r16
```

```
/* Set baud rate: 100 kHz at 3.333 MHz */
ldi   r16, TWI_BAUD_100K            /* 12                                     */
sts   TWI0_MBAUD, r16

/* Enable master */
ldi   r16, TWI_ENABLE_bm            /* 0x01                                   */
sts   TWI0_MCTRLA, r16
```

Writing `TWI_BUSSTATE_IDLE_gc` (= 0x01) to `TWI0_MSTATUS` forces the bus state machine to IDLE. Without this, after reset the state is UNKNOWN and the peripheral will not initiate transactions.

This is one of those TWI-specific startup details that feels odd until you remember the peripheral is tracking bus state, not just shifting bytes.

13.10 Master Write Transaction (Polling)

A complete write transaction — send one data byte to a slave device:

```
Step   Action
────────────────────────────────────────────────────────────────────────────
1.  Write (slave_addr << 1 | 0) to TWI0_MADDR  → generates START + address
2.  Wait for WIF set
3.  Check RXACK == 0 (slave ACK); if RXACK == 1 → NACK, abort
4.  Write data byte to TWI0_MDATA
5.  Wait for WIF set
6.  Check RXACK == 0
7.  Write TWI_MCMD_STOP_gc to TWI0_MCTRLB → generates STOP
```

That list is longer than the SPI case because TWI has more protocol structure: START, address, ACK/NACK, data, ACK/NACK, STOP.

In assembly (using r24 = slave address, r22 = data byte):

```
/*
 * twi_write_byte — write one byte to TWI slave
 * r24 = 7-bit slave address (unshifted, 0x00–0x7F)
 * r22 = data byte
 * Returns: r24 = 0 on success, 1 on NACK
 * Clobbers: r16, r17
 */
twi_write_byte:
    /* Send START + address + W (R/W bit = 0) */
    lsl    r24                 /* shift left: addr occupies bits[7:1], bit0=0  */
    sts    TWI0_MADDR, r24     /* write triggers START + 9-bit address phase   */

    /* Wait for WIF */
.Lwif_addr:
    lds    r17, TWI0_MSTATUS
    sbrs   r17, TWI_WIF_bp     /* WIF = bit 6                                  */
    rjmp   .Lwif_addr

    /* Check RXACK */
```

```
    sbrc  r17, TWI_RXACK_bp /* RXACK = bit 4; skip ret if ACK (bit = 0)     */
    rjmp  .Ltwi_nack

    /* Send data byte */
    sts   TWI0_MDATA, r22

    /* Wait for WIF again */
.Lwif_data:
    lds   r17, TWI0_MSTATUS
    sbrs  r17, TWI_WIF_bp
    rjmp  .Lwif_data

    /* Check RXACK for data */
    sbrc  r17, TWI_RXACK_bp
    rjmp  .Ltwi_nack

    /* Send STOP */
    ldi   r16, TWI_MCMD_STOP_gc    /* 0x03                                      */
    sts   TWI0_MCTRLB, r16

    ldi   r24, 0            /* return 0 = success                               */
    ret

.Ltwi_nack:
    ldi   r16, TWI_MCMD_STOP_gc
    sts   TWI0_MCTRLB, r16  /* release bus even on failure                      */
    ldi   r24, 1            /* return 1 = NACK error                            */
    ret
```

13.10.1 SBRC vs SBRS for RXACK

`RXACK = 0` means ACK received (success). The condition for *success* is bit = 0, so we use `SBRC` (Skip if Bit in Register is Cleared) to skip the NACK handler: if the bit is cleared (ACK), skip over `rjmp .Ltwi_nack`.

This is the opposite of USART's `SBRS` on `DREIF` — read the bit polarity carefully for each flag.

That is a general embedded rule: never assume status bits use the same success polarity across peripherals.

13.11 Example: Write to PCF8574 I²C I/O Expander

The PCF8574 is a popular 8-bit I²C I/O expander. Its 7-bit address is `0b0100xxx` where `xxx` is set by hardware pins A2:A0. With A2:A0 = 0b000, the address is `0x20`.

To write a byte to the PCF8574's output latch, a single data byte is sent after the address — no register address needed.

That simplicity is why expanders like the PCF8574 are common first TWI devices: the bus protocol is still there, but the device-side command format is minimal.

```
START | 0x20 W | ACK | 0b10101010 | ACK | STOP
               (slave)            (slave)
```

This drives outputs P7..P0 = 1,0,1,0,1,0,1,0 (alternating).

```
.equ PCF8574_ADDR, 0x20

main:
    /* TWI0 init (as shown in 13.9) */
    ldi   r16, TWI_BUSSTATE_IDLE_gc
    sts   TWI0_MSTATUS, r16
    ldi   r16, TWI_BAUD_100K
    sts   TWI0_MBAUD, r16
    ldi   r16, TWI_ENABLE_bm
    sts   TWI0_MCTRLA, r16

    /* Write 0xAA to PCF8574 */
    ldi   r24, PCF8574_ADDR     /* 0x20                                    */
    ldi   r22, 0b10101010       /* output data                             */
    rcall twi_write_byte

.Lhalt:
    rjmp  .Lhalt
```

13.11.1 Build

```
avr-gcc -mmcu=attiny3217 -nostartfiles -o pcf8574.elf pcf8574.S
avr-objcopy -O ihex pcf8574.elf pcf8574.hex
avrdude -c serialupdi -p t3217 -P /dev/ttyUSB0 -U flash:w:pcf8574.hex
```

13.12 SPI vs TWI Comparison

Feature	SPI	TWI (I²C)
Wires	4 (MOSI, MISO, SCK, CS×n)	2 (SDA, SCL) + pull-ups
Multi-slave	One CS pin per slave	Addressed (7-bit)
Duplex	Full duplex	Half duplex
Max speed here	833 kHz default, 1.667 MHz max	100 kHz / 400 kHz / 1 MHz
Protocol overhead	None (raw shift register)	START/STOP/ACK overhead
Hardware	Shift register + IF flag	State machine (WIF/RIF)
Complexity	Low	Medium
Typical devices	EEPROM, flash, ADC, display	Sensor ICs, I/O expanders

The practical split is usually this: use SPI when you want speed and simple byte streaming, and use TWI when you want several slower peripherals to share the same two wires.

13.13 Summary

```
SPI0 setup (master, mode 0, f/4, manual CS):
  SPI0_CTRLB  ← SPI_SSD_bm            ; disable hardware SS
  SPI0_CTRLA  ← SPI_MASTER_bm | SPI_ENABLE_bm
```

```
SPI transfer:
  STS SPI0_DATA, r16    ; start transfer
  poll SPI_IF_bp in SPI0_INTFLAGS
  LDS r16, SPI0_DATA    ; get received byte, clears IF

25LC010A write sequence:
  1. CS↓, send WREN (0x06), CS↑
  2. CS↓, send WRITE (0x02), addr, data, CS↑
  3. CS↓, send RDSR (0x05), dummy, CS↑ — check WIP (bit 0)
  4. Repeat step 3 until WIP = 0 (max 5 ms)

TWI0 setup (master, 100 kHz):
  TWI0_MSTATUS ← TWI_BUSSTATE_IDLE_gc  ; force idle
  TWI0_MBAUD   ← 12                    ; ~98 kHz at 3.333 MHz
  TWI0_MCTRLA  ← TWI_ENABLE_bm

TWI master write:
  STS TWI0_MADDR, (addr << 1)          ; START + address + W
  wait WIF, check RXACK == 0
  STS TWI0_MDATA, data_byte
  wait WIF, check RXACK == 0
  STS TWI0_MCTRLB, TWI_MCMD_STOP_gc   ; STOP

RXACK polarity: 0 = ACK (slave present), 1 = NACK (slave absent/busy)
```

13.14 Exercises

1. The `spi_transfer` function polls `SPI_IF_bp` in `SPI0_INTFLAGS`. At 3.333 MHz with SCK = f/4, how many CPU cycles elapse from writing `SPI0_DATA` until IF becomes set? How many poll iterations does the CPU execute during that time? (Use the `LDS+SBRS+RJMP` cycle count.)

2. The 25LC010A has a 16-byte page buffer. A page write starts with address 0x00 and fills up to 16 bytes before CS↑. Modify `eeprom_write_byte` into `eeprom_write_page` that accepts a pointer (Z) and a byte count (r20, max 16). Send WREN, then one WRITE transaction with all bytes inside the same CS-framed transfer. How does the CS framing differ from the single- byte version?

3. Change the SPI clock to the fastest value this 3.333 MHz CPU clock can generate while remaining below the 25LC010A's 10 MHz limit. Which combination of PRESC and CLK2X gives that SCK? Write the new `SPI0_CTRLA` value.

4. In `twi_write_byte`, the `BUSSTATE` field of `TWI0_MSTATUS` is not checked before sending. What value does it hold between transactions (after a STOP)? Under what conditions might starting a transaction with BUSSTATE = BUSY cause a problem? How would you guard against it?

5. Write `twi_read_byte`: read one byte from a TWI slave after sending its address with R/W = 1. After receiving the byte, send NACK + STOP (to tell the slave you don't want more bytes). Which `MCMD` value performs "receive byte + send NACK"? Which MSTATUS flag indicates the byte is ready?

6. Using `avr-objdump -d spi_eeprom.elf`, find the `STS SPI0_DATA, r16` instruction. What is its 32-bit encoding? Identify which bytes encode the destination address (0x0824) and

which bytes encode the source register (r16 = R16 = register 16). Recall that `STS` with a 16-bit address uses the encoding `1001 001d dddd 0000 | kkkk kkkk kkkk kkkk`.

Next: Chapter 14

Chapter 14

CRC and Data Integrity in Assembly

CRC routines are a compact way to detect corrupted messages, tables, and code images. This chapter stays in assembly: a software CRC-8 routine for small data blocks, then the ATtiny3217 CRCSCAN peripheral for checking program Flash.

The two jobs are different:

```
Software CRC routine:
  Runs over bytes you choose.
  Useful for packets, EEPROM records, protocol frames, and small tables.

CRCSCAN peripheral:
  Runs over Flash sections selected by hardware configuration.
  Useful for startup or runtime program-memory integrity checks.
```

Microchip documents CRCSCAN on ATtiny3217 as a CRC-16-CCITT memory scanner that can check the entire Flash, the application section, or the boot section. It can also be configured through `FUSE.SYSCFG0` to run during reset initialization before normal code execution starts.

14.1 CRC Basics

A cyclic redundancy check treats a byte stream as a polynomial over bits. Each incoming bit shifts the current remainder. When a one is shifted out, the remainder is XORed with a generator polynomial.

For the byte-oriented routines in this chapter:

```
CRC-8/SMBus:
  Width:        8 bits
  Polynomial:   0x07  (x^8 + x^2 + x + 1)
  Initial CRC:  0x00
  Bit order:    MSB first
```

CRC-8 is small enough to write directly as a bit loop. It is not as strong as CRC-16, but it is useful for short records and teaching the mechanics.

For an 8-bit CRC accumulator:

```
for each input byte:
    crc = crc XOR byte
    repeat 8 times:
        shift crc left
        if the old bit 7 was one:
            crc = crc XOR polynomial
```

On AVR, `LSL` shifts bit 7 into `SREG.C`. That means the branch condition is already available:

```
lsl   r24        ; old bit 7 moves into C
brcc  no_xor     ; if old bit 7 was 0, skip feedback
eor   r24, r25   ; apply polynomial
```

14.2 Software CRC-8 Register Plan

The chapter example computes a CRC over bytes stored in Flash.

```
Input:
  Z      byte address of first Flash byte
  r20    byte count

Output:
  r24    CRC result

Scratch:
  r18    current input byte
  r19    bit counter
  r25    polynomial constant
```

Because this is a standalone assembly routine, the interface is simply the register contract above. There is no hidden runtime setup and no external language call boundary.

14.3 Flash Test Data

The example uses the byte sequence `01 02 03 04`. For CRC-8/SMBus with polynomial `0x07` and initial value `0x00`, the expected CRC is `0xE3`.

```
.section .text
.Lmsg:
    .byte 0x01, 0x02, 0x03, 0x04
.balign 2
```

The `.balign 2` matters because the next instruction must start on an even byte address. Raw byte data in `.text` can leave the location counter odd.

14.4 CRC-8 Implementation

```
.equ CRC8_POLY, 0x07
```

```
.global crc8_flash
crc8_flash:
    ldi   r24, 0x00          /* CRC accumulator */
    ldi   r25, CRC8_POLY     /* polynomial */

    tst   r20
    breq  .Lcrc_done

.Lcrc_byte:
    lpm   r18, Z+            /* read next Flash byte */
    eor   r24, r18           /* mix byte into CRC */
    ldi   r19, 8             /* 8 bits per byte */

.Lcrc_bit:
    lsl   r24                /* old bit 7 -> C */
    brcc  .Lcrc_no_xor
    eor   r24, r25
.Lcrc_no_xor:
    dec   r19
    brne  .Lcrc_bit

    dec   r20
    brne  .Lcrc_byte

.Lcrc_done:
    ret
```

Important details:

- `LPM r18, Z+` reads program memory and advances the Flash byte pointer.
- `EOR` is used both to mix the input byte and to apply the polynomial.
- `LSL` sets carry from the old MSB, so no separate mask instruction is needed.
- `r24` is reused as the accumulator and final result.

For four bytes, the bit loop runs 32 times. Each bit takes:

```
No feedback path:  LSL + BRCC taken + DEC + BRNE
Feedback path:     LSL + BRCC not taken + EOR + DEC + BRNE
```

The exact cycle count depends on the data because the feedback path includes the extra `EOR` and different branch timing.

14.5 Standalone Demo

The full source is crc8.S. It computes the CRC and stores the result in SRAM for debugger inspection.

```
.section .bss
.global crc_result
crc_result:
    .skip 1

.global main
main:
```

```
    ldi   r16, hi8(RAMEND)
    out   SPH, r16
    ldi   r16, lo8(RAMEND)
    out   SPL, r16

    ldi   ZL, lo8(.Lmsg)
    ldi   ZH, hi8(.Lmsg)
    ldi   r20, 4
    rcall crc8_flash

    sts   crc_result, r24

.Lhalt:
    rjmp  .Lhalt
```

Build and inspect:

```
avr-gcc -mmcu=attiny3217 -x assembler-with-cpp -c -o crc8.o src/crc8.S
avr-gcc -mmcu=attiny3217 -nostartfiles -o crc8.elf crc8.o
avr-objdump -d crc8.elf
```

Expected signs in the disassembly:

```
lpm   r18, Z+
eor   r24, r18
lsl   r24
brcc  ...
eor   r24, r25
sts   <crc_result>, r24
```

The expected stored value is `0xE3`.

14.6 Table-Driven CRC

The bit loop is small, but it costs eight iterations per byte. A faster design uses a 256-byte lookup table in Flash:

```
index = crc XOR input_byte
crc = table[index]
```

On AVR, a table lookup uses `Z` and `LPM`. If the table starts on a 256-byte boundary, the index can be placed directly in `ZL` while `ZH` holds the page. If the table is not aligned, add the index to the full 16-bit table pointer.

Tradeoff:

```
Bit loop:
  Small code and no table.
  Cost grows with 8 bit-steps per input byte.

Table lookup:
  Larger Flash use: 256 bytes for CRC-8.
  Much faster per byte.
```

For short packets or rare checks, the bit loop is usually fine. For long streams, the table can be worth the Flash.

14.7 ATtiny3217 CRCSCAN Peripheral

The ATtiny3217 has a dedicated CRCSCAN peripheral at `0x0120`.

```
Register          Address   Purpose
----------------------------------------------------------------
CRCSCAN_CTRLA     0x0120    Enable, NMI enable, reset
CRCSCAN_CTRLB     0x0121    Source selection and Flash access mode
CRCSCAN_STATUS    0x0122    Busy and OK flags
```

`CTRLA` bits:

```
Bit 7  RESET   CRCSCAN_RESET_bm   = 0x80
Bit 1  NMIEN   CRCSCAN_NMIEN_bm   = 0x02
Bit 0  ENABLE  CRCSCAN_ENABLE_bm  = 0x01
```

`CTRLB` fields:

```
SRC[1:0]
  0x00  CRCSCAN_SRC_FLASH_gc        entire Flash
  0x01  CRCSCAN_SRC_APPLICATION_gc  boot + application section
  0x02  CRCSCAN_SRC_BOOT_gc         boot section

MODE[5:4]
  0x00  CRCSCAN_MODE_PRIORITY_gc    priority access to Flash
```

`STATUS` bits:

```
Bit 1  OK    CRCSCAN_OK_bm    = 0x02
Bit 0  BUSY  CRCSCAN_BUSY_bm  = 0x01
```

Microchip documents the hardware algorithm as CRC-16-CCITT:

```
Polynomial: x^16 + x^12 + x^5 + 1
Initial checksum register value: 0xFFFF
Input order: byte by byte, starting at byte 0, MSB first
```

The last location in the selected Flash section must contain the precomputed 16-bit checksum. If the calculated value matches the placed checksum, `CRCSCAN_STATUS.OK` is set.

14.8 Starting a CRCSCAN in Assembly

The software sequence is short:

```
1. Write CRCSCAN_CTRLB to select the Flash source.
2. Write CRCSCAN_CTRLA with ENABLE.
3. Poll STATUS.BUSY until it clears.
4. Test STATUS.OK.
```

Example: scan entire Flash without NMI.

```
crcscan_flash:
    ldi   r16, CRCSCAN_SRC_FLASH_gc
    sts   CRCSCAN_CTRLB, r16

    ldi   r16, CRCSCAN_ENABLE_bm
    sts   CRCSCAN_CTRLA, r16

.Lcrcscan_busy:
    lds   r16, CRCSCAN_STATUS
    sbrc  r16, CRCSCAN_BUSY_bp
    rjmp  .Lcrcscan_busy

    sbrs  r16, CRCSCAN_OK_bp
    rjmp  .Lcrcscan_failed
    ret

.Lcrcscan_failed:
    rjmp  .Lcrcscan_failed
```

Microchip notes that a software-started scan begins after three cycles. In priority mode, CRCSCAN has priority access to Flash and stalls the CPU until the scan is complete. That makes runtime scans easy to reason about but not free; schedule them where a pause is acceptable.

14.9 NMI on CRC Failure

CRCSCAN can trigger a Non-Maskable Interrupt when a scan fails:

```
ldi   r16, (CRCSCAN_ENABLE_bm | CRCSCAN_NMIEN_bm)
sts   CRCSCAN_CTRLA, r16
```

The CRCSCAN NMI vector is vector 1, byte address `0x0002`:

```
__vectors:
    rjmp  main              /* 0x0000 RESET */
    rjmp  crcscan_nmi       /* 0x0002 CRCSCAN_NMI */
```

A CRC failure means program memory may be unreliable, so the NMI handler should be extremely conservative. A typical response is to put outputs in a safe state and stop execution or force a reset path. Do not treat a failed program-memory CRC like an ordinary recoverable event.

14.10 Startup CRC Through Fuses

`FUSE_SYSCFG0` contains the `CRCSRC` field at bits 7:6. Microchip documents that this field can make CRCSCAN run during reset initialization. If the startup CRC check fails, normal code execution is not allowed to start. The failure can still be observed through the debug interface.

That mode is different from a software-started runtime scan:

```
Runtime scan:
```

```
  Code starts first, then software enables CRCSCAN.

Startup scan:
  Reset sequence checks Flash before normal code execution.
```

Use startup CRC for stronger boot-time assurance. Use runtime CRC when the program needs to periodically verify a Flash region while running.

14.11 Summary

```
Software CRC-8:
  Polynomial 0x07
  Initial value 0x00
  LSL moves old bit 7 into SREG.C
  BRCC decides whether to XOR the polynomial

ATtiny3217 CRCSCAN:
  Registers: CTRLA=0x0120, CTRLB=0x0121, STATUS=0x0122
  Hardware algorithm: CRC-16-CCITT
  Polynomial: x^16 + x^12 + x^5 + 1
  Initial value: 0xFFFF
  Sources: entire Flash, application, or boot section
  STATUS.BUSY bit 0, STATUS.OK bit 1
  Optional NMI on failure
  Optional startup scan through FUSE_SYSCFG0.CRCSRC
```

14.12 Exercises

1. Step through `crc8_flash` for the first byte `0x01`. After the `EOR`, what is the accumulator? Which bit iterations take the feedback path?
2. Modify `crc8_flash` to compute over SRAM instead of Flash. Which instruction changes, and how does the pointer setup differ?
3. Add a second test message in Flash and store two result bytes in SRAM.
4. Rewrite the CRC-8 routine so the polynomial is passed in `r21` instead of assembled as `CRC8_POLY`.
5. Write a `crcscan_boot` routine that selects `CRCSCAN_SRC_BOOT_gc`, starts a scan, waits for `BUSY=0`, and returns `r24=0` on OK or `r24=1` on failure.
6. Estimate the runtime cost of a CRCSCAN priority scan over 32 KB Flash using Microchip's note that the peripheral fetches one 16-bit word every third main clock cycle in priority mode.

Next: Chapter 15

Chapter 15

Optimisation Techniques

Assembly optimisation is useful only when it is measured. The ATtiny3217 is predictable enough that many improvements can be proven from the instruction listing alone: count the instructions on the hot path, apply the cycle counts from Microchip's AVR Instruction Set Manual, then compare the result with the flash and SRAM cost.

This chapter keeps the examples assembly-only. The benchmark harness in `src/bench_harness.S` calls the routines directly with registers, gives the linker real symbols to place, and leaves the result visible to `avr-objdump`, `avr-nm`, GDB, or MDB.

The techniques:

1. **Cycle counting** - build a baseline from the disassembly.
2. **Register residency** - keep hot values in registers, not SRAM.
3. **Loop unrolling** - trade flash for fewer branch and counter cycles.
4. **Flash lookup tables** - trade flash for much faster fixed-domain results.
5. **Size checks** - verify that speed did not quietly consume too much flash.

Measure first, optimise second.

15.1 Build and Inspect the Chapter Examples

Build from the chapter directory:

```
cd ch15_optimisation

avr-gcc -mmcu=attiny3217 -x assembler-with-cpp -g3 -Wa,--gdwarf-2 \
    -nostartfiles -o bench.elf \
    src/bench_harness.S src/memcpy_naive.S src/memcpy_fast.S src/lut_sin.S
```

Inspect symbols and sizes:

```
avr-nm --size-sort --print-size bench.elf
```

Inspect instructions:

```
avr-objdump -d -S bench.elf
```

Useful focused views:

```
avr-objdump -d bench.elf | grep -A 20 '<memcpy_naive>'
avr-objdump -d bench.elf | grep -A 40 '<memcpy_fast>'
avr-objdump -d bench.elf | grep -A 12 '<sin8>'
avr-objdump -s -j .text bench.elf
```

The examples are linked into one ELF so the addresses and code sizes are real. No generated PDF rebuild is required for this chapter edit.

15.2 Cycle Reference for This Chapter

The ATtiny3217 uses the AVRxt timing column in Microchip's AVR Instruction Set Manual. Always check the manual for instructions not listed here, especially when carrying code to older AVR, AVRxm, or reduced-core AVRrc devices.

Common timing used in this chapter:

```
Instruction              ATtiny3217 cycles     Notes
--------------------------------------------------------------------
NOP                      1
MOV, MOVW                1
LDI                      1                     r16-r31 only
CLR                      1                     encoded as EOR Rd,Rd
ADD, ADC                 1
AND, ANDI, EOR           1
INC, DEC                 1
LSR                      1
TST                      1                     encoded as AND Rd,Rd
BREQ/BRNE not taken      1
BREQ/BRNE taken          2
RJMP                     2
RCALL                    3
RET                      4                     larger PC-width AVRs can differ
LD Rd, Z+                2
ST X+, Rr                2
LPM Rd, Z                3
LPM Rd, Z+               3
STS k, Rr                2                     two-word instruction
```

Two details cause most bad cycle counts:

- A conditional branch costs 2 cycles when taken and 1 cycle when it falls through.
- Count the last loop iteration separately, because the loop branch usually falls through only once.

For a loop that executes `N` times:

```
branch cycles = (N - 1) * 2 + 1
```

15.3 Baseline: One-Byte Copy Loop

The file `src/memcpy_naive.S` copies one byte per loop iteration.

Register interface:

```
R25:R24  destination pointer
R23:R22  source pointer
R20      byte count
```

Inner loop:

```
.Lnaive_loop:
    ld    r18, Z+            /* 2 cycles */
    st    X+, r18            /* 2 cycles */
    dec   r20                /* 1 cycle  */
    brne  .Lnaive_loop       /* 2 taken, 1 final fall-through */
```

Taken-path loop cost:

```
2 + 2 + 1 + 2 = 7 cycles per byte
```

For `len = 16`, the branch is taken 15 times and falls through once:

```
15 taken iterations: 15 * 7 = 105
1 final iteration:   2 + 2 + 1 + 1 = 6
inner-loop total:    111 cycles
```

Full subroutine cost for `len = 16` on ATtiny3217:

```
setup:       movw + movw + tst + breq(not taken) = 4
copy loop:   111
ret:         4
subroutine:  119 cycles
with RCALL:  122 cycles from the harness call site
```

The important baseline is the loop body: **111 cycles for 16 bytes**, or about 6.94 cycles per byte including the final branch fall-through.

15.4 Keep Hot Values in Registers

SRAM is much slower than a register. Direct SRAM access also costs flash because `LDS` and `STS` are two-word instructions on this device family.

Bad pattern for hot loop state:

```
.Lloop:
    lds   r18, counter
    inc   r18
    sts   counter, r18
    dec   r20
    brne  .Lloop
```

That loop repeatedly pays for direct SRAM access. If the value is only needed by this routine, keep it in a register:

```
    clr   r18
.Lloop:
    inc   r18
    dec   r20
```

```
    brne  .Lloop
    sts   counter, r18       /* store once if an observable result is needed */
```

The same rule applies to pointers and accumulators. Plan register use before writing the loop:

```
Routine: block_sum
  Z       source pointer
  r20     byte count
  r18     loaded byte
  r24     accumulator/result
  r1      zero source for ADC carry propagation when needed
```

Example:

```
.global block_sum
.type   block_sum, @function

block_sum:
    movw  ZL, r22
    clr   r24
    tst   r20
    breq  .Lsum_done

.Lsum_loop:
    ld    r18, Z+
    add   r24, r18
    dec   r20
    brne  .Lsum_loop

.Lsum_done:
    ret
.size block_sum, .-block_sum
```

No SRAM is touched inside the loop. The accumulator stays live in `r24` until the routine returns.

When registers run short, do not spill in the inner loop by reflex. First try:

- reusing argument registers once their original value is no longer needed
- shortening the lifetime of temporary values
- splitting a large loop into smaller named phases
- saving a register once at entry and restoring it once at exit

One `PUSH`/`POP` pair outside a hot loop is often much cheaper than `LDS`/`STS` inside every iteration.

15.5 Loop Unrolling

Each iteration of the naive copy loop spends 3 cycles on loop overhead:

```
DEC  = 1
BRNE = 2 when taken
```

For a tiny loop body, that overhead is a large fraction of the total. Unrolling processes multiple bytes per branch.

The file `src/memcpy_fast.S` processes four bytes per block:

```
.Lfast_block:
    ld    r18, Z+            /* 2 */
    ld    r19, Z+            /* 2 */
    ld    r22, Z+            /* 2 */
    ld    r23, Z+            /* 2 */
    st    X+, r18            /* 2 */
    st    X+, r19            /* 2 */
    st    X+, r22            /* 2 */
    st    X+, r23            /* 2 */
    dec   r20                /* 1 */
    brne  .Lfast_block       /* 2 taken, 1 final */
```

Taken-path block cost:

```
4 loads + 4 stores + DEC + BRNE
= 8 + 8 + 1 + 2
= 19 cycles per 4 bytes
= 4.75 cycles per byte
```

For `len = 16`, there are four blocks:

```
3 taken blocks:      3 * 19 = 57
1 final block:       8 + 8 + 1 + 1 = 18
block-loop total:    75 cycles
```

The fast routine also has setup and tail handling:

```
setup before block loop: 9 cycles
block loop:             75 cycles
tail test, no tail:      3 cycles
ret:                     4 cycles
subroutine:             91 cycles
with RCALL:             94 cycles from the harness call site
```

Compare the hot copy work:

```
naive inner loop, len=16: 111 cycles
unrolled block loop:      75 cycles
speedup in copy loop:     111 / 75 = 1.48x
```

Compare the full call from the harness:

```
naive with RCALL:         122 cycles
fast with RCALL:           94 cycles
full-call speedup:        122 / 94 = 1.30x
```

Both numbers matter. The first tells you what happens on long buffers where setup fades away. The second tells you what this exact `len = 16` benchmark costs.

15.6 Tail Handling

Unrolling by four creates a remainder problem. Lengths such as 13 or 17 do not divide evenly by four.

The fast routine computes:

```
    mov   r21, r20
    andi  r21, 0x03         /* tail = len & 3 */
    lsr   r20
    lsr   r20               /* blocks = len >> 2 */
    breq  .Lfast_tail       /* len < 4 */
```

Then the block loop copies groups of four and the tail loop copies the last 0-3 bytes:

```
.Lfast_tail:
    tst   r21
    breq  .Lfast_done
    ld    r18, Z+
    st    X+, r18
    dec   r21
    brne  .Lfast_tail
```

This is the price of a general-purpose unrolled routine. The fast path is cheaper per byte, but the control flow is larger than the naive loop.

Unrolling is usually worth it when:

- the loop runs many times
- the useful work per iteration is small
- flash budget is available
- the tail path is rare or short

It is often not worth it when:

- the loop normally runs only a few times
- the loop body already contains expensive work
- the routine is cold startup code
- flash space is the limiting resource

15.7 Flash Lookup Tables

Some computations have a small input domain. If every possible input fits in one byte, a 256-entry table can replace a long calculation.

The file `src/lut_sin.S` contains a 256-entry unsigned sine table:

```
.section .progmem.data,"a",@progbits
.global sin_table
sin_table:
    .byte 128,131,134,137,140,143,146,149
    /* full 256-entry table in src/lut_sin.S */
.size sin_table, .-sin_table
```

The table value is:

```
128 + 127 * sin(n * 2 * pi / 256)
```

Rounded to bytes, angle 0 gives 128, angle 64 gives 255, angle 128 gives 128, and angle 192 gives 0.

The lookup routine:

```
.global sin8
.type   sin8, @function

sin8:
    ldi   ZL, lo8(sin_table)
    ldi   ZH, hi8(sin_table)
    add   ZL, r24
    adc   ZH, r1
    lpm   r24, Z
    ret
.size sin8, .-sin8
```

Cycle count on ATtiny3217:

```
LDI + LDI + ADD + ADC + LPM + RET
= 1 + 1 + 1 + 1 + 3 + 4
= 11 cycles

With RCALL from the harness: 14 cycles
```

The table costs 256 flash bytes. The routine costs 12 flash bytes. For a value that is used repeatedly, that is often a good trade.

15.7.1 Why `ADC ZH, r1` Works

The `ADD ZL, r24` instruction may carry from the low byte of the table address into the high byte. `ADC ZH, r1` adds that carry to `ZH`.

This requires `r1` to be zero. The chapter harness clears `r1` before calling the routines:

```
    clr     r1
```

If a standalone assembly program uses `r1` as a zero source, it must preserve that convention itself. There is no startup code in these examples to do it for you.

15.7.2 Flash Address Limits

`LPM` addresses the low 64 KB of program memory. The ATtiny3217 has 32 KB of flash, so ordinary `LPM` is sufficient. Larger AVR devices may require `ELPM` and `RAMPZ` for tables above the low 64 KB region.

15.7.3 Verifying Table Placement

The default AVR linker script places program-memory input sections into the final flash image. In the linked ELF, inspect the final `.text` output section:

```
avr-nm -n bench.elf | grep sin_table
avr-objdump -s -j .text bench.elf
```

Do not assume that an input section name such as `.progmem.data` remains a separate output section after linking.

15.8 Size Results from the Current Sources

With the current chapter files, `avr-nm --size-sort --print-size bench.elf` reports the relevant symbols as:

```
00000188 0000000c T sin8
00803820 00000010 B dst_fast
00803810 00000010 B dst_naive
00803800 00000010 B src_data
00000142 00000012 T memcpy_naive
00000154 00000034 T memcpy_fast
00000104 0000003e T main
00000004 00000100 T sin_table
```

Meaning:

Symbol	Size	Meaning
`memcpy_naive`	18 bytes	Smallest copy routine.
`memcpy_fast`	52 bytes	Faster for larger lengths, 34 bytes larger.
`sin8`	12 bytes	Lookup routine only.
`sin_table`	256 bytes	Flash table used by `sin8`.
`src_data`	16 bytes	SRAM source buffer for the harness.
`dst_naive`	16 bytes	SRAM destination for naive copy.
`dst_fast`	16 bytes	SRAM destination for unrolled copy.
`sin_sample`	1 byte	SRAM location storing one sine lookup result.

Speed is not free. In this example, the unrolled copy routine saves cycles by spending 34 extra flash bytes. The sine lookup spends 256 flash bytes to replace a much longer calculation with one table load.

15.9 Reading `avr-objdump` Correctly

Example disassembly:

```
0000014a <.Lnaive_loop>:
 14a:   21 91           ld      r18, Z+
 14c:   2d 93           st      X+, r18
 14e:   4a 95           dec     r20
 150:   e1 f7           brne    .-8
```

Read it as:

```
14a       byte address in flash
21 91     encoded instruction bytes, little-endian in the listing
ld ...    decoded instruction
```

AVR instructions are stored as 16-bit words, but the ELF view uses byte addresses. Most instructions advance by 2 bytes. Two-word instructions such as `STS`, `LDS`, `CALL`, and `JMP` advance by 4 bytes.

Use these commands often:

```
avr-objdump -d -S bench.elf
avr-objdump -h bench.elf
avr-nm -n bench.elf
avr-nm --size-sort --print-size bench.elf
```

If a symbol reports an unexpected size, check that the source has matching `.type` and `.size` directives.

15.10 Common Errors in Hand Optimisation

15.10.1 Optimising Cold Code

Speed work belongs on hot paths. A routine that runs once during setup is rarely worth making larger or harder to inspect. Use a timer, GPIO toggle, logic analyzer, simulator trace, or debugger stop points to identify where time is actually spent.

15.10.2 Ignoring Setup Cost

An unrolled loop can be faster in steady state but slower for tiny lengths. Do both calculations:

```
steady-state loop cost
full routine cost including setup, tail, and return
```

For `len = 16`, the unrolled copy is still faster. For very short lengths, the naive loop may win.

15.10.3 Forgetting the Final Branch

The last conditional branch usually costs one cycle, not two. Count taken and not-taken paths separately.

15.10.4 Using a Zero Register Without Clearing It

The lookup example uses `r1` as a zero source. In a fully standalone assembly program, clear it yourself before relying on it:

```
    clr r1
```

If a routine temporarily uses `r1` for some other purpose, restore it to zero before returning to code that expects the zero-register convention.

15.10.5 Putting Hot State in SRAM

Repeated `LDS`/`STS` inside a tight loop is often the first thing to remove. Keep counters, pointers, accumulators, masks, and temporary bytes in registers for as long as their live ranges allow.

15.10.6 Unrolling Until the Source Becomes Fragile

Unrolling x2 or x4 is often manageable. Unrolling x8 or x16 can become hard to audit, especially when there is a tail path, status flags, or overlapping source and destination ranges. Code that cannot be reviewed reliably is not a professional optimisation.

15.11 Optimisation Checklist

Use this sequence before changing a routine:

1. Mark the exact hot loop or hot path.
2. Build the ELF.
3. Disassemble with `avr-objdump -d -S`.
4. Count cycles for the current path.
5. Record flash size for the current routine.
6. Apply one change.
7. Rebuild and recount.
8. Check flash size again.
9. Keep the change only if the gain is worth the size and complexity.

For AVR assembly, this discipline works well because the machine is small and the instruction timings are documented.

15.12 Source Notes

Facts in this chapter are based on:

- Microchip AVR Instruction Set Manual DS40002198C, especially the AVRxt cycle timing column used by ATtiny3217-class devices.
- Microchip ATtiny3217 documentation for the 32 KB flash size that makes plain `LPM` sufficient for the chapter's lookup table.
- The local linked ELF produced from `src/bench_harness.S`, `src/memcpy_naive.S`, `src/memcpy_fast.S`, and `src/lut_sin.S`.

15.13 Exercises

1. Count the exact full-call cycle cost of `memcpy_naive` for `len = 8`, including `RCALL`, setup, loop, and `RET`.

2. Count the exact full-call cycle cost of `memcpy_fast` for `len = 8`. Does the unrolled routine still win?

3. Modify `memcpy_fast` to unroll by eight instead of four. Measure the new symbol size and calculate the new loop cost for `len = 64`.

4. Write `block_sum`, an assembly routine that sums `len` bytes from `Z` into `r24`. Compare the cycle count before and after unrolling by two.

5. Change the sine table to a 64-entry quarter-wave table and reconstruct the quadrant in code. How many flash bytes are saved, and how many cycles are added?

6. Use GDB or MDB to stop at `bench_done` and inspect `dst_naive`, `dst_fast`, and `sin_sample`. Explain what each byte proves about the benchmark.

Next: Chapter 16 - Bootloaders

Chapter 16

Self-Programming Flash: NVMCTRL on tinyAVR 1-Series

Classic AVR devices (ATmega328P and friends) use `SPM` from a classic boot section to write flash. The **ATtiny3217**, like other modern tinyAVR 1-series parts, takes a different approach: the `SPM` instruction does not exist and there is no `BOOTRST` fuse. Instead, a memory-mapped peripheral called **NVMCTRL** (Non-Volatile Memory Controller) handles self-programming, while the `BOOTEND` and `APPEND` fuses divide flash into BOOT, APPCODE, and APPDATA sections with directional write protection.

This chapter covers everything needed to write assembly code that programs its own flash on the ATtiny3217 Curiosity Nano board:

1. Why AVRXMEGA3 replaced SPM with NVMCTRL.
2. How NVMCTRL registers and commands work.
3. The page buffer and the exact write sequence.
4. Configuration Change Protection (CCP) — the timing-critical unlock.
5. A complete 64-byte page-write routine in assembly.
6. A UART-driven self-update loop that receives Intel HEX and writes it to flash live.
7. The BOOTEND and APPEND fuses: defining protected flash sections.
8. Building, uploading, and common pitfalls.

This is one of the more hardware-specific chapters in the book. The goal is to turn "the chip can rewrite its own flash" from a mysterious feature into a sequence you can reason about step by step.

16.1 tinyAVR 1-Series vs Classic AVR: No SPM, NVMCTRL Instead

16.1.1 Architecture Family: AVRXMEGA3

The ATtiny3217 belongs to the **AVRXMEGA3** sub-architecture. The three key differences that affect self-programming are:

Feature	ATmega328P (classic)	ATtiny3217 (AVRXMEGA3)
Self-program instr.	SPM (dedicated opcode)	none – use ST to flash

Boot section	fixed classic boot area	configurable BOOT section
BOOTRST fuse	yes	none
Flash write ctrl.	SPMCSR register	NVMCTRL peripheral
Flash address space	separate word-address	byte-address, unified
Page size	128 bytes (64 words)	64 bytes (32 words)
Write privileges	boot section only	BOOT can write APPCODE/APPDATA; APPCODE can write APPDATA
UPDI programming	no	yes (PA0, replaces ISP)

The AVRXMEGA3 memory map is **unified**: flash, SRAM, and peripherals all share the same byte-addressed data space. Flash byte address 0 appears at data address `0x8000` when accessed by ordinary `LD`/`ST` instructions. The CPU's program counter still addresses executable flash from `0x0000`. You will see both conventions in this chapter.

16.2 Microchip App Notes Used

The main Microchip reference for this chapter is **AN2634, Bootloader for tinyAVR 0- and 1-series, and megaAVR 0-series**. The relevant points from that application note are:

- tinyAVR 0/1-series and megaAVR 0-series devices program flash one page at a time.
- The page buffer has the same size as the flash page: 64 or 128 bytes depending on the device. The ATtiny3217 page size is 64 bytes.
- The target page must be erased before the page buffer is written to flash. Programming an unerased page corrupts its contents.
- `PAGEERASEWRITE` starts the erase and write as one NVMCTRL command.
- The page buffer is automatically cleared after NVMCTRL commands execute.
- For tinyAVR 0/1-series, flash is mapped at data address `0x8000` for normal load/store access. Microchip's headers call this `MAPPED_PROGMEM_START`.
- `BOOTEND` and `APPEND` define BOOT, APPCODE, and APPDATA sections in 256-byte units.
- The documented write privileges are directional: BOOT can write APPCODE and APPDATA; APPCODE can write APPDATA; APPDATA cannot write flash or EEPROM.
- AN2634 warns against issuing `CHIPERASE` during self-programming because it erases the flash array.

The ATtiny3216/3217 data sheet adds two important details:

- The page buffer write uses the least significant address bits as the offset in the page buffer, and new page-buffer data is combined with previous buffer contents by a binary AND.
- After the self-programming CCP signature is written to `CPU_CCP`, the protected NVMCTRL command must be executed within the next four instructions.

16.2.1 What Replaces SPM?

On tinyAVR 1-series, you fill the flash **page buffer** by writing normally to flash addresses using `ST` (Store Indirect). There is no special opcode. The NVMCTRL peripheral watches these writes and accumulates them in a 64-byte buffer. When you issue a command to `NVMCTRL_CTRLA`, it commits the buffer to the actual flash array.

```
Classic AVR SPM flow:          tinyAVR NVMCTRL flow:
──────────────────────         ──────────────────────

load R1:R0 ← data word         (nothing special)
write SPMCSR ← mode            wait FBUSY = 0
SPM  ← fills buffer            ST Z+, rn  ← fills buffer (just write!)
```

```
(repeat for each word)             (repeat for each word)
write SPMCSR ← PGERS               CCP unlock (0x9D → 0x0034)
SPM  ← erases page                 STS NVMCTRL_CTRLA ← 0x03
write SPMCSR ← PGWRT               wait FBUSY = 0
SPM  ← writes page
```

The NVMCTRL approach is simpler to understand: write your data to the target flash addresses, then send the “erase+write” command. The hardware does the rest.

The important mental shift is that self-programming no longer looks like a special instruction stream. It looks like ordinary memory-mapped peripheral control wrapped around a careful write sequence.

16.2.2 Programming Interface: UPDI

The ATtiny3217 has no ISP pins. All external programming is done over **UPDI** (Unified Program and Debug Interface), a single-wire protocol on **PA0**. The Curiosity Nano board’s on-board debugger bridges USB to UPDI automatically. Use `pymcuprog` for the on-board debugger:

```
pymcuprog write -d attiny3217 -t uart -f firmware.hex
```

Once the self-update firmware is running, the device reprograms itself — no external programmer needed for firmware updates.

16.3 NVMCTRL Overview

16.3.1 Flash Parameters

```
ATtiny3217 flash layout
─────────────────────────────────────────────────────
Total flash:       32 KB (32768 bytes)
Page size:         64 bytes
Number of pages:   512
Flash data space:  0x8000–0xFFFF (byte addresses when read as data)
Flash program:     0x0000–0x7FFF (word addresses for PC / LPM)
SRAM:              0x3800–0x3FFF (2 KB)
```

The 64-byte page size is critical: every NVMCTRL flash write operation commits one page. If your update stream ends with a short final page, pad the unwritten bytes in the SRAM page image with `0xFF` before issuing `PAGEERASEWRITE`.

That last point matters because it explains why page alignment and buffering show up everywhere in self-programming code. Flash is not rewritten one byte at a time, even if your firmware update logic receives bytes one at a time.

16.3.2 NVMCTRL Register Map

All NVMCTRL registers are outside the I/O space and require `LDS`/`STS`:

```
Register          Address   Width   Purpose
──────────────────────────────────────────────────────────
NVMCTRL_CTRLA     0x1000    1 byte  Command register — write command here
NVMCTRL_CTRLB     0x1001    1 byte  Config: BOOTLOCK, APCWP write-protect
```

```
NVMCTRL_STATUS     0x1002    1 byte  Status flags: FBUSY, EEBUSY, WRERROR
NVMCTRL_INTCTRL    0x1003    1 byte  Interrupt enable: EEREADY (bit 0)
NVMCTRL_INTFLAGS   0x1004    1 byte  Interrupt flags (write 1 to clear)
NVMCTRL_DATA       0x1006    2 bytes Data register (used for EEPROM word)
NVMCTRL_ADDR       0x1008    4 bytes Address register (used for EEPROM)
```

For flash programming, you only need `NVMCTRL_CTRLA` and `NVMCTRL_STATUS`.

16.3.3 NVMCTRL_STATUS — Status Register (0x1002)

```
Bit:   7      6      5      4      3      2        1        0
       —      —      —      —      —      WRERROR  EEBUSY   FBUSY

FBUSY   — Flash Busy: 1 while a flash erase/write operation is in progress.
           Poll this bit; do not issue a new command while FBUSY = 1.
EEBUSY  — EEPROM Busy: 1 while an EEPROM operation is in progress.
WRERROR — Write Error: set if a write was attempted to a protected region.
           Write 1 to clear (W1C).
```

16.3.4 NVMCTRL Commands (written to NVMCTRL_CTRLA)

```
Value   Name               Description
─────────────────────────────────────────────────────────────────────
0x00    NOCMD              No operation (also clears WRERROR)
0x01    PAGEWRITE          Write page buffer to flash (no erase first)
0x02    PAGEERASE          Erase one flash page (Z must point into page)
0x03    PAGEERASEWRITE     Erase + write in one atomic step  ← use this
0x04    PAGEBUFERASE       Clear the page buffer (all bytes → 0xFF)
0x05    CHIPERASE          Erase flash array; do not use in this updater
0x06    EEERASE            Erase EEPROM
0x07    FUSEWRITE          Write fuse bytes (restricted, rarely needed)
```

PAGEERASEWRITE (0x03) is the command you will use for firmware updates. It atomically erases the target page and writes the page buffer in one operation — safer than doing them separately, because a power failure between separate erase and write steps cannot leave a page half-erased.

In practice, that combined command is what makes the sequence robust enough to use in the field. Fewer separate steps means fewer partially completed states.

16.4 The Page Buffer and Write Sequence

16.4.1 How the Page Buffer Works

The NVMCTRL page buffer is a 64-byte staging area internal to the NVM controller. It is **not** directly addressable. You populate it by writing to the corresponding flash addresses in the data address space:

```
mapped_flash_address = 0x8000 + flash_byte_address
```

When the Z pointer points into flash address space and you execute `ST Z+, Rn`, the byte goes into the page buffer at the offset corresponding to Z's address. The hardware handles this automatically whenever the NVM controller recognises the write target as flash.

```
Page buffer write — step by step:

Flash page at byte address 0x0000 appears in data space at 0x8000.

  Z = 0x8000          ; points to byte 0 of flash page 0
  st Z+, r16          ; → page buffer[0] = r16
  st Z+, r17          ; → page buffer[1] = r17
  ...                 ; fill all 64 bytes
  st Z+, r30          ; → page buffer[63] = r30
                      ; Z = 0x8040 (past end of page)
```

The page buffer is erased to all ones after reset, after page write/erase commands, after a Clear Page Buffer command, and after wake-up from sleep. The least significant address bits select the offset inside the page buffer. If you write the same page-buffer byte more than once, the resulting value is the binary AND of the old buffer value and the new value.

That AND behavior matters. A partial page update is not automatically merged with the existing flash page for you. For a robust partial update, either fill unwritten bytes with `0xFF` when programming a fresh page, or read the old page into SRAM, modify the intended bytes, erase the flash page, and write the full merged 64-byte image.

16.4.2 Complete Flash Page Write Sequence

```
Step 1: Wait for any previous NVM operation to finish.
        Poll NVMCTRL_STATUS.FBUSY until it reads 0.

Step 2: Issue NOCMD to clear any stale error state.
        STS NVMCTRL_CTRLA, 0x00

Step 3: Fill the page buffer.
        Set Z = 0x8000 + (target_page × 64)    ; data-space address
        ST Z+, r0 ; ST Z+, r1 ; ...            ; 64 sequential writes

Step 4: Issue PAGEERASEWRITE (CCP-protected).
        LDI r16, 0x03        ; PAGEERASEWRITE command, prepared first
        LDI r17, 0x9D        ; CCP_SPM_gc key, prepared first
        STS 0x0034, r17      ; write to CCP register
        STS NVMCTRL_CTRLA, r16  ; next instruction: protected command

Step 5: Wait for FBUSY to clear again.
        Poll NVMCTRL_STATUS.FBUSY until 0.
```

16.4.3 The Z Pointer Convention

On ATtiny3217 the Z pointer (R31:R30) is a 16-bit register, and `0x8000` is well within that range. For page-buffer writes with `ST Z+, Rn`, set Z to the mapped program-memory address:

```
/* To write page at flash byte address 0x0200 (page 8): */
ldi   ZL, lo8(0x8200)
ldi   ZH, hi8(0x8200)
/* Now ST Z+, Rn writes to page buffer slots starting at offset 0 */
```

Equivalently:

```
ldi   ZL, lo8(0x0200)
ldi   ZH, hi8(0x0200)
subi  ZH, -0x80         /* add mapped flash base 0x8000 */
```

The examples in Sections 16.5 and 16.6 use this mapped-address convention.

That convention keeps the assembly easier to follow because the same flash byte address can be used consistently through buffering, writing, and later jumps.

16.5 CCP — Configuration Change Protection

16.5.1 Why CCP Exists

Some NVMCTRL commands (and other sensitive operations like changing clock settings) can corrupt firmware if executed accidentally — for example, if a runaway pointer writes to NVMCTRL_CTRLA, or if a flash write executes during a brownout. **CCP** (Configuration Change Protection) prevents accidental execution by requiring a timed unlock sequence before each protected write.

So CCP is not there to make your life difficult. It is there because an accidental flash-control write is serious enough that the hardware demands a deliberate two-step action.

16.5.2 The CCP Register (0x0034)

```
Address: 0x0034 (in I/O space: use OUT or STS)

CCP keys:
  0x9D  – CCP_SPM_gc    Unlocks NVM Self-Programming commands
  0xD8  – CCP_IOREG_gc  Unlocks protected I/O register writes (e.g. CLKCTRL)

After writing a key, the protected write must occur within the next four CPU
instructions. Interrupt requests are held pending during this short
configuration-change window.
```

16.5.3 The Four-Instruction Rule

The CCP window opens the moment you write the key. Use the strictest practical rule in assembly: make the protected `NVMCTRL_CTRLA` write the very next instruction after the `CPU_CCP` write:

```
sts   CPU_CCP, r17
sts   NVMCTRL_CTRLA, r16
```

Assembly idiom:

```
ldi   r16, 0x03             /* PAGEERASEWRITE command              */
ldi   r17, 0x9D             /* CCP_SPM_gc key                      */
sts   0x0034, r17           /* open the CCP window                 */
sts   NVMCTRL_CTRLA, r16    /* next instruction: protected write   */
```

Critical: do not put any instruction between `STS CPU_CCP` and `STS NVMCTRL_CTRLA` — not even a `NOP`.

This is one of the rare cases where instruction spacing is not just an optimization detail. It is part of the functional correctness of the code.

16.6 Flash Write Routine in Assembly

The routine `nvm_write_page` writes one 64-byte page. It expects:

- **Z** (R31:R30): mapped flash data address of the target page: `0x8000 + flash_byte_address` (must be 64-byte aligned).
- **X** (R27:R26): data address of the 64-byte source buffer in SRAM.

It clobbers R16, R17, R18 (scratch) and leaves Z past the end of the written page. The source buffer pointer X is incremented and left past the last byte.

Leaving both pointers advanced is a small design choice, but it makes the routine easier to chain when writing consecutive pages from a stream.

16.6.1 Register Allocation

```
Z  (R31:R30)   mapped flash destination — page base address, then advanced
X  (R27:R26)   SRAM source pointer — 64-byte buffer of new content
R16            scratch: NVMCTRL command, status poll byte
R17            loop counter (64 iterations)
R18            data byte being copied into page buffer
```

16.6.2 Full Listing: `src/nvmctrl_write.S`

See `src/nvmctrl_write.S` for the complete annotated source. Key routines:

```
/*
 * nvm_wait — spin until NVMCTRL_STATUS.FBUSY = 0
 *
 * Clobbers: R16
 * Cycles per loop: LDS (2) + SBRC (1 or 2) + RJMP (2) = 5 cycles per poll
 */
nvm_wait:
    lds   r16, NVMCTRL_STATUS          /* 2 cy: load status              */
    sbrc  r16, NVMCTRL_FBUSY_bp        /* 1 cy: skip next if FBUSY=0     */
    rjmp  nvm_wait                     /* 2 cy: still busy, loop         */
    ret                                /* 4 cy                            */
```

`SBRC Rd, b` — Skip if Bit in Register is Cleared. Takes 1 cycle when not skipping, 2 when skipping (pipeline flush). `NVMCTRL_FBUSY_bp` = 0 (bit position 0 of NVMCTRL_STATUS).

```
/*
 * nvm_write_page — write 64-byte SRAM buffer to one flash page
 *
 * Entry:  Z = mapped flash data address (0x8000 + flash byte address)
 *           and 64-byte aligned
 *         X = SRAM pointer to 64-byte source buffer
 * Exit:   Z advanced by 64, X advanced by 64
 * Clobbers: R16, R17, R18
 */
nvm_write_page:
    /* Step 1: wait for any previous NVM operation */
```

```
    rcall nvm_wait                    /* 4 + 5n cy (n polls)           */

    /* Step 2: clear previous error (NOCMD) */
    ldi   r16, NVMCTRL_CMD_NOCMD_gc   /* r16 = 0x00                    */
    sts   NVMCTRL_CTRLA, r16          /* 2 cy                          */

    /* Step 3: fill page buffer from SRAM source */
    ldi   r17, 64                     /* loop counter = page size       */
.Lfill_loop:
    ld    r18, X+                     /* 2 cy: load byte from SRAM      */
    st    Z+,  r18                    /* 2 cy: write byte to flash page buf   */
    dec   r17                         /* 1 cy: decrement counter        */
    brne  .Lfill_loop                 /* 2 cy taken / 1 cy fall-through      */
    /* Total fill: 63 × 7 + 6 = 447 cy                                  */

    /* Step 4: rewind Z to page start for PAGEERASEWRITE */
    subi  ZL, lo8(64)                 /* Z -= 64                        */
    sbci  ZH, hi8(64)

    /* Step 5: CCP-protected PAGEERASEWRITE command */
    cli                               /* 1 cy: disable interrupts       */
    ldi   r16, NVMCTRL_CMD_PAGEERASEWRITE_gc  /* r16 = 0x03             */
    ldi   r17, CCP_SPM_gc             /* r17 = 0x9D                     */
    sts   CPU_CCP, r17                /* 2 cy: unlock CCP window (cycles 1-2) */
    sts   NVMCTRL_CTRLA, r16          /* 2 cy: issue command (cycles 3-4)     */
    sei                              /* 1 cy: re-enable interrupts      */

    /* Step 6: wait for completion */
    rcall nvm_wait

    /* Z points to page start; advance past the page for caller convenience  */
    subi  ZL, lo8(-64)                /* Z += 64                        */
    sbci  ZH, hi8(-64)
    ret
```

16.6.3 Instruction Notes

Two instructions appear for the first time in this chapter:

```
SBIW Rd, K   – Subtract Immediate from Word
  Operation:  Rd+1:Rd ← Rd+1:Rd − K    (K = 0..63)
  Encoding:   1001 0111 KKdd KKKK   (2 bytes)
  Registers:  d ∈ {24,26,28,30} only (R25:R24, X, Y, Z)
  Flags:      C, Z, N, V, S
  Cycles:     2

ADIW Rd, K   – Add Immediate to Word
  Operation:  Rd+1:Rd ← Rd+1:Rd + K    (K = 0..63)
  Encoding:   1001 0110 KKdd KKKK   (2 bytes)
  Registers:  d ∈ {24,26,28,30} only
  Flags:      C, Z, N, V, S
  Cycles:     2
```

`SBIW` and `ADIW` are useful word operations, but their immediate range is 0..63. A 64-byte page adjustment must use another sequence. The source uses `SUBI`/`SBCI` pairs to subtract and add 64 from the 16-bit Z pointer.

16.7 Practical Firmware Update: UART-Driven Self-Update

With `nvm_write_page` in hand, we can build a self-update loop. The device receives new firmware as Intel HEX records over USART0, accumulates complete 64-byte pages in SRAM, then writes each page to flash.

At that point the problem stops being "how do I write flash?" and becomes a system problem: receive bytes reliably, group them into pages, commit them, and avoid damaging the updater itself.

16.7.1 Intel HEX Recap

```
:LLAAAATT[DD...]CC
 | |   | |       └─ checksum (2 hex ASCII chars)
 | |   | └─────────── data bytes (LL × 2 hex chars)
 | |   └───────────── record type: 00=data, 01=EOF
 | └───────────────── address (4 hex ASCII chars, big-endian)
 └─────────────────── byte count LL (2 hex ASCII chars)
 ↑ start code ':'

Checksum: two's complement of (LL + addr_hi + addr_lo + type + sum(data))
The sum of all bytes in the record including checksum = 0x00 (mod 256).
```

16.7.2 Architecture of `selfupdate.S`

```
selfupdate_main:
  │
  ├─ init: stack, USART0 (PB2 TX, PB3 RX, 9600 baud at 3.333 MHz)
  ├─ LED0 (PA3) on → indicates update mode active
  ├─ send banner "SU\r\n"
  │
  └─ record_loop:
       ├─ wait for ':' (discards noise before start code)
       ├─ recv_hex_byte × 4: byte_count, addr_hi, addr_lo, type
       ├─ if type = 0x01 (EOF): send "OK\r\n", wait FBUSY, IJMP to 0x0000
       ├─ if type = 0x00 (data):
       │    ├─ set Z = record address (flash byte address)
       │    ├─ receive LL data bytes → SRAM page buffer
       │    ├─ if buffer >= 64 bytes: call nvm_write_page
       │    └─ verify checksum → send '.' (ACK) or '!' (NACK)
       └─ repeat
```

16.7.3 Register Allocation for `selfupdate.S`

```
Z  (R31:R30)   flash write address (maintained across records)
X  (R27:R26)   SRAM page buffer pointer (buf_base .. buf_base+63)
R16            scratch / UART byte / NVMCTRL command
```

```
R17             checksum accumulator (zeroed at each record start)
R18             byte count from record header
R19             record type (00 or 01)
R24             address low byte from record header
R25             address high byte from record header
R20, R21        scratch in helpers
R15             saved high nibble in recv_hex_byte
```

16.7.4 USART0 Initialisation at 3.333 MHz

The ATtiny3217's default clock is 3.333 MHz (20 MHz internal oscillator with the default /6 prescaler in CLKCTRL). The USART0 fractional baud register for 9600 baud:

```
BAUD_REG = (f_cpu × 64) / (16 × baud)
         = (3333333 × 64) / (16 × 9600)
         = 213333312 / 153600
         = 1389   (0x056D)
```

```
.equ BAUD_REG, 1389                  /* 9600 baud @ 3.333 MHz, error < 0.01% */

usart_init:
    sbi   VPORTB_DIR, 2              /* PB2 = TXD output                    */
    ldi   r16, lo8(BAUD_REG)
    sts   USART0_BAUDL, r16
    ldi   r16, hi8(BAUD_REG)
    sts   USART0_BAUDH, r16
    ldi   r16, USART_TXEN_bm | USART_RXEN_bm
    sts   USART0_CTRLB, r16          /* enable TX and RX                    */
    ret
    /* CTRLC defaults to 8N1 — no write needed */
```

Note: `VPORTB_DIR` is at I/O address `0x04` (within `sbi`/`cbi` range) — we use `sbi` rather than `sts` to set the single bit without disturbing other PORTB direction bits.

16.7.5 Full Listing: `src/selfupdate.S`

The complete source is at `src/selfupdate.S`. The core hex-receive loop and page-write call are shown below for reference:

```
.Ldata_loop:
    rcall recv_hex_byte          /* receive next data byte from HEX record   */
    st    X+, r16                /* store in SRAM page buffer                */
    add   r17, r16               /* accumulate checksum                      */
    dec   r18                    /* decrement byte count                     */

    /* check if page buffer is full (64 bytes written) */
    mov   r20, XL
    andi  r20, 0x3F              /* XL & 63 — non-zero means not at boundary */
    brne  .Lcheck_done           /* not at 64-byte boundary: continue        */

    /* page buffer is full: save X, write page, restore */
    movw  r20, XL                /* save X                                   */
    rcall nvm_write_page         /* writes page, advances Z by 64            */
    movw  XL, r20                /* restore X                                */
```

```
.Lcheck_done:
    tst   r18
    brne  .Ldata_loop            /* more bytes in this record                */
```

16.7.6 Jumping to the Updated Application

After the EOF record is received and the last page is written:

```
.Leof:
    rcall recv_hex_byte          /* consume EOF checksum byte (not verified)  */
    rcall nvm_wait               /* ensure final write is complete            */
    /* turn off LED0 (PA3) to signal update complete */
    sbi   VPORTA_OUT, 3          /* PA3 high = LED off (active-low)          */
    /* send "OK\r\n" */
    ldi   r16, 'O'
    rcall usart_tx
    ldi   r16, 'K'
    rcall usart_tx
    ldi   r16, '\r'
    rcall usart_tx
    ldi   r16, '\n'
    rcall usart_tx
    /* jump to new application */
    clr   ZL
    clr   ZH
    ijmp                         /* PC ← Z = 0x0000 (new firmware entry)     */
```

`IJMP` (Indirect Jump): sets the program counter to Z (2 cycles). With Z = 0, execution continues from the reset vector of the freshly written firmware.

Conceptually, that final jump is just a handoff. The updater finishes its work, then transfers control to the program it just installed.

16.7.7 Uploading Firmware

Use the existing `src/send_hex.py` helper, or any serial terminal. The selfupdate firmware sends `SU\r\n` instead of `BL\r\n`:

The helper rewrites ordinary Intel HEX input into 64-byte page records before sending. That matches the small updater's assumptions: each data record starts on a page boundary, and a short final page is padded with `0xFF`.

```
# Flash the self-update firmware via the Curiosity Nano debugger (one-time):
pymcuprog write -d attiny3217 -t uart -f selfupdate.hex

# Reset the board — LED0 turns on (active-low, so PA3 = 0 = LED on)
# Then immediately send application firmware:
python3 src/send_hex.py /dev/ttyUSB0 app.hex
```

Expected output:

```
Waiting for self-updater on /dev/ttyUSB0...
Self-updater ready
  record 1/18 OK
  record 2/18 OK
  ...
```

```
  record 18/18 OK
Upload complete. Application starting.
```

LED0 turns off when the update completes, and the new firmware starts.

16.8 BOOTEND Fuse: Protecting a Bootloader Region

16.8.1 The Problem

The self-update firmware in Section 16.6 writes to any flash address, including address 0x0000 — which is where the self-update firmware itself lives. If the update process is interrupted after erasing page 0 but before writing the new content, the device is bricked (no valid firmware at the reset vector). It also means buggy application firmware could accidentally overwrite the updater.

16.8.2 BOOTEND: Defining a Protected Region

The ATtiny3217 provides the **BOOTEND** and **APPEND** fuses. These divide flash into BOOT, APPCODE, and APPDATA sections in 256-byte units. For the simple updater layout, set `BOOTEND` to reserve space for the updater and leave `APPEND = 0` so the rest of flash is APPCODE:

```
FUSES.BOOTEND value, with APPEND = 0:
  0x01  — 256 bytes BOOT (pages 0–3)
  0x02  — 512 bytes BOOT (pages 0–7)
  0x04  — 1024 bytes BOOT (pages 0–15)
  ...

Boot section:        0x0000 .. (BOOTEND × 256 - 1)
APPCODE section:     (BOOTEND × 256) .. 0x7FFF
```

With `BOOTEND = 0x04` (1 KB boot section), the self-update firmware occupies pages 0–15 (addresses 0x0000–0x03FF). The application firmware lives at 0x0400 onward. The NVMCTRL enforces section privileges. BOOT code can write APPCODE and APPDATA. APPCODE can write APPDATA. APPDATA cannot write flash or EEPROM. The CPU cannot write the BOOT section.

This is the safety boundary that turns a self-programming demo into something you can trust more. Without it, the updater can erase the very code that is needed to recover from a failed update.

16.8.3 Flash Layout With BOOTEND = 0x04

```
ATtiny3217 flash (32 KB) with BOOTEND = 0x04
─────────────────────────────────────────────────────────────

0x0000  ┌──────────────────────────────────┐
        │   Boot section (1 KB)            │
        │   selfupdate.S lives here        │
        │   CPU cannot write this region   │
        │   Update through UPDI            │
0x03FF  ├──────────────────────────────────┤
0x0400  │                                  │
        │   Application section (31 KB)    │
        │   Receives new firmware          │
```

```
        │   via self-update loop          │
        │                                 │
0x7FFF  └─────────────────────────────────┘
```

16.8.4 Application Entry Point

When `BOOTEND = 0x04`, the application reset vector is at **0x0400**. The self-update firmware must jump there (not to 0x0000) after a successful update:

```
/* With BOOTEND = 0x04: application starts at 0x0400 */
.equ APP_START, 0x0400

.Leof:
    rcall recv_hex_byte
    rcall nvm_wait
    ldi   ZL, lo8(APP_START)
    ldi   ZH, hi8(APP_START)
    ijmp                     /* jump to application section start            */
```

The application must be built with its `.text` section starting at `0x0400`:

```
avr-gcc -mmcu=attiny3217 -nostartfiles \
        -Wl,--section-start=.text=0x0400 \
        -o app.elf app.S
```

16.8.5 Run-Time Locks: BOOTLOCK and APCWP

`NVMCTRL_CTRLB` has two run-time lock bits that remain active until reset:

- `APCWP` prevents further writes to APPCODE.
- `BOOTLOCK` prevents reads and execution from BOOT.

Setting `APCWP` after the update completes provides defense-in-depth:

```
/* After successful update, lock application section against further writes  */
ldi   r16, 0x01              /* APCWP bit                                    */
sts   NVMCTRL_CTRLB, r16
```

This is cleared on every reset, so it does not permanently lock the device.

Think of `APCWP` as a temporary guardrail for the current boot session, not as a replacement for fuse-based protection.

16.9 Building and Flashing

16.9.1 Building nvmctrl_write.S (standalone test)

```
# Assemble the NVM write routine alone (produces an ELF for inspection)
avr-gcc -mmcu=attiny3217 -nostartfiles -nodefaultlibs \
        -o nvmctrl_write.elf src/nvmctrl_write.S

# Check sections and disassemble to verify instruction encoding
avr-objdump -h nvmctrl_write.elf
avr-objdump -d nvmctrl_write.elf
```

16.9.2 Building selfupdate.S

```
# Assemble the self-update firmware
# Without BOOTEND protection: starts at 0x0000
avr-gcc -mmcu=attiny3217 -nostartfiles -nodefaultlibs \
        -o selfupdate.elf src/selfupdate.S

# With BOOTEND=0x04: starts at 0x0000, app section starts at 0x0400
# The selfupdate firmware itself is the same; linker placement is 0x0000
avr-gcc -mmcu=attiny3217 -nostartfiles -nodefaultlibs \
        -o selfupdate.elf src/selfupdate.S

# Size check — .text must fit within chosen boot section (1 KB = BOOTEND 0x04)
avr-objdump -h selfupdate.elf
```

Expected section output excerpt:

```
Idx Name          Size      VMA
  1 .text         0000014a  00000000
  2 .bss          00000040  00803800
```

`0x14a` is 330 bytes of flash code, which fits comfortably within the 1 KB BOOT section. The 64-byte `.bss` allocation is the SRAM page buffer.

16.9.3 Converting to HEX

```
avr-objcopy -O ihex selfupdate.elf selfupdate.hex
avr-objcopy -O ihex app.elf app.hex
```

16.9.4 Flashing with pymcuprog

```
# Program selfupdate firmware via UPDI (requires Curiosity Nano USB connection)
pymcuprog write -d attiny3217 -t uart -f selfupdate.hex

# Set BOOTEND fuse to 0x04 (1 KB boot section)
# WARNING: verify the exact pymcuprog fuse syntax for your installed version.
# BOOTEND is fuse byte 8; write value 0x04.

# Read back fuse to verify
pymcuprog read -d attiny3217 -t uart
```

Warning: BOOTEND is in fuse byte 8 on ATtiny3217. An incorrect value can make the device appear to run no code. Always read back fuses after writing. UPDI can always recover the device regardless of fuse settings (unlike ISP on ATmega parts where the BOOTRST fuse can complicate recovery).

That recovery path is important operationally: experimenting with self-update code is much less risky when you know UPDI can still get you back in.

16.9.5 Verifying the Disassembly

After building, verify the CCP sequence is encoded correctly:

```
avr-objdump -d selfupdate.elf | grep -A5 "CPU_CCP\|0034"
```

Look for two back-to-back `sts` instructions with addresses `0x0034` (CCP) and `0x1000` (NVMCTRL_CTRLA). No instruction should appear between them in the listing.

16.9.6 Building an Application for the Protected Layout

```
# Application must be linked at 0x0400 when BOOTEND = 0x04
avr-gcc -mmcu=attiny3217 -nostartfiles -nodefaultlibs \
        -Wl,--section-start=.text=0x0400 \
        -o app.elf app.S
avr-objcopy -O ihex app.elf app.hex

# Send via self-update firmware:
python3 src/send_hex.py /dev/ttyUSB0 app.hex
```

16.10 Common Pitfalls

16.10.1 Pitfall 1 — CCP Window Violated

Symptom: NVMCTRL_CTRLA write appears to succeed (no error) but no flash write occurs; WRERROR is set in STATUS.

Cause: An instruction was inserted between `STS CPU_CCP, r16` and `STS NVMCTRL_CTRLA, r16`, or an interrupt fired inside the window and consumed cycles.

Fix: Use `CLI` before the CCP sequence and `SEI` immediately after the protected write. Ensure no instruction sits between the two `STS` calls.

```
/* WRONG — ldi between STS instructions wastes a cycle inside the window */
sts   CPU_CCP, r16
ldi   r16, 0x03           /* ← this is inside the window, shrinks margin    */
sts   NVMCTRL_CTRLA, r16

/* CORRECT — load command value BEFORE writing CCP key */
ldi   r16, 0x03           /* load command first (outside window)            */
...
ldi   r17, CCP_SPM_gc
sts   CPU_CCP, r17        /* open window                                    */
sts   NVMCTRL_CTRLA, r16  /* issue command (within window)                  */
```

Note the subtlety: if `r16` holds the CCP key when you write `CPU_CCP`, you must reload `r16` with the command before the protected write, or use two registers. The corrected pattern above uses two registers to keep the command value ready before opening the CCP window.

That is a good example of why “almost correct” is not good enough here. A sequence can look visually right and still fail because one register was reused at the wrong moment.

16.10.2 Pitfall 2 — Page Buffer Not Aligned

Symptom: The first few bytes of a written page are corrupted or contain old flash content.

Cause: The mapped flash address passed to `nvm_write_page` is not 64-byte aligned. The page buffer fill loop always fills from byte 0 to byte 63 of the chosen page. If Z points into the middle of a page, the first `ST Z+, Rn` writes to a mid-page buffer slot. The earlier slots contain the current page-buffer state, not an automatic copy of the old flash page.

Fix: Always pass a 64-byte aligned address:

```
/* Align mapped Z to 64-byte page boundary: clear bits 5:0 */
andi  ZL, 0xC0    /* mask: 1100 0000 — clear bits 5:0 */
```

Page alignment bugs are especially deceptive because most of the write may look correct. Only the offsets near the page boundary reveal the mistake.

16.10.3 Pitfall 3 — Writing to Flash During Interrupt

Symptom: Application ISR fires while `nvm_write_page` is between the page buffer fill and the PAGEERASEWRITE command. The ISR reads stale flash content via `LPM` from the page being written.

Cause: The page buffer is not yet committed. If an ISR attempts to read instruction or data from flash in the middle of a PAGEERASEWRITE, it reads the erased (0xFF) state during the erase phase.

Fix: Disable global interrupts before the entire write sequence and re-enable after:

```
cli
rcall nvm_write_page
sei
```

The `nvm_write_page` routine already disables interrupts around the CCP write, but a complete `CLI`/`SEI` around the entire page write is safer if you have ISRs active.

16.10.4 Pitfall 4 — Forgetting to Wait for FBUSY Before Each Command

Symptom: Random corruption of flash pages, or WRERROR flag set frequently.

Cause: A second `PAGEERASEWRITE` command was issued before the first completed. NVMCTRL_STATUS.FBUSY must be 0 before issuing any command.

Fix: The `nvm_wait` routine at the start of `nvm_write_page` ensures this. If you issue NVMCTRL commands from multiple places in your code, always call `nvm_wait` first.

16.10.5 Pitfall 5 — Writing the Boot Section From Application Code

Symptom: Write to pages 0–(BOOTEND-based boundary) triggers WRERROR but no actual write.

Cause: With `BOOTEND` set, NVMCTRL enforces section privileges. The CPU cannot write the BOOT section through NVMCTRL.

Fix: This is intentional behaviour — the boot section protection is working correctly. If you need to update the self-update firmware, use UPDI with `pymcuprog`.

In other words, `WRERROR` here is a feature, not a malfunction.

16.10.6 Pitfall 6 — UPDI Port Floating (PA0)

Symptom: `avrdude` cannot connect; "timeout communicating with programmer."

Cause: On the Curiosity Nano, PA0 is used for UPDI. If you configure PA0 as a GPIO output in your firmware, UPDI communication breaks. The on-board on-board debugger pulls PA0 to its own UPDI driver.

Fix: Never configure PA0 as GPIO output. Leave PA0 at its power-on default (UPDI function). If your application needs an extra GPIO and PA0 was used, remap to a different pin.

16.10.7 Pitfall 7 — Baud Rate at Non-Default Clock

Symptom: Garbage characters received; UART communication fails after changing CLKCTRL prescaler.

Cause: The BAUD_REG constant was calculated for 3.333 MHz (default clock). If your firmware changes CLKCTRL_MCLKCTRLB to select a different frequency before calling `usart_init`, the baud rate will be wrong.

Fix: Recalculate BAUD_REG for the actual f_cpu, or initialise USART before changing the clock. The self-update firmware deliberately runs at the default clock to avoid this dependency.

16.11 Summary

```
ATtiny3217 self-programming (AVRXMEGA3):

Flash geometry:
  32 KB total, 64-byte pages, 512 pages
  Data-space flash base address: 0x8000
  Page buffer fill: ST Zn+, Rn  (plain store instruction)

NVMCTRL registers:
  NVMCTRL_CTRLA   0x1000  command register
  NVMCTRL_STATUS  0x1002  FBUSY (bit 0), WRERROR (bit 2)

Write sequence for one 64-byte page:
  1. Poll NVMCTRL_STATUS until FBUSY = 0
  2. STS NVMCTRL_CTRLA, 0x00      (NOCMD — clear errors)
  3. 64 × ST Z+, Rn                (fill page buffer at mapped flash address)
  4. SUBI/SBCI Z, 64               (rewind Z to page base)
  5. CLI
     STS CPU_CCP, 0x9D             (unlock CCP)
     STS NVMCTRL_CTRLA, 0x03      (PAGEERASEWRITE as next instruction)
     SEI
  6. Poll NVMCTRL_STATUS until FBUSY = 0

CCP (Configuration Change Protection):
  Register: 0x0034 (CPU_CCP)
  Key for NVMCTRL: 0x9D (CCP_SPM_gc)
  Window: next four CPU instructions
  Rule: STS CPU_CCP must be immediately followed by STS NVMCTRL_CTRLA
```

```
BOOTEND fuse (fuse byte 8):
  0x04 = 1 KB boot section (selfupdate.S fits here)
  App section starts at BOOTEND × 256

Key instructions used:
  SUBI/SBCI   — 16-bit pointer adjustment for +/-64
  IJMP        — indirect jump through Z (2 cy)

Build:
  avr-gcc -mmcu=attiny3217 -nostartfiles -nodefaultlibs \
          -o selfupdate.elf src/selfupdate.S
  avr-objcopy -O ihex selfupdate.elf selfupdate.hex
  pymcuprog write -d attiny3217 -t uart -f selfupdate.hex
```

16.12 Exercises

1. The `nvm_wait` polling loop costs 5 cycles per iteration at 3.333 MHz. A PAGEERASEWRITE operation on ATtiny3217 takes approximately 2.5 ms. How many loop iterations does `nvm_wait` execute while waiting? At 3.333 MHz, what is the duration of each iteration in microseconds? Is a timer-based timeout (to detect a stuck NVM controller) practical within the same loop?

2. The CCP window is four CPU instructions. Why does the source place `STS NVMCTRL_CTRLA, r16` immediately after `STS CPU_CCP, r17` even though the hardware allows a small window?

3. The `nvm_write_page` routine fills the page buffer with 64 iterations of `LD r18, X+` + `ST Z+, r18`. Each pair takes `2 + 2 = 4` cycles. Calculate the total cycle count for the fill loop plus the surrounding overhead (wait, NOCMD write, pointer rewind, CCP sequence, final wait entry). What is the minimum time in milliseconds required to write 16 pages (1 KB)?

4. The self-update firmware in `selfupdate.S` does not verify the EOF record's checksum. Add checksum verification: if the EOF checksum is incorrect, send `'!'`, turn LED0 on (it was turned off on successful EOF), and loop back to `record_loop` without jumping to the application. What register holds the checksum accumulator at the point of the EOF handler, and what condition do you test?

5. The current `selfupdate.S` jumps to address `0x0000` on EOF (the reset vector). If `BOOTEND = 0x04`, the application section begins at `0x0400`. Modify the EOF handler to: (a) check that at least one page in the application section was written (track the highest written address in a register), (b) jump to `0x0400` instead of `0x0000`. Show the assembly changes.

6. Implement a **write-verify** step: after writing each page with `nvm_write_page`, read back the 64 bytes using `LPM Z+, r16` and compare against the SRAM buffer. If any byte mismatches, retransmit `'!'` (NACK) and re-request the record. Where in the `selfupdate.S` main loop would you add this? What additional register or memory is needed?

7. Add a page-address guard that rejects any incoming data record below `BOOTEND * 256`. On rejection, send `'!'` and discard the record data. Which two header bytes do you compare, and where should the check occur?

8. Using `avr-objdump -d selfupdate.elf`, locate the two `STS` instructions that form the CCP sequence. Record their addresses. What is the byte offset between them? Confirm that no other instruction appears at an address between them. Now deliberately insert a

`NOP` between them in the source and rebuild — does `avr-objdump` show the NOP between the two `STS` instructions? Describe the hardware consequence of this change.

Next: Chapter 17 — Bit Manipulation and Fixed-Point Arithmetic

Chapter 17

Bit Manipulation & Fixed-Point Math

AVR has no hardware divider and no floating-point unit. The ATtiny3217 does include the AVR hardware multiplier, but shift-and-add multiplication remains useful for understanding the machine model and for AVR parts that omit `MUL`. When computation must be fast and deterministic — sensor scaling, display formatting, signal processing — the answers live in bit manipulation and fixed-point arithmetic.

This chapter covers:

1. **Shift-and-add multiply** — multiplication using only shifts and adds; useful as a portable fallback and as a teaching algorithm.
2. **Fixed-point Q8.8 arithmetic** — representing and operating on fractional numbers in 16-bit register pairs.
3. **BCD arithmetic** — binary-coded decimal for human-readable display.
4. **Practical example** — convert a 16-bit binary sensor reading to BCD for a four-digit 7-segment display, using all three techniques.

The common thread is that all of these techniques trade mathematical elegance for predictability. On a small MCU, that is usually a good trade.

17.1 Shift Operations

AVR provides five shift/rotate instructions. All take a single register operand and execute in **1 cycle**.

Instruction	Operation	Flags
`LSL Rd`	`C ← Rd7; Rd ← Rd << 1; Rd0 = 0`	`C, Z, N, V, H`
`LSR Rd`	`Rd7 = 0; Rd ← Rd >> 1; C ← Rd0`	`C, Z, N, V`
`ASR Rd`	`Rd7 unchanged; Rd ← Rd >> 1; C ← Rd0`	`C, Z, N, V`
`ROL Rd`	`C ← Rd7; Rd ← Rd << 1; Rd0 = old C`	`C, Z, N, V, H`
`ROR Rd`	`Rd7 = old C; Rd ← Rd >> 1; C ← Rd0`	`C, Z, N, V`

LSL/LSR: logical shift, zero fill. `LSL Rd` multiplies by 2; `LSR Rd` divides by 2 (unsigned).

ASR: arithmetic shift right — sign-extends during right shift, preserving the sign of a signed value. `ASR Rd` divides a signed byte by 2 (rounds toward negative infinity).

ROL/ROR: rotate through carry. The carry flag acts as a 9th bit. This is how multi-byte shifts are chained:

That is the key idea behind most "wide" arithmetic on AVR. You do not get a 16-bit or 32-bit shift instruction; you build one byte at a time and let carry connect the pieces.

```
/* Logical left shift a 16-bit value in R25:R24 by 1 bit */
lsl   r24            /* shift low byte left; bit 7 → C        */
rol   r25            /* shift high byte left; C → bit 0       */
```

Multi-byte right shift (logical):

```
/* Logical right shift R25:R24 by 1 bit */
lsr   r25            /* shift high byte right; bit 0 → C      */
ror   r24            /* shift low byte right; C → bit 7       */
```

17.1.1 Register Pair Diagram — 16-bit LSL

```
Before:  R25[b7..b0]  R24[b7..b0]
                                  └── (bit 0 = 0 after LSL)
After:
  R24: b6 b5 b4 b3 b2 b1 b0  0   ← LSL r24 (b7 of R24 → C)
  R25: b6 b5 b4 b3 b2 b1 b0  C   ← ROL r25 (C → bit 0 of R25)
  C  = old R25[b7]
```

17.2 Shift-and-Add Multiplication

The `MUL` instruction (8 × 8 → 16 unsigned) exists on ATtiny3217 and many other AVR devices. Some smaller or older AVR parts omit hardware multiply, so the shift-and-add algorithm is still worth knowing.

The shift-and-add algorithm works on any AVR: for each bit in the multiplier, if the bit is set, add the multiplicand (shifted appropriately) to the accumulator.

17.2.1 Algorithm — Unsigned 8×8 → 16

```
product = 0
for bit = 0 to 7:
    if multiplier[bit] == 1:
        product += multiplicand << bit
    (equivalently: shift multiplicand left each iteration,
     add if LSB of multiplier is 1, then shift multiplier right)
```

17.2.2 Implementation

Instruction	Operands	Cycles	Flags
`LSL Rd`	Rd	1	C,Z,N,V,H
`LSR Rd`	Rd	1	C,Z,N,V
`ROL Rd`	Rd	1	C,Z,N,V,H
`ROR Rd`	Rd	1	C,Z,N,V
`ADD Rd,Rr`	Rd, Rr	1	C,Z,N,V,H

Instruction	Operands	Cycles	Flags
`ADC Rd,Rr`	Rd, Rr	1	C,Z,N,V,H
`BRCC k`	k (−64..63)	1/2	—
`DEC Rd`	Rd	1	Z,N,V

```
/*
 * mul8u — unsigned 8×8 → 16 multiply (shift-and-add)
 *
 * Entry:  R16 = multiplicand
 *         R17 = multiplier
 * Exit:   R25:R24 = R16 × R17 (unsigned 16-bit product)
 * Clobbers: R16, R17, R18, R20
 * Cycles: worst-case ~56
 */
mul8u:
    clr   r24           /* product = 0                                     */
    clr   r25
    clr   r20           /* multiplicand high byte = 0                      */
    ldi   r18, 8        /* 8 bits to process                               */
.Lmul8u_loop:
    lsr   r17           /* multiplier >>= 1; old bit 0 → C                */
    brcc  .Lmul8u_skip
    add   r24, r16      /* product_lo  += multiplicand_lo                  */
    adc   r25, r20      /* product_hi  += multiplicand_hi + carry          */
.Lmul8u_skip:
    lsl   r16           /* multiplicand_lo <<= 1; old bit 7 → C           */
    rol   r20           /* multiplicand_hi <<= 1; C → bit 0               */
    dec   r18
    brne  .Lmul8u_loop
    ret
```

The key detail is that the multiplicand is tracked as a 16-bit value (`R20:R16`), so left shifts preserve the carry-out from the low byte.

That is what makes the routine correct for products larger than 255. If you shifted only the low byte, the upper half of the multiplicand would vanish.

Trace — 3 × 5 (i.e. 0b00000011 × 0b00000101 = 15):

```
r16=3, r17=5, r20=0, r24=0, r25=0

Iter 1: lsr r17 → r17=2, C=1 (bit 0 of 5)
        add r24, r16 → r24=3; adc r25, r20 → r25=0
        lsl r16 → r16=6, C=0; rol r20 → r20=0

Iter 2: lsr r17 → r17=1, C=0 (bit 0 of 2)
        skip
        lsl r16 → r16=12; rol r20 → r20=0

Iter 3: lsr r17 → r17=0, C=1 (bit 0 of 1)
        add r24, r16 → r24=3+12=15; adc r25, r20 → r25=0
        lsl r16 → r16=24; rol r20 → r20=0

Iters 4-8: r17=0, C=0 each time → all skip
```

```
Result: r25:r24 = 0x000F = 15  OK
```

17.2.3 Signed Multiply — mul8s (8×8 → 16 signed)

Signed multiply extends naturally: run the unsigned loop for 7 bits, then treat the MSB of the multiplier as negative (two's complement weight −128):

```
/*
 * mul8s — signed 8×8 → 16 multiply
 *
 * Entry:  R16 = multiplicand (signed)
 *         R17 = multiplier   (signed)
 * Exit:   R25:R24 = R16 × R17 (signed 16-bit product)
 * Clobbers: R16, R17, R18, R20
 */
mul8s:
    clr   r24
    clr   r25
    /* sign-extend R16 into R20 */
    mov   r20, r16
    lsl   r20           /* old bit 7 → C                                  */
    sbc   r20, r20      /* r20 = 0xFF if negative, 0x00 if positive       */
    ldi   r18, 7        /* process 7 unsigned bits                        */
.Lmul8s_loop:
    lsr   r17
    brcc  .Lmul8s_skip
    add   r24, r16
    adc   r25, r20
.Lmul8s_skip:
    lsl   r16
    rol   r20
    dec   r18
    brne  .Lmul8s_loop
    /* bit 7 of multiplier: subtract instead of add (two's complement) */
    lsr   r17           /* bit 7 → C                                      */
    brcc  .Lmul8s_done
    sub   r24, r16      /* product -= (multiplicand << 7)                 */
    sbc   r25, r20
.Lmul8s_done:
    ret
```

`SBC Rd, Rr` subtracts Rr and the carry flag from Rd. Here, after `lsr r17`, C holds the sign bit of the original multiplier. If set, we subtract the current (shifted) multiplicand — the two's complement correction for the negative weight of bit 7.

This is one of those places where two's complement stops being abstract. The top bit is not just "another positive weight"; it contributes a negative term, so the final step has to subtract instead of add.

17.3 Fixed-Point Q8.8 Arithmetic

Fixed-point represents a real number as an integer scaled by a power of two. **Q8.8** uses a 16-bit register pair where the high byte is the integer part and the low byte is the fractional part:

The advantage is that the CPU still performs ordinary integer arithmetic. The "fractional" meaning comes from how you interpret the bits, not from special hardware support.

```
Q8.8 value = register_pair / 256

Bit layout (R25:R24 example):
  R25 [b15..b8]   R24 [b7..b0]
   |              |
   └── integer ───┘└── fraction ──┘
    (×1)              (×1/256)

Range: −128.0 to +127.99609375 (signed)
        0.0   to  255.99609375 (unsigned)

Resolution: 1/256 ≈ 0.0039
```

17.3.1 Converting to Q8.8

```
/* Integer 42 → Q8.8: shift left 8 bits (multiply by 256) */
ldi   r24, 0        /* fractional part = 0                                    */
ldi   r25, 42       /* integer part = 42                                      */
/* R25:R24 = 0x2A00 = 10752 raw = 42.0 in Q8.8 */

/* Fractional constant 0.5 → Q8.8 = 128 = 0x0080 */
ldi   r24, 128
clr   r25
/* R25:R24 = 0x0080 = 0.5 in Q8.8 */

/* Convert from Q8.8 back to integer (truncate) */
mov   r16, r25      /* integer part = high byte                               */
```

17.3.2 Q8.8 Addition

Identical to 16-bit integer addition — no adjustment needed:

That simplicity is one of the main reasons fixed-point is attractive. Once two values share the same scaling, addition and subtraction are just normal integer operations.

```
/* R25:R24 += R23:R22 (Q8.8 + Q8.8) */
add   r24, r22
adc   r25, r23
```

17.3.3 Q8.8 Multiplication (Q8.8 × Q8.8 → Q8.8)

Two Q8.8 values multiplied give a Q16.16 result in a 32-bit space. To get a Q8.8 result, extract bits [23:8] of the 32-bit product — effectively shift right 8 bits:

```
A = a15..a0 (Q8.8)   B = b15..b0 (Q8.8)
A × B = 32-bit Q16.16
Q8.8 result = (A × B) >> 8  = bits [23:8] of the 32-bit product
```

In practice, build the 32-bit product as four 8×8 partial products:

```
A = AH:AL   (high byte : low byte)
B = BH:BL

A × B = AH×BH (weight 2^16) + AH×BL (weight 2^8)
             + AL×BH (weight 2^8) + AL×BL (weight 2^0)

Q8.8 result = shift right 8 → take bits [23:8]:
  bits [15:8] of (AH×BL + AL×BH) + AH×BH (lower byte) + carry from AL×BL
```

The ATtiny3217 is an AVRxt device with the hardware multiply instructions available; Microchip's instruction set manual lists `MUL`, `MULS`, and `MULSU` as two-cycle operations that write the 16-bit product to `R1:R0`. The routine below deliberately uses `mul8u` from Section 17.2 so the byte decomposition is visible and so the same listing still works on smaller AVR devices that omit hardware multiply.

The bookkeeping looks tedious, but the idea is straightforward: break the 16-bit multiply into byte-sized pieces, then keep only the middle 16 bits that correspond to the Q8.8 result.

```
/*
 * mulq88 — unsigned Q8.8 × Q8.8 → Q8.8 using mul8u
 *
 * Entry:  R25:R24 = A (Q8.8),  R23:R22 = B (Q8.8)
 * Exit:   R25:R24 = A × B (Q8.8, truncated)
 */
mulq88:
    push  r12
    push  r13
    push  r14
    push  r15

    mov   r12, r24      /* AL                                              */
    mov   r13, r25      /* AH                                              */
    mov   r14, r22      /* BL                                              */
    mov   r15, r23      /* BH                                              */

    /* AL × BL */
    mov   r16, r12
    mov   r17, r14
    rcall mul8u
    mov   r21, r25      /* result low byte starts with high(AL×BL)         */
    clr   r22           /* result high byte                                */

    /* AL × BH */
    mov   r16, r12
    mov   r17, r15
    rcall mul8u
    add   r21, r24
    adc   r22, r25

    /* AH × BL */
    mov   r16, r13
    mov   r17, r14
    rcall mul8u
    add   r21, r24
    adc   r22, r25
```

```
    /* AH × BH */
    mov   r16, r13
    mov   r17, r15
    rcall mul8u
    add   r22, r24      /* low byte of AH×BH contributes at result high byte */

    mov   r24, r21
    mov   r25, r22
    pop   r15
    pop   r14
    pop   r13
    pop   r12
    ret
```

Full listing in `src/fixedpoint.S`.

17.4 BCD Arithmetic

Binary-Coded Decimal stores each decimal digit as a 4-bit nibble. Two formats:

```
Unpacked BCD: one digit per byte (low nibble used, high nibble = 0)
  Decimal 47  →  byte 0x04, byte 0x07

Packed BCD:   two digits per byte
  Decimal 47  →  single byte 0x47
```

17.4.1 Decimal Adjust on AVR

Unlike some 8-bit CPUs, AVR does **not** provide a general `DAA` instruction for packed-BCD correction. After a binary `ADD`/`ADC`, packed BCD must be corrected manually using the half-carry and carry flags.

17.4.2 H Flag and BCD Correction

The H (half-carry) flag is set when a carry propagates from bit 3 to bit 4 during addition — i.e., when the low nibble overflows. This is exactly the condition that requires BCD correction:

```
After binary ADD of two packed BCD bytes:
  If low nibble > 9 OR H flag set:  add 0x06 to correct low nibble
  If high nibble > 9 OR C flag set: add 0x60 to correct high nibble
```

```
/*
 * bcd_add — add two single packed BCD bytes
 *
 * Entry:  R16 = BCD byte A (e.g. 0x47 for decimal 47)
 *         R17 = BCD byte B
 * Exit:   R16 = BCD sum (low byte), C = 1 if result > 99
 * Clobbers: R18
 */
bcd_add:
    add   r16, r17      /* binary add                                        */
    ldi   r18, 0
```

```
    /* check low nibble: H flag set or low nibble > 9 */
    brhs  .Lbcd_fix_low /* H set → low nibble overflowed                    */
    mov   r18, r16
    andi  r18, 0x0F
    cpi   r18, 10
    brlo  .Lbcd_check_high
.Lbcd_fix_low:
    subi  r16, -6       /* add 6 to low nibble (subi x, -N = addi x, N)     */
.Lbcd_check_high:
    brcs  .Lbcd_fix_high  /* carry from low correction means high overflowed */
    mov   r18, r16
    andi  r18, 0xF0
    cpi   r18, 0xA0
    brlo  .Lbcd_done
.Lbcd_fix_high:
    subi  r16, -0x60    /* add 0x60 to high nibble                          */
.Lbcd_done:
    ret
```

`BRHS` (Branch if Half-carry Set) and `BRHC` (Branch if Half-carry Clear) branch directly on the H flag.

That matters because packed BCD correction is really nibble arithmetic. The H flag tells you whether the low decimal digit overflowed even when the full byte did not.

Instruction	Branch condition	Cycles
`BRHS k`	H = 1	1/2
`BRHC k`	H = 0	1/2

17.5 Binary to BCD Conversion

Converting a multi-digit binary number to BCD for display is a recurring task. The **double-dabble** (shift-and-add-3) algorithm converts an N-bit binary number to packed BCD in N iterations, requiring no division:

```
Algorithm:
  Place binary value in a shift register alongside N/4 BCD digits (all zero).
  Repeat N times:
    For each BCD digit: if digit > 4, add 3 to that digit.
    Shift the entire combined register left by 1 bit.
  After N iterations, BCD digits contain the decimal representation.
```

The "add 3" step ensures that a BCD digit that is about to overflow (>= 5) will produce the correct carry into the next BCD digit after the shift (since shifting left doubles, and 5×2=10 wraps in BCD; pre-adding 3 makes (5+3)×2=16, which carries correctly as 6 in the upper digit and 0 in the lower).

If double-dabble feels magical at first, focus on that one fact: digits 5..9 need help before the shift so that a binary left shift turns into a valid decimal carry.

17.5.1 16-bit Binary to 5-Digit BCD

A 16-bit value (0–65535) needs 5 BCD digits (0–9 each), packed into 3 bytes (two packed + one partial):

```
Packed BCD storage: [tens-thousands][thousands|hundreds][tens|ones]
Example: 65535 → 0x06, 0x55, 0x35
```

```
/*
 * bin16_to_bcd — convert 16-bit unsigned binary to 5 packed BCD digits
 *
 * Entry:  R25:R24 = 16-bit value (0–65535)
 * Exit:   R21:R20:R19 = packed BCD (5 digits)
 *           R21[3:0] = ten-thousands digit
 *           R20[7:4] = thousands digit
 *           R20[3:0] = hundreds digit
 *           R19[7:4] = tens digit
 *           R19[3:0] = ones digit
 * Clobbers: R16, R17, R22
 */
bin16_to_bcd:
    clr   r19           /* BCD digits [tens|ones] = 0                      */
    clr   r20           /* BCD digits [thousands|hundreds] = 0             */
    clr   r21           /* BCD digit  [ten-thousands] = 0                  */
    ldi   r16, 16       /* loop counter: 16 bits to process                */

.Lbcd_loop:
    /* For each BCD nibble: if > 4, add 3 */
    rcall bcd_adjust    /* adjust R21:R20:R19                              */

    /* Shift binary value and BCD digits left by 1 */
    lsl   r24           /* binary low byte: bit 7 → C                     */
    rol   r25           /* binary high byte: C → bit 0; bit 7 → C         */
    rol   r19           /* BCD byte 0: C in, bit 7 → C                    */
    rol   r20           /* BCD byte 1: C in, bit 7 → C                    */
    rol   r21           /* BCD byte 2: C in (no further shift needed)      */

    dec   r16
    brne  .Lbcd_loop
    ret

/*
 * bcd_adjust — add 3 to any BCD nibble > 4 in R21:R20:R19
 */
bcd_adjust:
    /* Check and adjust each nibble */
    /* R19 low nibble (ones) */
    mov   r22, r19
    andi  r22, 0x0F
    cpi   r22, 5
    brlo  .Lbcd_adj_r19h
    subi  r19, -3       /* add 3 to low nibble                             */
.Lbcd_adj_r19h:
    /* R19 high nibble (tens) */
    mov   r22, r19
```

```
    andi  r22, 0xF0
    cpi   r22, 0x50
    brlo  .Lbcd_adj_r20l
    subi  r19, -0x30
.Lbcd_adj_r20l:
    /* R20 low nibble (hundreds) */
    mov   r22, r20
    andi  r22, 0x0F
    cpi   r22, 5
    brlo  .Lbcd_adj_r20h
    subi  r20, -3
.Lbcd_adj_r20h:
    /* R20 high nibble (thousands) */
    mov   r22, r20
    andi  r22, 0xF0
    cpi   r22, 0x50
    brlo  .Lbcd_adj_r21
    subi  r20, -0x30
.Lbcd_adj_r21:
    /* R21 low nibble (ten-thousands) — max digit is 6, never > 4 until late */
    mov   r22, r21
    andi  r22, 0x0F
    cpi   r22, 5
    brlo  .Lbcd_adj_done
    subi  r21, -3
.Lbcd_adj_done:
    ret
```

17.5.2 Build and Test

```
avr-gcc -mmcu=attiny3217 -nostartfiles \
        -o bin2bcd.elf bin16_to_bcd.S
avr-objdump -d bin2bcd.elf
```

Simulate with `simavr`:

```
simavr -m attiny3217 bin2bcd.elf
```

17.6 Complete Example: Sensor Reading to 7-Segment Display

A 10-bit ADC reading (0–1023) from a temperature sensor is scaled to a °C value (0–99.9°C, one decimal place), then displayed on a four-digit 7-segment display connected over SPI.

17.6.1 Scaling: ADC → Temperature

Assume linear sensor: 0 → 0.0°C, 1023 → 99.9°C.

```
temp_Q8 = (adc_reading × 25344) >> 16
```

Where 25344 = round(99.9 / 1023 × 65536) — maps the full ADC range to 99.9 scaled as a 16-bit integer. The result is in units of 0.1°C.

Or, equivalently with Q8.8:

```
scale_factor = 99.9 × 256 / 1023 ≈ 24.99... ≈ Q8.8 value 0x18FF
temp_Q88 = adc_reading_Q88 × scale_factor_Q88 (Q8.8 multiply)
```

We use the simpler fixed shift approach here to avoid an extra Q8.8 multiply.

The full example is in `src/display.S`. Below are the key routines.

This example is intentionally end-to-end. It shows why bit tricks and numeric formats matter: eventually the value has to become something a human can read.

17.6.2 Register Map for display.S

```
R25:R24      ADC reading (10-bit, 0–1023)
R23:R22      scaled temperature × 10 (e.g. 365 = 36.5°C)
R21:R20:R19 BCD digits (five digits packed)
R18          digit index (0–3)
R17          segment data byte to SPI
R16          scratch
Z (R31:R30) pointer to 7-segment LUT in flash
```

17.6.3 7-Segment LUT

```
.section .rodata
seg7_lut:
    /* segments: gfedcba (bit 6..0), active high */
    .byte 0x3F   /* 0: abcdef  = 0b0111111                                  */
    .byte 0x06   /* 1: bc      = 0b0000110                                  */
    .byte 0x5B   /* 2: abdeg   = 0b1011011                                  */
    .byte 0x4F   /* 3: abcdg   = 0b1001111                                  */
    .byte 0x66   /* 4: bcfg    = 0b1100110                                  */
    .byte 0x6D   /* 5: acdfg   = 0b1101101                                  */
    .byte 0x7D   /* 6: acdefg  = 0b1111101                                  */
    .byte 0x07   /* 7: abc     = 0b0000111                                  */
    .byte 0x7F   /* 8: all     = 0b1111111                                  */
    .byte 0x6F   /* 9: abcdfg  = 0b1101111                                  */
```

17.6.4 Scale ADC to Temperature

The implementation in `src/display.S` uses a 32-bit intermediate for:

```
temp10 = (adc × 999 + 512) >> 10
```

That keeps the result accurate across the full 0-1023 ADC range and avoids the overflow problems that appear in a 16-bit-only sketch.

That is a practical fixed-point lesson by itself: the final answer may fit in 16 bits, but an intermediate step often needs more headroom.

17.6.5 Extract BCD Digits and Drive SPI

```
/*
 * display_temp – convert R23:R22 (temp×10) to BCD and send to SPI display
 *
 * Digit layout: [D3][D2][D1].[D0] (D1D0 = ones and tenths, decimal point
 *                after digit D1)
 * Entry:  R23:R22 = temperature × 10 (0–999)
 */
display_temp:
    movw  r24, r22
    rcall bin16_to_bcd  /* R21:R20:R19 = BCD digits                          */

    /* temp×10 = abc in decimal corresponds to display "ab.c".
       hundreds digit = R20[3:0]
       tens digit     = R19[7:4]
       ones digit     = R19[3:0]
       For example, 365 (meaning 36.5°C) displays as " 36.5". */

    /* Send digit 3 (leading digit; blank if zero) */
    mov   r16, r20
    andi  r16, 0x0F
    breq  .Ldisp_blank_d3
    rcall send_digit
    rjmp  .Ldisp_d2
.Ldisp_blank_d3:
    ldi   r16, 0x00     /* blank segment                                     */
    rcall spi_send

.Ldisp_d2:
    /* tens digit */
    mov   r16, r19
    swap  r16
    andi  r16, 0x0F
    rcall send_digit    /* send plain digit                                  */

    /* ones digit with decimal point */
    mov   r16, r19
    andi  r16, 0x0F
    rcall send_digit_dp /* send digit + decimal point bit                    */

    /* tenths digit comes from the ones digit of temp×10.
       In the full source, preserve that nibble before reusing it. */
    ret

/*
 * send_digit – look up 7-seg pattern for digit in R16, send via SPI
 */
send_digit:
    ldi   ZL, lo8(seg7_lut)
    ldi   ZH, hi8(seg7_lut)
    add   ZL, r16       /* index into LUT                                    */
    adc   ZH, r1
    lpm   r17, Z        /* r17 = segment pattern                             */
```

```
    rjmp  spi_send

/*
 * spi_send — send R17 via SPI (polling)
 */
spi_send:
    sts   SPDR, r17     /* load SPI data register                         */
.Lspi_wait:
    lds   r16, SPSR
    sbrs  r16, SPIF     /* wait for SPIF (transfer complete) bit          */
    rjmp  .Lspi_wait
    ret
```

17.6.6 Build

```
# display.S in this directory targets ATmega328P SPI hardware
avr-gcc -mmcu=atmega328p -nostartfiles \
        -o display.elf display.S fixedpoint.S bin16_to_bcd.S
avr-objcopy -O ihex display.elf display.hex
avr-size display.elf
```

17.7 Instruction Reference for This Chapter

Instruction	Syntax	Cycles	Flags	Notes
LSL	lsl Rd	1	C,Z,N,V,H	Rd ← Rd«1; C←Rd7
LSR	lsr Rd	1	C,Z,N,V	Rd ← Rd»1; C←Rd0
ASR	asr Rd	1	C,Z,N,V	Signed »1, sign extends
ROL	rol Rd	1	C,Z,N,V,H	Rotate left through C
ROR	ror Rd	1	C,Z,N,V	Rotate right through C
SWAP	swap Rd	1	—	Exchange high/low nibbles
MUL	mul Rd,Rr	2	Z,C	R1:R0 ← Rd×Rr unsigned
MULS	muls Rd,Rr	2	Z,C	Signed × signed
MULSU	mulsu Rd,Rr	2	Z,C	Signed × unsigned

Instruction	Syntax	Cycles	Flags	Notes
BRHS	brhs k	1/2	—	Branch if H set
BRHC	brhc k	1/2	—	Branch if H clear
ADIW	adiw Rd,K	2	Z,C,N,V,S	Add immediate (0–63) to register pair

`SWAP Rd` is extremely useful for packed BCD: it moves the high nibble to the low position for digit extraction without shifting:

That is much cleaner than doing four single-bit shifts just to expose one decimal digit.

```
/* Extract high nibble of R16 as a value 0–15 */
swap  r16           /* high nibble → low nibble                              */
andi  r16, 0x0F     /* mask off high bits                                    */
```

17.8 Common Pitfalls

17.8.1 Pitfall 1 — Using R1 Without Zeroing After MUL

`MUL`, `MULS`, and `MULSU` write the result to `R1:R0`. Many AVR assembly projects reserve `R1` as a zero register because it makes multi-byte arithmetic and pointer math simpler. If your project follows that convention, copy the product out and restore `R1` before code that expects it to be zero:

```
mul   r16, r17
mov   r24, r0       /* use result */
mov   r25, r1
clr   r1            /* restore project zero-register convention            */
```

17.8.2 Pitfall 2 — ASR vs LSR on Signed Values

`LSR` fills the MSB with 0, which is wrong for negative values:

```
-8 in 8-bit two's complement: 0b11111000 = 0xF8
LSR → 0b01111100 = 0x7C = +124  WRONG
ASR → 0b11111100 = 0xFC = -4    CORRECT (-8 / 2)
```

Always use `ASR` for signed right shifts.

If you pick the wrong shift instruction on signed data, the result is not “slightly off.” It is usually catastrophically wrong.

17.8.3 Pitfall 3 — Q8.8 Overflow

Q8.8 integer range is ±127 (signed) or 0–255 (unsigned). Multiplication of two large Q8.8 values easily overflows:

```
200.0 × 200.0 = 40000.0 — exceeds unsigned Q8.8 max (255.99)
```

Check that the mathematical range fits before choosing Q8.8. For larger ranges, use Q4.12 (4-bit integer, 12-bit fraction) or Q12.4.

Choosing a fixed-point format is mostly a range-versus-resolution trade. More fraction bits give you finer steps; more integer bits give you more headroom.

17.8.4 Pitfall 4 — BCD Correction Order

The BCD correction must check low nibble **before** high nibble. Correcting the high nibble first can leave the low nibble invalid after the correction carry propagates:

```
/* WRONG: high nibble corrected first */
subi  r16, -0x60    /* might corrupt low nibble                             */
subi  r16, -0x06

/* CORRECT: low nibble first */
subi  r16, -0x06
subi  r16, -0x60
```

17.8.5 Pitfall 5 — Shift Count for Multi-Byte Values

AVR shifts operate on single registers only. There is no `lsl r25:r24` opcode. Multi-byte shifts require one instruction per byte:

```
/* 24-bit left shift (R22:R21:R20) */
lsl   r20
rol   r21
rol   r22
```

Forget the middle `rol` and bits disappear silently.

That is a recurring AVR pattern: multi-byte arithmetic usually fails by losing a carry in the middle, not by producing an obvious syntax error.

17.9 Summary

```
Shift recap:
  LSL / LSR  — logical shift (unsigned), zero fill
  ASR        — arithmetic right shift (signed), sign extend
  ROL / ROR  — rotate through C; chain with LSL/LSR for multi-byte shifts
  SWAP       — swap nibbles in 1 cycle; essential for packed BCD

Shift-and-add multiply (portable fallback when hardware MUL is unavailable):
 8 iterations: lsr multiplier; brcc skip; add accumulator; shift 16-bit multiplicand
  ~56 cycles for 8×8 vs 2 cycles for MUL on AVRxt devices such as ATtiny3217

Fixed-point Q8.8:
  value = raw_int / 256
  add: same as 16-bit integer add
  multiply: 4 partial products, extract bits [23:8] of 32-bit result

BCD conversion (double-dabble):
  16 iterations of: add-3-if-nibble>4, then shift-left
```

```
  No division, no LUT, predictable cycle count at the cost of a relatively long loop

Binary-to-BCD use cases:
  ADC reading → decimal display
  Timer counter → MM:SS display
  Any sensor value needing human-readable output

Build:
 avr-gcc -mmcu=attiny3217 -nostartfiles -nodefaultlibs -o ch17.elf src/mul8u.S src/fixedpoint.S s
  avr-objdump -d ch17.elf   # verify cycle counts
  avr-size ch17.elf          # check flash budget

Microchip source notes:
  AVR Instruction Set Manual, device-core appendix: ATtiny3217 is AVRxt and
  its missing-instruction list does not include MUL/MULS/MULSU
  AVR Instruction Set Manual, MUL/MULS/MULSU descriptions: 8×8 products write
  R1:R0 and execute in 2 cycles on AVRxt
```

17.10 Exercises

1. The `mul8u` routine takes ~56 cycles worst-case. For a specific use case — multiplying an 8-bit value by the constant 17 — write a dedicated routine using only `LSL`/`ADD`/`MOV` that computes the result in fewer than 6 cycles. (Hint: 17 = 16 + 1.)
2. Q4.12 format stores 4 integer bits and 12 fractional bits in 16 bits. Write the Q4.12 multiply: two Q4.12 values → Q4.12 result. Where do you extract bits from the 32-bit intermediate product? What is the new maximum integer value?
3. The `bin16_to_bcd` routine calls `bcd_adjust` 16 times — once per shift. Each `bcd_adjust` call checks all 5 nibbles. How many of those 5-nibble checks do useful work on iteration 1 (when all BCD bytes are 0)? Optimise `bcd_adjust` to skip nibbles that cannot yet be > 4 based on loop iteration count. How many instruction words does this save?
4. Implement `bcd16_add` — add two 4-digit packed BCD values (two bytes each, representing 0000–9999) with carry out. Use the H flag and manual correction. Test with 9999 + 0001 (result: 0000, carry = 1).
5. The `seg7_lut` uses active-high segment encoding. Many 7-segment driver ICs use active-low (a 0 bit turns the segment on). Rewrite `send_digit` to invert the LUT byte before sending. Use a single `COM Rd` instruction (one's complement, 1 cycle) — do not rewrite the LUT itself.
6. Profile `display_temp` using `avr-objdump -d`. Count the cycles for the path where no digit is blank. Then count cycles for the path where the hundreds digit is zero (blanked). Which path is longer and by how much?
7. The double-dabble loop shifts the binary input value leftward until it is consumed. After 16 iterations, R25:R24 = 0. Verify this is always true regardless of the input value, and explain why it does not cause a problem that R25:R24 is destroyed during the conversion.

Next: Chapter 18 — Real-Time Patterns

Chapter 18

Real-Time Patterns

Real-time embedded systems must respond to events within bounded, predictable time. An AVR running bare-metal assembly has no OS scheduler, no kernel, and no hidden overhead — every cycle is accounted for. That is both its strength and its responsibility.

This chapter covers:

1. **Cooperative task scheduling** — a minimal round-robin scheduler in assembly; no dynamic allocation, no context switch overhead beyond register saves.
2. **Deterministic ISR latency analysis** — counting cycles from interrupt trigger to first ISR instruction; identifying and eliminating latency sources.
3. **Example: two-task scheduler with TCB0** — a TCB0 periodic interrupt fires every 1 ms; cooperative task switching drives an LED blinker and a USART heartbeat sender on the ATtiny3217 Curiosity Nano.

The main goal is not to build a tiny preemptive kernel. It is to show how timing, interrupts, and task structure fit together when you want behavior that stays bounded and explainable.

18.1 Real-Time Fundamentals

A real-time system is not necessarily fast — it is **predictable**. Two key properties:

Deadline: a task must complete before a specific time. Missing a deadline is a failure, regardless of whether the computation eventually finishes.

Determinism: given the same inputs, execution time is bounded and known in advance. The bound must hold under all interrupt patterns and all data values.

18.1.1 Two Scheduling Models

Preemptive: a higher-priority task can interrupt a lower-priority task at any instruction boundary. The interrupted task's full register state must be saved and restored, usually on a per-task stack.

Cooperative: tasks run to completion (or to an explicit yield point) before the scheduler runs the next task. No mid-task preemption. Context switch is cheaper; latency is bounded only by the longest task segment between yields.

That last clause is the defining tradeoff. Cooperative systems are simple because nothing interrupts a task halfway through, but that also means one task can delay everything behind it if it runs too long.

This chapter implements cooperative scheduling. It is appropriate when:

- All tasks have known, short run times.
- The system has only a few tasks (two to eight is typical on an 8-bit AVR).
- Worst-case response time = sum of all task segments + scheduler overhead, and that total meets all deadlines.

If you can write that sum down and it fits comfortably inside your timing budget, cooperative scheduling is often the cleanest answer on an AVR.

18.2 Cooperative Task Scheduler Design

18.2.1 Task Representation

Each task is a subroutine that runs, does some work, and returns. The scheduler calls each task in rotation. A task that needs to wait (e.g., for a UART byte) must either poll with a non-blocking check and return immediately, or use a flag set by an ISR.

```
Scheduler main loop:
  loop forever:
    call task_0
    call task_1
    call task_2
    ...
```

This is the simplest possible cooperative scheduler. Its properties:

- **No stack switch**: each task runs on the same stack, called directly.
- **No saved context**: tasks return normally; all register state is the task's own responsibility (save what you use).
- **Tick-driven variant**: tasks are only called when a timer tick has elapsed, providing a fixed time base.

Notice what is missing: there is no saved "task state" in CPU registers and no separate per-task stack. These tasks are really just well-behaved subroutines that run on a schedule.

18.2.2 Tick-Driven Scheduler

A TCB0 periodic interrupt fires every 1 ms (the *tick*). Each tick sets a flag. The main loop spins until the flag is set, clears it, then runs all tasks:

```
tcb0_isr:
    ldi   r16, 1
    sts   tick_flag, r16
    reti

main_loop:
.Lwait_tick:
    lds   r16, tick_flag
    tst   r16
    breq  .Lwait_tick
```

```
    clr   r16
    sts   tick_flag, r16
    rcall task_led
    rcall task_uart
    rjmp  main_loop
```

In assembly, `tick_flag` is a byte in SRAM (`.bss`). The ISR sets it; the main loop polls and clears it.

That one-byte handoff is a classic embedded pattern: ISR records that an event happened, main-loop code does the slower work later.

18.2.3 Task Counter Pattern

Each task maintains its own counter and acts only when its counter reaches its period:

```
task_led:
    increment led_counter
    compare led_counter with LED_PERIOD
    branch to task_led_done when below period
    clear led_counter
    toggle LED
task_led_done:
    ret

task_uart:
    increment uart_counter
    compare uart_counter with UART_PERIOD
    branch to task_uart_done when below period
    clear uart_counter
    send heartbeat byte
task_uart_done:
    ret
```

With a 1 ms tick: - `LED_PERIOD = 500` → LED toggles every 500 ms (1 Hz blink) - `UART_PERIOD = 1000` → UART byte sent every 1 s

This pattern scales well because each task keeps its own notion of time without needing a complex global scheduler.

18.3 TCB0 Configuration — 1 ms Tick

On the ATtiny3217 Curiosity Nano the CPU runs at **3.333 MHz** (internal 20 MHz oscillator divided by 6, set by `CLKCTRL_MCLKCTRLB`).

TCB0 in **Periodic Interrupt mode** (CNTMODE = 0) counts up from 0, reloads from CCMP when it matches, and fires an interrupt on each match. With no prescaler (CLK_PER):

```
CCMP = (F_CPU / F_tick) - 1
     = (3 333 333 / 1000) - 1
     = 3333 - 1
     = 3332  (0x0D04)
```

18.3.1 TCB0 Register Map

Address	Name	Description
0x0A40	TCB0_CTRLA	Control A — clock select, enable
0x0A41	TCB0_CTRLB	Control B — mode select
0x0A44	TCB0_INTCTRL	Interrupt enable (bit 0: CAPT)
0x0A45	TCB0_INTFLAGS	Interrupt flags (bit 0: CAPT; write 1 to clear)
0x0A4A	TCB0_CNTL	Counter low byte
0x0A4B	TCB0_CNTH	Counter high byte
0x0A4C	TCB0_CCMPL	Compare/capture match low byte
0x0A4D	TCB0_CCMPH	Compare/capture match high byte

TCB0_CTRLA bit layout:

Bit	Name	Description
0	ENABLE	1 = timer runs
3:1	CLKSEL	000 = CLK_PER (no prescaler)
		001 = CLK_PER/2
		110 = TCA0 CLK (divide-down from TCA0 prescaler output)

TCB0_CTRLB bit layout:

Bit	Name	Description
2:0	CNTMODE	000 = Periodic Interrupt (INT mode)
		001 = Timeout Check
		010 = Input Capture on Event
		011 = Input Capture Frequency Measurement
		100 = Input Capture Pulse-Width Measurement
		101 = Input Capture Frequency and Pulse-Width Measurement
		110 = Single Shot
		111 = 8-bit PWM

18.3.2 Initialization Code

```
/* tcb0_init — configure TCB0 for 1 ms periodic interrupt
 *
 * F_CPU = 3 333 333 Hz, CCMP = 3332 = 0x0D04
 * Clobbers: R16, R17
 */
tcb0_init:
    /* Set CCMP = 3332 (low byte first, high byte second — TCB locks on CCMPH write) */
    ldi   r16, lo8(3332)         /* 0x04                                          */
    sts   TCB0_CCMPL, r16
    ldi   r16, hi8(3332)         /* 0x0D                                          */
    sts   TCB0_CCMPH, r16

    /* Mode: periodic interrupt (CNTMODE = 0) */
    ldi   r16, 0
    sts   TCB0_CTRLB, r16

    /* Enable CAPT interrupt */
    ldi   r16, TCB_CAPT_bm       /* bit 0                                         */
```

```
    sts   TCB0_INTCTRL, r16

    /* Enable timer, clock = CLK_PER (no prescaler) */
    ldi   r16, TCB_ENABLE_bm     /* bit 0                                  */
    sts   TCB0_CTRLA, r16

    ret
```

Instruction	Cycles
`LDS Rd, k`	3
`STS k, Rr`	3

`LDS`/`STS` access SRAM-mapped peripherals (addresses > 0x001F). Each costs 3 cycles on avrxmega3.

That is an easy place to lose cycles in timer setup and ISR code on newer AVR parts: many peripheral registers are no longer reachable with the short I/O instructions.

18.4 ATtiny3217 Interrupt Vector Table

Vectors are 4 bytes each (one `JMP` instruction). The table starts at address `0x0000` in flash.

Byte addr	Vector #	Source	Description
0x0000	0	RESET	Power-on, external, WDT, BOD reset
0x0004	1	CRCSCAN_NMI	CRC scan non-maskable interrupt
0x0008	2	BOD_VLM	Brown-out detector voltage level monitor
0x000C	3	PORTA_PORT	Port A pin-change
0x0010	4	PORTB_PORT	Port B pin-change
0x0014	5	PORTC_PORT	Port C pin-change
0x0018	6	RTC_CNT	RTC counter overflow/compare
0x001C	7	RTC_PIT	RTC periodic interrupt timer
0x0020	8	TCA0_LUNF	TCA0 low byte underflow (split mode)
0x0024	9	TCA0_OVF/HUNF	TCA0 overflow / high byte underflow
0x0028	10	TCA0_CMP0	TCA0 compare match 0
0x002C	11	TCA0_CMP1	TCA0 compare match 1
0x0030	12	TCA0_CMP2	TCA0 compare match 2
0x0034	13	TCB0_INT	TCB0 capture/compare (our tick ISR)
0x0038	14	TCB1_INT	TCB1 capture/compare
0x003C	15	TWI0_TWIS	TWI0 slave
0x0040	16	TWI0_TWIM	TWI0 master
0x0044	17	SPI0_INT	SPI0
0x0048	18	USART0_RXC	USART0 receive complete
0x004C	19	USART0_DRE	USART0 data register empty
0x0050	20	USART0_TXC	USART0 transmit complete
0x0054	21	PORTMUX	Port multiplexer
0x0058	22	ADC0_RESRDY	ADC0 result ready
0x005C	23	ADC0_WCOMP	ADC0 window comparator match
0x0060	24	AC0_AC	Analog comparator 0
0x0064	25	DAC0_DACREADY	DAC0 ready

In assembly, the vector table uses `jmp` for each slot:

```
.section .vectors, "ax", @progbits
    jmp   reset_handler         /* 0x0000  RESET                               */
    jmp   default_isr           /* 0x0004  CRCSCAN_NMI                         */
    jmp   default_isr           /* 0x0008  BOD_VLM                             */
    jmp   default_isr           /* 0x000C  PORTA_PORT                          */
    jmp   default_isr           /* 0x0010  PORTB_PORT                          */
    jmp   default_isr           /* 0x0014  PORTC_PORT                          */
    jmp   default_isr           /* 0x0018  RTC_CNT                             */
    jmp   default_isr           /* 0x001C  RTC_PIT                             */
    jmp   default_isr           /* 0x0020  TCA0_LUNF                           */
    jmp   default_isr           /* 0x0024  TCA0_OVF                            */
    jmp   default_isr           /* 0x0028  TCA0_CMP0                           */
    jmp   default_isr           /* 0x002C  TCA0_CMP1                           */
    jmp   default_isr           /* 0x0030  TCA0_CMP2                           */
    jmp   tcb0_isr              /* 0x0034  TCB0_INT  ← our tick                */
    .fill 12 * 4, 1, 0xFF       /* remaining vectors — unused                  */
```

`.fill count, size, value` pads with the given byte value. Here it is only a compact placeholder for omitted vectors in the listing. For production code, prefer populating every vector explicitly (for example, with `jmp default_isr`) rather than relying on erased-flash filler bytes.

The explicit-table style is a little longer, but it makes interrupt ownership obvious when you inspect the binary later.

18.5 ISR Prologue and Epilogue

Every ISR on AVR must:

1. **Save SREG** — instructions inside the ISR affect flags; they must be restored before `RETI`.
2. **Save any registers it uses** — if the ISR uses R16–R29 or any other register that the interrupted code relies on.
3. **Acknowledge the interrupt source correctly** — for TCB0, write 1 to `TCB0_INTFLAGS` bit 0 (`TCB_CAPT_bm`). Other peripherals may clear their flags by different means, such as a data-register read.
4. **Restore saved registers and SREG**.
5. **`RETI`** — return from interrupt and let CPUINT leave the active interrupt state.

That save/restore discipline is the price you pay for asynchronous execution. If the ISR leaves flags or registers disturbed, the interrupted code resumes in an impossible state.

18.5.1 Minimal Tick ISR

The tick ISR only sets a 1-byte flag — `tick_flag` in SRAM:

```
/*
 * tcb0_isr — TCB0 periodic interrupt (1 ms tick)
 *
 * Sets tick_flag = 1.
 * Saves/restores: SREG, R16.
 * Latency from interrupt trigger to first useful instruction: see §18.6.
 */
tcb0_isr:
```

```
    push  r16               /* 2 cycles: save R16 to stack                  */
    in    r16, _SFR_IO_ADDR(SREG)   /* 1 cycle: save SREG                   */
    push  r16               /* 2 cycles                                     */

    /* clear interrupt flag (write 1 to TCB_CAPT_bm = bit 0) */
    ldi   r16, TCB_CAPT_bm
    sts   TCB0_INTFLAGS, r16    /* 3 cycles                                 */

    /* set tick flag */
    ldi   r16, 1
    sts   tick_flag, r16        /* 3 cycles                                 */

    pop   r16               /* 2 cycles: restore SREG value                 */
    out   _SFR_IO_ADDR(SREG), r16   /* 1 cycle: restore SREG                */
    pop   r16               /* 2 cycles: restore R16                        */
    reti                    /* 4 cycles on ATtiny3217                       */
```

Total ISR body: 2+1+2+1+3+1+3+2+1+2+4 = **22 cycles** from first instruction to end of `RETI`.

On AVRxt devices such as the ATtiny3217, `RETI` costs 4 cycles with the 16-bit program counter. Microchip's instruction set manual notes that CPUINT tracks interrupt state on AVRxm/AVRxt devices; `RETI` returns through that interrupt-controller state rather than being just a plain `RET`. Keep it only at the true end of the handler, after every register and flag has been restored.

18.6 ISR Latency Analysis

ISR latency = cycles from interrupt trigger condition to the first instruction of the ISR executing.

18.6.1 Sources of Latency

On ATtiny3217 (avrxmega3), the CPU-side interrupt response after the request has been recognized is:

Source	Cycles
Peripheral/request synchronization	0–2
Finish current instruction	1–4
Push PC to stack	2
Jump from vector to ISR (`jmp`)	3
Total (best case)	**6**
Total (worst case with 2 sync cycles and a 4-cycle instruction)	**11**

Microchip documents a minimum CPU interrupt response of 6 cycles on devices with more than 8 KB of flash: 1 cycle to finish a single-cycle instruction, 2 cycles to push the PC, and 3 cycles for the vector `jmp`. Peripheral-level synchronization can add a small device-specific delay before the CPU begins that sequence. In practice:

```
Interrupt asserted during instruction N
→ request is synchronized into the CPU domain (0–2 cycles)
```

```
→ instruction N completes (1–4 cycles total, depending on instruction length)
→ CPU pushes PC (2 cycles)
→ CPU executes JMP at vector (3 cycles)
→ first ISR instruction executes (cycle 6–11 from assertion)
```

This is the right way to think about interrupt latency on AVR: not as one magic number, but as a short chain of small, countable delays.

18.6.2 Push/Pop Overhead

The prologue `push r16 / in r16, SREG / push r16` costs **5 cycles** before any useful work. For a time-critical ISR (e.g., one that must sample a pin within N cycles), count:

```
latency_to_useful_work = ISR_entry_latency + prologue_cycles
                       = 6 (best) + 5 = 11 cycles minimum
                       = 11 (worst) + 5 = 16 cycles worst case
```

At 3.333 MHz, 16 cycles = **4.8 µs** worst-case latency to the first useful instruction.

18.6.3 Eliminating Sources of Latency

Technique	Cycles saved
Keep ISRs short — move work to main loop	Reduces re-entry latency
Use VPORT for output (1-cycle `sbi`/`cbi`)	0 — saves body cycles, not entry
Avoid long instructions (LPM, CALL) near critical ISR entry points	0–4
Reduce number of registers pushed in prologue	2 per register pair
Reserve a known scratch register for a dedicated ISR and document ownership	2 per omitted push/pop

For the tick ISR, latency is unimportant — the tick fires every 1 ms and a few microseconds of jitter is negligible. Latency matters for ISRs that must capture hardware events (e.g., `TCB0` in input-capture mode, or a pin-change ISR measuring pulse width).

So always ask "latency relative to what?" before optimizing an ISR. A scheduler tick and an edge-timestamp ISR care about very different things.

18.6.4 Measuring Latency

Toggle a debug pin at the start and end of the ISR, then use an oscilloscope or logic analyser to measure ISR entry and execution width relative to another observable system event:

```
tcb0_isr:
    sbi   VPORTB_OUT, 3     /* debug pin PB3 high — mark ISR entry          */
    push  r16
    ...
    cbi   VPORTB_OUT, 3     /* debug pin PB3 low  — mark ISR exit           */
    reti
```

`SBI VPORTB_OUT, 3` executes in **1 cycle** and does not affect SREG — safe before saving SREG, as long as the debug pin is not used by the application.

18.7 Complete Two-Task Scheduler

18.7.1 Hardware Setup — Curiosity Nano

```
LED0  = PA3, active low (HIGH = off, LOW = on)
USART0 TX = PB2 (to on-board debugger virtual COM port)
TCB0 tick = every 1 ms (CCMP = 3332, CLK_PER, F_CPU = 3.333 MHz)
```

18.7.2 Register Allocation

```
R2   — tick_flag mirror (ISR writes SRAM; main reads via LDS)
R24  — scratch / parameter
R25  — scratch
R26:R27 (X) — not used (reserved for future tasks)

SRAM variables (.bss):
  tick_flag   1 byte — set by ISR, cleared by main loop
  led_cnt     2 bytes — 16-bit counter for LED task
  uart_cnt    2 bytes — 16-bit counter for UART task
```

Using SRAM variables (accessed via LDS/STS) ensures the ISR and main loop share data correctly — SRAM is in the data address space accessible to both.

It also keeps the design honest: anything shared between asynchronous code paths should live in memory, not only in a caller's temporary register plan.

18.7.3 Source Code

```
/* scheduler.S — two-task cooperative scheduler, ATtiny3217 Curiosity Nano
 *
 * Tasks:
 *   task_led  — toggle LED0 (PA3) every 500 ms
 *   task_uart — send 'H' over USART0 every 1000 ms
 *
 * Tick: TCB0 periodic interrupt, 1 ms
 * F_CPU: 3 333 333 Hz (20 MHz / 6, default Curiosity Nano clock)
 */

#include <avr/io.h>

#define F_CPU         3333333UL
#define LED_PIN       3            /* PA3, Curiosity Nano LED0                 */
#define LED_PERIOD    500          /* ticks (= 500 ms)                         */
#define UART_PERIOD   1000         /* ticks (= 1 s)                            */

/* USART0 baud: 9600 at 3.333 MHz
 * BAUD register = 64 * F_CPU / (16 * baud) = 64 * 3333333 / (16 * 9600)
 *               = 213333333 / 153600 ≈ 1389 = 0x056D
 */
#define BAUD_9600     1389

/* ── Vector table ──────────────────────────────────────────────────────── */
.section .vectors, "ax", @progbits
    jmp   reset_handler         /* 0x0000  RESET                            */
```

```
    jmp   default_isr          /* 0x0004  CRCSCAN_NMI                    */
    jmp   default_isr          /* 0x0008  BOD_VLM                        */
    jmp   default_isr          /* 0x000C  PORTA_PORT                     */
    jmp   default_isr          /* 0x0010  PORTB_PORT                     */
    jmp   default_isr          /* 0x0014  PORTC_PORT                     */
    jmp   default_isr          /* 0x0018  RTC_CNT                        */
    jmp   default_isr          /* 0x001C  RTC_PIT                        */
    jmp   default_isr          /* 0x0020  TCA0_LUNF                      */
    jmp   default_isr          /* 0x0024  TCA0_OVF                       */
    jmp   default_isr          /* 0x0028  TCA0_CMP0                      */
    jmp   default_isr          /* 0x002C  TCA0_CMP1                      */
    jmp   default_isr          /* 0x0030  TCA0_CMP2                      */
    jmp   tcb0_isr             /* 0x0034  TCB0_INT                       */
    .fill (26 - 14) * 4, 1, 0xFF /* remaining 12 vectors unused          */

/* —— SRAM variables ——————————————————————————————————————————————————— */
.section .bss
tick_flag:  .byte 0            /* 1 = tick has fired; 0 = not yet       */
led_cnt:    .word 0            /* LED task counter (16-bit)             */
uart_cnt:   .word 0            /* UART task counter (16-bit)            */

/* —— TCB0 ISR ————————————————————————————————————————————————————————— */
.section .text

tcb0_isr:
    push  r16
    in    r16, _SFR_IO_ADDR(SREG)
    push  r16

    ldi   r16, TCB_CAPT_bm
    sts   TCB0_INTFLAGS, r16    /* clear interrupt flag                  */

    ldi   r16, 1
    sts   tick_flag, r16        /* signal tick to main loop              */

    pop   r16
    out   _SFR_IO_ADDR(SREG), r16
    pop   r16
    reti

default_isr:
    reti                       /* ignore unexpected interrupts           */

/* —— Initialization ——————————————————————————————————————————————————— */
reset_handler:
    /* Stack pointer: top of SRAM = 0x3FFF */
    ldi   r16, hi8(RAMEND)
    out   _SFR_IO_ADDR(SPH), r16
    ldi   r16, lo8(RAMEND)
    out   _SFR_IO_ADDR(SPL), r16
    clr   r1                  /* project zero register for ADC-with-carry use */

    /* LED0: PA3 output, initial state HIGH (LED off — active low) */
```

```
    sbi   VPORTA_DIR, LED_PIN
    sbi   VPORTA_OUT, LED_PIN

    /* USART0: set TX pin PB2 as output */
    sbi   VPORTB_DIR, 2

    rcall tcb0_init
    rcall usart0_init

    sei                         /* enable global interrupts                 */

    rjmp  main_loop

/* ── TCB0 setup ─────────────────────────────────────────────────────────── */
tcb0_init:
    ldi   r16, lo8(3332)
    sts   TCB0_CCMPL, r16
    ldi   r16, hi8(3332)
    sts   TCB0_CCMPH, r16
    ldi   r16, 0
    sts   TCB0_CTRLB, r16       /* CNTMODE = 0: periodic interrupt          */
    ldi   r16, TCB_CAPT_bm
    sts   TCB0_INTCTRL, r16     /* enable CAPT interrupt                    */
    ldi   r16, TCB_ENABLE_bm
    sts   TCB0_CTRLA, r16       /* CLK_PER, enable                          */
    ret

/* ── USART0 setup ───────────────────────────────────────────────────────── */
usart0_init:
    /* BAUD = 1389 = 0x056D */
    ldi   r16, lo8(BAUD_9600)
    sts   USART0_BAUDL, r16
    ldi   r16, hi8(BAUD_9600)
    sts   USART0_BAUDH, r16
    /* CTRLB: TX enable (TXEN bit 6) */
    ldi   r16, USART_TXEN_bm
    sts   USART0_CTRLB, r16
    /* CTRLC: async USART, 8N1 (default reset value = 0x03 = 8-bit, 1 stop)  */
    /* No explicit write needed — reset value is correct */
    ret

/* ── Main loop ──────────────────────────────────────────────────────────── */
main_loop:
    /* wait for tick */
.Lwait_tick:
    lds   r16, tick_flag
    tst   r16
    breq  .Lwait_tick

    /* clear flag */
    clr   r16
    sts   tick_flag, r16
```

```
    /* run tasks */
    rcall task_led
    rcall task_uart

    rjmp  main_loop

/* -- task_led ------------------------------------------------------------- */
task_led:
    /* increment 16-bit counter */
    lds   r24, led_cnt
    lds   r25, led_cnt+1
    adiw  r24, 1
    sts   led_cnt, r24
    sts   led_cnt+1, r25

    /* compare to LED_PERIOD (500) */
    cpi   r24, lo8(LED_PERIOD)
    ldi   r16, hi8(LED_PERIOD)
    cpc   r25, r16
    brlo  .Lled_done            /* counter < period: nothing to do           */

    /* reset counter */
    clr   r24
    clr   r25
    sts   led_cnt, r24
    sts   led_cnt+1, r25

    /* toggle PA3 via PORTA_OUTTGL (1-byte write toggles the pin) */
    ldi   r16, (1 << LED_PIN)
    sts   PORTA_OUTTGL, r16

.Lled_done:
    ret

/* -- task_uart ------------------------------------------------------------ */
task_uart:
    /* increment 16-bit counter */
    lds   r24, uart_cnt
    lds   r25, uart_cnt+1
    adiw  r24, 1
    sts   uart_cnt, r24
    sts   uart_cnt+1, r25

    /* compare to UART_PERIOD (1000) */
    cpi   r24, lo8(UART_PERIOD)
    ldi   r16, hi8(UART_PERIOD)
    cpc   r25, r16
    brlo  .Luart_done

    /* reset counter */
    clr   r24
    clr   r25
    sts   uart_cnt, r24
```

```
    sts   uart_cnt+1, r25

    /* send 'H' (0x48): wait for DREIF (Data Register Empty Flag) */
.Luart_wait_dreif:
    lds   r16, USART0_STATUS
    sbrs  r16, USART_DREIF_bp   /* skip next if DREIF=1 (ready)             */
    rjmp  .Luart_wait_dreif

    ldi   r16, 'H'
    sts   USART0_TXDATAL, r16

.Luart_done:
    ret
```

18.7.4 Build

```
MCU=attiny3217
avr-gcc -mmcu=${MCU} -nostartfiles -nodefaultlibs -o scheduler.elf src/scheduler.S
avr-size scheduler.elf
avr-objcopy -O ihex scheduler.elf scheduler.hex
avrdude -p t3217 -c serialupdi -P /dev/ttyUSB0 -U flash:w:scheduler.hex:i
```

18.8 Timing Analysis — Worst-Case Response

With two tasks per tick, the cooperative overhead before the main loop re-checks tick_flag is:

```
scheduler_overhead = wait_tick_poll (1 iteration) + clear_flag
                   = (3 + 1 + 1) + (1 + 3) = ~9 cycles

task_led_max = lds×4 + adiw + sts×4 + cpi + ldi + cpc + brlo + ... toggle path
             ≈ 4×3 + 1 + 4×3 + 1 + 1 + 1 + 2 + clr×2 + sts×4 + ldi + sts
             ≈ 12 + 1 + 12 + 5 + 8 + 4×3 + 1 + 3 = ~54 cycles (toggle path)
             ≈ 28 cycles (no-toggle path)

task_uart_max (no-send path) ≈ 28 cycles (same structure as task_led)
task_uart_send_fast_path ≈ 28 + 3 + 1 + 3 = ~35 cycles

total_cycles_per_tick (when USART is immediately ready) ≈ 9 + 54 + 35 = ~98 cycles maximum
time_at_3.333MHz = 98 / 3 333 333 ≈ 29 µs
```

The tick period is 1 ms = 3333 cycles. On the fast path, the scheduler consumes ≈ 98 of those cycles per tick — less than 3% CPU utilisation.

Worst-case tick latency (time between ISR firing and main loop responding):

```
worst_case_tick_latency_fast_path ≈ task_led_max + task_uart_send_fast_path
                                  ≈ 54 + 35 = ~89 cycles ≈ 27 µs
```

This occurs when both tasks do their full work in the tick that just preceded the one being measured. The scheduler handles one tick at a time; if both tasks fire simultaneously on the same tick, tick_flag is set for the next tick while the current work finishes.

Important caveat: the `scheduler.S` UART task is still **blocking**. If the code reaches `.Lu-art_wait_dreif` just after the USART has become busy, the loop can stall for almost a full character time. At 9600 baud with 8N1, one frame is about 10/9600 s ≈ **1.04 ms**, which is longer than the 1 ms tick. So the fast-path cycle count above is useful for instruction budgeting, but it is **not** a strict worst-case bound for the blocking UART design.

That is the most important design lesson in the chapter: a system is not truly bounded just because its arithmetic paths are short. One blocking peripheral wait can dominate the whole schedule.

18.9 Non-Blocking Task Extension

The fix is to decouple task execution from peripheral transmit timing. The non-blocking variant in `src/scheduler_nb.S` uses:

- a small SRAM ring buffer,
- a short `uart_enqueue` routine used by the task, and
- the `USART0_DRE` interrupt to feed bytes into `TXDATAL` whenever the UART is ready for another byte.

Conceptually:

```
task_uart:
    once per second:
        enqueue "Hbeat\r\n"
        enable DRE interrupt

usart0_dre_isr:
    compare tx_tail with tx_head
    if equal, disable DRE interrupt and reti
    load tx_buf[tx_tail]
    write byte to TXDATAL
    advance tx_tail
    reti
```

This keeps `task_uart` short and bounded: it copies bytes into the queue and returns. The USART hardware timing is handled asynchronously by the DRE ISR. In the example implementation, bytes are dropped if the ring buffer is full; that policy keeps the task non-blocking and deterministic.

This is a typical real-time trade: it is often better to lose optional output than to let one slow device destroy the timing guarantees of the whole system.

Full implementation in `src/scheduler_nb.S`.

Two details matter in that source file:

- The heartbeat string is placed in `.text`, not `.rodata`, so `LPM` reads from a flash byte address. It is aligned after the 7-byte string so the following AVR instructions still start on a word boundary.
- `uart_enqueue` saves `R18` because the caller uses it as the heartbeat string loop counter. A non-blocking helper is only deterministic if its register ownership is documented and obeyed.

18.10 Scaling to More Tasks

The round-robin scheduler scales by adding more `rcall task_N` instructions in the main loop. Each task must be **short** — its worst-case execution time must fit within the tick budget:

```
sum(task_worst_case_cycles) < tick_period_cycles
sum ≤ 3333 cycles (1 ms at 3.333 MHz)
```

For a task that exceeds this (e.g., a DFT computation), split it across multiple ticks using a state machine:

```
dft_task:
    /* process one input sample per tick — spread N-point DFT over N ticks */
    load dft_state
    branch to the matching state label
SAMPLE_0:
    compute partial sum for k=0
    store SAMPLE_1 into dft_state
    ret
SAMPLE_1:
    compute partial sum for k=1
    store SAMPLE_2 into dft_state
    ret
```

Breaking work into per-tick slices is one of the most useful habits in small embedded systems. It keeps long algorithms compatible with a simple scheduler.

18.10.1 When to Move to a Preemptive Kernel

A cooperative scheduler works until either: 1. Any task's worst-case time can exceed the tick period, or 2. Tasks have different priorities that require preemption.

For those cases, a preemptive kernel provides priority scheduling and proper stack switching. The assembly patterns in this chapter (ISR prologue/epilogue, timer setup, ISR-to-task signalling) are the same low-level building blocks.

So even if you eventually move to a preemptive kernel, the reasoning here still matters. The kernel is built out of the same timer and interrupt mechanics.

18.11 Summary

Topic	Key Point
Cooperative scheduling	Tasks run to completion; scheduler calls them in rotation
Tick source	TCB0 periodic interrupt, CCMP=3332 for 1 ms at 3.333 MHz
ISR entry latency	6–11 cycles (hardware) + 5 cycles (prologue)
SREG save	Mandatory in any ISR that uses arithmetic or branch instructions
RETI	Returns through CPUINT interrupt state; costs 4 cycles on ATtiny3217
VPORT	1-cycle `SBI`/`CBI` for fast output; does not affect SREG

Topic	Key Point
Tick budget	3333 cycles/ms; cooperative core uses < 100 cycles on the fast path
Scaling	Add `rcall task_N`; split long tasks into per-tick state machines

Microchip source notes:

- ATtiny3216/3217 data sheet, TCB Periodic Interrupt mode: TCB counts to the capture value, restarts from BOTTOM, and generates a CAPT interrupt/event.
- ATtiny3216/3217 data sheet, CPUINT Interrupt Response Time: devices with more than 8 KB Flash use a 3-cycle `jmp` vector, after 1 cycle to finish a single-cycle instruction and 2 cycles to store the PC.
- AVR Instruction Set Manual, `RETI`: AVRxt devices use CPUINT interrupt state on return; `RETI` is 4 cycles with a 16-bit program counter.
- Microchip USART guidance: asynchronous normal mode uses `BAUD = 64 * f_CLK_PER / (16 * f_BAUD)`.

Next: Chapter 19 — CORDIC on AVR

Chapter 19

CORDIC on AVR

CORDIC (COordinate Rotation DIgital Computer) is one of the classic tricks for small processors. It computes trigonometric, inverse-trigonometric, and vector rotation results using only:

- shifts,
- adds/subtracts,
- comparisons, and
- a small table of constants.

That makes it a natural fit for AVR. The ATtiny3217 includes the AVR hardware multiplier, but Microchip's instruction set still has no general divide instruction and the device has no floating-point execution unit. When you need sine, cosine, vector angle, or vector magnitude with predictable execution time, CORDIC is often a better fit than general software floating-point math.

This chapter covers:

1. **Why CORDIC is useful on AVR** — where it beats general math routines, and where a lookup table is still better.
2. **Rotation mode** — computing sine and cosine from an angle.
3. **Vectoring mode** — reducing a vector to magnitude and angle.
4. **Fixed-point formats** — a practical AVR-friendly choice: Q1.15 for vector components and BAM16 for angles.
5. **Examples** — AVR assembly stage examples, register plans, and practical integration patterns.

The important theme is the same one that runs through the rest of the book: replace expensive, hard-to-bound math with small deterministic steps.

19.1 Why Use CORDIC on AVR?

CORDIC is attractive on AVR for three practical reasons:

1. **No floating-point execution unit**. General software floating-point trigonometry is much larger and harder to budget than a fixed shift-add kernel.
2. **Fixed iteration count**. A 12-stage CORDIC always does 12 stages. That is useful when you care about real-time bounds.

3. **One core, many functions**. The same shift-add structure can be used for sine/cosine, vector rotation, `atan2`, and magnitude approximation.

Typical AVR uses:

- waveform generation in a DDS/NCO,
- phase tracking,
- I/Q demodulation,
- motor-control angle estimation,
- tilt/heading math from sensors, and
- display-oriented trig without a large general-purpose math package.

19.1.1 When Not to Use It

CORDIC is not magic. Sometimes a lookup table is better:

- If you only need sine values at 8-bit resolution, a 256-byte flash table is usually faster.
- If you need one-off trig rarely, a larger general-purpose math routine may be acceptable.
- If your MCU has enough flash and your timing budget is loose, a polynomial or LUT approach may be easier to maintain.

So the real question is not "is CORDIC clever?" It is "does repeated deterministic shift-add math fit this problem better than tables or general software arithmetic?"

19.2 The Core Idea

CORDIC works by applying a sequence of tiny rotations whose tangents are powers of two:

`atan(2^0), atan(2^-1), atan(2^-2), atan(2^-3), ...`

Because `tan(theta_i) = 2^-i`, each stage can be written without a multiply:

```
x' = x plus_or_minus  (y >> i)
y' = y minus_or_plus  (x >> i)
z' = z minus_or_plus  atan(2^-i)
```

The shifts replace a true multiply by `tan(theta_i)`.

There are two standard modes:

19.2.1 Rotation Mode

Start with a unit vector on the x-axis and rotate it toward the requested angle. After the final stage:

- `x` approximates `cos(theta)`
- `y` approximates `sin(theta)`

19.2.2 Vectoring Mode

Start with an input vector `(x, y)` and rotate it toward the x-axis. After the final stage:

- `z` approximates the vector angle (`atan2`)
- `x` approximates the vector magnitude, scaled by the CORDIC gain
- `y` approaches zero

This is why one algorithm can serve both "give me sine/cosine" and "tell me the angle/magnitude of this vector."

19.3 Fixed-Point Choices for AVR

CORDIC is almost always done in fixed-point on AVR.

19.3.1 Vector Format: Q1.15

Use signed 16-bit values for `x` and `y`:

```
Q1.15: signed fraction in the range about -1.0 to +0.99997

0x4000 = +0.5
0x7FFF = +0.99997
0x0000 =  0.0
0xC000 = -0.5
0x8000 = -1.0
```

This is a good fit for sine and cosine because those values naturally live in the interval `[-1, +1]`.

19.3.2 Angle Format: BAM16

For angles, a very AVR-friendly choice is **BAM16** (Binary Angular Measure):

```
Full turn = 65536 units
90°       = 16384 = 0x4000
180°      = 32768 = 0x8000
360°      = wraps to 0x0000
```

Why BAM16 works well:

- addition and wraparound are natural in 16-bit unsigned arithmetic,
- phase accumulators use the same representation,
- quadrants are easy to identify from the top two bits.

19.3.3 CORDIC Gain

The repeated pseudo-rotations scale the vector by a constant gain:

```
1 / K ≈ 1.646760258
K     ≈ 0.607252935
```

After 12 stages the gain is already close to the infinite-stage value:

```
K12 = 0.607252959
1/K12 = 1.646760193
```

For a 12-stage sine/cosine CORDIC, preload:

```
K_Q15 = round(0.607252959 × 32768) = 19898 = 0x4DBA
```

That means rotation mode typically starts with:

```
x = 0x4DBA   ; +0.60725 in Q1.15
y = 0x0000
```

and the final result lands near properly scaled sine/cosine outputs.

19.4 The Angle Table

For 12 stages in BAM16:

i	`atan(2^-i)`	BAM16 value
0	45.000°	`0x2000`
1	26.565°	`0x12E4`
2	14.036°	`0x09FB`
3	7.125°	`0x0511`
4	3.576°	`0x028B`
5	1.790°	`0x0146`
6	0.895°	`0x00A3`
7	0.448°	`0x0051`
8	0.224°	`0x0029`
9	0.112°	`0x0014`
10	0.056°	`0x000A`
11	0.028°	`0x0005`

This table is tiny. Even stored as 16-bit words in flash, 12 stages only cost 24 bytes.

19.4.1 Convergence Range

Basic rotation-mode CORDIC converges only for angles near `±99.88°`, not over the full 360° range. On AVR, the usual fix is **quadrant reduction**:

1. Determine the quadrant from bits `[15:14]` of the BAM16 angle.
2. Reflect the angle into the first quadrant `[0°, 90°]`.
3. Run the CORDIC core there.
4. Apply the final sign correction to `sin` and `cos`.

That is why BAM16 is convenient: the top two bits already tell you the quadrant.

19.5 Rotation Mode for Sine and Cosine

Register-transfer view using Q1.15 vectors and BAM16 angles:

```
x = K_Q15
y = 0
z = reduced_angle       ; first quadrant, 0..90 degrees

repeat for stages i = 0..11:
    temp_x = arithmetic_shift_right(x, i)
    temp_y = arithmetic_shift_right(y, i)
```

```
    test sign bit of z
    when z is non-negative:
        x = x - temp_y
        y = y + temp_x
        z = z - atan_table[i]
    when z is negative:
        x = x + temp_y
        y = y - temp_x
        z = z + atan_table[i]
```

After the loop:

- `x` is the first-quadrant cosine
- `y` is the first-quadrant sine

Then apply quadrant signs:

Quadrant	Reduced angle	`cos` sign	`sin` sign
0	`angle`	`+`	`+`
1	`180°-angle`	`-`	`+`
2	`angle-180°`	`-`	`-`
3	`360°-angle`	`+`	`-`

19.5.1 Example Values

With 12 stages and `K_Q15 = 0x4DBA`:

- `30°` (`0x1555`) gives about `cos ≈ 0.8660`, `sin ≈ 0.5001`
- `45°` (`0x2000`) gives about `cos ≈ 0.7070`, `sin ≈ 0.7072`
- `300°` (`0xD555`) gives about `cos ≈ 0.5001`, `sin ≈ -0.8660`

That is easily good enough for control loops, DDS, UI graphics, and many sensor applications on AVR.

19.6 AVR Assembly Structure

An AVR-assembly CORDIC routine should be organized around three fixed pieces:

- quadrant reduction on the BAM16 input angle,
- a 12-entry `atan` table in flash, and
- either a counted loop or a fully unrolled set of stages.

19.6.1 Table Layout

One straightforward flash table is:

```
cordic_atan_bam16:
    .word 0x2000, 0x12E4, 0x09FB, 0x0511
    .word 0x028B, 0x0146, 0x00A3, 0x0051
    .word 0x0029, 0x0014, 0x000A, 0x0005
```

Put this table in `.text` or another flash-resident section and align it to a word boundary. When reading it with `LPM`, remember that `LPM` fetches bytes; read the low byte and high byte separately, then advance Z to the next word.

For rotation mode, preload the vector with `K_Q15 = 0x4DBA`:

```
    ldi  r24, lo8(0x4DBA)    ; x low
    ldi  r25, hi8(0x4DBA)    ; x high
    clr  r22                 ; y low
    clr  r23                 ; y high
```

19.6.2 Core Loop Skeleton

At a high level, the loop does this each stage:

```
1. form temp_x = x >> i      using signed shifts
2. form temp_y = y >> i      using signed shifts
3. if z >= 0:
       x = x - temp_y
       y = y + temp_x
       z = z - atan[i]
   else:
       x = x + temp_y
       y = y - temp_x
       z = z + atan[i]
4. advance i and table pointer
```

That description is intentionally close to the register transfers you will actually code. On AVR, the main implementation choice is whether `i` is variable inside a loop or fixed in an unrolled stage sequence.

19.7 How This Maps to AVR Assembly

At the algorithm level, CORDIC is simple. At the assembly level, the main cost is the **variable arithmetic shift** by `i`.

For a general loop, `x >> i` and `y >> i` mean:

1. copy the 16-bit value to a temporary pair,
2. shift it right `i` times using `ASR` on the high byte and `ROR` on the low byte,
3. use the shifted temporary in the add/subtract step.

That is why many hand-tuned AVR CORDIC routines are either:

- partially unrolled, or
- fully unrolled.

When `i` is a compile-time constant, the shifts become fixed instruction sequences and the loop-control overhead disappears.

The file `src/cordic_helpers.S` contains a buildable support routine for the variable signed 16-bit shift:

```
/* Entry: R25:R24 = signed value, R17 = count. Exit: R25:R24 shifted. */
cordic_asr16:
    tst   r17
```

```
    breq  .Lasr16_done
.Lasr16_loop:
    asr   r25
    ror   r24
    dec   r17
    brne  .Lasr16_loop
.Lasr16_done:
    ret
```

19.7.1 Example: One Unrolled Stage (i = 3)

Suppose:

- R25:R24 = x
- R23:R22 = y
- R21:R20 = z

One positive-rotation stage for `i = 3` looks like:

```
/* temp_y = y >> 3  (signed shift) */
movw  r19, r22          /* r19:r18 = y copy                              */
asr   r19
ror   r18
asr   r19
ror   r18
asr   r19
ror   r18

/* x = x - temp_y */
sub   r24, r18
sbc   r25, r19

/* temp_x = old x >> 3
 * In a real implementation, preserve old x before modifying it. */
```

The mechanics are exactly the same as any other multi-byte signed shift on AVR: `ASR` the high byte, `ROR` the low byte, repeat as many times as needed.

19.7.2 Practical Register Plan

For a hand-written AVR version, a reasonable plan is:

```
R25:R24   x
R23:R22   y
R21:R20   z
R19:R18   temp shifted copy
R17       stage counter i
R16       scratch / table byte
Z         pointer into atan table
```

This is workable, but you can see why register pressure gets real quickly. CORDIC is a good example of a routine where AVR assembly quality depends as much on register planning as on the arithmetic itself.

19.8 Vectoring Mode: Magnitude and Angle

Vectoring mode starts with an input vector `(x, y)` and rotates it toward the x-axis:

```
z = 0
repeat for stages i = 0..11:
    temp_x = arithmetic_shift_right(x, i)
    temp_y = arithmetic_shift_right(y, i)
    test sign bit of y
    when y is non-negative:
        x = x + temp_y
        y = y - temp_x
        z = z + atan_table[i]
    when y is negative:
        x = x - temp_y
        y = y + temp_x
        z = z - atan_table[i]
```

After the last stage:

- `z` approximates the four-quadrant vector angle
- `x` approximates `magnitude / K`
- `y` is near zero

The magnitude is scaled upward by the CORDIC gain `1/K`, so you either:

- multiply the result by `K` afterwards,
- pre-scale the input vector by `K`, or
- calibrate the rest of the application around the scaled value.

This gain matters for overflow. A Q1.15 input vector with magnitude near 1.0 cannot safely be fed directly into a 16-bit vectoring core, because the final `x` wants to grow to about 1.64676. Either pre-scale the vector by `K`, use a wider internal format such as 24-bit or 32-bit fixed point, or accept a smaller input range.

19.8.1 Where AVR Uses This

Vectoring mode is especially useful for:

- phase estimation from I/Q samples,
- converting Cartesian sensor axes to angle,
- simple magnitude estimation without a square root.

This is where CORDIC often beats ad-hoc math on AVR: one fixed-iteration loop replaces both angle extraction and square-root style magnitude calculation.

19.9 Example 1: DDS / NCO Sine Generator

A common AVR use is a numerically controlled oscillator:

1. keep a 16-bit phase accumulator,
2. add a phase step each sample period,
3. feed the phase into CORDIC rotation mode,
4. send the sine result to a PWM or DAC.

```
phase += phase_step;
call cordic_sincos_bam16
use high byte of sine as PWM sample
```

Why CORDIC can make sense here:

- no large sine table in flash,
- easy frequency control through `phase_step`,
- constant run time every sample.

If you need the very highest sample rate, a LUT is still usually faster. But if flash is tight and you want both sine and cosine from the same phase input, CORDIC is an attractive compromise.

19.10 Example 2: Tilt Angle from Two Axes

Suppose an AVR reads two already-filtered accelerometer channels:

```
x_axis = forward/back tilt component
y_axis = vertical tilt component
```

You want an angle estimate:

```
tilt = four-quadrant angle from x_axis and y_axis
```

Vectoring CORDIC gives the AVR a fixed-time integer path:

1. load `(x, y)` into Q1.15,
2. run vectoring mode,
3. interpret `z` in BAM16,
4. convert BAM16 to degrees only if the UI needs it.

That avoids general software floating-point math and keeps the control-loop representation in a format that is cheap to update.

19.11 Accuracy, Flash, and Cycle Tradeoffs

19.11.1 Iteration Count

More stages mean more accuracy and more cycles.

Typical choices on AVR:

- 8 stages: quick, coarse
- 12 stages: good practical middle ground
- 16 stages: near the limit of 16-bit usefulness

With Q1.15 outputs, 12 stages is usually enough because the result resolution is already limited by the 16-bit format.

19.11.2 LUT vs CORDIC

Rough comparison:

Method	Flash	Cycles	Strength
256-byte sine LUT	moderate	very low	fastest for sine only
CORDIC (12 stages)	small table + code	moderate, fixed	sine/cosine/angle/magnitude from one core
Polynomial fixed-point	small to moderate	data-dependent unless carefully bounded	good for one narrow function

The main advantage of CORDIC is not "fewest cycles." It is that one deterministic integer core can cover several mathematically different jobs.

19.12 Common Pitfalls

19.12.1 Pitfall 1 — Forgetting Quadrant Reduction

Basic rotation mode does not converge over the full 360° range.

If you feed `210°` directly into the core without reducing it first, the result will be wrong even if the stage math itself is perfect.

19.12.2 Pitfall 2 — Using Logical Right Shift on Signed Values

The shifts inside CORDIC are signed for `x` and `y`.

On AVR assembly:

```
asr   r25
ror   r24
```

is correct for a signed 16-bit right shift.

Using:

```
lsr   r25
ror   r24
```

would break negative vector components.

19.12.3 Pitfall 3 — Ignoring the Gain

CORDIC results are scaled. If you forget to preload with `K` in rotation mode, your sine/cosine amplitude will be too large. If you forget the gain in vectoring mode, your magnitude estimate will be off by a constant factor.

19.12.4 Pitfall 4 — Overflow in Narrow Formats

Q1.15 is ideal for sine/cosine outputs, but it is tight for vectoring-mode inputs.

If the input magnitude can approach 1.0, pre-scale by `K` or move to a wider fixed-point format before running vectoring CORDIC. Otherwise the internal `x = magnitude / K` result can overflow even though both original components were inside `[-1, +1)`.

19.12.5 Pitfall 5 — Hand-Writing the Loop Before Proving Correctness

CORDIC is simple enough to look obvious and subtle enough to get wrong.

On AVR, prove the math with a small set of known angles and compare the assembly against those expected values stage by stage. That is faster than debugging a broken fully-unrolled assembly version from scratch.

19.13 Summary

```
CORDIC on AVR:
  uses shifts + adds/subtracts + compare + small atan table
  no floating-point execution unit required
  fixed iteration count → deterministic timing

Useful formats:
  x,y     = Q1.15 signed fractions
  angle   = BAM16 (65536 units per turn)
  K_Q15   = 0x4DBA for 12-stage rotation mode

Rotation mode:
  input  = angle
  output = cos(theta), sin(theta)

Vectoring mode:
  input  = x,y vector
  output = angle and scaled magnitude

Main AVR tradeoff:
  slower than a small LUT for sine only
  more flexible than a LUT when you need sine/cosine/angle/magnitude

Build:
 avr-gcc -mmcu=attiny3217 -nostartfiles -nodefaultlibs -o ch19.elf src/cordic_helpers.S
  avr-objdump -d ch19.elf
```

Microchip source notes:

- ATtiny3217 product documentation lists an 8-bit AVR CPU with hardware multiplier, up to 20 MHz operation, 32 KB Flash, 2 KB SRAM, and 256 bytes EEPROM.
- The AVR Instruction Set Manual device-core appendix lists ATtiny3217 as AVRxt. Its missing-instruction list does not include `MUL`, but the AVR instruction set has no general divide instruction.
- The AVR Instruction Set Manual defines `ASR` as an arithmetic right shift and `ROR` as rotate right through carry; together they implement signed multi-byte right shifts for CORDIC stage terms.

19.14 Exercises

1. Implement an 8-stage CORDIC sine/cosine routine and compare its maximum error against the 12-stage version. How many cycles do you save?
2. Store the BAM16 `atan` table in flash and read it with `LPM` through Z. Compare the flash cost and cycle cost against hard-coding the constants in a fully unrolled routine.
3. Write an assembly helper that performs a signed 16-bit arithmetic right shift by a variable count in `R17`. Use `ASR`/`ROR` in a loop. How many cycles does it take for counts 0, 1, 4, and 8?
4. Write a sine-only wrapper around the rotation-mode assembly core. Does a dedicated calling convention save any useful register moves, or are sine and cosine effectively "free" once the same 12 stages have already run?
5. Implement vectoring mode for inputs pre-scaled by `K` and test it on a vector proportional to `(0.6, 0.8)`. What BAM16 angle do you get, and how close is the corrected magnitude to 1.0?
6. Compare three ways to generate a PWM sine on AVR:
   a. 256-entry flash LUT
   b. 12-stage CORDIC
   c. fixed-point polynomial approximation Measure flash usage with `avr-size` and inspect the call paths with `avr-objdump -d`.

Next: Appendices

Chapter 20

ATtiny3217 Register Reference

Quick lookup for registers used throughout this book. The ATtiny3217 is an AVRXMEGA3 device. All peripheral registers are memory-mapped and require `LDS`/`STS` unless noted otherwise.

> **VPORT exception**: Virtual PORT registers at 0x0000–0x001F are within the I/O space and can be accessed with `IN`/`OUT`, `SBI`/`CBI`, `SBIS`/`SBIC` in a single cycle. All other peripheral registers require `LDS`/`STS`.
>
> **CPU registers**: SREG (0x003F), SPH (0x003E), SPL (0x003D) are accessible with `IN`/`OUT` using their I/O addresses directly (no +0x20 offset needed on AVRXMEGA3 — these addresses are already the memory-mapped addresses, and the I/O address equals the memory address for registers at 0x0000–0x003F).

20.1 CPU Registers

Register	Mem Addr	Access	Description
SREG	0x003F	IN/OUT	Status Register (I T H S V N Z C)
SPH	0x003E	IN/OUT	Stack Pointer High
SPL	0x003D	IN/OUT	Stack Pointer Low
CCP	0x0034	LDS/STS	Configuration Change Protection

SREG, SPH, and SPL sit within the range 0x0000–0x003F and are accessible via `IN`/`OUT` using the address directly. The `CCP` register requires `LDS`/`STS`.

20.1.1 SREG Bit Layout

```
Bit:  7    6    5    4    3    2    1    0
Name: I    T    H    S    V    N    Z    C
      │    │    │    │    │    │    │    └─ Carry
      │    │    │    │    │    │    └────── Zero
      │    │    │    │    │    └─────────── Negative
      │    │    │    │    └──────────────── Overflow (two's complement)
      │    │    │    └───────────────────── Sign (N XOR V)
      │    │    └────────────────────────── Half-carry (nibble carry)
      │    └─────────────────────────────── Bit Copy Storage (T flag)
      │
```

```
└──────────────────────────────────── Global Interrupt Enable
```

20.1.2 Configuration Change Protection (CCP)

Certain registers are protected against accidental writes. Before writing to a protected register, write the appropriate key to CCP. The protected register must then be written within 4 CPU cycles.

Key	Value	Protects
IOREG	0xD8	I/O registers (e.g. CLKCTRL, BOD, OSC settings)
SPM	0x9D	NVM (flash/EEPROM) write commands

Example — disable clock prescaler to run at 20 MHz:

```
    ldi   r16, 0xD8          ; CCP key for I/O register changes
    sts   0x0034, r16        ; write to CCP
    sts   0x0061, r1         ; write 0 to CLKCTRL_MCLKCTRLB (clear PEN)
                              ; must complete within 4 cycles of CCP write
```

20.2 CLKCTRL — Clock Controller

Base address: 0x0060. All registers require LDS/STS and CCP protection where noted.

Register	Mem Addr	CCP?	Description
CLKCTRL_MCLKCTRLA	0x0060	Yes	Main clock source select
CLKCTRL_MCLKCTRLB	0x0061	Yes	Main clock prescaler
CLKCTRL_MCLKLOCK	0x0062	Yes	Lock register (prevents further changes)
CLKCTRL_MCLKSTATUS	0x0063	No	Status (read-only)

20.2.1 MCLKCTRLA — Main Clock Source Select

```
Bit:  7      6:2    1:0
      CLKOUT  –     CLKSEL
```

CLKSEL	Source
0x0	20 MHz internal oscillator ← factory default
0x1	32 KHz internal ULP oscillator
0x2	32.768 KHz external crystal (XOSC32K)
0x3	External clock on CLKI pin

Setting CLKOUT (bit 7) routes the main clock to the CLKOUT pin.

20.2.2 MCLKCTRLB — Main Clock Prescaler

```
Bit:  7:5    4:1    0
       –     PDIV   PEN

PEN    Prescaler enable (1 = prescaler active, 0 = disabled)

PDIV   Division factor (when PEN=1)
  0x0   /2
```

```
0x1   /4
0x2   /8
0x3   /16
0x4   /32
0x5   /64
0x6   /6   ← factory default (20 MHz / 6 = 3.333 MHz)
0x7   /10
0x8   /12
0x9   /24
0xA   /48
```

Factory default: PDIV=0x6 (divide by 6), PEN=1 → **3.333 MHz** system clock.

To run at full **20 MHz**: write 0x00 to MCLKCTRLB (clears PEN) — requires CCP key 0xD8 first. To run at **10 MHz**: write PEN=1, PDIV=0x1 (divide by 2).

20.2.3 MCLKSTATUS — Status (read-only)

Bit:	7	6	5	4	3	2	1	0
	EXTS	XOSC32KS	SOSC	OSC32KS	OSC20MS	–	–	SOSC

Bit name	Description
OSC20MS	1 = 20 MHz oscillator is stable and running
OSC32KS	1 = 32 KHz oscillator is stable
SOSC	1 = System oscillator is stable
EXTS	1 = External clock source is active

20.3 GPIO Registers

The AVRXMEGA3 GPIO model is completely different from classic AVR. Each port has a full set of direction/output/input/control registers at a fixed base address. Atomic set/clear/toggle registers eliminate the need for read-modify-write sequences.

20.3.1 Port Base Addresses

Port	Base Addr
PORTA	0x0400
PORTB	0x0420
PORTC	0x0440

20.3.2 Per-Port Register Map (offset from base)

Offset	Register	Description
+0x00	PORTx_DIR	Data direction register (1=output, 0=input)
+0x01	PORTx_DIRSET	Write 1 to set direction bits (atomic)
+0x02	PORTx_DIRCLR	Write 1 to clear direction bits (atomic)
+0x03	PORTx_DIRTGL	Write 1 to toggle direction bits (atomic)
+0x04	PORTx_OUT	Output value register
+0x05	PORTx_OUTSET	Write 1 to set output bits (atomic)

+0x06	PORTx_OUTCLR	Write 1 to clear output bits (atomic)
+0x07	PORTx_OUTTGL	Write 1 to toggle output bits (atomic)
+0x08	PORTx_IN	Input value (reads current pin state)
+0x09	PORTx_INTFLAGS	Pin-change interrupt flags (W1C)
+0x0A	PORTx_PORTCTRL	Port control register
+0x10	PORTx_PIN0CTRL	Pin 0 config (pull-up, invert, ISC)
+0x11	PORTx_PIN1CTRL	Pin 1 config
+0x12	PORTx_PIN2CTRL	Pin 2 config
+0x13	PORTx_PIN3CTRL	Pin 3 config
+0x14	PORTx_PIN4CTRL	Pin 4 config
+0x15	PORTx_PIN5CTRL	Pin 5 config
+0x16	PORTx_PIN6CTRL	Pin 6 config
+0x17	PORTx_PIN7CTRL	Pin 7 config

20.3.3 PORTA Absolute Addresses

Register	Mem Addr	Description
PORTA_DIR	0x0400	Direction
PORTA_DIRSET	0x0401	Set direction (atomic)
PORTA_DIRCLR	0x0402	Clear direction (atomic)
PORTA_DIRTGL	0x0403	Toggle direction (atomic)
PORTA_OUT	0x0404	Output value
PORTA_OUTSET	0x0405	Set output (atomic)
PORTA_OUTCLR	0x0406	Clear output (atomic)
PORTA_OUTTGL	0x0407	Toggle output (atomic)
PORTA_IN	0x0408	Input value
PORTA_INTFLAGS	0x0409	Interrupt flags
PORTA_PORTCTRL	0x040A	Port control
PORTA_PIN0CTRL	0x0410	PA0 config (UPDI pin — do not reconfigure)
PORTA_PIN1CTRL	0x0411	PA1 config (SPI MOSI default)
PORTA_PIN2CTRL	0x0412	PA2 config (SPI MISO default)
PORTA_PIN3CTRL	0x0413	PA3 config (LED0 — Curiosity Nano)
PORTA_PIN4CTRL	0x0414	PA4 config (SPI SS default)
PORTA_PIN5CTRL	0x0415	PA5 config
PORTA_PIN6CTRL	0x0416	PA6 config
PORTA_PIN7CTRL	0x0417	PA7 config

20.3.4 PORTB Absolute Addresses

Register	Mem Addr	Description
PORTB_DIR	0x0420	Direction
PORTB_DIRSET	0x0421	Set direction (atomic)
PORTB_DIRCLR	0x0422	Clear direction (atomic)
PORTB_DIRTGL	0x0423	Toggle direction (atomic)
PORTB_OUT	0x0424	Output value
PORTB_OUTSET	0x0425	Set output (atomic)
PORTB_OUTCLR	0x0426	Clear output (atomic)
PORTB_OUTTGL	0x0427	Toggle output (atomic)
PORTB_IN	0x0428	Input value
PORTB_INTFLAGS	0x0429	Interrupt flags
PORTB_PORTCTRL	0x042A	Port control
PORTB_PIN0CTRL	0x0430	PB0 config (TWI SDA default)

PORTB_PIN1CTRL	0x0431	PB1 config (TWI SCL default)
PORTB_PIN2CTRL	0x0432	PB2 config (USART0 TX – Curiosity Nano)
PORTB_PIN3CTRL	0x0433	PB3 config (USART0 RX – Curiosity Nano)
PORTB_PIN4CTRL	0x0434	PB4 config
PORTB_PIN5CTRL	0x0435	PB5 config
PORTB_PIN6CTRL	0x0436	PB6 config
PORTB_PIN7CTRL	0x0437	PB7 config

20.3.5 PORTC Absolute Addresses

Register	Mem Addr	Description
PORTC_DIR	0x0440	Direction
PORTC_DIRSET	0x0441	Set direction (atomic)
PORTC_DIRCLR	0x0442	Clear direction (atomic)
PORTC_DIRTGL	0x0443	Toggle direction (atomic)
PORTC_OUT	0x0444	Output value
PORTC_OUTSET	0x0445	Set output (atomic)
PORTC_OUTCLR	0x0446	Clear output (atomic)
PORTC_OUTTGL	0x0447	Toggle output (atomic)
PORTC_IN	0x0448	Input value
PORTC_INTFLAGS	0x0449	Interrupt flags
PORTC_PORTCTRL	0x044A	Port control
PORTC_PIN0CTRL	0x0450	PC0 config
PORTC_PIN1CTRL	0x0451	PC1 config
PORTC_PIN2CTRL	0x0452	PC2 config
PORTC_PIN3CTRL	0x0453	PC3 config
PORTC_PIN4CTRL	0x0454	PC4 config
PORTC_PIN5CTRL	0x0455	PC5 config
PORTC_PIN6CTRL	0x0456	PC6 config (not present on 20-pin package)
PORTC_PIN7CTRL	0x0457	PC7 config (not present on 20-pin package)

20.3.6 PINnCTRL Bit Layout

Each `PORTx_PINnCTRL` register configures one pin independently.

```
Bit:  7      6:3    3        2:0
      INVEN   —     PULLUPEN  ISC[2:0]
      |              |         |
      |              |         └─ Input Sense Configuration (see below)
      |              └────────── 1 = enable internal pull-up resistor
      └───────────────────────── 1 = invert pin (input and output)
```

ISC[2:0]	Input Sense Configuration	
0x0	INTDISABLE	No interrupt; digital input buffer enabled
0x1	BOTHEDGES	Interrupt on both rising and falling edges
0x2	RISING	Interrupt on rising edge only
0x3	FALLING	Interrupt on falling edge only
0x4	INPUT_DISABLE	Digital input buffer disabled (reduces power)
0x5	LEVEL	Interrupt on low level

20.3.7 Virtual PORT Registers (VPORT) — Fast I/O

VPORTs mirror the key PORT registers into I/O space (0x0000–0x001F). They can be accessed with `IN`/`OUT` (1 cycle), `SBI`/`CBI`, and `SBIS`/`SBIC`. Writing to `VPORT_IN` toggles the corresponding output bit (same as `OUTTGL`).

```
Register         Addr    Mirrors
------------------------------------------------------------------
VPORTA_DIR       0x0000  PORTA_DIR   (1=output)
VPORTA_OUT       0x0001  PORTA_OUT
VPORTA_IN        0x0002  PORTA_IN    (write 1 to toggle output)
VPORTA_INTFLAGS  0x0003  PORTA_INTFLAGS

VPORTB_DIR       0x0004  PORTB_DIR
VPORTB_OUT       0x0005  PORTB_OUT
VPORTB_IN        0x0006  PORTB_IN
VPORTB_INTFLAGS  0x0007  PORTB_INTFLAGS

VPORTC_DIR       0x0008  PORTC_DIR
VPORTC_OUT       0x0009  PORTC_OUT
VPORTC_IN        0x000A  PORTC_IN
VPORTC_INTFLAGS  0x000B  PORTC_INTFLAGS
```

Use VPORT registers with `IN`/`OUT` for single-cycle access. Use the full PORT registers with `LDS`/`STS` when you need atomic set/clear/toggle or pin config.

20.4 Curiosity Nano Pin Assignments

```
Signal          Port Pin   Mem Addr (PINnCTRL)   Notes
------------------------------------------------------------------
LED0            PA3        0x0413                Active low — drive LOW to light
USART0 TX       PB2        0x0432                To on-board debugger virtual COM port
USART0 RX       PB3        0x0433                From on-board debugger virtual COM port
UPDI            PA0        0x0410                Programming/debug — do not reconfigure
```

20.4.1 LED and Button Quick Reference

```
LED0 on  (light):  STS PORTA_OUTCLR, (1<<3)   ; clear PA3 (active low)
LED0 off (dark):   STS PORTA_OUTSET, (1<<3)   ; set PA3
LED0 toggle:       STS PORTA_OUTTGL, (1<<3)   ; toggle PA3

The ATtiny3217 Curiosity Nano does not provide a dedicated user pushbutton.
For button examples, wire an external active-low button to a free GPIO and
enable that pin's internal pull-up.
```

20.5 TCA0 — Timer/Counter Type A (16-bit)

Base address: 0x0A00 (single-slope mode, the default). TCA0 is a general-purpose 16-bit timer supporting normal, CTC, and PWM modes.

Register	Mem Addr	Description
TCA0_SINGLE_CTRLA	0x0A00	Prescaler select (CLKSEL), enable (ENABLE)
TCA0_SINGLE_CTRLB	0x0A01	Waveform mode (WGMODE), compare enables
TCA0_SINGLE_CTRLC	0x0A02	Force compare outputs
TCA0_SINGLE_CTRLD	0x0A03	Split mode enable
TCA0_SINGLE_CTRLECLR	0x0A04	Control E clear
TCA0_SINGLE_CTRLESET	0x0A05	Control E set
TCA0_SINGLE_INTCTRL	0x0A06	Interrupt enable (OVF, CMP0, CMP1, CMP2)
TCA0_SINGLE_INTFLAGS	0x0A07	Interrupt flags (W1C)
TCA0_SINGLE_EVCTRL	0x0A09	Event control
TCA0_SINGLE_DBGCTRL	0x0A0E	Debug control
TCA0_SINGLE_TEMP	0x0A0F	Temp register for 16-bit read/write
TCA0_SINGLE_CNT	0x0A20	Counter value (16-bit, write low byte first)
TCA0_SINGLE_PER	0x0A26	Period / TOP value (16-bit)
TCA0_SINGLE_CMP0	0x0A28	Compare/capture 0 (16-bit)
TCA0_SINGLE_CMP1	0x0A2A	Compare/capture 1 (16-bit)
TCA0_SINGLE_CMP2	0x0A2C	Compare/capture 2 (16-bit)
TCA0_SINGLE_PERBUF	0x0A36	Period buffer (double-buffered)
TCA0_SINGLE_CMP0BUF	0x0A38	Compare 0 buffer
TCA0_SINGLE_CMP1BUF	0x0A3A	Compare 1 buffer
TCA0_SINGLE_CMP2BUF	0x0A3C	Compare 2 buffer

20.5.1 CTRLA — Prescaler and Enable

```
Bit:  7:4    3:1       0
       —     CLKSEL    ENABLE
```

CLKSEL	Prescaler	
0x0	DIV1	(no prescaling)
0x1	DIV2	
0x2	DIV4	
0x3	DIV8	
0x4	DIV16	
0x5	DIV64	
0x6	DIV256	
0x7	DIV1024	

Example — 1 Hz overflow at 3.333 MHz with DIV64 (prescaled clock = 52,083 Hz, PER = 52082):

```
    ldi  r16, 0x0B              ; CLKSEL=DIV64 (bits 3:1 = 101), ENABLE=1
    sts  0x0A00, r16            ; TCA0_SINGLE_CTRLA
    ldi  r16, lo8(52082)
    sts  0x0A26, r16            ; TCA0_SINGLE_PER low byte
    ldi  r16, hi8(52082)
    sts  0x0A27, r16            ; TCA0_SINGLE_PER high byte
    ldi  r16, 0x01              ; OVF interrupt enable
    sts  0x0A06, r16            ; TCA0_SINGLE_INTCTRL
```

20.5.2 CTRLB — Waveform Mode

WGMODE[2:0] (bits 2:0)	Mode

0x0	Normal (count up, overflow at PER)
0x1	Frequency (toggle WO, reset at CMP0)
0x2	Single-slope PWM
0x5	Dual-slope PWM (overflow at BOTTOM and TOP)
0x6	Dual-slope PWM (overflow at TOP only)
0x7	Dual-slope PWM (overflow at BOTTOM only)

20.6 TCB0 / TCB1 — Timer/Counter Type B (16-bit)

Type B timers are 16-bit, single-channel timers suited for periodic interrupts, input capture, and PWM. TCB0 and TCB1 share the same register layout.

```
TCB0 base: 0x0A40
TCB1 base: 0x0A50
```

Offset	Register	Description
+0x00	TCBn_CTRLA	Clock select, enable
+0x01	TCBn_CTRLB	Compare/capture mode, output enable
+0x04	TCBn_EVCTRL	Event control
+0x05	TCBn_INTCTRL	CAPT interrupt enable (bit 0)
+0x06	TCBn_INTFLAGS	CAPT interrupt flag (bit 0, W1C)
+0x07	TCBn_STATUS	RUN flag (bit 0, read-only)
+0x08	TCBn_DBGCTRL	Debug control
+0x09	TCBn_TEMP	Temp register for 16-bit read/write
+0x0A	TCBn_CNTL	Counter low byte
+0x0B	TCBn_CNTH	Counter high byte
+0x0C	TCBn_CCMPL	Compare/capture low byte
+0x0D	TCBn_CCMPH	Compare/capture high byte

20.6.1 CTRLA — Clock Select and Enable

```
Bit:  7:3    2:1       0
       —     CLKSEL    ENABLE
```

CLKSEL	Clock source	
0x0	CLKPER	(same as system clock)
0x1	CLKPER/2	
0x2	TCA0	(TCB clocked from TCA0 overflow)

20.6.2 CTRLB — Mode (CMODE)

CMODE[2:0] (bits 2:0)	Mode
0x0	Periodic Interrupt ← default, most common
0x1	Timeout Check
0x2	Input Capture on Event
0x3	Input Capture Frequency Measurement
0x4	Input Capture Pulse-Width Measurement
0x5	Input Capture Freq+PW Measurement
0x6	Single Shot
0x7	8-bit PWM

In Periodic Interrupt mode (CMODE=0), the counter counts up to CCMP, fires the CAPT interrupt, then resets to zero. CCMP sets the period (TOP).

Example — 1 kHz periodic interrupt at 3.333 MHz:

```
    ldi  r16, lo8(3332)         ; CCMP = (f_cpu / f_irq) - 1 = 3332
    sts  0x0A4C, r16            ; TCB0_CCMPL
    ldi  r16, hi8(3332)
    sts  0x0A4D, r16            ; TCB0_CCMPH
    ldi  r16, 0x01              ; CAPT interrupt enable
    sts  0x0A45, r16            ; TCB0_INTCTRL
    ldi  r16, 0x01              ; CLKSEL=CLKPER, ENABLE=1
    sts  0x0A40, r16            ; TCB0_CTRLA
```

20.7 USART0

Base address: 0x0800. On the Curiosity Nano board, USART0 is connected to the on-board debugger (PB2=TX, PB3=RX), providing a virtual COM port over USB.

Register	Mem Addr	Description
USART0_RXDATAL	0x0800	Receive data (low byte, 8-bit data here)
USART0_RXDATAH	0x0801	Receive data high + error flags
USART0_TXDATAL	0x0802	Transmit data (write byte here to send)
USART0_TXDATAH	0x0803	Transmit data high (9-bit mode only)
USART0_STATUS	0x0804	Status flags
USART0_CTRLA	0x0805	Interrupt enables
USART0_CTRLB	0x0806	Receiver/transmitter enable
USART0_CTRLC	0x0807	Frame format (default = 8N1)
USART0_BAUDL	0x0808	Baud rate register low byte
USART0_BAUDH	0x0809	Baud rate register high byte
USART0_CTRLD	0x080A	Auto-baud, IRCOM
USART0_DBGCTRL	0x080B	Debug control
USART0_EVCTRL	0x080C	Event control
USART0_TXPLCTRL	0x080D	TX pin LINAUTO
USART0_RXPLCTRL	0x080E	RX pin LINAUTO

20.7.1 STATUS Bits

```
Bit:  7      6      5      4      3       2      1:0
      RXCIF  TXCIF  DREIF  FERR   BUFOVF  PERR   —

RXCIF   Receive Complete Interrupt Flag  (1 = unread data in RXDATA)
TXCIF   Transmit Complete Interrupt Flag (1 = TX shift register empty)
DREIF   Data Register Empty Flag        (1 = OK to write next byte to TXDATA)
FERR    Frame Error
BUFOVF  Buffer Overflow
PERR    Parity Error
```

20.7.2 CTRLA — Interrupt Enables

```
Bit:  7      6      5      4:2    1      0
```

```
      RXCIE  TXCIE  DREIE  —      LBME   ABEIE

RXCIE   Receive Complete IE
TXCIE   Transmit Complete IE
DREIE   Data Register Empty IE (triggers when ready for next transmit byte)
```

20.7.3 CTRLB — Enable Bits

```
Bit:  7    6    5:4    3       2      1:0
      RXEN TXEN MPCM   RXMODE  ODME   —

RXEN    1 = enable receiver
TXEN    1 = enable transmitter
RXMODE  0x0=NORMAL, 0x1=CLK2X, 0x2=GENAUTO, 0x3=LINAUTO
```

20.7.4 CTRLC — Frame Format

```
Default value: 0x03 = 8N1 (8 data bits, no parity, 1 stop bit)

Bit:  7:6       5:4     3     2:0
      CMODE     PMODE   SBMODE CHSIZE

CMODE   0x0=ASYNCHRONOUS, 0x1=SYNCHRONOUS, 0x2=IRCOM, 0x3=MSPI
PMODE   0x0=DISABLED, 0x2=EVEN, 0x3=ODD
SBMODE  0=1 stop bit, 1=2 stop bits
CHSIZE  0x0=5-bit, 0x1=6-bit, 0x2=7-bit, 0x3=8-bit, 0x4-0x6=reserved, 0x7=9-bit
```

20.7.5 Baud Rate Formula and Common Values

Normal asynchronous mode:

```
BAUD_REG = (f_cpu × 64) / (16 × baud_rate)
         = (f_cpu × 4) / baud_rate
```

Baud Rate	f_cpu = 3.333 MHz BAUD_REG (hex)	f_cpu = 20 MHz BAUD_REG (hex)
9600	0x056D (1389)	0x2098 (8344)
19200	0x02B7 (694)	0x104C (4172)
38400	0x015B (347)	0x0826 (2086)
57600	0x00E2 (226)	0x0571 (1393)
115200	0x0074 (116)	0x02B9 (697)

20.8 SPI0

Base address: 0x08C0. Default pin assignments: MOSI=PA1, MISO=PA2, SCK=PA3, SS=PA4.

Register	Mem Addr	Description
SPI0_CTRLA	0x08C0	Control A: DORD, MASTER, CLK2X, PRESC[1:0], ENABLE
SPI0_CTRLB	0x08C1	Control B: BUFEN, SSD, MODE[1:0]

```
SPI0_INTCTRL    0x08C2     Interrupt enables: IE, SSIE, TXCIE, RXCIE
SPI0_INTFLAGS   0x08C3     Interrupt flags: IF(7), WRCOL(0)
SPI0_DATA       0x08C4     TX/RX data register
```

20.8.1 CTRLA Bits

```
Bit:  7      6      5      4:3      2      1      0
      DORD   MASTER CLK2X  PRESC    –      –      ENABLE

DORD    0=MSB first, 1=LSB first
MASTER  0=slave, 1=master
CLK2X   1=double clock speed
PRESC   0x0=DIV4, 0x1=DIV16, 0x2=DIV64, 0x3=DIV128
ENABLE  1=enable SPI
```

20.8.2 SPI Clock Rates

PRESC	CLK2X=0	CLK2X=1
0x0	f_cpu / 4	f_cpu / 2
0x1	f_cpu / 16	f_cpu / 8
0x2	f_cpu / 64	f_cpu / 32
0x3	f_cpu / 128	f_cpu / 64

20.9 TWI0 — Two-Wire Interface (I2C)

Base address: 0x08A0. Default pins: SDA=PB0, SCL=PB1. Requires external 4.7 kΩ pull-up resistors on SDA and SCL lines.

Register	Mem Addr	Description
TWI0_CTRLA	0x08A0	SDA hold time, fast-mode plus enable
TWI0_DUALCTRL	0x08A1	Dual-mode control
TWI0_MCTRLA	0x08A2	Master: RIEN, WIEN, QCEN, TIMEOUT, SMEN, ENABLE
TWI0_MCTRLB	0x08A3	Master: FLUSH, ACKACT, MCMD[1:0]
TWI0_MSTATUS	0x08A4	Master status flags
TWI0_MBAUD	0x08A5	Master baud rate
TWI0_MADDR	0x08A6	Master address (write triggers START condition)
TWI0_MDATA	0x08A7	Master data register (read/write)
TWI0_SCTRLA	0x08A8	Slave: DIEN, APIEN, PIEN, PMEN, SMEN, ENABLE
TWI0_SCTRLB	0x08A9	Slave: ACKACT, SCMD[1:0]
TWI0_SSTATUS	0x08AA	Slave status flags
TWI0_SADDR	0x08AB	Slave address
TWI0_SDATA	0x08AC	Slave data
TWI0_SADDRMASK	0x08AD	Slave address mask

20.9.1 MSTATUS Bits

```
Bit:  7    6    5        4      3        2       1:0
      RIF  WIF  CLKHOLD  RXACK  ARBLOST  BUSERR  BUSSTATE
```

```
RIF       Read Interrupt Flag
WIF       Write Interrupt Flag
CLKHOLD   Clock hold (SCL stretched by master)
RXACK     Received ACK (0=ACK received, 1=NACK)
ARBLOST   Arbitration lost
BUSERR    Bus error
BUSSTATE  0x0=UNKNOWN, 0x1=IDLE, 0x2=OWNER, 0x3=BUSY
```

20.9.2 MCTRLB — Master Command

MCMD[1:0]	Command	
0x0	NOACT	No action
0x1	REPSTART	Issue repeated START
0x2	RECVTRANS	Continue (ACK) / send byte
0x3	STOP	Issue STOP condition

20.9.3 Baud Rate Formula

```
BAUD = (f_cpu / f_scl - 10) / 2        (for f_cpu in Hz, f_scl in Hz)

100 kHz at 3.333 MHz:  BAUD = (3333333 / 100000 - 10) / 2 ≈ 12
400 kHz at 3.333 MHz:  BAUD = (3333333 / 400000 - 10) / 2 ≈ 2 (use fast-mode+)
100 kHz at 20 MHz:     BAUD = (20000000 / 100000 - 10) / 2 = 95
400 kHz at 20 MHz:     BAUD = (20000000 / 400000 - 10) / 2 = 20
```

20.10 ADC0

Base address: 0x0600. The ATtiny3217 ADC is 10-bit (or 8-bit selectable).

Register	Mem Addr	Description
ADC0_CTRLA	0x0600	Enable, resolution, free-run, left-adjust
ADC0_CTRLB	0x0601	Sample accumulation (SAMPNUM)
ADC0_CTRLC	0x0602	Prescaler, reference select, sample capacitor
ADC0_CTRLD	0x0603	Init delay, auto-sample delay
ADC0_CTRLE	0x0604	Window comparator mode
ADC0_SAMPCTRL	0x0605	Sample length extension
ADC0_MUXPOS	0x0606	Input mux (selects channel)
ADC0_COMMAND	0x0608	Write bit 0 (STCONV) to start conversion
ADC0_EVCTRL	0x0609	Event trigger enable
ADC0_INTCTRL	0x060A	RESRDY(0) and WCMP(1) interrupt enables
ADC0_INTFLAGS	0x060B	Interrupt flags (W1C)
ADC0_RESL	0x060C	Result low byte
ADC0_RESH	0x060D	Result high byte
ADC0_WINLT	0x060E	Window comparator low threshold
ADC0_WINHT	0x0610	Window comparator high threshold
ADC0_CALIB	0x0612	Calibration

20.10.1 CTRLA Bits

```
Bit:  7      6:5    4         3        2       1        0
```

```
     –      –      LEFTADJ   RESSEL   FREERUN RUNSTDBY  ENABLE

LEFTADJ   1=result left-adjusted in 16-bit register (8-bit in RESH)
RESSEL    0=10-bit result, 1=8-bit result
FREERUN   1=continuous conversions
ENABLE    1=enable ADC
```

20.10.2 CTRLC — Prescaler and Reference

```
Bit:  7:6    5:4      3        2:0
      –      REFSEL   SAMPCAP  PRESC

REFSEL   0x0=INTREF (internal), 0x1=VDDIO2/10, 0x2=reserved, 0x3=VREFA (external)
SAMPCAP  1=reduced sample capacitance (for high-impedance sources)
PRESC    ADC clock prescaler (see table below)
```

PRESC	Division
0x0	DIV2
0x1	DIV4
0x2	DIV8
0x3	DIV16
0x4	DIV32
0x5	DIV64
0x6	DIV128
0x7	DIV256

ADC requires 50–1500 kHz clock. At 3.333 MHz use DIV8 (417 kHz) or DIV16 (208 kHz). At 20 MHz use DIV32 (625 kHz) or DIV64 (312 kHz).

20.10.3 MUXPOS — Input Mux

MUXPOS	Channel
0x00	AIN0 (PA0 – also UPDI, avoid)
0x01	AIN1 (PA1)
0x02	AIN2 (PA2)
0x03	AIN3 (PA3)
0x04	AIN4 (PA4)
0x05	AIN5 (PA5)
0x06	AIN6 (PA6)
0x07	AIN7 (PA7)
0x08	AIN8 (PB5)
0x09	AIN9 (PB4)
0x0A	AIN10 (PB1)
0x0B	AIN11 (PB0)
0x1B	DACREF0
0x1C	TEMPSENSE (internal temperature sensor)
0x1D	GND
0x1E	VDDIO2/10
0x1F	INTREF (internal reference voltage)

20.10.4 Internal Reference Voltage

The internal reference is configured via VREF peripheral (base 0x00A0):

Register	Mem Addr	Description
VREF_CTRLA	0x00A0	ADC0 and DAC0 reference select
VREF_CTRLB	0x00A1	ADC0 reference force enable

```
VREF_CTRLA bits 2:0 (ADC0REFSEL):
  0x0   0.55V
  0x1   1.1V
  0x2   2.5V
  0x3   4.3V
  0x4   1.5V
```

Default internal reference: 1.1V.

20.11 NVMCTRL — Non-Volatile Memory Controller

Base address: 0x1000. Writing flash or EEPROM requires writing the CCP_SPM key (0x9D) to CCP before the command, then completing the write within 4 cycles.

Register	Mem Addr	Description
NVMCTRL_CTRLA	0x1000	Command register (write CCP key first)
NVMCTRL_CTRLB	0x1001	Boot lock, application code write protect
NVMCTRL_STATUS	0x1002	Status flags (read-only)
NVMCTRL_INTCTRL	0x1003	EEREADY interrupt enable
NVMCTRL_INTFLAGS	0x1004	EEREADY interrupt flag (W1C)
NVMCTRL_DATA	0x1006	Data register (16-bit)
NVMCTRL_ADDR	0x1008	Address register (16-bit)

20.11.1 CTRLA — NVM Commands

CMD[2:0] (bits 2:0)	Command	
0x00	NOCMD	No operation
0x01	PAGEWRITE	Write page buffer to flash
0x02	PAGEERASE	Erase one page of flash
0x03	PAGEERASEWRITE	Erase and write in one operation
0x04	PAGEBUFCLR	Clear page write buffer
0x05	CHIPERASE	Erase entire chip (flash + EEPROM)
0x06	EEERASE	Erase EEPROM
0x07	FUSEWRITE	Write fuse bytes

20.11.2 STATUS Bits

```
Bit:  2        1       0
      WRERROR  EEBUSY  FBUSY

FBUSY    1 = flash write/erase in progress
EEBUSY   1 = EEPROM write/erase in progress
WRERROR  1 = write error occurred
```

20.11.3 Memory Layout

Region	Start	End	Size	Notes
Flash	0x0000	0x7FFF	32 KB	Program memory (word-addressed by PC)
SRAM	0x3800	0x3FFF	2 KB	Data memory (avr-ld origin 0x803800)
EEPROM	0x1400	0x14FF	256 B	(memory-mapped for read; use NVMCTRL to write)
Fuses	0x1280	0x128A		Written via UPDI/avrdude

Flash page size: 64 bytes. EEPROM page size: 32 bytes.

20.12 Interrupt Vector Table

All vectors are 4-byte (2-word) entries. Use `JMP` (4-byte instruction) for targets anywhere in the 32 KB flash, or `RJMP` + `NOP` (2+2 bytes) to keep the same slot size.

Vector	Byte Addr	Name	Description
0	0x0000	RESET	Power-on / external / WDT reset
1	0x0004	CRCSCAN_NMI	CRC scan non-maskable interrupt
2	0x0008	BOD_VLM	Brown-out detection voltage level monitor
3	0x000C	PORTA_PORT	Port A pin-change interrupt
4	0x0010	PORTB_PORT	Port B pin-change interrupt
5	0x0014	PORTC_PORT	Port C pin-change interrupt
6	0x0018	RTC_CNT	RTC overflow / compare match
7	0x001C	RTC_PIT	RTC periodic interrupt timer
8	0x0020	TCA0_OVF / TCA0_LUNF	TCA0 overflow (single) / low underflow (split)
9	0x0024	TCA0_HUNF	TCA0 high byte underflow (split mode)
10	0x0028	TCA0_CMP0 / TCA0_LCMP0	TCA0 compare 0 match
11	0x002C	TCA0_CMP1 / TCA0_LCMP1	TCA0 compare 1 match
12	0x0030	TCA0_CMP2 / TCA0_LCMP2	TCA0 compare 2 match
13	0x0034	TCB0_INT	TCB0 capture interrupt
14	0x0038	TCB1_INT	TCB1 capture interrupt
15	0x003C	TWI0_TWIS	TWI0 slave interrupt
16	0x0040	TWI0_TWIM	TWI0 master interrupt
17	0x0044	SPI0_INT	SPI0 transfer complete
18	0x0048	USART0_RXC	USART0 receive complete
19	0x004C	USART0_DRE	USART0 data register empty
20	0x0050	USART0_TXC	USART0 transmit complete
21	0x0054	AC0_AC	Analog comparator interrupt
22	0x0058	ADC0_RESRDY	ADC0 result ready
23	0x005C	ADC0_WCMP	ADC0 window comparator match
24	0x0060	CCL_CCL	Configurable custom logic interrupt
25	0x0064	NVMCTRL_EE	EEPROM ready

Minimal startup with vector table in assembly:

```
    .section .vectors,"ax",@progbits
    jmp   _start              ; 0x0000 RESET — must be first
    jmp   _unhandled          ; 0x0004 CRCSCAN_NMI
    ; ... one jmp per vector ...

    .section .text
_unhandled:
```

```
    rjmp _unhandled          ; spin on unhandled interrupts

_start:
    ; ... init stack, SRAM, then jump to main
```

20.13 Fuses

Fuses are one-time-programmable configuration bytes written via UPDI using `avrdude`. Factory defaults are shown. Unlike ATmega, there is no separate CKDIV, BOOTRST, or BOOTSZ fuse.

```
Fuse     Mem Addr   Default   Description
─────────────────────────────────────────────────────────────────────
FUSE0    0x1280     0x00      APPEND[7:0]   — application code end page
FUSE1    0x1281     0x00      BOOTEND[7:0]  — boot section end page (0=none)
FUSE2    0x1282     0x02      OSCCFG        — oscillator config
FUSE4    0x1284     0xF6      SYSCFG0       — CRC, reset pin, UPDI pin config
FUSE5    0x1285     0x07      SYSCFG1       — startup time SUT[2:0]
FUSE6    0x1286     0x00      (same as FUSE0 on this device)
FUSE7    0x1287     0x00      (same as FUSE1)
FUSE8    0x1288     0xAA      LOCKBIT
```

20.13.1 FUSE2 — OSCCFG

```
Bit:  1       0
      FREQSEL FREQSEL

FREQSEL  0=16 MHz internal oscillator base
         1=20 MHz internal oscillator base  ← factory default (0x02)
```

The clock prescaler in CLKCTRL_MCLKCTRLB further divides this frequency. Factory default: 20 MHz / 6 = 3.333 MHz.

20.13.2 FUSE4 — SYSCFG0

```
Bit:  7:6    5:4       3:2        1:0
      CRCSRC RSTPINCFG UPDIPINCFG —

RSTPINCFG  0x0=GPIO, 0x1=UPDI, 0x2=RESET  (0xF6 default = RESET=GPIO, UPDI active)
CRCSRC     0x3=no CRC check (default)
```

20.13.3 Writing Fuses with avrdude

```
avrdude -c serialupdi -p t3217 -P /dev/ttyUSB0 -b 115200 \
        -U fuse2:w:0x02:m

avrdude -c serialupdi -p t3217 -P /dev/ttyUSB0 -b 115200 \
        -U fuse0:w:0x00:m -U fuse1:w:0x00:m -U fuse2:w:0x02:m \
        -U fuse4:w:0xF6:m -U fuse5:w:0x07:m
```

> **Warning**: Setting RSTPINCFG incorrectly can disable the UPDI programming interface, effectively bricking the device. Never modify FUSE4 unless certain.

Chapter 21

AVR Instruction Set Summary

Complete instruction set for AVR (classic core, enhanced core with MUL). Columns: **Opcode** — mnemonic and operands; **Operation** — what it does; **Cycles** — clock cycles (branch cycles shown as taken/not-taken); **Flags** — SREG bits modified (I T H S V N Z C).

`Rd` = destination register (R0–R31 unless noted). `Rr` = source register. `K` = 8-bit constant (0–255). `k` = relative branch offset (−64..63 for 7-bit, −2048..2047 for 12-bit). `b` = bit index (0–7). `A` = I/O address (0–63). `q` = displacement (0–63) for Y/Z indirect with displacement.

21.1 How to Read the AVR Instruction Set Manual

This appendix is a quick reference. The Microchip instruction set manual is the authoritative source. If you only use the summary table, you can write code. If you learn to read the manual pages directly, you can answer harder questions:

- Why does this instruction only work on `r16–r31`?
- Why is this branch 1 cycle sometimes and 2 cycles other times?
- Why does `SBIS` skip differently from `BRNE`?
- Why does `OUT` work here but `STS` is required there?
- Why does an instruction update `Z` but not `C`?

The manual is not hard once you know what each field is trying to tell you. The problem is that it reads like a hardware document, not a tutorial.

21.1.1 What One Instruction Page Contains

Each instruction in the manual is usually presented in the same structure:

- **Description / operation**: what value changes logically.
- **Syntax**: the assembly form, such as `ADD Rd, Rr`.
- **Operands**: legal ranges, such as `0 <= d <= 31`.
- **Program Counter**: how `PC` changes after execution.
- **Opcode**: the encoded bit pattern.
- **Status Register (SREG) and Boolean Formula**: which flags change and why.
- **Words / Cycles**: code size and execution time, often by AVR core family.

When reading a page, do not read it top to bottom like prose. Read it in this order instead:

1. **Syntax**: what form the instruction has.

2. **Operands**: which registers or constants are legal.
3. **Operation**: what value is computed.
4. **SREG effects**: which later branches or compares it can support.
5. **Words / Cycles**: whether it is a good fit for tight loops.
6. **Opcode**: only when you want to understand encoding limits or disassembly.

That order matches how you actually use the information while programming.

21.1.2 Start with Syntax, but Trust the Operand Limits

Beginners often read `LDI Rd, K` as "load any register with any 8-bit constant". The syntax alone does not tell the whole story. The **Operands** field does:

```
LDI Rd, K
Operands: 16 <= d <= 31, 0 <= K <= 255
```

That means `ldi r0, 1` is impossible, not merely discouraged. The reason is in the opcode encoding: `LDI` only has four bits for the register number, so the encoded register is really `Rd - 16`.

This pattern appears all over AVR:

- `LDI`, `ANDI`, `ORI`, `SUBI`, `SBCI`, `CPI` only work on `r16`–`r31`.
- `ADIW` and `SBIW` only work on specific register pairs.
- `MOVW` only uses even-numbered register pairs.
- `SBI`, `CBI`, `SBIC`, `SBIS` only work on low I/O addresses.

If the syntax looks general but the operand range is narrow, believe the range. The encoding is the real reason.

21.1.3 Read "Words" Before "Cycles"

AVR documentation uses **words**, not bytes, as the natural unit of program storage. One word is 16 bits, so:

- **1 word** = 2 bytes
- **2 words** = 4 bytes

This matters because AVR instructions are either one word or two words. That affects all of these at once:

- code size in flash
- instruction fetch behaviour
- skip-instruction timing
- absolute vs relative addressing choices

Examples:

- `OUT` is one word; `STS` is two words.
- `RJMP` is one word; `JMP` is two words.
- `RCALL` is one word; `CALL` is two words.

So when the manual says an instruction is "Words: 2", translate that immediately into "this costs 4 bytes and may also affect skip timing".

21.1.4 Relative Addresses Are Not Absolute Addresses

AVR manuals use several similar-looking operand letters:

- `K` often means an immediate constant.

- `k` often means a relative code offset.
- `q` means a small displacement from `Y` or `Z`.
- `A` means a low I/O address.

Case matters. `K` and `k` are not interchangeable.

For branches and relative calls:

- `RJMP k` means jump relative to the current `PC`
- `RCALL k` means call relative to the current `PC`
- `BRNE k` means branch a short relative distance if `Z = 0`

The manual usually writes this as:

```
PC <- PC + k + 1
```

That `+ 1` exists because the PC normally advances to the next instruction word before the relative offset is applied. The important practical result is:

- `RJMP` and `RCALL` do not encode a full absolute address.
- `BRxx` instructions have much shorter range than `RJMP`.
- Disassemblers may show targets as labels, but the hardware stores offsets.

21.1.5 Read the SREG Section as "What Can I Test Next?"

The **Status Register (SREG)** is not just bookkeeping. It determines what branches are meaningful after an instruction.

The bits are:

- `I`: global interrupt enable
- `T`: transfer bit for `BST` / `BLD`
- `H`: half-carry, mainly for nibble/BCD-style carries
- `S`: sign bit, defined as `N xor V`
- `V`: signed overflow
- `N`: negative
- `Z`: zero
- `C`: carry / borrow

When reading an instruction page, ask:

- Does this instruction update flags at all?
- Which flags does it update?
- Are those flags enough for unsigned comparison, signed comparison, or both?

Examples:

- `ADD` updates `Z`, `C`, `N`, `V`, `S`, `H`, so it supports later arithmetic decisions.
- `INC` does **not** update `C`, so it is not a drop-in replacement for `ADD Rd, 1` if carry matters.
- `TST` is really a flag-producing logical test, useful before `BREQ` or `BRMI`.
- `CP` changes flags without changing registers, which is why branches usually follow it directly.

The fastest way to understand a flag table is to map it to branch usage:

- For **unsigned** comparisons, care mostly about `C` and `Z`.
- For **signed** comparisons, care mostly about `S`, `V`, `N`, and `Z`.
- For equality, care about `Z`.

That is why AVR has both unsigned-style branches (`BRLO`, `BRSH`) and signed-style branches (`BRLT`, `BRGE`).

21.1.6 Do Not Panic About the Boolean Formulas

The manual often gives flag equations in Boolean form. Those formulas are precise, but you usually do not need to derive them by hand while writing ordinary code.

Use them in three situations:

1. When a flag result surprises you.
2. When writing multi-byte arithmetic where carry/borrow chains matter.
3. When reviewing whether two instructions are truly interchangeable.

For most programming, this simpler interpretation is enough:

- `C`: carry out of bit 7 for addition, borrow behaviour for subtraction
- `Z`: result was zero
- `N`: result bit 7 is 1
- `V`: signed overflow happened
- `S`: signed comparison helper, `N xor V`
- `H`: carry from bit 3 to bit 4

If you are not doing decimal-style arithmetic or bit-exact arithmetic tricks, `H` is usually less important than `Z`, `C`, and `S`.

21.1.7 Cycles Are Family-Specific

The instruction set manual often shows cycle counts across multiple AVR core families. This book targets the ATtiny3217, which is an **AVRxt** device, so the AVRxt column is the one that matters most here.

Do not assume:

- a timing number from an older ATmega necessarily matches AVRxt
- all loads and stores cost the same on every AVR family
- every manual example was written with your exact core in mind

When timing matters, read the correct family column first, then read any note attached to it. Notes are where the manual hides the details that break naive assumptions.

21.1.8 Skip Instructions Need Extra Attention

`CPSE`, `SBRC`, `SBRS`, `SBIC`, and `SBIS` do not branch. They **skip the next instruction** if the condition is true.

That sounds simple, but the manual's word count matters here:

- If the next instruction is 1 word, the skip jumps over 2 bytes.
- If the next instruction is 2 words, the skip jumps over 4 bytes.
- That is why skip instructions have multiple cycle counts.

So this is safe:

```
sbrc r16, 0
rjmp bit_was_set
```

But this behaves differently from a true branch because the skip is based on the size of the *next* instruction, not on a stored relative destination.

When reading the manual, any cycle count like `1/2/3` or `2/3/4` should make you ask: "is this a skip instruction, and is the next instruction one or two words long?"

21.1.9 Learn the Address Spaces Separately

AVR documentation is much easier once you stop treating "memory" as one thing. The manual switches among several address spaces:

- **register file**: `r0–r31`
- **I/O space**: low peripheral addresses used by `IN` / `OUT`
- **data space**: SRAM and memory-mapped locations used by `LD` / `ST` / `LDS` / `STS`
- **program memory**: flash, read with `LPM`

Many confusing instruction choices become obvious once you ask which space is being accessed.

Example:

- `OUT PORTA_DIR, r16` only works if that register is in low I/O space.
- `STS PORTA_DIR, r16` works with a data-space address, but costs more.
- `LPM` is for flash, not SRAM.

If two instructions seem to do the same job, check whether they are really talking to different address spaces.

21.1.10 Aliases and Pseudo-Instructions Can Hide the Real Opcode

Assembly source often uses friendly names that are not separate hardware instructions.

Examples:

- `CLR Rd` is assembled as `EOR Rd, Rd`.
- `SER Rd` is assembled as `LDI Rd, 0xFF`.
- `LSL Rd` is effectively `ADD Rd, Rd`.
- named branches like `BREQ` and `BRNE` are readable aliases for specific `BRBS` / `BRBC` conditions.

That matters because the manual may document one spelling while the assembler accepts another. If something looks inconsistent, check whether you are dealing with:

- a real opcode
- an alias mnemonic
- a pseudo-instruction expanded by the assembler

21.1.11 A Good Manual-Reading Workflow

When you meet an unfamiliar instruction, use this checklist:

1. Read the syntax and operand limits.
2. Identify the address space involved: register, I/O, SRAM, or flash.
3. Check whether it is 1 word or 2 words.
4. Check which SREG flags it updates.
5. Check whether the cycle count depends on branch taken/not taken, skip size, or AVR core family.
6. Check whether there is a shorter equivalent instruction for your case.

That method is enough to decode most of the manual quickly.

21.1.12 What This Means for the Rest of the Book

Use Chapter 4 to learn the common instruction patterns, then come back to this appendix when you need the full table. Use the Microchip manual when a question depends on one of these details:

- exact flag behaviour
- exact operand limits
- instruction encoding constraints
- family-specific cycle counts
- special notes about program memory, skip timing, or core differences

Once you start reading the manual this way, it stops looking like a wall of tables and starts looking like a set of answers to very specific questions.

21.2 Arithmetic and Logic

Opcode	Operation	Cycles	Flags
ADD Rd, Rr	Rd ← Rd + Rr	1	H S V N Z C
ADC Rd, Rr	Rd ← Rd + Rr + C	1	H S V N Z C
ADIW Rd, K	Rd+1:Rd ← Rd+1:Rd + K (0–63)	2	S V N Z C
SUB Rd, Rr	Rd ← Rd − Rr	1	H S V N Z C
SUBI Rd, K	Rd ← Rd − K	1	H S V N Z C
SBC Rd, Rr	Rd ← Rd − Rr − C	1	H S V N Z C
SBCI Rd, K	Rd ← Rd − K − C	1	H S V N Z C
SBIW Rd, K	Rd+1:Rd ← Rd+1:Rd − K (0–63)	2	S V N Z C
AND Rd, Rr	Rd ← Rd AND Rr	1	S V N Z
ANDI Rd, K	Rd ← Rd AND K (R16–R31)	1	S V N Z
OR Rd, Rr	Rd ← Rd OR Rr	1	S V N Z
ORI Rd, K	Rd ← Rd OR K (R16–R31)	1	S V N Z
EOR Rd, Rr	Rd ← Rd XOR Rr	1	S V N Z
COM Rd	Rd ← 0xFF − Rd (bitwise NOT)	1	S V N Z C
NEG Rd	Rd ← 0x00 − Rd (two's comp)	1	H S V N Z C
INC Rd	Rd ← Rd + 1	1	S V N Z
DEC Rd	Rd ← Rd − 1	1	S V N Z
MUL Rd, Rr	R1:R0 ← Rd × Rr (unsigned)	2	Z C
MULS Rd, Rr	R1:R0 ← Rd × Rr (signed)	2	Z C
	Rd, Rr: R16–R31 only		
MULSU Rd, Rr	R1:R0 ← Rd × Rr (sgn × uns)	2	Z C
	Rd, Rr: R16–R23 only		
FMUL Rd, Rr	R1:R0 ← (Rd × Rr) << 1 (uns)	2	Z C
FMULS Rd, Rr	R1:R0 ← (Rd × Rr) << 1 (sgn)	2	Z C
FMULSU Rd, Rr	R1:R0 ← (Rd × Rr) << 1 (s×u)	2	Z C

21.3 Shift and Rotate

Opcode	Operation	Cycles	Flags
LSL Rd	Rd ← Rd << 1; Rd0=0; C←Rd7	1	H S V N Z C
LSR Rd	Rd ← Rd >> 1; Rd7=0; C←Rd0	1	S V N Z C
ASR Rd	Rd ← Rd >> 1; Rd7 unchanged	1	S V N Z C
ROL Rd	Rd ← Rd << 1; Rd0=C; C←Rd7	1	H S V N Z C
ROR Rd	Rd ← Rd >> 1; Rd7=C; C←Rd0	1	S V N Z C

SWAP Rd	Rd[7:4] ↔ Rd[3:0]	1	–

21.4 Bit Operations

Opcode	Operation	Cycles	Flags
SBI A, b	I/O[A][b] ← 1 (A: 0–31)	2	–
CBI A, b	I/O[A][b] ← 0 (A: 0–31)	2	–
BST Rd, b	T ← Rd[b]	1	T
BLD Rd, b	Rd[b] ← T	1	–
SEC	C ← 1	1	C
CLC	C ← 0	1	C
SEN	N ← 1	1	N
CLN	N ← 0	1	N
SEZ	Z ← 1	1	Z
CLZ	Z ← 0	1	Z
SEI	I ← 1 (global interrupt on)	1	I
CLI	I ← 0 (global interrupt off)	1	I
SES	S ← 1	1	S
CLS	S ← 0	1	S
SEV	V ← 1	1	V
CLV	V ← 0	1	V
SET	T ← 1	1	T
CLT	T ← 0	1	T
SEH	H ← 1	1	H
CLH	H ← 0	1	H
BSET b	SREG[b] ← 1	1	(b)
BCLR b	SREG[b] ← 0	1	(b)

21.5 Compare and Test

Opcode	Operation	Cycles	Flags
CP Rd, Rr	Rd − Rr (flags only)	1	H S V N Z C
CPC Rd, Rr	Rd − Rr − C (flags only)	1	H S V N Z C
CPI Rd, K	Rd − K (flags only, R16–R31)	1	H S V N Z C
TST Rd	Rd AND Rd (flags only)	1	S V N Z

21.6 Branch Instructions

Opcode	Condition	Cycles	Notes
RJMP k	–	2	PC ← PC + k + 1; k: −2048..2047
JMP k	–	3	PC ← k; 32-bit instruction
IJMP	–	2	PC ← Z (16-bit)
EIJMP	–	2	PC ← EIND:Z (>128KB flash)

```
RCALL k          –                   3        Push PC, PC ← PC + k + 1
CALL  k          –                   4        Push PC, PC ← k; 32-bit
ICALL            –                   3        Push PC, PC ← Z
EICALL           –                   3        Push PC, PC ← EIND:Z
RET              –                   4        Pop PC
RETI             –                   4        Pop PC, I ← 1

BRBS b, k        SREG[b] = 1        1/2      Branch if bit set; 1=not taken, 2=taken
BRBC b, k        SREG[b] = 0        1/2      Branch if bit clear

/* Named branch mnemonics (aliases for BRBS/BRBC) */
BREQ k           Z = 1              1/2      Branch if equal
BRNE k           Z = 0              1/2      Branch if not equal
BRCS k           C = 1              1/2      Branch if carry set
BRCC k           C = 0              1/2      Branch if carry clear
BRSH k           C = 0              1/2      Branch if same or higher (unsigned ≥)
BRLO k           C = 1              1/2      Branch if lower (unsigned <)
BRMI k           N = 1              1/2      Branch if minus
BRPL k           N = 0              1/2      Branch if plus
BRGE k           S = 0              1/2      Branch if greater or equal (signed ≥)
BRLT k           S = 1              1/2      Branch if less than (signed <)
BRHS k           H = 1              1/2      Branch if half-carry set
BRHC k           H = 0              1/2      Branch if half-carry clear
BRTS k           T = 1              1/2      Branch if T set
BRTC k           T = 0              1/2      Branch if T clear
BRVS k           V = 1              1/2      Branch if overflow set
BRVC k           V = 0              1/2      Branch if overflow clear
BRIE k           I = 1              1/2      Branch if interrupt enabled
BRID k           I = 0              1/2      Branch if interrupt disabled
```

21.6.1 Skip Instructions

Opcode	Condition	Cycles	Notes
CPSE Rd, Rr	Rd = Rr → skip next	1/2/3	skip 1 or 2-word instruction
SBRC Rd, b	Rd[b] = 0 → skip	1/2/3	
SBRS Rd, b	Rd[b] = 1 → skip	1/2/3	
SBIC A, b	I/O[A][b] = 0 → skip	2/3/4	A: 0–31 only
SBIS A, b	I/O[A][b] = 1 → skip	2/3/4	A: 0–31 only

Skip cycle counts: 1 = no skip; 2 = skip 1-word instruction; 3 = skip 2-word instruction (LDS, STS, CALL, JMP). SBIC/SBIS add 1 due to I/O read.

21.7 Load and Store

Opcode	Operation	Cycles	Notes
MOV Rd, Rr	Rd ← Rr	1	
MOVW Rd, Rr	Rd+1:Rd ← Rr+1:Rr	1	Rd, Rr: even registers
LDI Rd, K	Rd ← K (R16–R31 only)	1	
LDS Rd, k	Rd ← SRAM[k]	2	k: 16-bit address

STS k, Rr	SRAM[k] ← Rr	2	k: 16-bit address
/* X register (R27:R26) indirect */			
LD Rd, X	Rd ← SRAM[X]	2	
LD Rd, X+	Rd ← SRAM[X]; X ← X+1	2	
LD Rd, −X	X ← X−1; Rd ← SRAM[X]	2	
ST X, Rr	SRAM[X] ← Rr	2	
ST X+, Rr	SRAM[X] ← Rr; X ← X+1	2	
ST −X, Rr	X ← X−1; SRAM[X] ← Rr	2	
/* Y register (R29:R28) indirect */			
LD Rd, Y	Rd ← SRAM[Y]	2	
LD Rd, Y+	Rd ← SRAM[Y]; Y ← Y+1	2	
LD Rd, −Y	Y ← Y−1; Rd ← SRAM[Y]	2	
LDD Rd, Y+q	Rd ← SRAM[Y+q] q: 0–63	2	
ST Y, Rr	SRAM[Y] ← Rr	2	
ST Y+, Rr	SRAM[Y] ← Rr; Y ← Y+1	2	
ST −Y, Rr	Y ← Y−1; SRAM[Y] ← Rr	2	
STD Y+q, Rr	SRAM[Y+q] ← Rr q: 0–63	2	
/* Z register (R31:R30) indirect */			
LD Rd, Z	Rd ← SRAM[Z]	2	
LD Rd, Z+	Rd ← SRAM[Z]; Z ← Z+1	2	
LD Rd, −Z	Z ← Z−1; Rd ← SRAM[Z]	2	
LDD Rd, Z+q	Rd ← SRAM[Z+q] q: 0–63	2	
ST Z, Rr	SRAM[Z] ← Rr	2	
ST Z+, Rr	SRAM[Z] ← Rr; Z ← Z+1	2	
ST −Z, Rr	Z ← Z−1; SRAM[Z] ← Rr	2	
STD Z+q, Rr	SRAM[Z+q] ← Rr q: 0–63	2	

21.7.1 Load from Program Memory (Flash)

Opcode	Operation	Cycles	Notes
LPM	R0 ← FLASH[Z]	3	Z = byte address
LPM Rd, Z	Rd ← FLASH[Z]	3	
LPM Rd, Z+	Rd ← FLASH[Z]; Z ← Z+1	3	
ELPM	R0 ← FLASH[RAMPZ:Z]	3	>64KB flash devices
ELPM Rd, Z	Rd ← FLASH[RAMPZ:Z]	3	
ELPM Rd, Z+	Rd ← FLASH[RAMPZ:Z]; Z++	3	
SPM	FLASH[Z] ← R1:R0	varies	Boot section only

21.8 I/O Instructions

Opcode	Operation	Cycles	Notes
IN Rd, A	Rd ← I/O[A]	1	A: 0–63
OUT A, Rr	I/O[A] ← Rr	1	A: 0–63
PUSH Rr	SRAM[SP] ← Rr; SP ← SP−1	2	
POP Rd	SP ← SP+1; Rd ← SRAM[SP]	2	

21.9 Miscellaneous

Opcode	Operation	Cycles	Notes
NOP	No operation	1	
SLEEP	Enter sleep mode	1	MCU-defined behaviour
WDR	Reset watchdog timer	1	
BREAK	Stop for on-chip debug system	1	

21.10 Instruction Encoding Summary

Most AVR instructions are 16-bit (one word). 32-bit instructions:

```
32-bit instructions (2 words):
  LDS Rd, k    — word 1: opcode+Rd; word 2: 16-bit address k
  STS k, Rr    — word 1: opcode+Rr; word 2: 16-bit address k
  CALL k       — word 1: 0x940E+kH; word 2: kL (22-bit address)
  JMP  k       — word 1: 0x940C+kH; word 2: kL (22-bit address)
```

All other instructions are 16-bit. The assembler handles encoding automatically.

21.11 GCC ABI Register Conventions

Registers	Role	Save by
R0	Temporary, MUL result	Caller (assume destroyed)
R1	Always zero (GCC)	Callee must restore if modified
R2–R17	Call-saved	Callee (push/pop if used)
R18–R27	Call-clobbered	Caller (assume destroyed)
R28–R29 (Y)	Frame pointer	Callee
R30–R31 (Z)	Call-clobbered	Caller

```
Return values:
  8-bit:  R24
  16-bit: R25:R24
  32-bit: R25:R24:R23:R22 (little-endian: R22=lowest byte)

Arguments (first to last):
  arg1: R25:R24  (or R24 for 8-bit)
  arg2: R23:R22
  arg3: R21:R20
  arg4: R19:R18
  More: pushed on stack (rightmost first)
```

After `MUL`/`MULS`/`MULSU`: always restore `R1 = 0` before returning or calling any C function. `R0` is implicitly scratch.

Chapter 22

GAS Directive Reference

GNU Assembler (`avr-as`) directives used in AVR assembly. Directives begin with `.` and are not instructions — they control how the assembler lays out output.

22.1 Section Directives

```
Directive                     Effect
─────────────────────────────────────────────────────────────────────────────
.section name [,"flags"[,@type]]
                              Switch to named section. Common flags:
                                a = allocatable
                                w = writable
                                x = executable
                              Types: @progbits (data), @nobits (BSS, zero-init)

.text                         Switch to .text (code, flash)
.data                         Switch to .data (initialised SRAM variables)
.bss                          Switch to .bss  (zero-initialised SRAM)
.rodata                       Read-only data (constants in flash on AVR)
```

Common section usage on AVR:

```
.section .text          /* executable code (flash) */
.section .data          /* initialised variables (SRAM; copied from flash at startup) */
.section .bss           /* zero-initialised variables (SRAM) */
.section .rodata        /* constants (flash; access via LPM) */
.section .vectors,"ax",@progbits   /* interrupt vector table */
.section .noinit,"aw",@nobits      /* SRAM variables NOT initialised at reset */
```

22.2 Symbol Directives

```
Directive                     Effect
─────────────────────────────────────────────────────────────────────────────
.global symbol                Export symbol (visible to linker / other files)
```

.local	symbol	Mark symbol local (file scope only)
.weak	symbol	Weak symbol: overridden if another file defines it
.type	symbol, @function	Mark symbol as function (for debugging/ELF info)
.type	symbol, @object	Mark symbol as data object
.size	symbol, size	Set symbol size (bytes)
.set	symbol, expr	Define symbol = expr (reassignable)
.equ	symbol, expr	Define symbol = expr (not reassignable; synonym: .equiv)

Example:

```
.global uart_init
.type   uart_init, @function
uart_init:
    /* ... */
    ret
.size uart_init, . - uart_init   /* size = current address minus symbol start */
```

22.3 Data Directives

Directive	Size per item	Example
.byte expr[,…]	1 byte	.byte 0x3F, 42, 'A'
.2byte expr[,…]	2 bytes LE	.2byte 0x1234 /* = 0x34, 0x12 in memory */
.word expr[,…]	2 bytes LE	.word 1000 /* AVR word = 2 bytes */
.long expr[,…]	4 bytes LE	.long 0xDEADBEEF
.quad expr[,…]	8 bytes LE	.quad 0
.ascii "str"	n bytes	.ascii "hello" /* no null terminator */
.asciz "str"	n+1 bytes	.asciz "hello" /* null terminated */
.string "str"	n+1 bytes	.string "hello" /* alias for .asciz */
.space n [,fill]	n bytes	.space 16, 0xFF /* fill with 0xFF */
.zero n	n zero bytes	.zero 64 /* alias for .space n, 0 */
.skip n [,fill]	n bytes	.skip 4 /* alias for .space */
.fill reps,size,val		.fill 8, 1, 0xAA /* 8 bytes of 0xAA */

22.4 Alignment Directives

Directive	Effect
.align n	Align to 2^n byte boundary (with NOP padding in .text)
.balign n [,fill]	Align to n byte boundary
.p2align n	Align to 2^n boundary (same as .align on most targets)
.org expr	Set location counter to expr (absolute address)

On AVR, `.align 1` aligns to 2-byte boundary (one AVR word). `.align 2` aligns to 4 bytes. Useful before jump tables and 16-bit data.

22.5 Conditional Assembly

Directive	Effect
`.ifdef sym`	`Include if sym is defined`
`.ifndef sym`	`Include if sym is NOT defined`
`.if expr`	`Include if expr != 0`
`.else`	`Else branch`
`.elif expr`	`Else-if`
`.endif`	`End conditional block`
`.error "msg"`	`Abort assembly with message`
`.warning "msg"`	`Emit warning, continue`

Example — select multiply implementation at assemble time:

```
#ifdef NOMUL
    rcall mul8u_shiftadd
#else
    mul   r16, r17
    movw  r24, r0
    clr   r1
#endif
```

When using `avr-gcc` as the front-end, `#ifdef`/`#define` (C preprocessor) work in `.S` files. For plain `avr-as`, use `.ifdef`/`.equ`.

22.6 Macro Directives

Directive	Effect
`.macro name [params]`	`Begin macro definition`
`.endm`	`End macro definition`
`.exitm`	`Exit macro early`
`.rept n`	`Repeat enclosed block n times`
`.endr`	`End .rept block`
`.irp sym, values`	`Iterate: sym takes each value in turn`
`.endr`	
`.irpc sym, chars`	`Iterate over characters of a string`
`.endr`	

Example — macro for a 16-bit load:

```
.macro  ldw  reg_h, reg_l, value
    ldi   \reg_l, lo8(\value)
    ldi   \reg_h, hi8(\value)
.endm

/* Usage */
ldw   r25, r24, 1999    /* expands to: ldi r24, lo8(1999); ldi r25, hi8(1999) */
```

Example — `.rept` for vector table padding:

```
__vectors:
    rjmp  reset_handler
```

```
    .rept 25
    rjmp  dummy_isr        /* fill remaining 25 vectors */
    .endr
```

22.7 Include Directives

Directive	Effect
.include "file"	Textually include file at this point
#include <file>	C preprocessor include (only when assembled via avr-gcc)

For AVR with `avr-gcc -x assembler-with-cpp` (the default for `.S` files):

```
#include <avr/io.h>        /* defines I/O register names, bit names, RAMEND, etc. */
```

`<avr/io.h>` selects the correct device header based on `-mmcu=` flag.

22.8 Listing and Debug Directives

Directive	Effect
.file "name"	Set source file name (for debug info)
.line n	Set source line number
.loc file line [col]	DWARF location (used by avr-gcc; rarely hand-written)
.stabs "str",n,n,n,n	STABS debug info (legacy)
.ident "string"	Embed comment string in .comment ELF section

22.9 Expression Operators

GAS expressions can use these operators in directives and immediate operands:

Operator	Example	Meaning
+	.byte 1+2	Addition
-	.byte 5-3	Subtraction
*	.word 16*1000	Multiplication
/	.byte 255/2	Integer division
%	.byte 17%10	Modulo
<<	.word 1<<8	Left shift
>>	.word 256>>1	Right shift
&	.byte 0xFF & val	Bitwise AND
|	.byte flags | mask	Bitwise OR
^	.byte a ^ b	Bitwise XOR
~	.byte ~mask	Bitwise NOT
lo8(x)	ldi r16, lo8(val)	Low byte of 16-bit value
hi8(x)	ldi r16, hi8(val)	High byte of 16-bit value
hlo8(x)		Byte 2 (bits 23:16) of 32-bit value
hhi8(x)		Byte 3 (bits 31:24) of 32-bit value

```
pm(x)      .word pm(label)     Program memory word address (byte addr >> 1)
.          (dot)               Current location counter (address)
```

22.10 Common Patterns

22.10.1 Define a constant

```
.equ  F_CPU,   16000000
.equ  BAUD,    9600
.equ  UBRR_V,  (F_CPU / (16 * BAUD)) - 1   /* = 103 */
```

22.10.2 Reserve SRAM variable

```
.section .bss
my_var:  .byte 0    /* 1-byte variable */
my_word: .2byte 0   /* 2-byte variable, aligned */
my_buf:  .space 64  /* 64-byte buffer */
```

22.10.3 Initialised variable (copied to SRAM at startup by avr-libc __init)

```
.section .data
init_val: .byte 42
```

22.10.4 Constant in flash (accessed via LPM)

```
.section .rodata
my_table:
    .byte 0, 1, 1, 2, 3, 5, 8, 13   /* Fibonacci */
```

22.10.5 Local labels

```
func:
    ldi   r16, 10
1:                  /* local label — referenced as 1f (forward) or 1b (back) */
    dec   r16
    brne  1b        /* branch back to label 1 */
    ret
```

Local labels (numeric) can be reused across the file; `1f` means "next `1:` going forward", `1b` means "previous `1:` going backward". They do not pollute the symbol table.

Chapter 23

Linker Script Walkthrough

The linker script tells `avr-ld` where to place each section in the output ELF file and ultimately in the device's address space. For most projects `avr-gcc` selects a built-in script automatically via `-mmcu=`. Understanding linker scripts is necessary when:

- Writing a bootloader (section placement at a specific flash address).
- Placing variables in a specific SRAM region.
- Using `.noinit` for variables that survive a software reset.
- Adding custom sections (e.g., a parameter page at a known flash address).

23.1 Anatomy of a Linker Script

A linker script has three main constructs:

```
MEMORY { }        — declares memory regions (name, origin, length)
SECTIONS { }      — maps input sections to output sections and memory regions
PROVIDE() / ENTRY() — define special symbols and the entry point
```

23.1.1 Minimal AVR Linker Script

```
/* minimal_avr.ld — bare-bones script for ATmega328P */

MEMORY
{
  flash  (rx)  : ORIGIN = 0x000000, LENGTH = 32K
  sram   (rw!x): ORIGIN = 0x800100, LENGTH = 2K
}

SECTIONS
{
  /* Interrupt vector table — must be at flash origin */
  .vectors :
  {
    KEEP(*(.vectors))
  } > flash
```

```
  /* Executable code */
  .text :
  {
    *(.text)
    *(.text.*)
  } > flash

  /* Read-only data (constants accessed via LPM) */
  .rodata :
  {
    *(.rodata)
    *(.rodata.*)
  } > flash

  /* Initialised data: stored in flash, copied to SRAM at startup */
  .data :
  {
    _data_start = .;
    *(.data)
    *(.data.*)
    _data_end = .;
  } > sram AT > flash          /* VMA in sram, LMA in flash */

  /* Zero-initialised data: only reservation in SRAM */
  .bss :
  {
    _bss_start = .;
    *(.bss)
    *(.bss.*)
    *(COMMON)
    _bss_end = .;
  } > sram

  /* Variables preserved across software reset (not zeroed at startup) */
  .noinit (NOLOAD) :
  {
    *(.noinit)
  } > sram

  /* Stack: grows downward from RAMEND */
  _stack_top = ORIGIN(sram) + LENGTH(sram) - 1;
}
```

23.2 MEMORY Region Flags

```
r   — readable
w   — writable
x   — executable
!   — negate following flags

Common combinations:
```

```
(rx)   — read-execute (flash)
(rw!x) — read-write, not executable (SRAM)
(rx!w) — read-execute, not writable (flash, explicit)
```

23.3 Address Spaces on AVR

AVR uses a **Harvard architecture** with separate address spaces:

```
Flash (program memory):
  avr-ld uses byte addresses with origin 0x000000
  ATmega328P: 0x000000–0x007FFF (32 KB)

SRAM (data memory):
  avr-ld uses byte addresses with origin 0x800000 + hardware_offset
  ATmega328P hardware SRAM starts at 0x0100
  avr-ld SRAM origin: 0x800100

  Registers:   0x0000–0x001F (R0–R31) — not in linker script
  I/O:         0x0020–0x005F — not in linker script
  Ext I/O:     0x0060–0x00FF — not in linker script
  SRAM:        0x0100–0x08FF (2 KB on ATmega328P)
```

The `0x800000` offset is an avr-ld convention to distinguish data addresses from code addresses in the ELF file. The hardware only sees the lower 16 bits.

23.4 VMA vs LMA

VMA (Virtual Memory Address): where the section lives at runtime (SRAM for `.data`).

LMA (Load Memory Address): where the section is stored in the ELF output (flash for `.data`, since flash is what gets programmed).

```
.data :
{
  _data_start = .;        /* VMA: SRAM address where data lives at runtime */
  *(.data)
  _data_end = .;
} > sram AT > flash        /* LMA: in flash, immediately after .rodata      */
```

The startup code (`__init` in avr-libc) copies `.data` from its LMA (flash) to its VMA (SRAM) before `main()` runs:

```
/* avr-libc __init pseudo-code */
    ldi   ZL, lo8(_data_lma_start)     /* source: flash LMA */
    ldi   ZH, hi8(_data_lma_start)
    ldi   YL, lo8(_data_start)         /* dest: SRAM VMA */
    ldi   YH, hi8(_data_start)
1:  lpm   r0, Z+
    st    Y+, r0
    cpi   ZL, lo8(_data_lma_end)
    brne  1b
```

If you use `-nostartfiles`, **you must perform this copy yourself** if any `.data` variables are used. The simplest approach: avoid initialised variables entirely, or use `.equ` constants in flash and load them explicitly.

23.5 KEEP()

The garbage-collector (`--gc-sections`) removes unreferenced sections. `KEEP()` prevents this:

```
KEEP(*(.vectors))    /* never discard the vector table */
```

When building with `avr-gcc -Wl,--gc-sections` (common for size optimisation), sections not reachable from the entry point are removed. `KEEP()` forces retention regardless of reachability.

23.6 Bootloader Placement

Place the bootloader at the boot section start address by overriding `.text` origin:

```
avr-gcc -Wl,--section-start=.text=0x7C00 ...
```

Or in a dedicated linker script:

```
/* bootloader.ld */
MEMORY
{
  boot_flash (rx) : ORIGIN = 0x007C00, LENGTH = 1K
  sram       (rw!x): ORIGIN = 0x800100, LENGTH = 2K
}

SECTIONS
{
  .text : { *(.vectors) *(.text) *(.text.*) } > boot_flash
  .bss  : { *(.bss) }                         > sram
}
```

```
avr-gcc -mmcu=atmega328p -nostartfiles \
        -T bootloader.ld \
        -o bootloader.elf bootloader.S
```

23.7 Parameter Page at Fixed Flash Address

Store calibration data at a known, fixed flash address so the application can find it regardless of code size changes:

```
SECTIONS
{
  /* ... normal sections ... */

  .params :
```

```
  {
    *(.params)
  } > flash
  . = 0x7000;         /* force next section to start at 0x7000 */
  .calibration :
  {
    KEEP(*(.calibration))
  } > flash
}
```

In source:

```
.section .calibration
cal_offset: .word 0x0032    /* factory calibration: ADC offset */
cal_gain:   .word 0x0100    /* Q8.8 gain factor */
```

Access at runtime:

```
ldi   ZL, lo8(cal_offset)
ldi   ZH, hi8(cal_offset)
lpm   r24, Z+
lpm   r25, Z        /* r25:r24 = cal_offset value */
```

23.8 .noinit Section

Variables in `.noinit` are **not zeroed** at startup — they retain whatever value was in SRAM before reset. Useful for reset reason tracking:

```
.section .noinit,"aw",@nobits
reset_reason: .byte 0    /* written before software reset; survives it */
```

Application writes a code before resetting:

```
ldi   r16, 0xAA
sts   reset_reason, r16
/* ... trigger watchdog reset ... */
```

After reset, check `reset_reason` before the BSS clear to determine why the device restarted. BSS clear (zeroing `.bss`) happens in `__init` — `.noinit` is placed in a separate section that `__init` never touches.

23.9 Useful Linker Symbols

These symbols are defined by the default avr-gcc linker script and available in assembly via `lo8()/hi8()`:

Symbol	Value
`__data_start`	VMA start of .data in SRAM
`__data_end`	VMA end of .data in SRAM
`__data_load_start`	LMA of .data in flash (copy source for __init)
`__bss_start`	Start of .bss in SRAM

```
__bss_end            End of .bss in SRAM
__heap_start         Start of heap (after .bss)
__stack              Initial stack pointer value (= RAMEND)
__TEXT_REGION_ORIGIN__ Start of .text (usually 0)
__DATA_REGION_ORIGIN__ Start of SRAM (usually 0x800100)
```

Access in assembly:

```
ldi   r26, lo8(__bss_start)
ldi   r27, hi8(__bss_start)
ldi   r24, lo8(__bss_end - __bss_start)  /* BSS length */
```

23.10 Inspecting Linker Output

```
# Show section layout (addresses, sizes)
avr-objdump -h firmware.elf

# Show all symbols with addresses
avr-nm --numeric-sort firmware.elf

# Show memory map (verbose linker output)
avr-gcc -Wl,-Map=firmware.map ... && cat firmware.map

# Show disassembly with source interleaved
avr-objdump -dS firmware.elf

# Verify flash usage
avr-size --mcu=atmega328p --format=avr firmware.elf
```

Sample avr-size output with AVR format:

```
AVR Memory Usage
----------------
Device: atmega328p

Program:    486 bytes (1.5% Full)
(.text + .data + .bootloader)

Data:         3 bytes (0.1% Full)
(.data + .bss + .noinit)
```

23.11 Common Linker Errors

Error	Cause / Fix
undefined reference to `symbol'	Symbol not defined in any linked file; add the .S or .o file that defines it.
relocation truncated to fit: R_AVR_13_PCREL against `symbol'	Branch offset exceeds range (e.g. RJMP only reaches ±2KB); use JMP or

```
                                       reorganise code.

section `.text' will not fit in        Total code exceeds flash; reduce size
region `flash'                         or use a larger device.

cannot find entry symbol reset_handler Define reset_handler or use
                                       -e reset_handler linker flag.

multiple definition of `__vector_N'   Two .S files both define the same ISR
                                       vector entry; remove the duplicate.
```

Chapter 24

avrdude Flashing Cheat-Sheet

avrdude is the standard tool for programming AVR microcontrollers. It communicates with the device via a programmer (USB, serial, SPI ISP) and can read/write flash, EEPROM, and fuse bytes.

24.1 Basic Syntax

```
avrdude -p PARTNO -c PROGRAMMER [-P PORT] [-b BAUD] -U MEMTYPE:OP:FILE[:FORMAT]
```

```
-p PARTNO     Device (e.g. m328p, t3217, m2560). See Section E.4 for list.
-c PROGRAMMER Programmer type. See Section E.3.
-P PORT       Port (e.g. /dev/ttyUSB0, /dev/ttyACM0, usb).
-b BAUD       Baud rate (for serial programmers; default depends on programmer).
-U            Memory operation (can be repeated for multiple operations).
-v            Verbose output. -vv for more detail.
-n            Dry run: do not write anything.
-e            Erase flash before programming.
-F            Override signature check (use with caution).
```

24.1.1 -U Flag Format

```
MEMTYPE:OP:FILE[:FORMAT]

MEMTYPE:
  flash    – program memory
  eeprom   – EEPROM
  lfuse    – low fuse byte
  hfuse    – high fuse byte
  efuse    – extended fuse byte
  lock     – lock bits
  signature – device signature (read only)
  calibration – oscillator calibration (read only)

OP:
  r – read from device, write to FILE
  w – write FILE to device
```

```
  v – verify device matches FILE

FORMAT (optional, auto-detected if omitted):
  i – Intel HEX
  s – Motorola S-record
  r – raw binary
  e – ELF
  h – hex string
  b – byte value (for single-byte ops like fuses)
  m – immediate value (e.g. 0xDE)
```

24.2 Most-Used Commands

24.2.1 Flash a HEX file

```
# Arduino Uno (ATmega328P, USB-serial bootloader on /dev/ttyUSB0)
avrdude -p m328p -c arduino -P /dev/ttyUSB0 -b 115200 \
        -U flash:w:firmware.hex:i

# USBasp programmer (no port needed)
avrdude -p m328p -c usbasp \
        -U flash:w:firmware.hex:i

# AVRISP mkII (USB)
avrdude -p m328p -c avrispmkII \
        -U flash:w:firmware.hex:i
```

24.2.2 Read flash to file

```
avrdude -p m328p -c usbasp \
        -U flash:r:backup.hex:i
```

24.2.3 Verify only (no write)

```
avrdude -p m328p -c usbasp \
        -U flash:v:firmware.hex:i
```

24.2.4 Erase then write

```
avrdude -p m328p -c usbasp -e \
        -U flash:w:firmware.hex:i
```

24.2.5 Write EEPROM

```
avrdude -p m328p -c usbasp \
        -U eeprom:w:data.hex:i
```

24.2.6 Read fuse bytes

```
avrdude -p m328p -c usbasp \
        -U lfuse:r:-:h -U hfuse:r:-:h -U efuse:r:-:h
# Output: e.g. 0xff 0xd9 0x07 (lfuse hfuse efuse)
# '-' = stdout, 'h' = hex string format
```

24.2.7 Write fuse bytes

```
# WARNING: incorrect fuse values can brick the device.
# Always calculate fuses with an online calculator and verify before writing.

# Set hfuse = 0xDE (BOOTRST=0, BOOTSZ=10 for 1KB bootloader section)
avrdude -p m328p -c usbasp \
        -U hfuse:w:0xDE:m

# Restore ATmega328P factory fuses (lfuse=0xFF, hfuse=0xD9, efuse=0x07)
avrdude -p m328p -c usbasp \
        -U lfuse:w:0xFF:m \
        -U hfuse:w:0xD9:m \
        -U efuse:w:0x07:m
```

Warning: Fuses control clock source, brown-out detection, bootloader region, and JTAG. Setting CKSEL bits for an external crystal when no crystal is connected will make the device unresponsive — recovery requires an ISP programmer that provides a clock signal (High-Voltage Serial Programming).

24.2.8 Multiple operations in one command

```
avrdude -p m328p -c usbasp \
        -U flash:w:firmware.hex:i \
        -U eeprom:w:eeprom_data.hex:i
```

24.3 Common Programmers

Programmer ID	Description
arduino	Arduino bootloader via serial (auto-reset on DTR)
arduino-ft232r	Arduino via FTDI FT232R cable
avrisp	Atmel AVRISP v1 (serial)
avrisp2	Atmel AVRISP mkII (USB HID); also: avrispmkII
usbasp	USBasp (cheap USB ISP clone, widely available)
usbtiny	USBtiny / Adafruit USBtinyISP
dragon_isp	AVR Dragon in ISP mode
dragon_jtag	AVR Dragon in JTAG mode
jtag2isp	JTAGICE mkII in ISP mode
jtag2	JTAGICE mkII
atmelice_isp	Atmel-ICE in ISP mode
atmelice_pdi	Atmel-ICE in PDI mode (XMEGA)
atmelice_updi	Atmel-ICE in UPDI mode (tinyAVR/megaAVR 0/1/2-series)

```
pymcuprog       Microchip MPLAB Snap / Curiosity Nano via pymcuprog bridge
serialupdi      UPDI via USB-serial adapter (cheap, 1-wire)
```

24.3.1 UPDI Devices (ATtiny3217 and tinyAVR 1-series)

The ATtiny3217 uses UPDI (Unified Program and Debug Interface), a single-wire protocol. Use `serialupdi` with a USB-serial adapter connected to the UPDI pin via a 4.7kΩ resistor:

```
# ATtiny3217 via serialupdi on /dev/ttyUSB0
avrdude -p t3217 -c serialupdi -P /dev/ttyUSB0 \
        -U flash:w:firmware.hex:i

# Read fuses
avrdude -p t3217 -c serialupdi -P /dev/ttyUSB0 \
        -U fuse0:r:-:h -U fuse1:r:-:h -U fuse2:r:-:h \
        -U fuse4:r:-:h -U fuse5:r:-:h -U fuse6:r:-:h -U fuse7:r:-:h
```

UPDI wiring (1-wire):

```
USB-serial TX ──┐
                ├── 4.7kΩ ── UPDI pin (ATtiny3217 pin 16)
USB-serial RX ──┘
USB-serial GND ─── GND
```

Alternatively, use the Atmel-ICE or MPLAB PICkit 4 for UPDI.

24.4 Device Part Numbers

Part Number	Device
m328p	ATmega328P (Arduino Uno, Nano)
m328pb	ATmega328PB
m2560	ATmega2560 (Arduino Mega)
m32u4	ATmega32U4 (Arduino Leonardo, Pro Micro)
m168p	ATmega168P
m88p	ATmega88P
m48p	ATmega48P
t3217	ATtiny3217 (book target for tinyAVR 1-series)
t1616	ATtiny1616
t816	ATtiny816
t416	ATtiny416
t85	ATtiny85
t84	ATtiny84
t45	ATtiny45
t13	ATtiny13
x128a4u	ATxmega128A4U

Full list: `avrdude -p ?`

24.5 Fuse Byte Reference (ATmega328P)

24.5.1 Low Fuse (LFUSE)

```
Bit   Name     Description
---------------------------------------------------------------------------
7     CKDIV8   Divide clock by 8 (0 = enabled; factory default)
6     CKOUT    Clock output on PB0 (0 = enabled)
5:4   SUT1:0   Start-up time (10 = default 14CK + 65ms)
3:0   CKSEL3:0 Clock source (0010 = internal 8MHz RC; 1111 = external crystal)

Factory default (internal RC, /8): LFUSE = 0x62
No divide (internal 8MHz full speed): LFUSE = 0xE2
External 16MHz crystal (Arduino): LFUSE = 0xFF
```

24.5.2 High Fuse (HFUSE)

```
Bit   Name      Description
---------------------------------------------------------------------------
7     RSTDISBL  Disable reset pin (0 = PC6 becomes I/O; DANGEROUS)
6     DWEN      debugWIRE enable (0 = enabled; disables ISP when set)
5     SPIEN     SPI programming enable (0 = enabled; keep this 0)
4     WDTON     Watchdog always on (0 = always on)
3     EESAVE    Preserve EEPROM during chip erase (0 = preserved)
2:1   BOOTSZ1:0 Boot section size (11=256B, 10=512B, 01=1KB, 00=2KB)
0     BOOTRST   Boot reset vector (0 = reset jumps to boot section)

Factory default: HFUSE = 0xD9
  (SPI enabled, WDT off, EEPROM erased, 2KB boot, reset to 0x0000)
Bootloader (1KB boot, reset to boot section): HFUSE = 0xDE
```

24.5.3 Extended Fuse (EFUSE)

```
Bit   Name       Description
---------------------------------------------------------------------------
2:0   BODLEVEL2:0 Brown-out detection level
      111 = BOD disabled (factory)
      110 = 1.8V
      101 = 2.7V
      100 = 4.3V

Factory default: EFUSE = 0x07 (BOD disabled; only bits 2:0 used)
```

24.5.4 Online Fuse Calculator

The Engbedded AVR Fuse Calculator provides a GUI for calculating fuse values. Search "AVR fuse calculator" — cross-check with the device datasheet before programming.

24.6 Generating Intel HEX from ELF

```
avr-objcopy -O ihex firmware.elf firmware.hex

# EEPROM section separately
avr-objcopy -O ihex -j .eeprom \
            --set-section-flags=.eeprom=alloc,load \
            --change-section-lma .eeprom=0 \
            firmware.elf eeprom.hex
```

24.7 Complete Build + Flash Script

```
#!/bin/bash
# build_flash.sh — assemble, link, and flash ATmega328P
set -e

MCU=atmega328p
PORT=/dev/ttyUSB0
BAUD=115200
SRC=firmware.S

avr-gcc -mmcu=${MCU} -nostartfiles -o firmware.elf ${SRC}
avr-size firmware.elf
avr-objcopy -O ihex firmware.elf firmware.hex
avrdude -p ${MCU} -c arduino -P ${PORT} -b ${BAUD} \
        -U flash:w:firmware.hex:i
echo "Done."
```

```
# For ATtiny3217 via serialupdi
MCU=attiny3217
PORT=/dev/ttyUSB0

avr-gcc -mmcu=${MCU} -nostartfiles -o firmware.elf firmware.S
avr-objcopy -O ihex firmware.elf firmware.hex
avrdude -p ${MCU} -c serialupdi -P ${PORT} \
        -U flash:w:firmware.hex:i
```

24.8 Troubleshooting

```
Problem                           Likely cause / fix
------------------------------------------------------------------------------
avrdude: stk500_recv(): timeout   Wrong port, wrong baud, board not reset.
                                  Try -P /dev/ttyACM0, check USB cable.

avrdude: Expected signature       Wrong -p part number; use avrdude -p ? to list
for ATmega328P is 1E 95 0F        all; read signature with -U signature:r:-:h.

avrdude: verification error       Flash write failed; bad connection, insufficient
```

	power, or write-protected region.
avrdude: error: usbasp, Error: Could not find USB device	USBasp firmware too old for target; update USBasp firmware, or add -B 10 flag (slow SCK).
avrdude: initialization failed rc=-1; check connections and try again, or use -F	Target not responding; check power, RESET pin, JTAG enable fuse (JTAGEN=0 disables ISP on some devices).
Can't open device /dev/ttyUSB0 Permission denied	Add user to dialout group: sudo usermod -a -G dialout $USER Then log out and back in.

24.8.1 USBasp Slow Clock

If ISP communication fails with a freshly-soldered board or a device that has been running at low speed (CKDIV8 fuse set):

```
# Slow down SCK to 125 kHz (-B = bitclock period in µs; -B 8 = 125kHz SCK)
avrdude -p m328p -c usbasp -B 8 \
        -U lfuse:w:0xFF:m   # then reprogram fuses for 16MHz crystal
```

24.8.2 Arduino Auto-Reset Circuit

Arduino boards use a 100 nF capacitor on DTR/RST. `avrdude -c arduino` toggles DTR to reset the MCU before uploading. If the reset does not trigger:

```
# Manual: press reset button 2 seconds before avrdude starts, then release
avrdude -p m328p -c arduino -P /dev/ttyUSB0 -b 115200 \
        -U flash:w:firmware.hex:i
```

Or disable the auto-reset capacitor (cut the RESET EN trace on the Arduino PCB) when running a custom bootloader that does not expect the timing.